

Université de Montréal

**Highly Optimizing Ahead-of-Time Compilers for
Dynamic Languages**

par

Olivier Melançon

Département d'informatique et de recherche opérationnelle
Faculté des arts et des sciences

Thèse présentée en vue de l'obtention du grade de
Philosophiæ Doctor (Ph.D.)
en Informatique

18 septembre 2026

Université de Montréal

Faculté des arts et des sciences

Cette thèse intitulée

**Highly Optimizing Ahead-of-Time
Compilers for Dynamic Languages**

présentée par

Olivier Melançon

a été évaluée par un jury composé des personnes suivantes :

Eugene Syriani

(président-rapporteur)

Marc Feeley

(directeur de recherche)

Manuel Serrano

(codirecteur)

Sébastien Roy

(membre du jury)

Matthew Flatt

(examineur externe)

Résumé

Le *versionnement de blocs de base* anticipé est une technique de compilation servant à optimiser les programmes. Elle repose sur la duplication de blocs de base afin de générer, au sein d'un programme, des chemins spécialisés où certaines propriétés, telles que les types, peuvent être garanties à la compilation, ce qui permet des optimisations comme l'élimination des vérifications de type dans un langage dynamique.

Le versionnement de blocs de base anticipé s'appuie sur la théorie de l'interprétation abstraite, soit l'exécution d'un programme à la compilation sur un domaine de valeurs abstraites. Cette dernière calcule une approximation de la sémantique d'un programme, ensuite exploitée à des fins d'optimisation. Cette approximation se révèle souvent insuffisante dans le cas des langages dynamiques, où l'éventail des comportements possibles d'un programme varie largement selon ses entrées à l'exécution, menant à des approximations trop larges et moins utiles. Grâce à la duplication de code, le versionnement de blocs de base retrouve des approximations précises, permettant une meilleure génération de code. La duplication de code soulève cependant de nouveaux défis, puisque l'interprétation abstraite ne s'effectue plus sur un graphe de flot de contrôle statique, mais sur un graphe dynamique, ce qui requiert un cadre pour garantir la convergence de l'analyse, déterminer les contextes pour lesquels le programme doit être spécialisé et éviter l'explosion de la taille du code.

Cette thèse propose un cadre permettant de raisonner sur l'application conjointe d'interprétation abstraite et de duplication de code. Elle présente deux algorithmes : le *versionnement de blocs de base statique*, une technique de spécialisation de code qui élimine les vérifications de type des chemins de code critiques, et le *lambda-versionnement*, une extension interprocédurale qui permet l'optimisation de programmes entiers au moyen de fonctions à entrées et retours multiples. Dans certains cas, le lambda-versionnement génère des programmes spécialisés de façon optimale sur les types, c'est-à-dire des programmes qui vérifient le type de leurs entrées une seule fois à l'exécution, puis basculent vers un chemin d'exécution spécialisé ne nécessitant aucune vérification de type supplémentaire.

Mots-clés : interprétation abstraite, langages de programmation dynamiques, analyse de code, compilation anticipée, optimisation, duplication de code, fonctions à retours multiples

Abstract

Ahead-of-time *Basic Block Versioning* is a compilation technique for optimizing programs. It relies on the duplication and specialization of basic blocks to generate efficient paths in a program where certain properties, such as the type of values, can be guaranteed at compile-time, enabling optimizations such as type check elimination in dynamic languages.

Ahead-of-time Basic Block Versioning builds upon the theory of abstract interpretation, the execution of a program at compile-time over a domain of abstract values. Abstract interpretation finds a sound approximation of a program’s semantics that can be used for optimization. This approximation, however, frequently falls short for dynamic languages, where the range of possible program behavior varies widely depending on a program’s run-time inputs, leading to overly broad and significantly less useful approximations. By leveraging code duplication, Basic Block Versioning recovers precise approximations, enabling high-quality, ahead-of-time code generation for dynamic languages. Code duplication introduces new challenges, however, since abstract interpretation no longer takes place on a static control-flow graph but on a dynamic one, requiring a new framework to ensure that the analysis converges, to decide which contexts the program should be specialized for, and to avoid code bloat.

This thesis provides a framework for reasoning about joint abstract interpretation and code duplication. It presents two algorithms that apply such specialization: *Static Basic Block Versioning*, a technique for ahead-of-time code specialization that eliminates type checks from critical code paths, and *Lambda Versioning*, an interprocedural extension that enables whole-program optimization through multi-entry/multi-return functions. In some cases, Lambda Versioning generates optimally type-specialized programs, that is, programs that check the type of their input only once at run-time, then switch to a specialized code path that requires no further type checks.

Keywords: abstract interpretation, dynamic programming languages, code analysis, ahead-of-time compilation, optimization, code duplication, multi-return functions

Contents

Résumé	v
Abstract	vii
List of Figures	xiii
List of Abbreviations	xv
Remerciements	xvii
Introduction	1
Contributions	2
Structure of this Thesis	2
Chapter 1. Background	3
1.1. Abstract Interpretation	3
1.1.1. Union and widening	3
1.1.2. Narrowing	6
1.2. Dynamic Programming Languages	7
1.2.1. Abstract Interpretation of Dynamic Languages	8
1.2.2. Abstract Interpretation of Highly Dynamic Languages	8
1.3. Basic Block Versioning	9
1.3.1. Eager Basic Block Versioning	10
1.3.2. Lazy Basic Block Versioning	10
1.4. Supplementary Related Work	12
1.4.1. Just-in-Time Code Generation	13
1.4.1.1. Tracing JIT	13
1.4.1.2. Meta-Tracing JIT	13
1.4.1.3. Truffle/Graal	13
1.4.2. Ahead-of-Time Program Analysis	14

1.4.2.1.	Control-Flow Analysis	14
1.4.2.2.	Adaptive Type Analysis.....	14
1.4.2.3.	Code Duplication	14
1.4.2.4.	Inlining.....	14
1.4.3.	Runtime Optimizations	14
1.4.3.1.	Object Representation	14
1.4.3.2.	Inline Caches	15
1.4.3.3.	Superinstructions	15
Chapter 2.	Static Basic Block Versioning.....	17
2.1.	Overview	17
2.1.1.	Motivating Example	18
2.2.	The Static Basic Block Versioning Algorithm.....	19
2.2.1.	Data Structures.....	19
2.2.2.	The Algorithm.....	20
2.2.3.	Merge Heuristic.....	24
2.3.	Evaluation.....	25
2.3.1.	Dynamic Checks.....	26
2.3.2.	Program Size	27
2.3.3.	Execution Time.....	27
2.3.4.	Compile Time	28
2.4.	Limitations	29
2.5.	Conclusion.....	30
Chapter 3.	Lambda Versioning.....	31
3.1.	Motivating Examples	32
3.1.1.	Simple Example	32
3.1.2.	Arithmetic Operators	34
3.1.3.	Recursive Functions.....	36
3.2.	Hyperfunctions	37
3.2.1.	Rewriting Motivating Examples as Hyperfunctions.....	37
3.2.1.1.	<code>square</code> and <code>power4</code>	37

3.2.1.2. Arithmetic Operators	39
3.2.1.3. Recursive fact	39
3.2.2. Representation of Hyperfunctions	40
3.3. The Lambda Versioning Algorithm	43
3.3.1. Source Code Preprocessing	43
3.3.2. Multi-Source Reachability Algorithm	44
3.3.3. The Algorithm	44
3.4. An Improved Merge Heuristic	50
3.5. Evaluation	51
3.5.1. Experimental Setup	51
3.5.2. Type Checks	52
3.5.3. Compile Time	55
3.6. Hyperfunctions Implementation Related Work	58
3.6.1. Fortran 77	58
3.6.2. Multi-Return Function Calls	59
3.6.3. Contextual Dispatch	60
3.6.4. Extended Flat Closures	60
3.6.5. Call Site Splitting and Call Interface	60
3.6.6. Second-Class Continuations and Contification	61
3.6.7. Hardware-Level Implementation	61
3.7. Limitations	61
3.8. Conclusion	62
Chapter 4. Future Work	65
4.1. Data Structures, Mutability, and Escapability	65
4.2. Context Memoization and Constant Folding	68
4.3. Untagged Immediate Values	69
4.4. Application to Highly Dynamic Languages	69
4.5. Float Self-Tagging	70
4.6. Compiler Generation	71

Chapter 5. Conclusion	73
References	75
Appendix A. Merge Heuristics	83
A.1. Arithmetic Merge Heuristic.....	83
A.2. Similarity Merge Heuristic.....	84
A.3. Random Merge Heuristic	85
Appendix B. Hyperfunction Macro.....	87
Appendix C. Lambda Versioning Experimental Results.....	91
Appendix D. Static Basic Block Versioning Paper	117
Contribution Statement	117
Appendix E. Float Self-Tagging Paper	145
Contribution Statement	145

List of Figures

1	Control-flow graph of the sum-to-n function	4
2	Interval lattice	5
3	Type lattice	6
4	Control-flow graph of the fact function	7
5	Control-flow graph of fact after LBBV	12
6	Specialized control-flow graph of the find function	19
7	Data structures used in the SBBV algorithm	20
8	Separability of merge heuristic	24
9	Static Basic Block Versioning dynamic checks	26
10	Static Basic Block Versioning program size	27
11	Static Basic Block Versioning execution time	28
12	Static Basic Block Versioning compile time	29
13	Control-flow graph of the power4 function	34
14	Specialized control-flow graph of the power4 function	35
15	Specialized control-flow graph of the square function	36
16	Specialized control-flow graph of the fact function	37
17	Data structures used in the Λ V algorithm	45
18	Lambda Versioning micro benchmarks: type checks	52
19	Lambda Versioning macro benchmarks: type checks	53
20	Lambda Versioning micro benchmarks: compile time	54
21	Lambda Versioning macro benchmarks: compile time	55
22	Lambda Versioning micro benchmarks: code duplication	56
23	Lambda Versioning macro benchmarks: code duplication	57

24	Benchmark descriptions.....	92
25	Detailed micro benchmarks: type checks, arithmetic , inlined.....	93
26	Detailed micro benchmarks: type checks, similarity , inlined.....	94
27	Detailed micro benchmarks: type checks, random , inlined.....	95
28	Detailed micro benchmarks: type checks, arithmetic , hyperfunction.....	96
29	Detailed micro benchmarks: type checks, similarity , hyperfunction.....	97
30	Detailed micro benchmarks: type checks, random , hyperfunction	98
31	Detailed macro benchmarks: type checks, arithmetic , inlined	99
32	Detailed macro benchmarks: type checks, similarity , inlined	100
33	Detailed macro benchmarks: type checks, random , inlined	101
34	Detailed macro benchmarks: type checks, arithmetic , hyperfunction	102
35	Detailed macro benchmarks: type checks, similarity , hyperfunction	103
36	Detailed macro benchmarks: type checks, random , hyperfunction	104
37	Detailed micro benchmarks: compile time, arithmetic , inlined	105
38	Detailed micro benchmarks: compile time, similarity , inlined	106
39	Detailed micro benchmarks: compile time, random , inlined	107
40	Detailed micro benchmarks: compile time, arithmetic , hyperfunction	108
41	Detailed micro benchmarks: compile time, similarity , hyperfunction	109
42	Detailed micro benchmarks: compile time, random , hyperfunction	110
43	Detailed macro benchmarks: compile time, arithmetic , inlined.....	111
44	Detailed macro benchmarks: compile time, similarity , inlined.....	112
45	Detailed macro benchmarks: compile time, random , inlined.....	113
46	Detailed macro benchmarks: compile time, arithmetic , hyperfunction.....	114
47	Detailed macro benchmarks: compile time, similarity , hyperfunction.....	115
48	Detailed macro benchmarks: compile time, random , hyperfunction.....	116

List of Abbreviations

AOT	Ahead-of-time
AST	Abstract syntax tree
BBV	Basic Block Versioning
CFA	Control-flow analysis
CFG	Control-flow graph
CPS	Continuation-passing style
DSL	Domain-specific language
EBBV	Eager Basic Block Versioning
JIT	Just-in-time
LBBV	Lazy Basic Block Versioning
ΛV	Lambda Versioning
SBBV	Static Basic Block Versioning

Remerciements

En 2016, j'ai obtenu mon baccalauréat, de justesse et au bord de l'épuisement. J'ai alors tout laissé derrière moi pour explorer le monde. À l'époque, je me disais que c'était ça, la *vraie vie*. Je me souviens distinctement de ces matins où je me réveillais à bord de Galadriel, mon voilier, dans les îles Gulf de Colombie-Britannique, sans âme qui vive à des kilomètres à la ronde, persuadé d'avoir bien choisi : le monde universitaire n'est pas pour moi. Je n'y remettrai plus jamais les pieds.

Avoir le privilège de relever cette ironie en soumettant aujourd'hui ma thèse, je le dois à toutes ces personnes qui m'ont permis de m'engager intensément dans mes projets, de changer d'avis, d'incarner mes passions tout comme mes doutes, qui ont su quand me pousser à me dépasser et quand me permettre de me déposer, et qui ont transformé ce parcours doctoral en une si belle aventure. Je vous remercie du fond du cœur.

À Marc Feeley, merci pour ta disponibilité, ta générosité, ta passion et ta rigueur, et pour m'avoir permis de forger mon identité de chercheur.

À Manuel Serrano, merci pour ton ouverture, ton soutien et ton humour, et pour avoir été un modèle d'équilibre et d'humanité.

À tous les membres du Laboratoire de traitement parallèle : Aurel, Cassandre, Dominick, Frédéric, Jean-Alexandre, Jonathon, Laurent, Léonard, Luchino, Maël, Marc-André, Simon, Stefan. On dit parfois que le doctorat est une expérience solitaire. Grâce à vous, je ne me suis jamais senti seul.

À mes amis : Alexandre, Lorie-Anne (les deux), Martin, Maude, Nicolas, Olivier (les deux aussi !), Victor. Merci d'avoir été présents au fil des années, de m'avoir soutenu, de m'avoir conseillé. À Lorie-Anne et Victor, discuter de nos parcours respectifs au doctorat a été une précieuse source de réconfort. À Maude, merci pour nos traditionnels déjeuners qui ont si souvent été le meilleur moment de ma semaine.

À mes parents, je ne pense pas qu'il existe un seul projet dans lequel je puisse me lancer sans que vous croyiez en moi. Il n'existe donc aucun projet dans lequel je puisse me lancer sans croire en moi-même. À Nicolas et Pascale, avoir une telle admiration pour mon frère et ma sœur est une source d'inspiration constante dans ma vie.

Cette thèse a été financée en partie par la Fondation Arbour et le Fonds de recherche du Québec. L'aventure qu'est la recherche ne peut avoir lieu sans ces organismes qui soutiennent la science. À ceux-ci et à tous les gens qui leur permettent d'exister, je suis reconnaissant.

Introduction

The last decades have seen the rise of dynamic programming languages, both in popularity and implementation efforts, with millions of work hours invested in compilers such as V8, SpiderMonkey and JavaScriptCore (JavaScript), CPython (Python), and CRuby (Ruby). The majority of this effort has been directed toward performant, multi-tiered, just-in-time (JIT) compilers, in particular for JavaScript, that offer good trade-offs between peak performance and warm-up. However, this success comes at the cost of increasingly complex compilers that a single programmer, or even a small team, would have no hope of constructing within a human lifetime [21]. This complexity also has an impact on soundness, as semantic discrepancies between compilation tiers lead to critical security bugs.¹

The push toward JIT compilation is motivated by the premise that efficient compilation of dynamic languages, such as JavaScript, Python, and Ruby, requires information that can be obtained only at run-time. These languages allow, and even encourage, highly dynamic features such as dynamic code loading, mutation of unsealed modules, and introspection. The looming possibility that such features may be used prevents static analysis from inferring run-time behavior, thereby limiting effective whole-program analysis and optimization.

While the strength of JIT compilation lies in its capacity to identify the run-time context it should optimize, an ahead-of-time (AOT) compiler can achieve a similar effect by optimizing for multiple contexts through code duplication. This has many advantages: peak performance without warm-up, portability to systems prohibiting run-time code generation, and single-tiered compilation that avoids semantic-discrepancy bugs affecting JITs.

This thesis presents an in-depth study of Static Basic Block Versioning (SBBV), a novel AOT compilation technique that leverages abstract interpretation and code duplication to generate optimized code for dynamic languages, together with its interprocedural extension, Lambda Versioning (Λ V). SBBV and Λ V unify multiple optimization techniques within a single framework, including constant folding, dead code elimination, loop unrolling, function inlining, tail duplication, and multiple dispatch. One strength of this approach is its reduction of whole-program optimization to a sequence of local choices (called *merges*) regarding

1. One recent example is a bug in V8 that allowed out-of-bounds memory writes [46].

which run-time contexts should be prioritized for optimization. This enables scalable, high-quality code generation guided by hand-written heuristics that are simple to reason about.

The primary metric used in this thesis is dynamic checks eliminated during program execution, including type, array bounds, and overflow checks. This measure reflects how effectively run-time contexts that warrant optimization are identified. Code size is also considered as an indicator of code quality, alongside execution time, which remains the ultimate performance metric targeted by optimizing compilers.

In this thesis, SBBV and AV are applied to the Scheme programming language, in particular within the Bigloo and Gambit Scheme compilers, which have been used as backends for JavaScript and Python AOT compilers. It is discussed how they can be extended so that optimizations performed in the host language (Scheme) may translate into optimizations in the source languages (JavaScript and Python).

Contributions

This thesis presents two main contributions.

- Static Basic Block Versioning, a technique for AOT optimizing compilation of dynamic languages.
- Lambda Versioning, an interprocedural extension of SBBV.

It also presents three minor contributions.

- An interface for multi-entry/multi-return functions, called *hyperfunctions*, in Scheme.
- An approach to implementing AV as a source-to-source transformation in Scheme.
- An empirical evaluation of SBBV and AV with three merge heuristics.

Structure of this Thesis

The remainder of this thesis is structured as follows:

- Chapter 1: background necessary to understand the rest of this thesis.
- Chapter 2: SBBV, an AOT optimizing compilation technique for dynamic languages.
- Chapter 3: AV, the interprocedural extension of SBBV.
- Chapter 4: overview of future research avenues.
- Chapter 5: concluding remarks.

This thesis also includes two published papers: *Static Basic Block Versioning* [42] and *Float Self-Tagging* [43], in Appendices D and E, respectively. While the main body of this thesis is self-contained, these are included as they contain relevant supplementary details.

Chapter 1

Background

This chapter presents background information relevant to the rest of this work. Section 1.1 discusses the theory of abstract interpretation upon which AOT Basic Block Versioning is built. Section 1.2 discusses what makes dynamic languages resistant to abstract interpretation. Section 1.3 presents *Lazy* Basic Block Versioning, a JIT variant of Basic Block Versioning, and *Eager* Basic Block Versioning, a distant and inconclusive attempt at implementing AOT Basic Block Versioning. Section 1.4 presents other relevant related work.

1.1. Abstract Interpretation

AOT Basic Block Versioning builds upon the theory of abstract interpretation [20], a framework for sound approximation of a program’s semantics. Abstract interpretation formalizes the relationship between a program’s concrete semantics (its exact run-time behavior) and its abstract semantics (an approximation of its behavior computed at compile-time). This approximation exposes information that an AOT compiler can exploit for optimization.

1.1.1. Union and widening

As an example, consider the following Scheme function that sums integers up to n .

```
1 (define (sum-to-n n)
2   (let loop ((i 0)
3             (acc 0))
4     (if (<= i n)
5         (loop (+ i 1) (+ acc i))
6         acc)))
```

This function is guaranteed to return a non-negative integer.¹ Abstract interpretation allows a compiler to deduce this guarantee by interpreting the function over an abstract

1. Scheme supports arbitrary size integers, hence there is no risk of wrap around due to overflow.

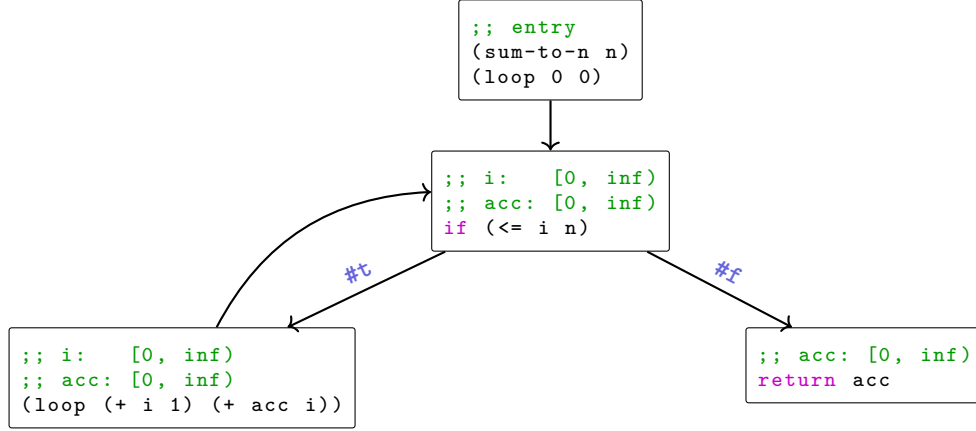


Figure 1 – Control-flow graph of the `sum-to-n` function. The annotation at the top of a block indicates the range in which values of the variables `i` and `acc` fall at run-time.

domain of objects that represent ranges of values instead of concrete ones (e.g., integer intervals rather than integers). This gives rise to a fixed-point algorithm that traverses the control-flow graph (CFG) of the function, updating the abstract objects at each basic block’s entry to be *at least* the union of all objects flowing out of its predecessors. The algorithm iterates until a fixed-point is reached.

One of the key insights of [20] is that the chosen domain of abstract objects must form a lattice where each pair of objects has a common upper bound, so that the union operation is always well-defined. This, however, is not sufficient to guarantee convergence for lattices with infinite height. To address this, the lattice must be accompanied by an operator that applies union *with widening*. This operator takes two abstract objects and returns an upper bound, with the additional property that ascending chains under widening eventually stabilize.

In practice, such operators are easy to construct: one can simply define a relatively small set of *relevant* abstract objects and a widening operator that maps any irrelevant object to the least upper bound among relevant objects. For instance, returning to the `sum-to-n` example, the lattice of all integer intervals can be used (shown in Figure 2). For most compiler optimizations, the precise range represented by an abstract object is irrelevant unless the object represents an exact singleton. Relevant properties of an integer may include its sign and whether it is representable in 8, 32, or 64 bits.

For instance, Figure 2 shows a lattice over integer intervals suitable for analyzing the `sum-to-n` function. The analysis uses a widening function that preserves only the sign and whether the integer is representable in 8, 32, or 64 bits. As abstract interpretation begins, the only abstract value encountered for `i` at the entry of the `loop` is the singleton 0. The red edges in Figure 2 represent the sequence of unions with widening for that variable, converging

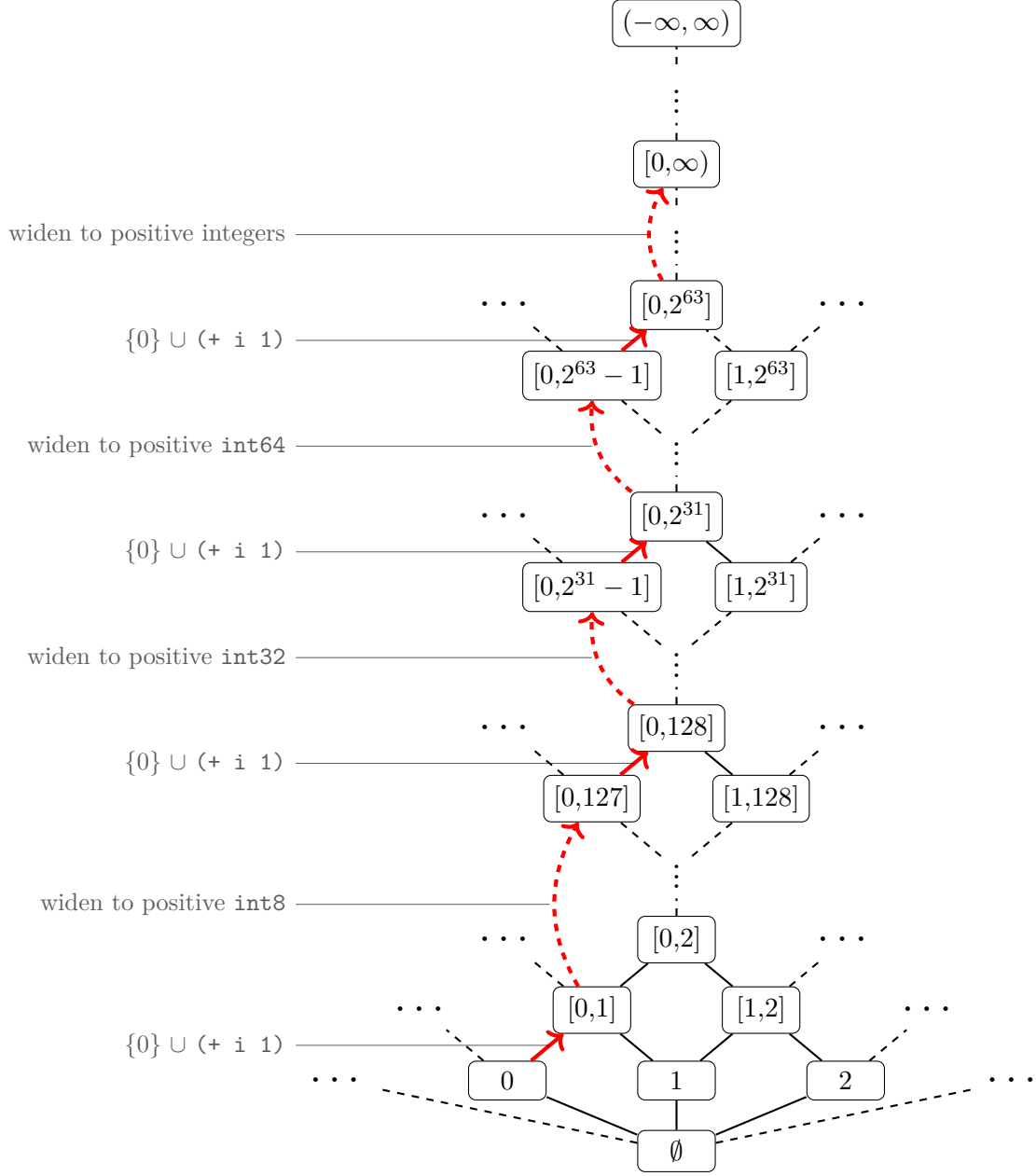


Figure 2 – Lattice abstracting integers as intervals $[a, b]$, including unbounded intervals. As the full lattice has infinite height, only relevant intervals are shown. The bottom layer contains the empty interval, representing an abstract object that can represent no concrete integer. The top layer contains the set of all integers, representing an abstract object covering any integer. Intermediate layers contain intervals of increasing size. The red edges show the sequence of unions with widening occurring at the entry of the `loop` during abstract interpretation of the CFG from Figure 1. Solid red edges correspond to union, and dashed red edges correspond to widening.

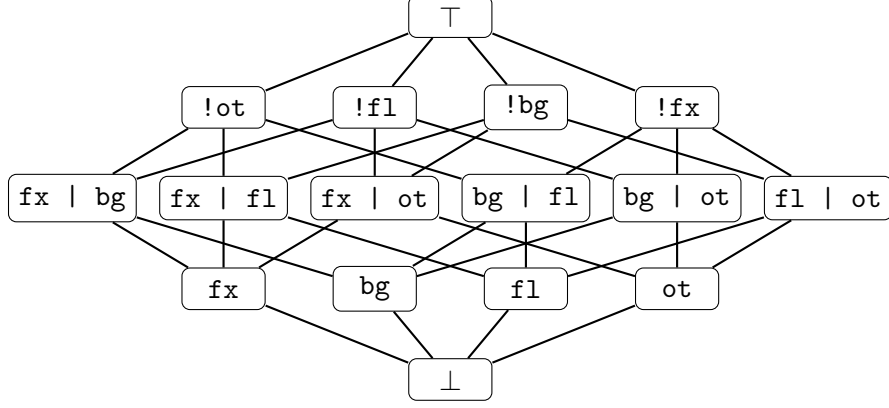


Figure 3 – A lattice of abstract objects representing the possible types of run-time values. The types are **fx** (fixnum, i.e., small integers), **bg** (big integers), **fl** (flonum, i.e., floating-point numbers), and **ot** representing all other types. The bottom layer contains $\perp$ (bottom), which represents an unreachable state (for instance, this is the type inferred for the return value of functions that always throw an exception). The second layer from the bottom contains exactly known types. The third layer contains unions of two types; for instance, **fx** | **fl** is an abstract object representing either a fixnum or a flonum at run-time. The fourth layer contains unions of three types, expressed concisely using the complement operator **!**. The top layer contains $\top$ (top), the union of all types.

to the interval $[0, \infty)$, which includes values exceeding the `int64` range. Hence, a compiler can deduce that the value of `i` is always non-negative, and similarly for the return value `acc`.

In the context of dynamic languages, where types are generally unknown statically and must be carried at run-time, abstract interpretation can discover type information about concrete values. One can apply abstract interpretation over a lattice of types, such as that shown in Figure 3, to narrow the possible types of run-time values to a set of types. Observe that the lattice of types has height equal to the number of types it contains; therefore, widening is not strictly necessary to guarantee convergence. However, the height of the lattice still influences the speed at which abstract interpretation reaches a fixed-point. Hence, widening can still be applied to speed up convergence.

1.1.2. Narrowing

While the union operation ensures that abstract interpretation yields a sound approximation of a program’s semantics and widening ensures that abstract interpretation terminates, information about the program’s semantics is acquired through *narrowing*. Narrowing refines abstract values at conditional branch points based on the outcome of the condition being

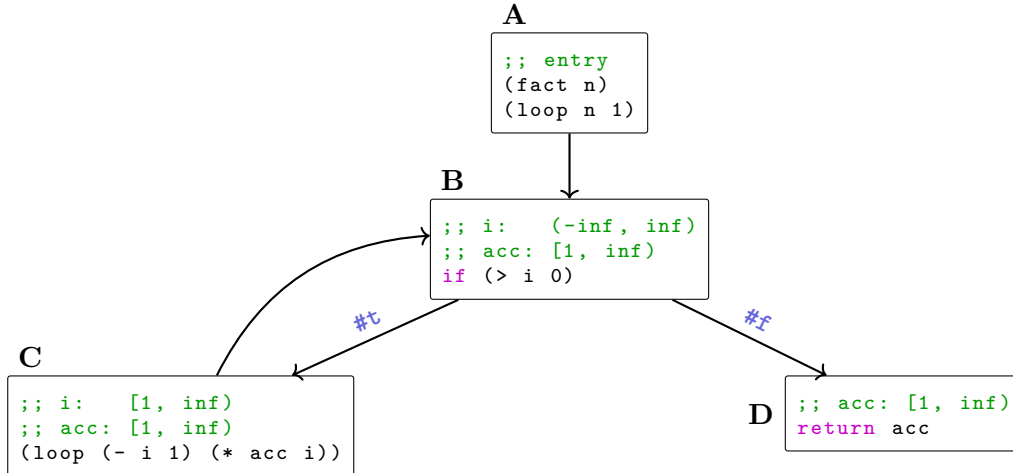


Figure 4 – Control-flow graph of the `fact` function. The annotation at the top of a block indicates the range in which values of the variables `i` and `acc` fall at run-time. For simplicity, the parameter `n` of the function is assumed to be an integer.

tested. On each branch, the abstract value of any variable involved in the condition can be restricted to the subset consistent with that branch.

As an example, consider the following factorial function in Scheme.

```

1 (define (fact n)
2   (let loop ((i n) (acc 1))
3     (if (> i 0)
4         (loop (- i 1) (* acc i))
5         acc)))

```

In the `sum-to-n` example from Section 1.1.1, the non-negativity of the result could be inferred directly from union, as the abstract values of `i` and `acc` always fell within non-negative intervals. The `fact` function differs because `i` takes an arbitrary parameter `n` (assumed to be an integer for simplicity) as its initial value and *decreases* with each iteration. Union with widening alone would widen `i` to $(-\infty, \infty)$. However, after the condition `(> i 0)` is tested, the abstract object representing `i` can be restricted to $[1, \infty)$ in the true branch. Figure 4 shows the annotated CFG of the `fact` function after abstract interpretation with narrowing. Abstract interpretation determines that `(* acc i)` always multiplies two positive integers, and that the return value is always positive.

1.2. Dynamic Programming Languages

Dynamic programming languages form a family of languages typically associated with dynamic typing, late binding, and dynamic code loading. Yet, in the nearly 70 years since Lisp was introduced as the first dynamic language [40], they have evolved so much that it

is useful to distinguish between early dynamic languages, such as Lisp and Scheme, and the more modern family of *highly* dynamic languages such as Python, Ruby and, to a lesser extent, JavaScript. Highly dynamic programming languages form a loosely defined subfamily of dynamic languages that allow *meta-dynamism*, which is the capability of programs to modify their own behavior at run-time, for instance by overloading arithmetic operators, rebinding standard library functions, or updating variables from outside their scope. Coupled with dynamic code loading, this leaves few guarantees to an optimizing AOT compiler, resulting in redundant run-time checks and dispatches, none of which can be safely eliminated.

1.2.1. Abstract Interpretation of Dynamic Languages

Dynamic languages pose particular challenges for abstract interpretation. Any value first encountered during abstract interpretation must be conservatively assigned the abstract type $\top$ (the top of the type lattice) unless more specific information can be established through narrowing. Furthermore, since abstract interpretation provides a sound approximation of a program's behavior through the union operator, broad execution contexts tend to propagate. Importantly, this is not a failure of abstract interpretation to reach a sufficiently precise approximation, it is an inherent property of many dynamic languages: valid inputs exist that can produce a wide range of execution behaviors the generated code must account for.

Again, consider the `fact` example from Section 1.1.2, in particular its CFG (Figure 4). In block B, the abstract object representing `i` *must* be $\top$, since the initial value of `i` is unconstrained. In block C, the type of `i` can be narrowed to `fx` | `fl` | `bg` since `(> i 0)` throws an exception on non-numeric values. Yet this still leaves three distinct number representations that arithmetic operators must handle. Consequently, while the return value is correctly narrowed to a positive number, whether it is represented as a fixnum, flonum, or bignum remains unknown at compile-time, requiring run-time checks to handle each case. This is not an overly broad approximation, as calling `(fact 3.0)` is semantically valid, even if this is unlikely to be the programmer's intent.

1.2.2. Abstract Interpretation of Highly Dynamic Languages

Highly dynamic languages pose extra challenges as meta-dynamism further hinders narrowing. For instance, consider the following Python program.

```
1 # 'main.py' module
2 import math
3
4 s = 0
5 for i in range(10):
6     s += i
7 print(s)
```

Because of late binding and dynamic code loading, this seemingly simple program can be made to print anything by placing the following `math.py` file in the directory of `main.py`.

```
1 # 'math.py' module
2 import builtins
3 def myrange(n): return [-42]
4 builtins.range = myrange
```

This rebinds `range` to a function that returns the list `[-42]`, contradicting any analysis guaranteeing a non-negative `s` variable. However unlikely in typical programs, such tampering is valid behavior the compiler must support.

The previous examples highlight some challenges of applying abstract interpretation to dynamic languages that may be summarized as follows.

- Dynamic typing frequently introduces $\top$ in the analysis.
- Polymorphic operators prevent narrowing values to a single type.
- Function calls lead to loss of compile-time guarantees due to possible meta-dynamism.

This thesis addresses these three issues by presenting an abstract interpretation framework that uses code duplication to preserve compile-time guarantees and narrow run-time behavior to a set of finer contexts instead of one general approximation, using Scheme as its subject language. Despite not falling in the category of highly dynamic languages, Scheme can mimic their behavior through programs where operators are part of a mutable global state such as the one below.

```
1 (define stdlib (vector > - *))
2
3 (define (fact n)
4   (let loop ((i n) (acc 1))
5     (if ((vector-ref stdlib 0) i 0) ;; >
6         (loop ((vector-ref stdlib 1) i 1) ;; -
7               ((vector-ref stdlib 2) acc i)) ;; *
8     acc)))
```

This approach has been validated in prior work implementing an optimizing Python-to-Scheme compiler [41]. How to analyze such programs is discussed in Chapter 4.

1.3. Basic Block Versioning

The challenges presented in Section 1.2 can be seen as a tug of war between the union and narrowing operations of abstract interpretation. In the context of dynamic languages, the union operator has the advantage over narrowing. As established in Section 1.2, this is a consequence of the semantics of such languages. One solution is to suppress union by allowing code duplication, thus eliminating join points in the CFG of a program and preserving the information acquired through narrowing. This immediately raises the issue of how to prevent code bloat caused by code duplication.

Basic Block Versioning (BBV) [14] is a compilation technique that provides a framework that operates at the level of basic blocks of a program’s CFG to reason about code duplication while bounding code bloat. Prior to the work in this thesis, two BBV algorithms existed. Section 1.3.1 presents *Eager* Basic Block Versioning (EBBV), an AOT variant of BBV that propagates type information and clones basic blocks for each encountered type context. However, this early variant did not show execution time improvements and was deemed inconclusive. Section 1.3.2 presents the first functioning BBV variant: *Lazy* Basic Block Versioning (LBBV), a JIT variant that uses run-time information to guide code duplication.

For brevity, the umbrella term BBV is used to refer to the AOT BBV techniques presented in this thesis, namely Static Basic Block Versioning and Lambda Versioning, while prior techniques are explicitly named *Eager* and *Lazy* BBV.

1.3.1. Eager Basic Block Versioning

Eager Basic Block Versioning (EBBV) [13] is an AOT compilation technique that generates type-specialized copies of basic blocks by attaching type information to each block and propagating it to successors. In EBBV, function entry points start with the $\top$ type assigned to parameters, and use branches to narrow the type context of each block similarly to abstract interpretation. However, contrary to abstract interpretation, EBBV does not compute the union of type contexts that can reach a block, but rather systematically duplicates basic blocks for each of these type contexts; each of these copies is called a *version*. EBBV copes with code bloat by setting a version limit beyond which new versions are no longer created for a given block. When the limit for a given block is reached, new type contexts will no longer generate a new version, but will instead use an existing, compatible one. If none exists, then a fully generic version is created. EBBV was shown to significantly reduce the number of run-time type checks, however, this was not accompanied by performance improvement due to a significant code size increase.

After the shortcomings of EBBV were discovered, research turned toward JIT compilation to guide code duplication with run-time context. This thesis explores a different avenue by refocusing on EBBV and addressing its flaws by augmenting it with a proper union with widening operator. While this introduces challenges that were absent from EBBV, the latter can nonetheless be seen as the precursor to the BBV algorithms presented in this thesis.

1.3.2. Lazy Basic Block Versioning

Lazy Basic Block Versioning (LBBV) is a JIT variant of BBV. In LBBV, function entry points also start with the $\top$ type assigned to parameters, but, rather than exploring type contexts at compile-time, every untraversed edge jumps to a *stub* which invokes the code

generator. When a stub is activated, the code generator either creates a new type-specialized version, or patches the incoming jump to target an existing version that matches this type context. This ensures versions are generated only for type contexts actually encountered at run-time. To prevent pathological cases where an excessive number of versions is generated, LBBV also enforces a version limit beyond which the code generator will not create a new version, but instead selects a compatible over-approximation of the requested type context. If none exists, a fully generic version is created.

As an example, consider the following `fact` function. For the purpose of LBBV, arithmetic operators are inlined to expose underlying type checks and dispatches to specialized arithmetic operators. For instance, `fixnum?` checks whether a number is a small integer, and `fx>` is the greater-than operator that operates only on small integers. Overflow checks are also inlined: `fx+?` performs addition on small integers, but returns `#f` (false) on overflow.

```

1 (define (fact n)
2   (let loop ((i n) (acc 1))
3     (if (cond
4         ((fixnum? i) (fx> i 0))
5         ((flonum? i) (fl> i 0.))
6         (else (> i 0)))
7       (loop
8         (cond
9           ((fixnum? i)
10            (or (fx-? i 1)
11                (- i 1))) ;; overflow
12            ((flonum? i)
13             (fl- i 1.))
14            (else
15             (- i 1)))
16          (cond
17            ((and (fixnum? acc) (fixnum? i))
18             (or (fx*? acc i)
19                 (* acc i))) ;; overflow
20            ((and (flonum? acc) (flonum? i))
21             (fl* acc i))
22            (else
23             (* acc i))))
24     acc)))

```

Figure 5 shows the CFG generated by LBBV after calling `fact` with a small integer (`fx`) that causes no overflow. The first loop iteration is unrolled by creating two versions of the `loop` entry, B.1 and B.2. The resulting hot loop is fully specialized for small integers, containing no `fixnum?` check. Unexplored paths remain as stubs, eliminating code bloat.

LBBV offers a trade-off that sacrifices peak performance slightly for significantly faster warm-up times. LBBV is also a single-tiered technique, avoiding the complexity and security issues that affect many multi-tiered JIT compilers.

However, it comes with limitations. By design, LBBV immediately generates specialized versions for type contexts encountered at run-time. Consequently, the code generator may produce versions for contexts encountered only during initialization and unused thereafter.

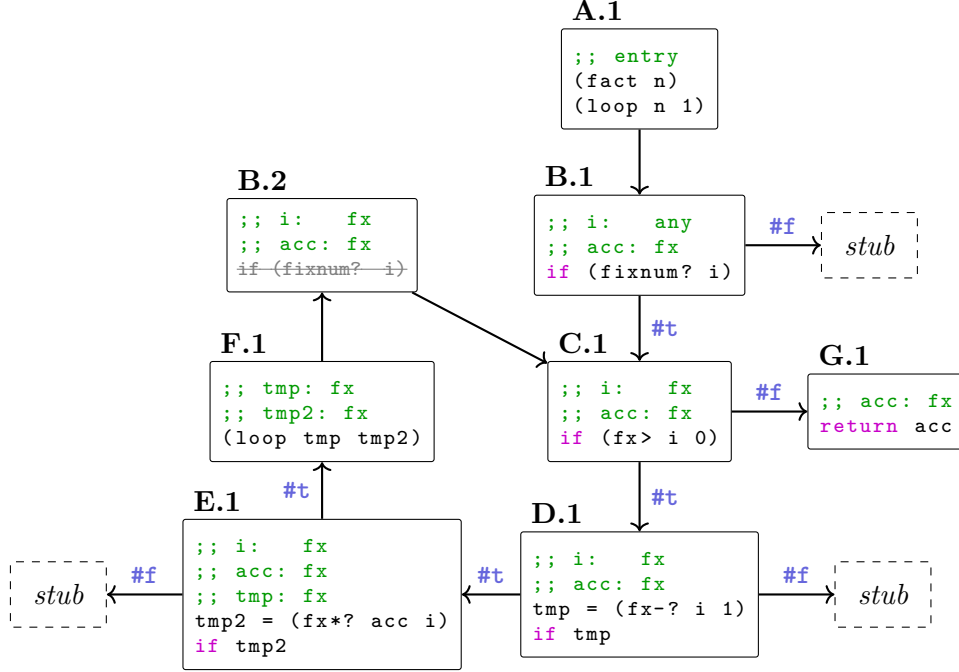


Figure 5 – Control-flow graph of the `fact` function as specialized by LBBV after a call to `fact` with no overflow. Two copies of the loop entry are created: B.1 (`i: any`) and B.2 (`i: fx`). The result is a specialized loop `C.1 → D.1 → E.1 → F.1 → B.2` that contains no type checks and only overflows need to be tested. Paths that were not executed at run-time remain stubs.

To avoid early saturation of the version limit, LBBV must refrain from generating overly specialized blocks, such as for constants or numeric intervals. In other words, LBBV lacks the union operator of abstract interpretation. LBBV also has limited support for interprocedural context propagation. Propagation of argument types from caller to a known callee is supported by duplicating entry points, as is the propagation of return types (limited to functions with a single return type [15]). It supports neither propagation of argument types from callee to caller, nor duplication of return blocks for multiple return contexts, which are important for precise specialization. These limitations are addressed in this thesis.

LBBV has been applied in the context of JavaScript [14], Ruby [16], and Scheme [50].

1.4. Supplementary Related Work

This section offers a quick survey of existing optimization techniques applicable to dynamic languages. They are divided into three categories: just-in-time code generation (Section 1.4.1), ahead-of-time program analysis (Section 1.4.2), and runtime system optimizations that minimize the cost associated with run-time checks (Section 1.4.3).

1.4.1. Just-in-Time Code Generation

1.4.1.1. Tracing JIT. A tracing just-in-time compiler [5] monitors the run-time behavior of a program to identify frequently executed loops, called *hot loops*. Upon identifying such a loop, the compiler records a trace of the concrete execution and generates optimized machine code for the observed run-time context. For instance, if a variable is consistently observed to be an integer, the compiler speculatively emits highly optimized code under the assumption that this variable will be an integer on subsequent executions of that loop. The loop’s entry point is then replaced by a guard that validates this assumption at run-time and jumps to the optimized code, or falls back to unoptimized code if the assumption is invalidated. A tracing JIT typically offers good peak performance, at the cost of significant, sometimes unpredictable, warm-up time [6].

1.4.1.2. Meta-Tracing JIT. A limitation of tracing JIT compilers is that they must be reimplemented for each source language, demanding significant engineering effort. Meta-tracing JIT compilers [10] address this by lifting the tracing mechanism to the level of an interpreter. Rather than tracing the execution of a user program, the JIT compiler traces the execution of an *interpreter*. Hot paths through the interpreter loop, corresponding to frequently executed bytecodes, are traced and compiled to machine code that, among other optimizations, eliminates the overhead of the bytecode dispatch loop. This strategy reduces the effort required to implement a JIT optimizing compiler, deriving it from a bytecode interpreter, but comes at the cost of inheriting the warm-up overhead of a tracing JIT.

1.4.1.3. Truffle/Graal. Truffle/Graal [62] is a framework for implementing dynamically typed languages that derives a JIT compiler from an abstract syntax tree (AST) interpreter with partial evaluation. A language implementation is written as an AST walker in Truffle, the underlying language of the framework. AST nodes are self-optimizing: upon execution, each node observes its run-time context and operand types, then replaces itself with a guarded, type-specialized variant. When a guard detects an invalidated assumption, the node is deoptimized and re-specialized to a more general variant. Once the AST has stabilized, the Graal compiler partially evaluates the interpreter against the specialized AST to produce optimized native code from which bytecode-dispatch overhead has been eliminated.

This design reduces the implementation of a high-performance JIT compiler to the simpler task of writing an AST interpreter. While this approach also suffers from significant warm-up overhead [17], it is widely regarded as the state of the art in JIT compiler generation, and has been used to implement high-performance JavaScript [61], Python [60], and Ruby [44] compilers.

1.4.2. Ahead-of-Time Program Analysis

1.4.2.1. Control-Flow Analysis. Similarly to abstract interpretation, control-flow analysis (CFA) statically approximates the set of values, or types, that expressions may take in a program. However, CFA focuses on which closures may flow to which call sites, enabling interprocedural analysis. In particular, k -CFA [55] identifies each closure and call site by its syntactic location *and* the last k call sites on the call stack at the time of its creation or invocation, allowing the same function to be analyzed under different execution contexts for more precise results. This, however, comes at the cost of prohibitive complexity [45] that generally prevents deep analysis ($k > 1$). More recently, on-demand [31] and context-sensitive [59] variants of CFA limit the analysis to facts deemed useful to the compiler, greatly reducing the cost of such analyses.

1.4.2.2. Adaptive Type Analysis. Discovering the type of variables at every point of a program is expensive, and may waste resources discovering information that does not lead to better code generation, either because it is unused or because it is too imprecise. Adaptive type analysis [24] adds constraints to the abstract interpretation that indicate which type information may be needed, demanding precise approximation only where it is relevant.

1.4.2.3. Code Duplication. Many techniques use code duplication to enable code optimizations. For instance, loop unrolling [22] duplicates the first few iterations of a loop, which hoists dynamic checks out of the loop and enables the generation of specialized code for the rest of the iterations. Other techniques such as loop peeling [57], tail duplication [47], and inlining [51] allow for a similar effect. This motivated frameworks for selecting duplication targets within programs, such as splitting [12], dominance-based duplication [39], and the BBV techniques from Section 1.3.

1.4.2.4. Inlining. Inline substitution, or inlining, is a code duplication technique that replaces function invocations by a copy of their target’s body [51]. Inlining is extensively used by optimizing compilers as it both removes the overhead of function calls at run-time and unlocks interprocedural optimization potential for otherwise intraprocedural techniques. For instance, the initial variant of Static Basic Block Versioning presented in Chapter 2 is purely intraprocedural and relies on inlining for propagating type information across abstraction barriers.

1.4.3. Runtime Optimizations

1.4.3.1. Object Representation. Efficient object representation allows for fast type checks at run-time. A prevalent technique is to reserve a few bits on each value and pointer to store type information. Bit patterns are reserved for the most frequently used types, such

as immediate small integers, enabling fast checks on common operations. Other representations such as NaN-tagging [38] and self-tagging [43] have also been developed for fast type checks without heap-allocating floats.

1.4.3.2. Inline Caches. Initially developed in the context of Smalltalk [23] and Self [33], inline caches are instruction sequences that test the type of operands of a polymorphic operation and dispatch to type-specialized code. The cache is updated at run-time whenever a new type is observed. In contexts where an operator is polymorphic yet used monomorphically, this yields efficient execution without type analysis. Inline caches are extensively used in JavaScript [18, 52, 54].

1.4.3.3. Superinstructions. In high-level languages, constructs that are often used in combination, yet defined separately for implementation simplicity, may introduce redundant run-time checks [41]. Superinstructions can be synthesized for frequent combinations derived from language profiling data and inlined, either speculatively ahead-of-time [41] or just-in-time via quickening [8].

Chapter 2

Static Basic Block Versioning

This chapter presents Static Basic Block Versioning (SBBV), an AOT optimization technique built upon the theory of abstract interpretation. It was first published at the *38th European Conference on Object-Oriented Programming* (ECOOP’24) [42]; the full paper can be found in Appendix D. This chapter is therefore written as a complement to the paper: Section 2.1 first offers an overview of the technique with a motivating example; Section 2.2 describes the SBBV algorithm and includes remarks that were not present in the initial paper; Section 2.3 summarizes the experimental results; Section 2.4 discusses limitations of the technique as presented in this chapter; Section 2.5 ends with concluding remarks.

Several details are relegated to Appendix D, including variable aliasing, integer interval analysis, elimination of vector bounds checks, implementation details of SBBV in the Gambit and Bigloo Scheme compilers, and a comparison to other optimizing compilers.

2.1. Overview

Abstract interpretation is a framework for the sound approximation of a program’s semantics, providing compile-time guarantees that can be exploited for optimization. Dynamic languages, however, pose a challenge for abstract interpretation because different inputs can lead to a wide range of behavior, yielding broad, less useful approximations.

The core idea of SBBV is to delay the union of contexts discovered by abstract interpretation by applying systematic tail-duplication to create multiple *versions* of basic blocks specialized for narrower run-time contexts. To avoid code bloat and allow the algorithm to terminate, a *version limit* dictates how many versions are allowed for each block. When the limit is reached, a heuristic decides which versions should be preserved and which should be *merged* into a more general approximation using union with widening. Importantly, this heuristic can be entirely local, considering only the versions at hand and performing no further program analysis, and still yield performance improvements.

2.1.1. Motivating Example

Consider the following `find` function, which returns the first element satisfying the predicate `p`, or false (`#f`) if none does.

```
1 (define (find p lst)
2   (if (pair? lst)
3       (if (p (car lst))
4           (car lst)
5           (find p (cdr lst)))
6       #f))
```

To ensure the safety of this code, all operations (`car`, `cdr`, and the invocation of `p`) are guarded by run-time checks that validate argument types and safely fail if incorrect values are provided. This can be implemented by inlining these checks within the source code, as shown below.

```
1 (define (find p lst)
2   (if (pair? lst)
3       (if ((if (procedure? p) p (fail)) (if (pair? lst) (car lst) (fail)))
4           (if (pair? lst) (car lst) (fail))
5           (find p (if (pair? lst) (cdr lst) (fail))))
6       #f))
```

In this example, the call to `p` is guarded by a `procedure?` check that ensures `p` is a function, and all pair accesses are guarded by a `pair?` check. In case of type mismatch, a `fail` statement safely stops execution.

Classical abstract interpretation can determine that the `pair?` tests at lines 3, 4, and 5 are redundant, since the value of `lst` need only be checked once per iteration. The same cannot be said, however, for the `procedure?` check. Although `p` need only be verified to be a function on its first invocation, the code of that invocation is shared across all iterations, so it is checked redundantly on subsequent iterations as well.

SBBV resolves this by duplicating the loop entry: once abstract interpretation has verified `p` to be a function on the first iteration, it reaches the loop entry with an abstract context in which `p` is known to be a function, and a specialized version is created for that context.

Figure 6 shows the specialized CFG resulting from this duplication. The source CFG (left) contains redundant `pair?` tests in blocks **D**, **G**, and **J**, all of which are found to be unreachable and eliminated from the specialized CFG (right). Block **A** in the source CFG has been duplicated into **A1**, a generic loop entry point, and **A2**, a specialized loop entry in which `p` is known to be a function. At run-time, this yields a hot loop **A2** $\rightarrow$ **F** $\rightarrow$ **A2** in which the `procedure?` check has been safely removed.

The `find` function provides a satisfying example of SBBV: a single block is duplicated and several redundant checks are eliminated, yielding code that is more efficient and shorter. In general, however, a strategy to avoid excessive, or even infinite, duplication is needed to curb code bloat and ensure that the process converges.

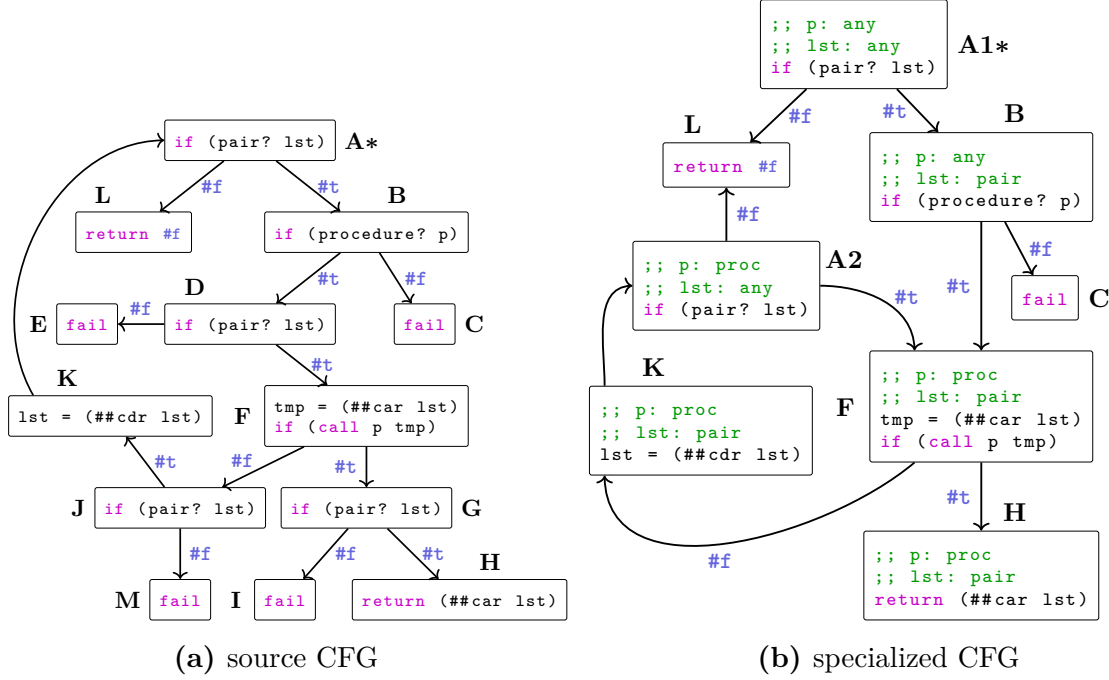


Figure 6 – Source (left) and specialized (right) control-flow graphs of the `find` function. In the specialized CFG, basic blocks reachable from different type contexts, refined through narrowing at conditional branches, are duplicated. For instance, block **A** has two versions: **A1**, a generic entry point, and **A2**, a specialized loop entry in which `p` is known to be a procedure. Type contexts are commented at the top of each block and are used to remove type checks; for instance, `;; p: procedure` denotes a context in which `p` is a procedure, the type of Scheme functions. Jump cascades and unreachable code, such as redundant `pair?` tests and their `fail` branches, are eliminated in the process. Entry points are denoted by `*`.

2.2. The Static Basic Block Versioning Algorithm

To unify abstract interpretation and code duplication, a mechanism to limit bloat and ensure convergence is required. In SBBV, this mechanism is *merging*: when the number of versions generated for a basic block exceeds a predetermined limit, some versions are selected for merging. A new context is computed from the union with widening of their contexts, and a new version is created for this more general context. The merged versions are then replaced by this new version. In that sense, SBBV does not eliminate the union of contexts from abstract interpretation, but rather delays it as long as the version limit permits.

2.2.1. Data Structures

Before describing SBBV in full, some terminology is needed. In SBBV, a *block* refers to a basic block in the source program. As specialization progresses, a specialized CFG

context
└ <i>types</i> : mapping from variables to their abstract type .
└ <i>equivalences</i> : equivalence classes among variables.
type
└ the set of concrete run-time types a variable may hold.
block
└ <i>body</i> : the instructions constituting the block's code.
└ <i>allVersions</i> : mapping from a context to the corresponding versions of the block.
└ <i>reachableVersions</i> : same as <i>allVersions</i> , but restricted to reachable contexts .
instruction
└ <i>kind</i> : instruction kind, for instance "if" or "goto".
└ ...: other properties relevant to the instruction kind.
version
└ <i>block</i> : the block from which this version was specialized.
└ <i>context</i> : the context for which this version was specialized.
└ <i>body</i> : the instructions constituting the version's specialized code.
└ <i>merged</i> : if this version was merged, points to the resulting merged version .

Figure 7 – Data structures used in the SBBV algorithm.

is constructed, whose basic blocks are specializations of blocks in the source CFG and are referred to as *versions*. Each version has a source block and a *context*, which maps variables to their abstract types. As the algorithm progresses, some blocks accumulate more versions than the limit permits, leading to merges. Merging versions corresponds to applying union with widening to their contexts; the merged versions are then no longer reachable in the specialized CFG and are replaced by the version resulting from their merge. Each block therefore has two nested mappings from contexts to versions: *allVersions*, which contains every version generated for the block, including merged ones, and *reachableVersions*, which contains only unmerged, reachable versions. These data structures are summarized in Figure 7.

2.2.2. The Algorithm

SBBV specializes a program by performing a breadth-first traversal of its CFG. Algorithm 1.1 shows the main function, which initiates the traversal at a given block, typically the entry of the program or of a function to specialize. As traversal progresses, more versions are enqueued for specialization until a fixed-point is reached. When a version is dequeued, it is first checked whether the number of versions of its block exceeds the version limit, given

Input: (<i>entry</i> : block, <i>limit</i> : integer)	Output: —
<pre> 1 define staticBasicBlockVersioning (<i>entry</i>, <i>limit</i>): 2 getOrCreateVersion (<i>entry</i>, $\top$) 3 while <i>version</i> $\leftarrow$ <i>queue</i>.dequeue(): 4 if <i>version</i> is not reachable or <i>version</i> is done : continue 5 if <i>version</i>.block.reachableVersions > <i>limit</i> : mergeSome (<i>version</i>.block) 6 if <i>version</i> is reachable : specialize (<i>version</i>) </pre>	

Algorithm 1.1: The main function initiates a breadth-first traversal from the given entry point. Since the queue can contain unreachable or already processed versions, some are skipped (line 4). If a version's block has exceeded its version limit, a merge is requested (line 5). The test at line 6 is needed to check if the version was merged.

as a parameter to SBBV. If so, some versions of that block are selected for merging before specialization proceeds.

Whereas classical abstract interpretation operates on a fixed CFG, SBBV modifies the specialized CFG as it progresses, through merges, making it a dynamic graph algorithm. Since merges remove edges from the specialized CFG, SBBV requires a single-source reachability algorithm that tracks which nodes remain reachable from the entry of a function. Before a version is traversed, it is therefore checked whether it is still reachable in the specialized CFG. Conversely, as the traversal progresses, previously unreachable versions may be reconnected to the specialized CFG and requeued for specialization.

Single-source reachability already has well-known, efficient solutions. The key idea, attributed to Even and Shiloach [25], is to embed a spanning tree in the CFG, rooted at its entry block, which assigns a rank to each block. On edge insertion or deletion, these ranks are used to detect whether the operation invalidates the tree and, if so, which part of it must be reconstructed. The implementation used in SBBV adapts the algorithm of [2], a single-source shortest path algorithm on weighted graphs, reduced here to its unweighted variant.¹ To avoid cluttering the algorithm, the tracking of version reachability is left implicit in this presentation.

Algorithms 1.2 and 1.3 describe what happens when a block is reached with a given context. This includes the ancillary steps of creating a new version if the context is newly reached, adding that version to its block's record, and obtaining the most up-to-date version when the requested context is that of a previously merged version.

Algorithms 1.4, 1.5, and 1.6 describe the specialization of a dequeued version. The instructions of the source block are traversed and specialized for the version's context. Each instruction is dispatched to a specialized that returns a new instruction, possibly a "no-op" if

1. The resulting single-source reachability algorithm was later improved sufficiently to serve as the basis for a garbage collector that immediately collects cycles [36]. These improvements were developed after the publication of SBBV and are therefore not reflected in the compilation times presented in this thesis.

Input: (<i>block</i> : <i>block</i> , <i>context</i> : <i>context</i>)	Output: <i>version</i>
<pre> 1 define <i>getOrCreateVersion</i> (<i>block</i>, <i>context</i>): 2 <i>version</i> ← <i>getVersion</i> (<i>block</i>, <i>context</i>) 3 if <i>version</i> : return <i>version</i> 4 <i>newVersion</i> ← new <i>Version</i>(<i>block</i>, <i>context</i>) 5 <i>block.allVersions</i>[<i>context</i>] ← <i>newVersion</i> 6 <i>queue.enqueue</i>(<i>newVersion</i>) 7 return <i>newVersion</i> </pre>	

Algorithm 1.2: This function is called to request a version for a given block and context. If a version exists, it is returned (line 3). Otherwise, a new version is added to the record of existing versions of this block, enqueued, and returned.

Input: (<i>block</i> : <i>block</i> , <i>context</i> : <i>context</i>)	Output: <i>version</i>
<pre> 1 define <i>getVersion</i> (<i>block</i>, <i>context</i>): 2 <i>version</i> ← <i>block.allVersions</i>[<i>context</i>] 3 if not <i>version</i> : return <i>false</i> 4 while <i>version.merged</i> : <i>version</i> ← <i>version.merged</i> 5 return <i>version</i> </pre>	

Algorithm 1.3: This function looks for an existing version for a given block and context. If a version exists, the chain of merges in its *merged* field (line 4) is unrolled to get and return its latest generalization. Otherwise *false* is returned.

Input: (<i>version</i> : <i>version</i>)	Output: —
<pre> 1 define <i>specialize</i> (<i>version</i>): 2 <i>context</i> ← <i>version.context</i> 3 for <i>instr</i> in <i>version.block.body</i> : 4 match <i>instr.kind</i> : 5 case "goto" : <i>context</i>, <i>newInstr</i> ← <i>specializeGoto</i> (<i>version</i>, <i>context</i>, <i>instr</i>) 6 case "if" : <i>context</i>, <i>newInstr</i> ← <i>specializeIf</i> (<i>version</i>, <i>context</i>, <i>instr</i>) 7 ... 8 <i>version.body.append</i>(<i>newInstr</i>) 9 if <i>newInstr</i> always halts : break 10 <i>version.contextAfter</i> ← <i>context</i> </pre>	

Algorithm 1.4: This function specializes a version by dispatching instructions to auxiliary functions. Only interesting cases, "goto" and "if", are shown.

the instruction can be discarded in that context, together with the context resulting from that specialized instruction. The most interesting case is the "if" instruction, whose specialized is shown in Algorithm 1.6. The version's context is narrowed using the condition of the "if", yielding two outgoing contexts that are propagated to the branch targets. If the condition is invariant in the given context, the "if" is specialized into a "goto", eliminating the test altogether.

Finally, Algorithm 1.7 describes the merging of versions when a block exceeds the version limit. In that case, a heuristic selects the versions to merge; the specifics of this heuristic are deferred to Section 2.2.3. A new context is computed from the union with widening of the

Input: (*version*:*version*, *context*:*context*, *instruction*:*instruction*)
Output: *context* × *instruction*

```

1 define specializeGoto (version, context, instruction):
2   targetContext ← context.filter(var ⇒ var is in instruction.target.parameters)
3   targetVersion ← getOrCreateVersion (instruction.target, targetContext)
4   return (context, new Instruction("goto", targetVersion.id))

```

Algorithm 1.5: This function propagates the context from a "goto" instruction to its target. If no specialization for this target and context exists, it is created and enqueued by **getOrCreateVersion**.

Input: (*version*:*version*, *context*:*context*, *instruction*:*instruction*)
Output: *context* × *instruction*

```

1 define specializeIf (version, context, instruction):
2   condition ← instruction.condition
3   trueContext ← context.narrowConditionTrue(condition)
4   falseContext ← context.narrowConditionFalse(condition)
5   if trueContext = ⊥ :
6     falseContext ← falseContext.filter(var ⇒ var is in instruction.targetFalse.parameters)
7     targetVersion ← getOrCreateVersion (instruction.targetFalse, falseContext)
8     return (context, new Instruction("goto", targetVersion.id))
9   else if falseContext = ⊥ :
10    trueContext ← trueContext.filter(var ⇒ var is in instruction.targetTrue.parameters)
11    targetVersion ← getOrCreateVersion (instruction.targetTrue, trueContext)
12    return (context, new Instruction("goto", targetVersion.id))
13  else
14    trueContext ← trueContext.filter(var ⇒ var is in instruction.targetTrue.parameters)
15    falseContext ← falseContext.filter(var ⇒ var is in instruction.targetFalse.parameters)
16    targetTrue ← getOrCreateVersion (instruction.targetTrue, trueContext)
17    targetFalse ← getOrCreateVersion (instruction.targetFalse, falseContext)
18    return (context, new Instruction("if", condition, targetTrue.id, targetFalse.id))

```

Algorithm 1.6: This function propagates narrowed contexts from an "if" instruction to its targets. When the conditional is always true, or false, in the current context, the "if" instruction is transformed into a "goto".

Input: (*block*:*block*)

Output: —

```

1 define mergeSome (block):
2   versions ← mergeHeuristic (block.reachableVersions)
3   newVersion ← getOrCreateVersion (block,  $\bigcup_{v \in \text{versions}} v.\text{context}$ )
4   for version in (versions \ {newVersion}) :
5     version.merged ← newVersion

```

Algorithm 1.7: When a block's version limit is exceeded, a heuristic selects versions to merge (union with widening, represented by $\bigcup$). Bookkeeping is required to update references to merged versions, done in practice with the **merged** field of each version, and is left implicit throughout the algorithm.

selected versions' contexts, and a new, corresponding version is requested for this context. The merged versions are then made permanently unreachable as the new version replaces

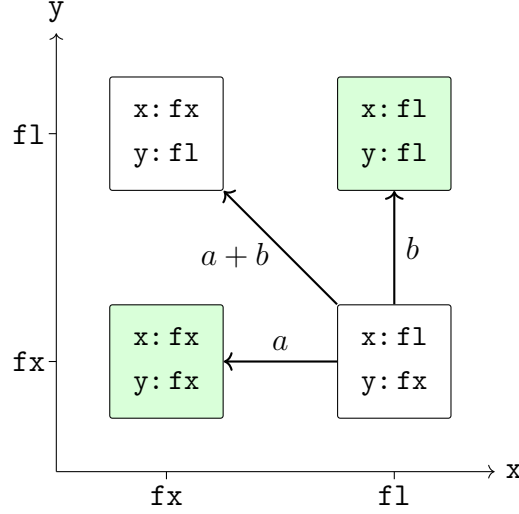


Figure 8 – A merge heuristic that preserves the monomorphic contexts (in green) by merging the two mixed contexts cannot be expressed by a separable distance function. Taking the `fl/fx` context as an example, its distances to each monomorphic context (a and b) are smaller than its distance to the other mixed context ($a + b$), so a separable distance never makes the two mixed contexts the closest pair.

them. The only exception arises when one of the selected versions’ contexts supersedes all the others, in which case the result of the merge may be one of the merged versions.

When the algorithm terminates, the specialized CFG is embedded in the *reachableVersions* tables of blocks in the source CFG. Post-processing steps are then applied, such as the removal of jump cascades, which frequently appear as branches are eliminated, followed by liveness recomputation and register allocation. The resulting CFG is finally compiled to machine code.

2.2.3. Merge Heuristic

The merge heuristic of the SBBV algorithm dictates which versions should be merged when the version limit is exceeded. The implementation evaluated in this chapter uses a merge by *similarity* that computes the Hamming distance between each pair of contexts. This Hamming distance is computed per variable, as the number of concrete types on which the variable’s abstract types disagree between the two contexts, and summed over all variables. A bias is added to steer the heuristic toward merging contexts that are either too broad or too specific. The closest pair of contexts under this biased distance is then merged. The implementation of this distance function can be found in Appendix A.

In an idealized scenario, the merge heuristic would identify the contexts most likely to be used at run-time and avoid merging them as long as the version limit permits. Empirically,

this would appear as the number of type checks decreasing with increasing version limits, then rapidly stabilizing once the heuristic preserves precise contexts for the versions that are executed at run-time.

As shown in the evaluation of Section 2.3, this is not the behavior observed with this **similarity** heuristic: many benchmarks require a high version limit before stabilization occurs. This is problematic, since the resulting code bloat outweighs the benefit of type check elimination. Despite offering some improvement, this heuristic therefore performs poorly.

The specific issue with the **similarity** heuristic is that, over contexts that share the same specificity bias, it is separable over its coordinates: the distance decomposes as a sum of per-variable type distances. To illustrate why this matters, consider a program point with two live variables, `x` and `y`, where the heuristic must select two versions to merge from among the four contexts in which `x` and `y` are each either a `fixnum` or a `flonum`, as shown in Figure 8. The desirable outcome is for the heuristic to merge the mixed-type contexts, preserving the `fixnum/fixnum` and `flonum/flonum` contexts, because the relative cost of type checks is higher in monomorphic arithmetic than in mixed arithmetic, which already incurs the cost of converting numbers to a common representation (here, a `flonum`). With a separable distance function, however, the closest pairs among the four contexts are the monomorphic-mixed pairs, so the heuristic systematically favors merging a monomorphic context into a mixed one, a suboptimal outcome.

Fortunately, nothing constrains the merge heuristic to be separable, and the result merely shows that SBBV can offer performance improvements *despite* a poor heuristic. As demonstrated in Chapter 3, a more effective heuristic can be designed by hand.

2.3. Evaluation

The SBBV algorithm presented in this thesis was implemented and evaluated in both the Bigloo [53] and Gambit [27] Scheme compilers, which apply a wide range of state-of-the-art AOT optimization techniques. The R7RS benchmark suite [1] is used for evaluation. Benchmarks are split into two categories: micro benchmarks (fewer than 400 lines of code) and macro benchmarks (more than 400 lines of code). In this experiment, SBBV contexts were extended to track value ranges and vector bounds, permitting the elimination of not only type checks but also overflow and array bounds checks. This section shows results aggregated over micro and macro benchmarks. Appendix D describes the context extensions and the experimental setup, along with per-benchmark results.

Overall, the evaluation shows that SBBV reduces the number of run-time checks by 54% (Bigloo) and 62% (Gambit) relative to these state-of-the-art techniques, with a low limit

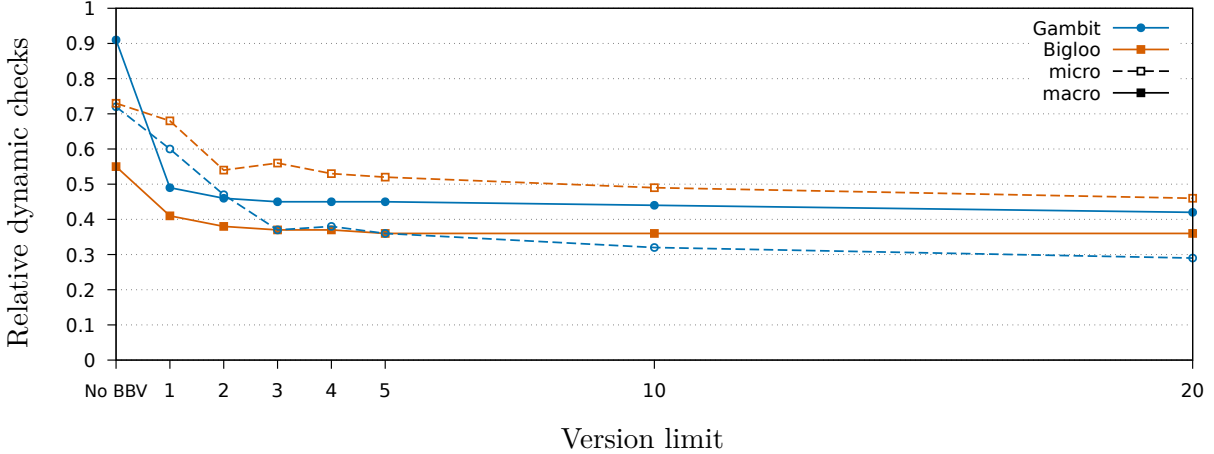


Figure 9 – Relative dynamic checks (type, overflow, and array bounds checks) with increasing version limits in Bigloo (orange) and Gambit (blue), on macro benchmarks (solid lines) and micro benchmarks (dashed lines). The x-axis is the version limit; the leftmost entry corresponds to compilation without SBBV. The y-axis is the ratio of run-time dynamic checks relative to compilation without any optimization (geometric mean). Lower is better.

of 2 versions. This translates into a 10% acceleration of execution time. This section breaks down these results, presenting the effect of SBBV on dynamic checks (Section 2.3.1), program size (Section 2.3.2), execution time (Section 2.3.3), and compile time (Section 2.3.4).

2.3.1. Dynamic Checks

Figure 9 shows the dynamic checks eliminated by SBBV with increasing version limits. The proportion of checks eliminated varies between Bigloo and Gambit, despite both compilers applying the same algorithm. The overall patterns are similar, but their magnitudes differ, presumably because of differences in each compiler’s implementation of the algorithm, its merge heuristic, and compilers’ internal representations. Bigloo also eliminates more checks without SBBV, leaving fewer checks for SBBV to remove.

Overall, SBBV eliminates checks beyond what Bigloo’s and Gambit’s existing optimizations already remove, and this number decreases further as the version limit rises. At a limit of one, each version’s context is a sound approximation of all reachable contexts, which corresponds to abstract interpretation without code duplication. The decrease observed beyond that limit therefore demonstrates that duplication eliminates checks that abstract interpretation alone cannot.

As noted in Section 2.2.3, however, the number of checks continues to decrease slowly even beyond a limit of 10, never fully stabilizing. This illustrates the weakness of the `similarity` heuristic, which requires many versions to cover all run-time contexts. The

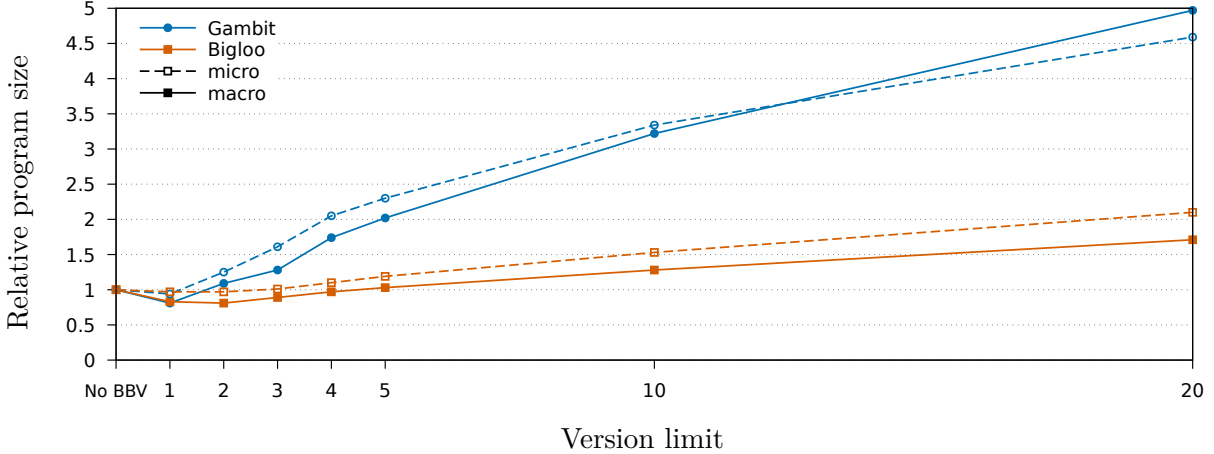


Figure 10 – Program size with increasing version limits in Bigloo (orange) and Gambit (blue), on macro benchmarks (solid lines) and micro benchmarks (dashed lines). The x-axis is the version limit; the leftmost entry corresponds to compilation without SBBV. The y-axis is program size relative to that entry (geometric mean). Lower is better.

effect is particularly apparent on micro benchmarks (dashed lines), which are mostly small, monomorphic programs expected to stabilize at a low version limit. Improvements to the merge heuristic are discussed in Chapter 3.

2.3.2. Program Size

Figure 10 shows the size of programs compiled with SBBV at increasing version limits. Program size is measured in bytes and compared against executables compiled without SBBV. As expected, size increases with the version limit, except at low limits (< 5 with Bigloo, < 2 with Gambit), where the resulting executables are smaller than those compiled without SBBV. At these limits, check elimination outweighs code duplication, as in the example of Section 2.1.1.

Program size is the main reason the version limit cannot be raised indefinitely to eliminate all checks: there is a trade-off between the checks eliminated and the size of the resulting program. In this experiment, that trade-off is suboptimal because of the poor `similarity` heuristic. Ideally, a merge heuristic would identify the useful versions and eliminate most checks within a low version limit.

2.3.3. Execution Time

Figure 11 shows the execution time of programs compiled with SBBV at increasing version limits. The aforementioned trade-off is apparent, especially in Gambit, where execution time drops rapidly at low version limits, reaching a minimum at a limit of 2 versions (89%

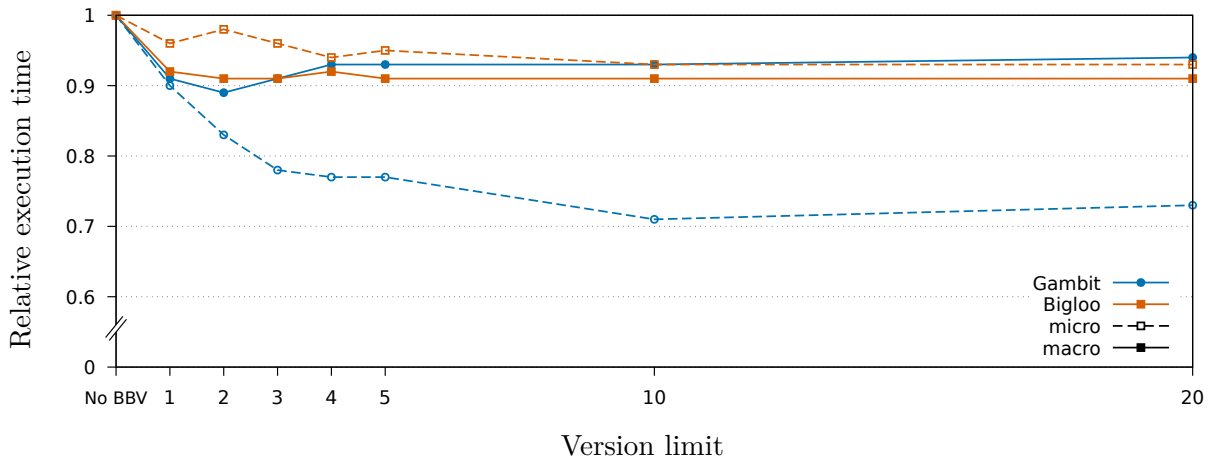


Figure 11 – Execution time with increasing version limits in Bigloo (orange) and Gambit (blue), on macro benchmarks (solid lines) and micro benchmarks (dashed lines). The x-axis is the version limit; the leftmost entry corresponds to compilation without SBBV. The y-axis is execution time relative to that entry (geometric mean). Lower is better.

of the execution time without SBBV). Beyond that limit, execution time slowly rises before stabilizing (at around 93% for Gambit and 91% for Bigloo). One exception is micro benchmarks under Gambit, for which execution time decreases steadily up to a limit of about 3 versions and continues to decrease slightly beyond it (down to about 71% of the execution time without SBBV).

Inspection of individual benchmarks (see Appendix D) reveals, however, that the optimal version limit varies between benchmarks. When optimizing for speed, a single static version limit across all programs is therefore suboptimal. It may be preferable to give the programmer manual control over the limit, possibly with different per-function limits.

Finally, Bigloo and Gambit implement inexpensive type checks using pointer tagging. SBBV could therefore have a greater performance impact in implementations with more costly dynamic checks, such as the object representations of Python and Ruby, where checking a type requires a memory access.

2.3.4. Compile Time

Figure 12 shows the compile time of programs compiled with SBBV at increasing version limits. Overall, compile time increases roughly linearly with the version limit. This increase is significantly steeper in Gambit, which compiles each Scheme module as a single C function, so that code duplication puts additional strain on the underlying C compiler. Conversely, at a limit of 1 version, where programs are smaller, compilation is faster.

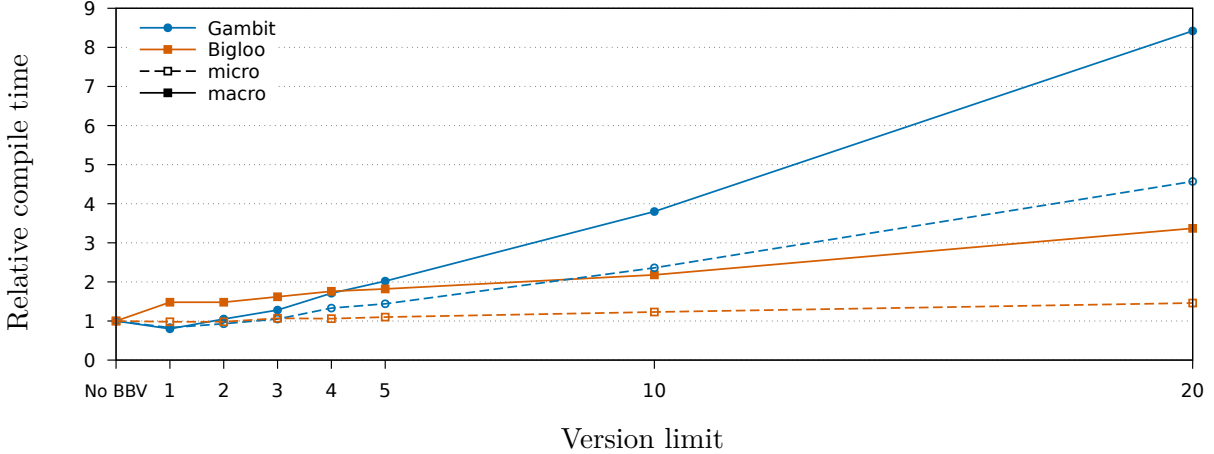


Figure 12 – Compile time with increasing version limits in Bigloo (orange) and Gambit (blue), on macro benchmarks (solid lines) and micro benchmarks (dashed lines). The x-axis is the version limit; the leftmost entry corresponds to compilation without SBBV. The y-axis is compile time relative to that entry (geometric mean). Lower is better.

2.4. Limitations

As noted throughout this chapter, a significant limitation of the SBBV implementation is the inefficiency of its merge heuristic at eliminating type checks with a low version limit. This inefficiency was discovered after the publication of [42], which is why it is barely mentioned in the paper.

Another limitation of SBBV is its entirely intraprocedural nature, which cannot propagate information across abstraction barriers. This is especially limiting when optimizing small recursive functions (such as `fib` or `ack`), since the type returned by the recursive call is not propagated. The implementation described in the paper mitigates this by applying function inlining *before* SBBV, which is suboptimal because inlining decisions cannot then benefit from type information. As a result, small recursive functions still execute redundant type checks at every recursive call.

A further limitation is the lack of portability, due to the reliance on the internal representations of the Bigloo and Gambit compilers. Integrating SBBV into each compiler proved especially tedious, requiring months of work by a team of three programmers familiar with their internals.

Chapter 3 addresses all three limitations. It proposes an improved merge heuristic and compares it with the `similarity` heuristic presented here; it largely resolves the intraprocedural limitation, generating control flow for small recursive functions with no redundant type checks; and it describes a source-to-source implementation in Scheme, allowing the analysis to be reused across host compilers.

2.5. Conclusion

This chapter introduced Static Basic Block Versioning, an optimization technique built upon abstract interpretation and extended with code duplication to address its shortcomings when applied to dynamic languages. SBBV is a variant of BBV that determines, through a fixed-point program analysis, a set of basic block versions that are appropriate for the program and that cover all possible contexts without exceeding a given limit. This gives control over the code bloat induced by code duplication in a way that avoids falling back on an unoptimized generic version of the basic block when the version limit is reached.

At the core of the SBBV algorithm is a heuristic that drives the merging of previously generated versions, keeping their number within the allocated limit. Experiments show that even a suboptimal merge heuristic eliminates dynamic checks in practice and reduces program execution time.

SBBV is, however, an intraprocedural technique: it cannot propagate contextual information across abstraction barriers. Consequently, each function must be specialized separately, precluding whole-program analysis. This limitation is the main concern addressed in Chapter 3.

Chapter 3

Lambda Versioning

This chapter addresses limitations of the Static Basic Block Versioning (SBBV) algorithm by introducing Lambda Versioning (ΛV), an interprocedural variant of SBBV. As a preparatory example, consider this simple function.

```
1 (define (f x)
2   (* x (g x)))
```

In this example, the function `g` operates on `x` and returns a value, most likely a number. To optimize the multiplication in `f`, a compiler needs information about the type of `x` and the type of the value returned by `(g x)`. SBBV cannot provide this information, being intraprocedural by nature: any information obtained within `g` is lost when exiting the function. This is unfortunate, because at the exit site of `g`, precise information about both `x` and the returned value may be available, but is lost on return. Type checks are therefore required on both operands of the multiplication in `f`.

The main obstacle to interprocedural code specialization by SBBV is that, while it can version function exit sites, it has no mechanism to duplicate and specialize return points.¹ This obstacle has two facets. The first is mechanical: a multi-return function must be passed multiple return points, so that each specialized exit site can jump to its corresponding return point. The second is more fundamental: SBBV propagates information strictly forward, always specializing entry points before exit sites. Specializing return points, however, demands that call sites be updated *after* new specialized exit sites are generated, which SBBV's forward-only model cannot accommodate.

To support invocations of subroutines with multiple return points, this chapter introduces the notion of *hyperfunction*, a generalization of functions supporting multiple entry

1. An *entry point* is the location within a function body to which execution jumps on invocation; a *return point* is the location to which control returns once a function invocation completes; a *call site* is the location from which a function is invoked; and an *exit site* is the location from which it returns. A *point* is thus the target of a jump and a *site* its origin: a call site jumps to an entry point, and an exit site to a return point.

points and return points. In Scheme, hyperfunctions can be implemented via a source-to-source transformation on which BBV, specifically AV, is applied, yielding a pure-Scheme representation of versioned code suitable for any Scheme compiler.

Section 3.1 begins with motivating examples. Section 3.2 introduces hyperfunctions. Section 3.3 presents the Lambda Versioning algorithm, a SBBV variant that generates specialized exit sites and corresponding specialized return points. Section 3.4 discusses an improved merge heuristic that performs better than the one used in Chapter 2. Section 3.5 presents an empirical evaluation demonstrating that AV eliminates more type checks, with a lower version limit, than SBBV. Section 3.6 discusses the efficient implementation of hyperfunctions. Section 3.7 discusses limitations. Section 3.8 offers concluding remarks.

3.1. Motivating Examples

The inability to propagate information interprocedurally prevents SBBV from inferring the types of values returned by functions and the types of arguments discovered within the function body. This limitation stems from the fact that a function must be given a single return point at each call site. In SBBV, this is circumvented by applying function inlining before specialization. This enables some interprocedural type propagation but is suboptimal. First, since inlining takes place before SBBV, the selection of call sites to inline cannot benefit from type analysis. Second, some call sites cannot be inlined at all; recursive calls, for instance, can only be partially unrolled.

Another approach is to lift the single-return restriction, allowing return points duplication to propagate type information back to a function’s caller. Prior work applied interprocedural multi-return functions to Lazy Basic Block Versioning (LBBV); this was achieved, however, by choosing return offsets for a fixed set of types shared across all call sites [50]. This therefore precludes per-call-site specialization, including the propagation of argument types discovered within the callee’s body. This section demonstrates that precise, per-call-site specialization is achievable without speculation, by generating specialized return points as new exit contexts are discovered within the callee. The following examples are hand-written, but Section 3.3 presents an algorithm for deriving them automatically from source code.

3.1.1. Simple Example

Consider the following program defining the functions `square`, which squares its parameter, and `power4`, which computes the fourth power of its parameter by calling `square` twice.

```

1 (define (square x)
2   (* x x))
3
4 (define (power4 y)
5   (square (square y)))

```

Inlining arithmetic operators to expose type checks yields the following code.

```

1 (define (square x)
2   (cond
3     ((fixnum? x) (or (fx*? x x) (*** x x)))
4     ((flonum? x) (fl* x x))
5     (else (*** x x))))
6
7 (define (power4 y)
8   (square (square y)))

```

The `fixnum?` and `flonum?` primitives check whether their argument is a fixnum or a flonum, respectively.² The `fx*?` primitive multiplies two fixnums but returns `#f` on overflow. The `fl*` primitive multiplies two flonums. Finally, `***` computes generic multiplication out-of-line, throwing an exception on invalid types.

Note that `square` has four exit sites: one returning a fixnum (`(fx*? x x)`, line 3), one returning a flonum (line 4), one returning a bignum (`(*** x x)`, line 3), and one returning a value of any numeric type (line 5). Figure 13 shows the CFG of `square`, where these exit sites appear as blocks **V**, **Z**, **W**, and **Y** respectively. Since each call site to `square` has a single return point (**B** and **C** in Figure 13), this precise type information is lost: the best achievable approximation is a return point expecting any numeric type. These return points thus act as bottlenecks for SBBV, forcing the type of the returned value to be widened to any number. This yields suboptimal code where the type of the returned value must be checked redundantly in the second call to `square`.

Given a calling scheme that permits a single call site to pass multiple return points to a function, this key limitation can be addressed by duplicating and specializing return points, and passing these specialized return points to the function. Yet this does not resolve the question of which return points should be created for a given call site. This problem stems from the fundamentally forward-only model of BBV: when specializing a call site, the callee’s exit sites may not yet have been specialized. Interprocedural BBV therefore requires a backward component that respecializes call sites as new exit sites are discovered. This is achieved by maintaining, for each function, the set of all known call sites and the set of exit sites reachable from each entry point. Whenever a new exit site becomes reachable from a given entry point, all corresponding call sites are updated to create a matching specialized return point. Conversely, when exit sites become unreachable due to merges, call sites

2. Scheme has multiple numeric datatypes. However, for simplicity, this chapter considers only fixnums (small integers), bignums (arbitrary-precision integers), and flonums (floating-point numbers).

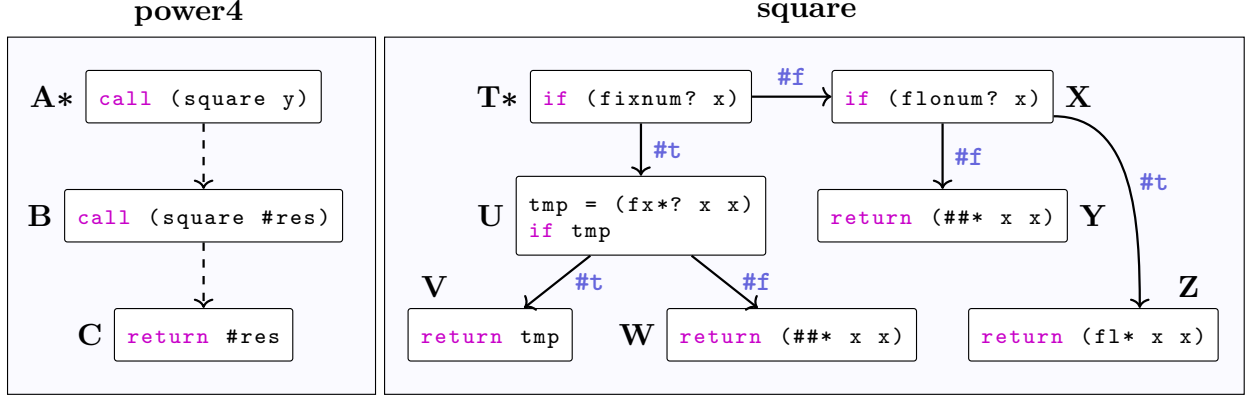


Figure 13 – Control-flow graphs of the `power4` (left, labeled **A-C**) and `square` (right, labeled **T-Z**) functions. Dashed edges represent return points of `call` instructions. Function entry points are denoted by `*`.

are notified to remove the now-obsolete return points. This backward propagation of exit contexts forms the core of the Lambda Versioning algorithm.

Figure 14 shows the result of this process applied to the CFG of Figure 13: a specialized `power4` that tests the type of its argument only once, in the first call to `square`, barring any overflow. The first call to `square` (block **A**) has three distinct, indexed return points: **B1**, specialized for fixnums and labeled `[0]`, is used by exit site **V**; **B2**, specialized for flonums and labeled `[1]`, is used by **Z**; and **B3**, a generic return point labeled `[2]`, is used by either **W** or **Y**. These return points propagate the type of the returned value to the argument type of the second call to `square`. For instance, **B2**, specialized to receive a flonum, invokes an entry point of `square` that directly applies floating-point multiplication (block **Z**).

Note that not all entry points require three return points. For instance, the **Z** entry point can only reach an exit site returning a flonum at return point `[1]`; return points `[0]` and `[2]` can therefore be left unspecified.

Beyond return type information, return points can also retrieve type information discovered about a call site’s arguments. For instance, at return points **B1**, **B2**, and **B3**, the variable `y` is now known to be a fixnum, flonum, and number, respectively. This information could be further propagated by the exit sites of `power4` to generate specialized return points and eliminate additional type checks.

3.1.2. Arithmetic Operators

In the previous example, arithmetic operators were inlined to expose type checks. This approach was necessary for SBBV, which can only propagate type information within a function’s body. However, inlining operators adds a layer of code duplication on top of

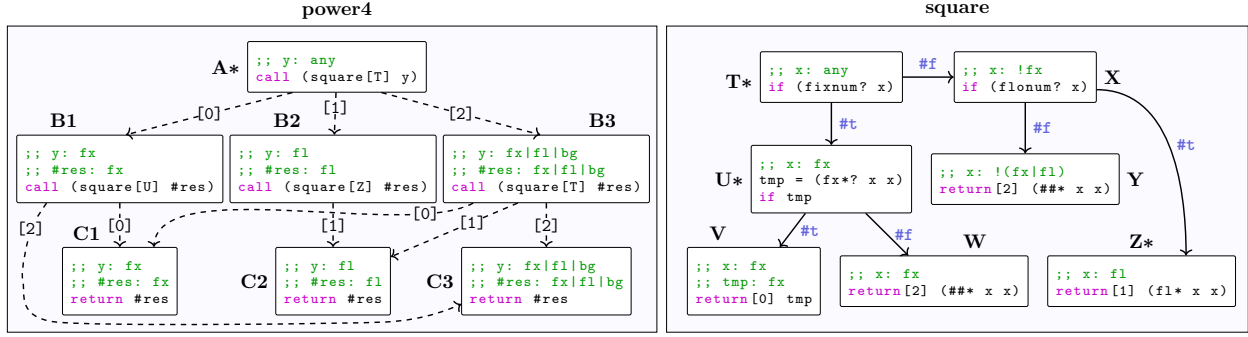


Figure 14 – Specialized control-flow graphs of `power4` (blocks **A-C**) and `square` (blocks **T-Z**), featuring multiple return points. Dashed edges represent return points associated with `call` instructions, each labeled by an index `[i]`. Each `return` specifies the index `[i]` of the return point to invoke. Each `call` additionally specifies a target entry point `[P]`.

SBBV. Since ΛV is capable of interprocedural context propagation, inlining is no longer necessary; operators can instead be represented as ordinary functions and versioned like any other. For instance, rather than inlining the `*` operator as in the previous example, the following `*` function can be added to the runtime, which tests the types of its arguments and dispatches to the appropriate multiplication primitive.³

```

1 (define (* a b)
2   (cond
3     ((and (fixnum? a) (fixnum? b))
4      (or (fx*? a b) (*** a b)))
5     ((and (flonum? a) (flonum? b))
6      (fl* a b))
7     (else
8      (*** a b))))

```

An advantage of this approach is that type specialization of operators can be reused across call sites. When calling an operator, if a specialized entry point and exit contexts have already been computed, specialized return points can be generated without reanalyzing the operator’s body. By removing the version limit on arithmetic operators, all call sites can be guaranteed a precisely specialized entry point, just as if the operator had been inlined, except that call sites with identical calling contexts share the same specialized entry point. It may still be desirable to inline specialized operators at code generation to eliminate function call overhead, but this inlining can now take place *after* BBV, benefiting from type analysis.

Figure 15 shows the specialized CFG of `square` using a multi-entry/multi-return representation of `*`. Note that propagating type information alone is insufficient: without additional context, the types of both `a` and `b` must be checked separately, even though they refer to the same value when calling `(* x x)`. Entry points are therefore specialized for contexts in which

3. Most Scheme arithmetic operators are variadic, but only binary operators are discussed for simplicity.

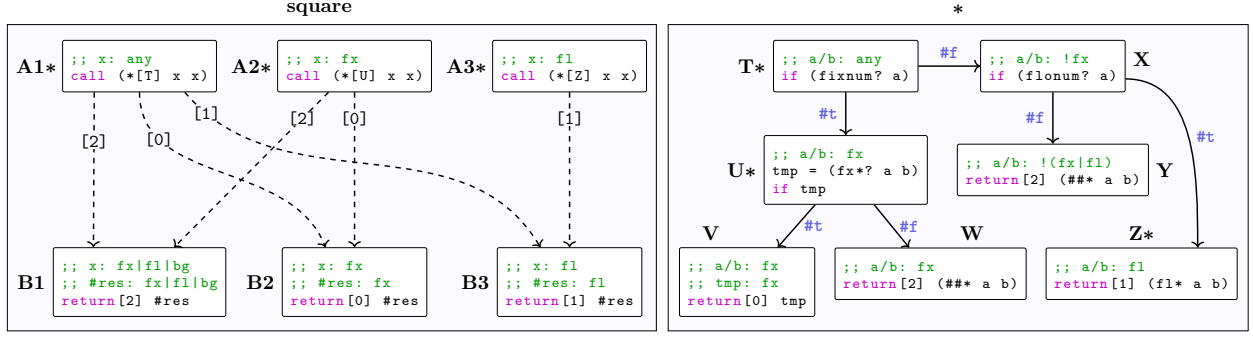


Figure 15 — Specialized control-flow graphs of `square` using a multi-return function to represent the `*` operator. Dashed edges represent the indexed return points of `call` instructions. The notation `:: a/b: type` indicates contexts where `a` and `b` are in the same equivalence class.

`a` and `b` belong to the same equivalence class, indicating that both variables contain the same value and allowing the type of both parameters to be determined from a single type check.

3.1.3. Recursive Functions

The previous examples were small enough that an optimizing compiler would likely inline them. Recursive functions are a case where this cannot be done: recursion can only be unrolled to a limited depth. This is particularly problematic for small recursive functions, where every recursive call redundantly performs type checks. This limitation is evident in Figure 9, which shows that small recursive functions optimized with SBBV execute redundant type checks even under high version limits, despite the fact that, barring any overflow, the argument’s type need only be checked once at the initial call.

As an example, consider the following recursive implementation of the factorial function.

```
1 (define (fact n)
2   (if (= n 0)
3       1
4       (* n (fact (- n 1)))))
```

Without specialized entry points, the type of `n` must be checked at every recursive call by the `=` operator. Without specialized return points, the type of the value returned by each recursive call must also be checked by `*`. The latter is especially problematic: even with thorough whole-program analysis establishing that `n` is always, say, a fixnum, the return type still depends on the exact value of `n`, since `fact` may return a bignum on overflow.

Figure 16 shows the result of specializing the CFG of `fact` with ΛV . The specialized function has two entry points: a generic one (A1), and one expecting a fixnum (A2). Both accept two return points: a fixnum-specialized one indexed [0], invoked when both `n` and the returned value are fixnums, and a generic one indexed [1], returning any numeric type. As a result, a call to `fact` with an unknown argument checks its type once in the `=` operator,

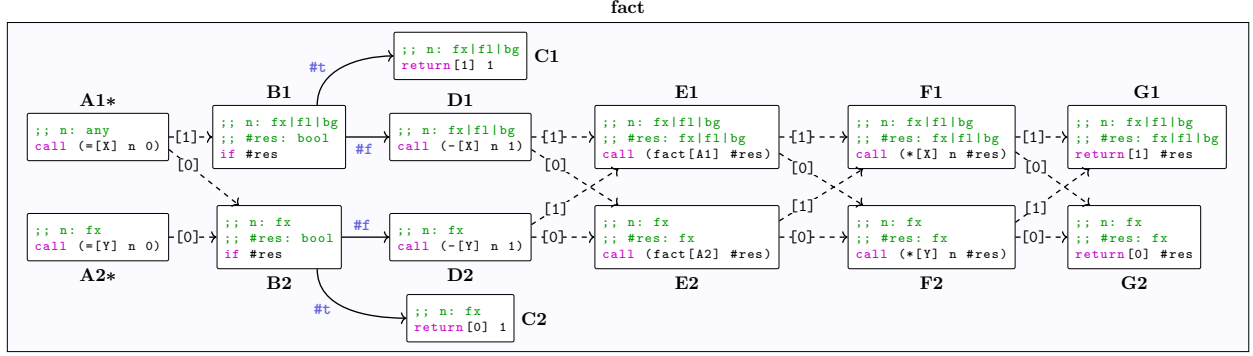


Figure 16 — Specialized control-flow graphs of the recursive **fact** function with multiple return points. Each call to the `=`, `-`, and `*` operators uses either a generic entry point `X` or a fixnum-specialized entry point `Y` with corresponding return points.

then transfers execution to branch `A2-G2`, where operators perform small-integer arithmetic without any type checks. Only overflows need to be checked, upon which execution falls back to the generic `A1-G1` branch. For simplicity, this example uses only two versions of each block, but could be extended to include a flonum-specialized entry point and return point.

3.2. Hyperfunctions

Examples in the previous section rely on the ability to generate functions with multiple entry points and return points. A *hyperfunction*⁴ is a generalization of a function possessing multiple specialized entry points, each accepting multiple specialized return points. This section describes an interface for defining and invoking hyperfunctions in Scheme. A native implementation of hyperfunctions is beyond the scope of this thesis, but Section 3.6 surveys existing implementations, and Appendix B presents the pure-Scheme implementation used in this thesis to experiment with type check removal.

3.2.1. Rewriting Motivating Examples as Hyperfunctions

A call interface for hyperfunctions requires three components: the list of available entry points, the list of return points required by each entry point, and the typing context expected at each entry point and return point.

3.2.1.1. square and power4. Consider **square** from Figure 14, which offers three entry points: a generic one, one specialized for a fixnum argument, and one for a flonum. Each entry point requires a set of indexed return points. The generic entry point, for instance, requires three: one at index `[0]`, invoked when the argument is found to be a fixnum and a

4. The term *hyperfunction* is inspired by *hypergraph*, a graph generalization with multi-target edges.

fixnum is returned; a similar one for flonums at index [1]; and a generic one at index [2], invoked when the argument is known to be a number and a number is returned. This indexing matters because some return points are shared by multiple entry points and must therefore agree on a common ordering.

In Scheme, a hyperfunction is declared with the `hyperfunction` macro, whose implementation details (“...” in the hyperfunction body) are deferred for now. Its entry points are listed under the `entries:` keyword, each labeled with the properties expected to hold at any call site jumping to it. Below is the hyperfunction definition of `square`.

```

1 (define square
2   (hyperfunction
3     entries:
4       (any
5         K: (fx→fx   fl→fl   fx|fl|bg→fx|fl|bg))
6         (fx
7           K: (fx→fx   #f      fx→bg))
8         (fl
9           K: (#f      fl→fl))
10        ...))

```

The labels `fx` and `fl` on the second and third entry points indicate that they expect their parameter to be a fixnum or a flonum, respectively, while the first entry point, `any`, accepts an argument of any type. Each entry point lists its expected return points after the `K:` keyword (the reason for that name will become apparent shortly). For instance, the `any` entry point requires three return points: `fx→fx`, `fl→fl`, and `fx|fl|bg→fx|fl|bg`. The point `fx→fx` is used when the hyperfunction determines that its parameter is a fixnum and the returned value is also a fixnum, and likewise `fl→fl` for flonums. Otherwise, `fx|fl|bg→fx|fl|bg` is used, where `fx|fl|bg` denotes the union of fixnum, flonum, and bignum (`bg`).

To conform to the indexing of return points, some are left empty. For instance, index [1] of the `fx` entry point is unspecified (`#f`), since no return point using that index is reachable from this entry. Note that index [2] of the `fx` entry point is more specialized than index [2] of the `any` entry: the latter serves two distinct exit sites, one for generic multiplication and one for an overflowing fixnum multiplication, whereas the former serves only overflows.

This hyperfunction definition specifies a calling interface: which entry points are available and which return points each entry point requires. However, some call sites inevitably have polymorphic or unknown callees. To accommodate these, every hyperfunction has an implicit fully generic entry point that accepts a single return point. This generic entry point forwards any call to the first entry point listed under `entries:`, supplying the generic return point for each of its expected return points.

The `power4` function from Figure 14 can likewise be defined as a hyperfunction. This one is simpler, as it has a single, generic entry point in this example.

```

1 (define power4
2   (hyperfunction
3     entries:
4       (any
5         K: (fx → fx   fl → fl   fx | fl | bg → fx | fl | bg))
6         ...))

```

3.2.1.2. Arithmetic Operators. Like any function, the `*` operator from Figure 15 can be converted into a hyperfunction.

```

1 (define *
2   (hyperfunction
3     entries:
4       (any × arg[0]
5         K: (fx × arg[0] → fx   fl × arg[0] → fl   fx | fl | bg × arg[0] → fx | fl | bg))
6       (fx × arg[0]
7         K: (fx × arg[0] → fx   #f                               fx × arg[0] → bg))
8       (fl × arg[0]
9         K: (#f                               fl × arg[0] → fl))
10      ...))

```

Since `*` is a binary operator, its entry points are labeled with two types separated by `×`. For instance, `fx × fx` denotes an entry point specialized for two fixnum arguments. However, in the example from Figure 15, removing all but one type check requires a more precise context in which both arguments belong to the same equivalence class. This is written `type × arg[0]`, indicating that the second argument is equivalent to the first. By convention, the representative of an equivalence class is always the earliest argument in the signature, to avoid having multiple ways to write the same label. For instance, `any × arg[0]` is an entry point where both arguments are the same value, of unknown type.

The `=` comparison operator from Figure 16 can likewise be converted into a hyperfunction. In this particular example, equivalence classes were not needed, so no entry point with an equivalence is listed. Note, however, that the choice of entry points depends on usage during program specialization: a large program performing arithmetic on fixnums, flonums, and mixed types would result in more entry points for arithmetic operators.

```

1 (define =
2   (hyperfunction
3     entries:
4       (any × any
5         K: (fx × fx → bool   fx | fl | bg × fx | fl | bg → bool))
6       (fx × fx
7         K: (fx × fx → bool))
8       ...))

```

3.2.1.3. Recursive fact. As a final example of a call interface, here is the hyperfunction definition of `fact` from Figure 16. It has two entry points: a generic one (`any`) and a fixnum-specialized one (`fx`). The `fx` entry point requires two return points: one for a fixnum result,

and one for the case where the multiplication overflows, in which case execution switches to a generic `fx|fl|bg` exit site.

```

1 (define fact
2   (hyperfunction
3     entries:
4       (any
5         K: (fx → fx   fx | fl | bg → fx | fl | bg))
6         (fx
7           K: (fx → fx   fx → fx | fl | bg))
8         ...))

```

3.2.2. Representation of Hyperfunctions

The previous section described an interface for hyperfunctions, treating them as black boxes. This section focuses on their internal representation. A key observation is that a typical imperative programming style is ill-suited for representing calls to hyperfunctions, since it does not support multiple return points. More appropriate is a continuation-passing style (CPS) representation, where return points are expressed explicitly as continuations. This representation is particularly compelling, as it directly encodes a CFG with multiple entry points and return points, making hyperfunctions well-suited for BBV. For instance, the hyperfunction below corresponds to the CFG of `*` from Figure 15.⁵

```

1 (define *
2   (hyperfunction
3     entries:
4       (any × arg[0]: A
5         K: (fx × arg[0] → fx   fl × arg[0] → fl   fx | fl | bg × arg[0] → fx | fl | bg))
6       (fx × arg[0]: B
7         K: (fx × arg[0] → fx   #f                               fx × arg[0] → bg))
8       (fl × arg[0]: D
9         K: (#f                               fl × arg[0] → fl))
10    blocks:
11      A: (lambda (K a b) ;; a/b: any
12          (if (fixnum? a)
13              (B K a b)
14              (C K a b)))
15      B: (lambda (K a b) ;; a/b: fx
16          (let ((tmp (fx*? a b)))
17            (if tmp
18                (K[0] tmp)
19                (K[2] (##* a b)))))
20      C: (lambda (K a b) ;; a/b: !fx
21          (if (flonum? a)
22              (D K a b)
23              (E K a b)))
24      D: (lambda (K a b) ;; a/b: fl
25          (K[1] (fl* a b)))
26      E: (lambda (K a b) ;; a/b: !(fx|fl)
27          (K[2] (##* a b))))

```

5. A CPS representation is not strictly necessary as other CFG-like representations would also permit this, but it proves convenient and portable, making it the representation adopted in this thesis.

The body of the hyperfunction is represented as a list of continuations under the `blocks:` keyword. These continuations were obtained by applying a CPS transformation to the body of the original `*` function, with lambda-lifting applied to prevent administrative closures from being introduced. The result is that each `lambda` in the `blocks:` section explicitly receives all live variables at that point in the program. For readability, all continuations representing return points are denoted by `κ`. When returning from the hyperfunction, the invocation of a specific return point is denoted `κ[i]`, where `i` is the index of that return point.

With the `*` hyperfunction in place, the internals of `square` can now be defined.

```

1 (define square
2   (hyperfunction
3     entries:
4       (any: A1
5         K: (fx → fx   fl → fl   fx | fl | bg → fx | fl | bg))
6       (fx: A2
7         K: (fx → fx   #f       fx → bg))
8       (fl: A3
9         K: (#f       fl → fl))
10    blocks:
11      A1: (lambda (K x)
12          (call *.any × arg[0]
13                fx × arg[0] → fx: K[0]
14                fl × arg[0] → fl: K[1]
15                fx | fl | bg × arg[0] → fx | fl | bg: K[2]
16                x x))
17      A2: (lambda (K x)
18          (call *.fx × arg[0]
19                fx × arg[0] → fx: K[0]
20                fx × arg[0] → bg: K[2]
21                x x))
22      A3: (lambda (K x)
23          (call *.fl × arg[0]
24                fl × arg[0] → fl: K[1]
25                x x))))

```

The `square` hyperfunction dispatches execution to the appropriate entry point of `*`. This is done with the `call` macro, which takes a specialized entry point, a return point for each required continuation, and the call arguments. Here, the call to `*` is a tail call, so the continuations `K[0]` to `K[2]` are passed through to `*`. This illustrates an interesting phenomenon of multi-return functions: the emergence of super-tail calls, that is when only a subset of the caller's return points is passed through at some call sites, potentially shrinking the stack [56]. This is discussed in the related work (Section 3.6.2).

In comparison, the `fact` hyperfunction corresponding to the CFG from Figure 16 is shown below. It contains both non-tail calls (blocks `A1` through `C2`) and tail calls (blocks `D1` and `D2`). The former are represented by passing a `lambda` as continuation. Although this `lambda` appears to introduce a closure, knowing that it represents a return point allows the compiler to avoid a heap-allocated closure and instead push its free variables onto the stack, implementing it as a stack frame.

```

1 (define fact
2   (hyperfunction
3     entries:
4       (any: A1
5         K: (fx→fx   fx|fl|bg→fx|fl|bg))
6         (fx: A2
7           K: (fx→fx   fx→fx|bg))
8     blocks:
9       A1:
10        (lambda (K n) ;; n:any
11          (call =.any × any
12                fx × fx→bool: (lambda (r) (B2 K n r))
13                fx|fl|bg × fx|fl|bg→bool: (lambda (r) (B1 K n r))
14                n 0))
15        A2:
16        (lambda (K n) ;; n:fx
17          (call =.fx × fx
18                fx × fx→bool: (lambda (r) (B2 K n r))
19                n 0))
20        B1:
21        (lambda (K n r) ;; n: fx|fl|bg / r: bool
22          (if r
23              (K[1] 1)
24              (call -.any × any
25                    fx × fx→fx: (lambda (r) (C2 K n r))
26                    fx|fl|bg × fx|fl|bg→fx|fl|bg: (lambda (r) (C1 K n r))
27                    n 1)))
28        B2:
29        (lambda (K n r) ;; n: fx / r: bool
30          (if r
31              (K[0] 1)
32              (call -.fx × fx
33                    fx × fx→fx: (lambda (r) (C2 K n r))
34                    fx × fx→bg: (lambda (r) (C1 K n r))
35                    n 1)))
36        C1:
37        (lambda (K n r) ;; n: fx|fl|bg / r: fx|fl|bg
38          (call fact.any
39                fx→fx: (lambda (r) (D2 K n r))
40                fx|fl|bg→fx|fl|bg: (lambda (r) (D1 K n r))
41                r))
42        C2:
43        (lambda (K n r) ;; n: fx / r: fx
44          (call fact.fx
45                fx→fx: (lambda (r) (D2 K n r))
46                fx→fx|bg: (lambda (r) (D1 K n r))
47                r))
48        D1:
49        (lambda (K n r) ;; n: fx|fl|bg / r: fx|fl|bg
50          (call *.any × any
51                fx × fx→fx: K[0]
52                fx|fl|bg × fx|fl|bg→fx|fl|bg: K[1]
53                n r))
54        D2:
55        (lambda (K n r) ;; n: fx / r: fx
56          (call *.fx × fx
57                fx × fx→fx: K[0]
58                fx × fx→bg: K[1]
59                n r))))

```

3.3. The Lambda Versioning Algorithm

This section presents the Lambda Versioning (AV) algorithm, a compilation technique that optimizes program execution by duplicating and specializing basic blocks, similarly to Static Basic Block Versioning (SBBV). It addresses two key limitations of SBBV: its intraprocedural nature and its reliance on a compiler’s internal CFG representation. The goal of AV is to transform a source program into a specialized one in which every function becomes a hyperfunction whose entry and return points are tailored to its uses in the program. AV operates directly on source code, Scheme in this case, and produces a transformed program that is itself valid Scheme. It can then be compiled by any Scheme compiler implementing the hyperfunction interface, benefiting from the specialization performed by AV.

The algorithm is presented in three parts. Section 3.3.1 covers the preprocessing steps that transform source code into a format suitable for AV. Section 3.3.2 describes the underlying multi-source reachability algorithm required by AV. Unlike in SBBV (Chapter 2), where the reachability algorithm was left implicit, it plays a central role in AV and warrants an explicit treatment. Section 3.3.3 then presents the algorithm itself.

3.3.1. Source Code Preprocessing

Before applying the AV algorithm, some preprocessing is performed. In Scheme, this includes desugaring, alpha conversion, and assignment conversion to make mutable free variables appear explicitly as boxes. These steps are not strictly necessary, but they simplify the algorithm. The crux of preprocessing is a CPS transformation followed by lambda-lifting, which together produce an intermediate representation in which every `lambda` in the source code becomes an unspecialized hyperfunction. For instance, consider this program.

```
1 (define (fact n) (if (= n 0) 1 (* n (fact (- n 1)))))
2 (display (fact (read)))
```

It is transformed into the code below. Note that the top-level code is wrapped into a top-level hyperfunction, which later serves as the entry point for AV.

```
1 (define fact (hyperfunction
2   entries: (any: A K: (any→any))
3   blocks:
4     A: (lambda (K n) (call-generic = (lambda (r) (B K n r)) n 0))
5     B: (lambda (K n r) (if r (K 1) (call-generic - (lambda (r) (C K n r)) n 1)))
6     C: (lambda (K n r) (call-generic fact (lambda (r) (D K n r)) r))
7     D: (lambda (K n r) (call-generic * K n r))))
8
9 (define top-level (hyperfunction
10  entries: (∅: A K: (∅→∅))
11  blocks:
12    A: (lambda () (call-generic fact (lambda (r) (B r)) (read)))
13    B: (lambda (r) (display r))))
```

3.3.2. Multi-Source Reachability Algorithm

Since AV requires listing the specialized exit sites reachable from each specialized entry point, it necessitates an underlying multi-source reachability algorithm. This is a stronger requirement than the *single*-source reachability algorithm implicitly assumed in the algorithm description of Chapter 2.

Due to its more prominent role in this interprocedural variant of BBV, the operations on the multi-source reachability graph used by AV are made explicit in Section 3.3.3. The table below describes the interface this algorithm must expose.

Operation	Effect	Output
addEdge	Add an edge in the graph and enqueue any reconnected versions if necessary.	Entry/exit pairs that became connected.
removeEdge	Remove an edge from the graph.	Entry/exit pairs that became disconnected.
redirectEdges	Find all edges pointing to a given set of blocks, and redirect them to another block.	Newly connected and newly disconnected entry/exit pairs.
markExit	Mark a block as an exit site.	All entry points from which this block is reachable.
markEntry	Mark a block as an entry point.	—

For the experiments in this thesis, this algorithm was implemented by maintaining one instance of the spanning tree described in Section 2.2.2 per unmerged entry point. This is acceptable in practice, since the number of unmerged entry points is approximately proportional to the version limit, which is expected to remain low.

3.3.3. The Algorithm

This section presents the Lambda Versioning (AV) algorithm. Like SBBV, AV specializes code via a breadth-first traversal of the CFG, here represented by hyperfunctions. It differs in that AV also crosses abstraction barriers, requiring two new components to the algorithm: a forward one that generates specialized entry points for argument types at call sites, and a backward one that generates specialized return points for each discovered specialized exit site. To emphasize these components, the AV algorithm is presented as a diff of SBBV (Algorithms 1.1-1.7) where new parts are highlighted in green in Algorithms 2.1-2.10.

The data structures used in the algorithm are presented in Figure 17. Algorithms 2.1-2.10 define their inputs and outputs in terms of these data structures. Most central are the **block** data structure, representing basic blocks in unspecialized hyperfunctions, and the **version**

context
└ <i>types</i> : mapping from variables to their abstract type .
└ <i>equivalences</i> : equivalence classes among variables.
type
└ <i>primitive</i> : set of primitive types a value can take.
└ <i>hyperfunctions</i> : set of hyperfunctions a value can be.
hyperfunction
└ <i>entry</i> : entry block of this hyperfunction.
└ <i>blocks</i> : set of all blocks of this hyperfunction.
block
└ <i>body</i> : the instructions constituting the block's code.
└ <i>allVersions</i> : mapping from a context to the corresponding versions of the block.
└ <i>reachableVersions</i> : same as <i>allVersions</i> , but restricted to reachable contexts .
instruction
└ <i>kind</i> : instruction kind, for instance "if" or "goto".
└ ...: other properties relevant to the instruction kind.
version
└ <i>block</i> : the block from which this version was specialized.
└ <i>context</i> : the context for which this version was specialized.
└ <i>contextAfter</i> : the exit context after execution of this version's instructions.
└ <i>body</i> : the instructions constituting the version's specialized code.
└ <i>merged</i> : if this version was merged, points to the resulting merged version .
└ <i>callSites (entry points only)</i> : call sites (versions) known to jump to this entry point.
└ <i>exitSites (entry points only)</i> : exit sites (versions) reachable from this entry point.

Figure 17 – Data structures used in the AV algorithm.

data structure, representing basic blocks specialized for a given context, i.e., basic blocks belonging to a specialized hyperfunction. A **context** is a mapping from variables to types, augmented with equivalence classes that identify variables containing the same value. Types are sets of primitive types a value can take at run-time, such as **fx**, **pair**, or **hyperfunction**, but do not include the type of values contained in structures, such as the **car** and **cdr** of a pair, or closure of a hyperfunction. In AV, contexts also track the type of each parameter of the current hyperfunction, even if that parameter is not live at the given block, which is necessary to propagate parameter type information to return points.

Algorithm 2.1 is the main function of AV. It traverses and specializes the blocks of a hyperfunction starting from its entry point. The initial context at this entry point is fully generic, represented by $\top$, and is narrowed as abstract interpretation progresses. In

Input: (<i>entry</i> : block, <i>limit</i> : integer)	Output: —
<pre> 1 define lambdaVersioning (<i>entry</i>, <i>limit</i>): 2 <i>entryVersion</i> ← getOrCreateVersion (<i>entry</i>, $\top$) 3 markEntry (<i>entryVersion</i>) 4 while <i>version</i> ← <i>queue.dequeue</i>() : 5 if <i>version</i> is not reachable or <i>version</i> is done : continue 6 if <i>version.block.reachableVersions</i> > <i>limit</i> : mergeSome (<i>version.block</i>) 7 if <i>version</i> is reachable : specialize (<i>version</i>) </pre>	

Algorithm 2.1: The main function initiates a breadth-first traversal from the given entry point. Since the queue can contain unreachable or already processed versions, some are skipped (line 5). If a version’s block has exceeded its version limit, a merge is requested (line 6). The test at line 7 is needed to check if the version was merged.

Input: (<i>block</i> : block, <i>context</i> : context)	Output: version
<pre> 1 define getOrCreateVersion (<i>block</i>, <i>context</i>): 2 <i>version</i> ← getVersion (<i>block</i>, <i>context</i>) 3 if <i>version</i> : return <i>version</i> 4 <i>newVersion</i> ← new Version(<i>block</i>, <i>context</i>) 5 <i>block.allVersions</i>[<i>context</i>] ← <i>newVersion</i> 6 <i>queue.enqueue</i>(<i>newVersion</i>) 7 return <i>newVersion</i> </pre>	

Algorithm 2.2: This function is called to request a version for a given block and context. If a version exists, it is returned (line 3). Otherwise, a new version is added to the record of existing versions of this block, enqueued, and returned.

Input: (<i>block</i> : block, <i>context</i> : context)	Output: version
<pre> 1 define getVersion (<i>block</i>, <i>context</i>): 2 <i>version</i> ← <i>block.allVersions</i>[<i>context</i>] 3 if not <i>version</i> : return false 4 while <i>version.merged</i> : <i>version</i> ← <i>version.merged</i> 5 return <i>version</i> </pre>	

Algorithm 2.3: This function looks for an existing version for a given block and context. If a version exists, the chain of merges in its **merged** field (line 4) is unrolled to get and return its latest generalization. Otherwise **false** is returned.

a program where all call sites have a uniquely known callee, invoking **lambdaVersioning** on the program’s entry point suffices to apply whole-program specialization. Since some call sites inevitably have unknown callees, however, **lambdaVersioning** is also invoked on each hyperfunction, ensuring the existence of fallback generic entry points. After ΛV , the specialized CFG of the program is embedded in the *reachableVersions* tables of blocks in the original CFG, from which specialized hyperfunctions are reconstructed.

The backward component of ΛV is present in Algorithms 2.5, 2.6, 2.7, and 2.10. When an edge is added to or removed from the specialized CFG, the multi-source reachability algorithm returns the sets of paths added/removed between entry points and exit sites. These are used to update the set of exit sites reachable from each specialized entry point, stored in their

Input: $(version : version)$	Output: —
<pre> 1 define specialize (<i>version</i>): 2 <i>context</i> $\leftarrow$ <i>version</i>.<i>context</i> 3 for <i>instr</i> in <i>version</i>.<i>block</i>.<i>body</i> : 4 match <i>instr</i>.<i>kind</i> : 5 case "goto" : <i>context</i>, <i>newInstr</i> $\leftarrow$ specializeGoto (<i>version</i>, <i>context</i>, <i>instr</i>) 6 case "exit" : <i>context</i>, <i>newInstr</i> $\leftarrow$ specializeExit (<i>version</i>, <i>context</i>, <i>instr</i>) 7 case "call" : <i>context</i>, <i>newInstr</i> $\leftarrow$ specializeCall (<i>version</i>, <i>context</i>, <i>instr</i>) 8 ... 9 <i>version</i>.<i>body</i>.append(<i>newInstr</i>) 10 if <i>newInstr</i> always halts : break 11 <i>version</i>.<i>contextAfter</i> $\leftarrow$ <i>context</i> </pre>	

Algorithm 2.4: This function specializes a version by dispatching its instructions to auxiliaries. Only some cases are shown; omitted is "if", which is similar to "goto" but with narrowing. In this Scheme implementation, *block.body* is an *sexp*.

Input: $(version : version, \text{ context : context, instruction : instruction})$	Output: $\text{context} \times \text{instruction}$
<pre> 1 define specializeGoto (<i>version</i>, <i>context</i>, <i>instruction</i>): 2 <i>targetContext</i> $\leftarrow$ <i>context</i>.filter(<i>var</i> $\Rightarrow$ <i>var</i> is in <i>instruction</i>.<i>target</i>.<i>parameters</i>) 3 <i>targetVersion</i> $\leftarrow$ getOrCreateVersion (<i>instruction</i>.<i>target</i>, <i>targetContext</i>) 4 <i>newPaths</i> $\leftarrow$ addEdge (<i>version</i>, <i>targetVersion</i>) 5 for (<i>in</i>, <i>out</i>) in <i>newPaths</i> : addReturnPoints (<i>in</i>, <i>out</i>) 6 return (<i>context</i>, new Instruction("goto", <i>targetVersion</i>.<i>id</i>)) </pre>	

Algorithm 2.5: This function propagates the context from a "goto" instruction to its target. This adds an edge in the specialized CFG with **addEdge**, which returns pairs of newly connected entry/exit (*in/out* at line 5). When an entry point acquires a new exit site, its call sites are notified with **addReturnPoints**.

Input: $(version : version, \text{ context : context, instruction : instruction})$	Output: $\text{context} \times \text{instruction}$
<pre> 1 define specializeExit (<i>version</i>, <i>context</i>, <i>instruction</i>): 2 <i>entries</i> $\leftarrow$ markExit (<i>version</i>) 3 for <i>entry</i> in <i>entries</i> : addReturnPoints (<i>entry</i>, <i>version</i>) 4 return (<i>context</i>, new Instruction("exit")) </pre>	

Algorithm 2.6: The specialization of an "exit" indicates that this version is an exit site. It is marked by **markExit**, which returns entry points from which it is reachable. Call sites of these entry points are notified with **addReturnPoints**.

exitSites field. The corresponding return points are then updated by **addReturnPoints** (Algorithm 2.8) and **removeReturnPoints** (Algorithm 2.9). Since adding/removing a return point corresponds to adding/removing an edge in the specialized CFG, these two functions are recursive: a single addition or removal can cascade across multiple hyperfunctions. To ensure termination, all newly reachable return points are added before any unreachable return points are removed.

Input: (*version*: *version*, *context*: *context*, *instruction*: *instruction*)
Output: *context* $\times$ *instruction*

```

1 define specializeCall (version, context, instruction):
2   hyperfunctions  $\leftarrow$  context[instruction.target].hyperfunctions
3   if |hyperfunctions| = 1 :
4     args  $\leftarrow$  instruction.arguments
5     params  $\leftarrow$  hyperfunctions[0].parameters
6     calleeContext  $\leftarrow$  new Context(p: context[a] for (p, a) in params  $\otimes$  args)
7     entry  $\leftarrow$  getOrCreateVersion (hyperfunctions[0].entry, calleeContext)
8     markEntry (entry)
9     entry.callSites.add(version)
10    for exitSite in entry.exitSites :
11      exitContext  $\leftarrow$  context  $\cap$  exitSite.contextAfter
12      if exitContext =  $\perp$  : continue
13      returnPoint  $\leftarrow$  getOrCreateVersion (instruction.returnPoint, exitContext)
14      newPaths  $\leftarrow$  addEdge (version, returnPoint)
15      for (in, out) in newPaths : addReturnPoints (in, out)
16    return (context, new Instruction("specialized-call", entry))
17  else return (context, new Instruction("generic-call", instruction.target))

```

Algorithm 2.7: Specialization of “call” instructions. When the target is a specific hyperfunction (line 3), a specialized entry point is requested. Contexts for specialized return points are computed with $\cap$, which narrows argument types at the call site with that discovered for parameters at exit sites. Edges to return points (line 14) may connect new entry/exit pairs, which are forwarded to **addReturnPoints**.

Input: (*entryPoint*: *version*, *exitSite*: *version*) **Output:** —

```

1 define addReturnPoints (entryPoint, exitSite):
2   if exitSite in entryPoint.exitSites : return
3   entryPoint.exitSites.add(exitSite)
4   for callSite in entryPoint.callSites :
5     exitContext  $\leftarrow$  callSite.contextAfter  $\cap$  exitSite.contextAfter
6     if exitContext =  $\perp$  : continue
7     callInstruction  $\leftarrow$  callSite.body.last
8     returnPoint  $\leftarrow$  getOrCreateVersion (callInstruction.returnPoint, exitContext)
9     newPaths  $\leftarrow$  addEdge (callSite, returnPoint)
10    for (in, out) in newPaths : addReturnPoints (in, out)

```

Algorithm 2.8: This function is called when a specialized exit site becomes reachable from an entry point. Specialized return points are generated for call sites of this entry. Return point contexts are computed by $\cap$, which narrows types at the call site with those at the exit site. When adding edges, this function is called recursively. The recursion terminates because it is limited to reconnecting existing return points.

The forward interprocedural component of AV is highlighted in the **specializeCall** function (Algorithm 2.7). When a call site is specialized, the type of the call target is looked up. Each type contains the set of hyperfunctions the target could be at that specialized call site. If this set contains a single hyperfunction, a specialized entry point is requested, and created if necessary, for that hyperfunction. This may seem overly restrictive, since many

Input: (<i>entryPoint</i> :version, <i>exitSite</i> :version)	Output: —
<pre> 1 define removeReturnPoints (<i>entryPoint</i>, <i>exitSite</i>): 2 if <i>exitSite</i> not in <i>entryPoint</i>.<i>exitSites</i> : return 3 <i>entryPoint</i>.<i>exitSites</i>.remove(<i>exitSite</i>) 4 for <i>callSite</i> in <i>entryPoint</i>.<i>callSites</i> : 5 <i>exitContext</i> $\leftarrow$ <i>callSite</i>.<i>contextAfter</i> $\cap$ <i>exitSite</i>.<i>contextAfter</i> 6 if <i>exitContext</i> = $\perp$: continue 7 <i>callInstruction</i> $\leftarrow$ <i>callSite</i>.<i>body</i>.last 8 <i>returnPoint</i> $\leftarrow$ getVersion (<i>callInstruction</i>.<i>returnPoint</i>, <i>exitContext</i>) 9 <i>oldPaths</i> $\leftarrow$ removeEdge (<i>callSite</i>, <i>returnPoint</i>) 10 for (<i>in</i>, <i>out</i>) in <i>oldPaths</i> : removeReturnPoints (<i>in</i>, <i>out</i>) </pre>	

Algorithm 2.9: This function is called when a specialized exit site becomes unreachable from an entry point. Specialized return points are disconnected for all call sites of this entry point. When removing edges, this function is called recursively. The recursion terminates because there are finitely many return points to be removed.

Input: (<i>block</i> :block)	Output: —
<pre> 1 define mergeSome (<i>block</i>): 2 <i>versions</i> $\leftarrow$ mergeHeuristic (<i>block</i>.<i>reachableVersions</i>) 3 <i>newVersion</i> $\leftarrow$ getOrCreateVersion (<i>block</i>, $\bigcup_{v \in \text{versions}} v.\text{context}$) 4 for <i>version</i> in (<i>versions</i> $\setminus$ {<i>newVersion</i>}): 5 <i>version</i>.<i>merged</i> $\leftarrow$ <i>newVersion</i> 6 <i>newPaths</i>, <i>oldPaths</i> $\leftarrow$ redirectEdges (<i>versions</i>, <i>newVersion</i>) 7 for (<i>in</i>, <i>out</i>) in <i>newPaths</i> : addReturnPoints (<i>in</i>, <i>out</i>) 8 for (<i>in</i>, <i>out</i>) in <i>oldPaths</i> : removeReturnPoints (<i>in</i>, <i>out</i>) </pre>	

Algorithm 2.10: When a block’s version limit is exceeded, a heuristic selects versions to merge (union with widening, represented by $\bigcup$), updating call sites accordingly. Bookkeeping is required to update references to merged versions, done in practice with the **merged** field of each version, and is left implicit throughout the algorithm.

call sites have polymorphic targets; however, ΛV generates multiple specialized versions of each call site, allowing versions to target a single callee. Figuratively, where techniques such as k -CFA specialize call sites along a *depth* axis (the call stack depth k), ΛV specializes them along a *width* axis, governed by the version limit. Yet, despite this unique-target constraint being less restrictive than it may seem, there is room for improvement. For instance, hyperfunctions with identically specialized entry points and exit sites could share a single call site. This is further discussed in Section 3.7.

Algorithms 2.2, 2.3, and 2.4 essentially mirror the SBBV algorithm from Chapter 2. They are included for completeness, but do not warrant further discussion.

Once **lambdaVersioning** (Algorithm 2.1) has been called on the top-level entry point of the program and on the entry point of each hyperfunction, ΛV is complete and specialized source code is generated from the versions embedded in the *reachableVersions* of each block. At this point, the required return points for each specialized call site are known and an index allocation phase computes an ordering for the continuations of hyperfunctions.

Input: (<i>contexts</i> : set of contexts)	Output: set of contexts
<pre> 1 define <i>mergeHeuristic</i> (<i>contexts</i>): 2 <i>selection</i>, <i>distance</i> $\leftarrow$ <i>null</i>, ∞ 3 for (<i>ctx1</i>, <i>ctx2</i>) in combinations of <i>contexts</i> : 4 <i>pairDistance</i> $\leftarrow$ <i>contextDistance</i> (<i>ctx1</i>, <i>ctx2</i>) 5 if <i>pairDistance</i> < <i>distance</i> : <i>selection</i>, <i>distance</i> $\leftarrow$ (<i>ctx1</i>, <i>ctx2</i>), <i>pairDistance</i> 6 return <i>selection</i> </pre>	

Algorithm 2.11: A broad family of local merge heuristics can be defined by selecting the pair of contexts minimizing some distance function.

3.4. An Improved Merge Heuristic

This section presents a simple yet effective merge heuristic for eliminating type checks with AV. In general, a merge heuristic is any function that takes the set of reachable versions of a basic block and outputs the set of versions to merge. The heuristic in this section, however, remains strictly local: its inputs are limited to the merge candidates and their contexts. This reduces whole-program optimization to a sequence of inexpensive merge decisions that are simpler to reason about.

One family of local heuristics takes the form shown in Algorithm 2.11: given a distance function *contextDistance*, versions to merge are selected by choosing the closest pair under that distance, reducing the selection problem to comparing contexts two at a time.⁶

A distance function effective at eliminating type checks is shown in Appendix A. It was designed specifically to reduce type checks in arithmetic computations, since most polymorphic operators in Scheme are arithmetic. It is built upon the Hamming distance between contexts, with biases that prioritize or deprioritize the merging of certain context pairs.

First, a bias is applied so that versions containing a possible bignum (e.g., *bg* or *fx|bg*) are closest, and thus merged first. This is motivated by the assumption that overflows are rare, so such versions are unlikely to execute, and merging them frees space for more useful ones. They remain obtainable, however, with increased version limits.

Second, a bias is applied so that versions containing only pure arithmetic, i.e., where all numeric variables are either *fx* or *f1*, are farthest from other versions. This is motivated by the assumption that pure arithmetic is common. While this may not hold universally, it proves true for many hot loops in the benchmarks presented in Section 3.5. When two pairs of contexts share the same bias, the Hamming distance serves as a tiebreaker.

Section 3.5 presents empirical results demonstrating that this heuristic significantly outperforms the merge by similarity from Chapter 2. While specific to Scheme and unlikely to generalize to other languages, this nonetheless demonstrates that hand-crafting an effective distance function is feasible.

6. The *merge by similarity* from Chapter 2 belongs to this family; its distance function is in Appendix A.

3.5. Evaluation

This section demonstrates that Lambda Versioning achieves significantly better program specialization than Static Basic Block Versioning. In some cases, AV produces optimal programs, i.e., programs where each input’s type is checked exactly once at run-time, after which execution enters a specialized path requiring no further type checks.

Merge heuristics are also compared, for both AV and SBBV, demonstrating that the heuristic from Section 3.4 is more effective at eliminating run-time type checks.

3.5.1. Experimental Setup

The primary metric is the number of type checks eliminated by AV. Since no native implementation of hyperfunctions exists yet, the implementation from Appendix B is used. This adds overhead to hyperfunction calls, preventing meaningful execution time measurements: programs in hyperfunction representation always execute slower than their source representation. More efficient implementations are discussed in Section 3.6.

A subset of the R7RS benchmark suite [1] is used, detailed in Appendix C. Benchmarks are split into two categories: micro benchmarks (fewer than 400 lines of code) and macro benchmarks (more than 400 lines of code). Each benchmark is first executed without specialization under profiling to measure the total number of type checks, then profiled after specialization by AV with version limits from 1 to 10. Three merge heuristics are evaluated:

- **random**: picks versions to merge at random.
- **similarity**: merge by similarity from Chapter 2.
- **arithmetic**: monomorphic arithmetic-preserving merge from Section 3.4.

Two representations of built-in operators are tested with AV:

- **Inlined**: operators are inlined, exposing type checks directly as in Figure 14.
- **Hyperfunction**: operators are specialized as hyperfunctions as in Figure 15.

The profiled AV implementation is compared to SBBV. Since AV is an extension of SBBV, the latter can be reimplemented by disabling the interprocedural components of AV, corresponding to the algorithm parts **highlighted in green** in Section 3.3.3. This makes it possible to measure exactly what proportion of type checks is removed by interprocedural type propagation. The experiment from Chapter 2 suffered from some noise because type check removal resulted from both SBBV and other compiler optimizations such as inlining and constant folding. In contrast, type check elimination in this experiment stems solely from AV and SBBV, demonstrating that AV subsumes these optimizations. Finally, only type checks are eliminated and profiled: no interval analysis is applied, so overflow and bounds checks are not tracked, unlike in Chapter 2.

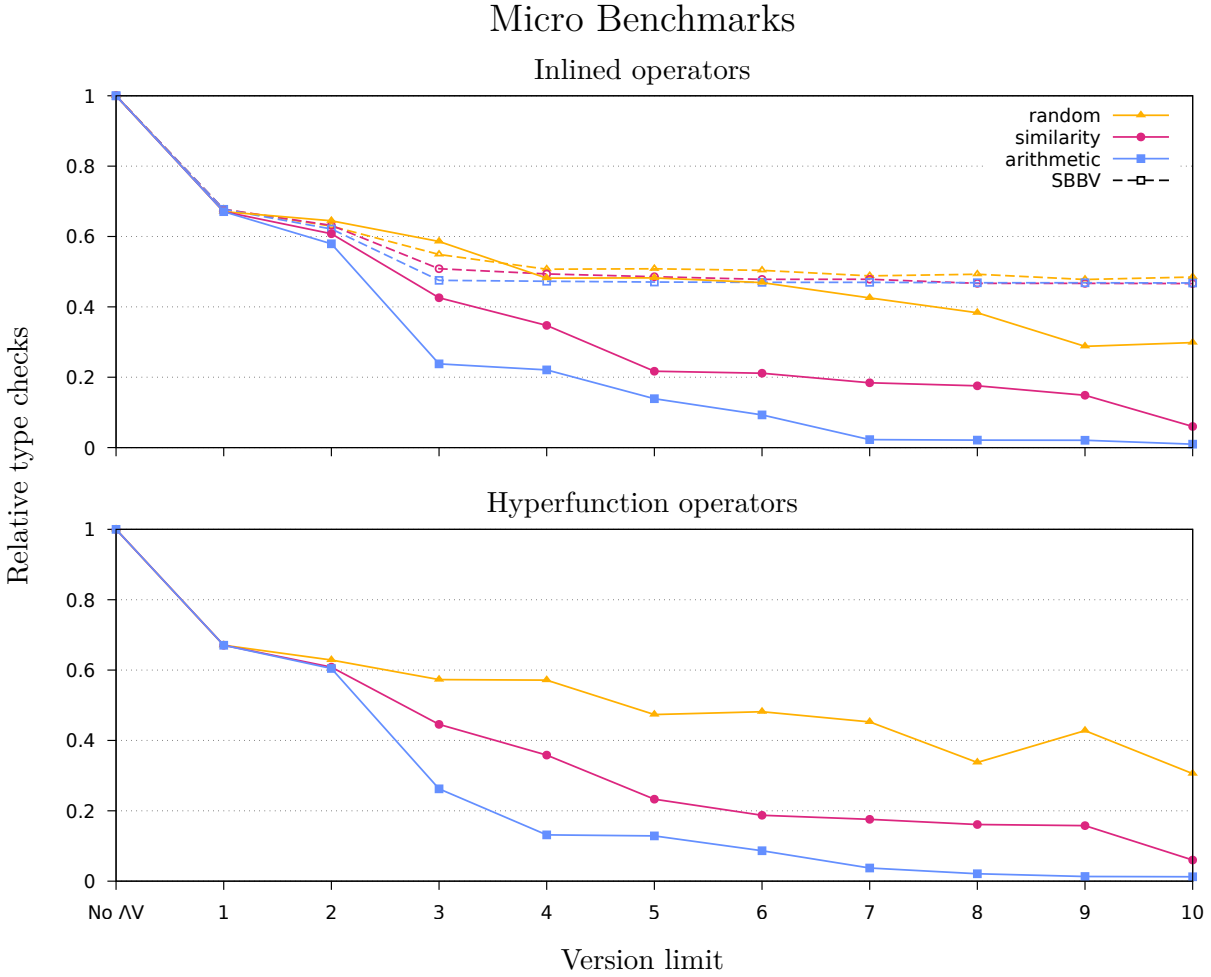


Figure 18 – Type check removal by ΛV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom, ΛV only), using three merge heuristics on micro benchmarks. The x-axis is the version limit; the leftmost entry corresponds to compilation without ΛV /SBBV. The y-axis is the ratio of run-time type checks relative to compilation without ΛV /SBBV (geometric mean). Lower is better.

3.5.2. Type Checks

Figure 18 shows aggregated type check removal across all micro benchmarks. Micro benchmarks are programs that execute short, typically recursive, functions. While not representative of real-world programs, they allow assessing how ΛV optimizes such functions.

One clear observation is that, given the right heuristic (`arithmetic`), ΛV eliminates all redundant type checks in most cases with a limit of about 7 versions. The result is an optimized recursive function that checks its argument types once, then enters a hot loop requiring no further type checks, barring any overflow. Examples of small recursive functions for which this is observed (see Appendix C) include `ack` (optimal with a limit of 5 versions),

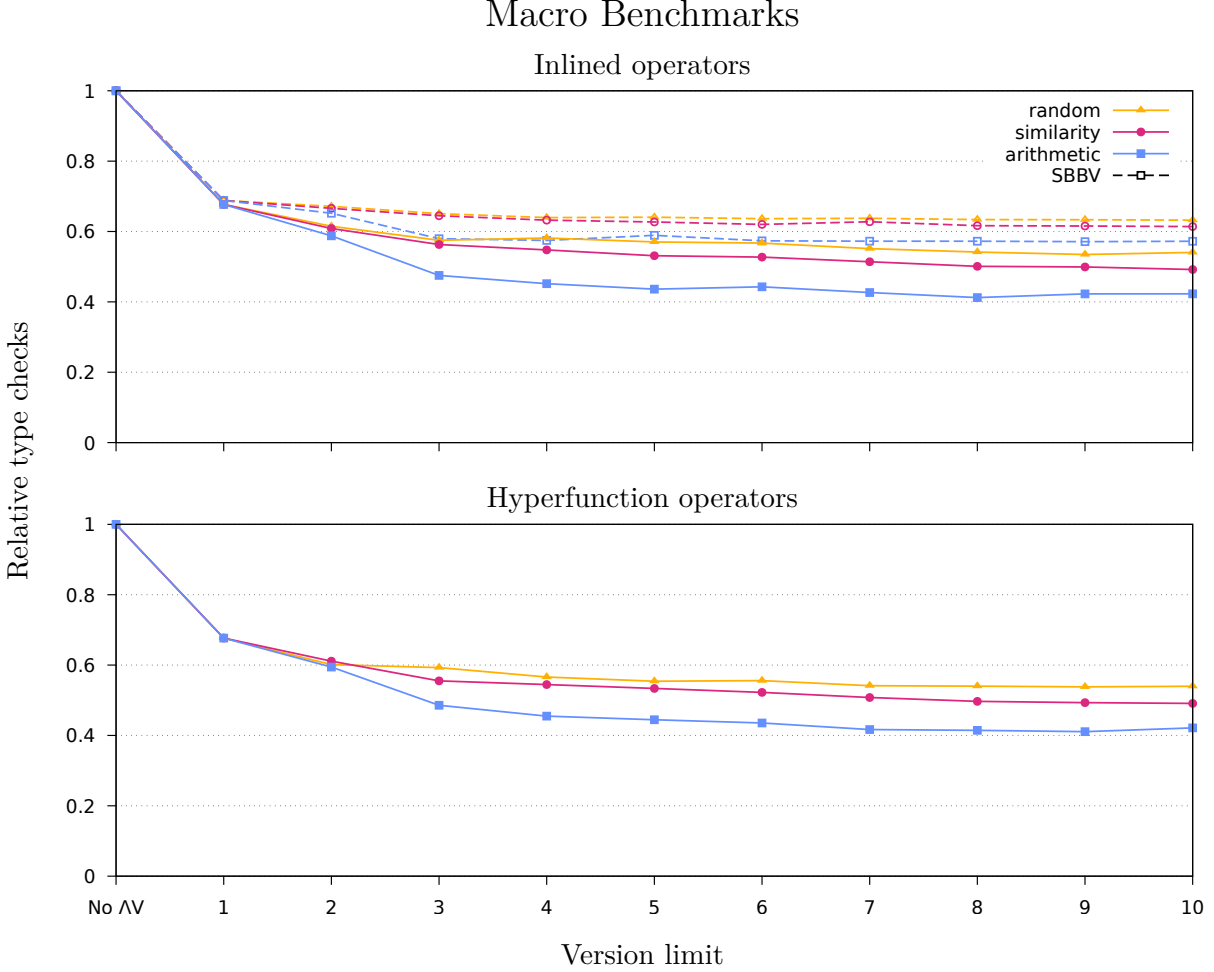


Figure 19 – Type check removal by ΛV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom, ΛV only), using three merge heuristics on macro benchmarks. The x-axis is the version limit; the leftmost entry corresponds to compilation without ΛV /SBBV. The y-axis is the ratio of run-time type checks relative to compilation without ΛV /SBBV (geometric mean). Lower is better.

`array1` (6), `fact20` (3), `fib` (3), `fibfp` (3), `quicksortfl` (10), `sum` (7), `sumfp` (7), and `tak` (10). Such optimal specializations are unattainable with SBBV, which cannot propagate return types or argument types across abstraction barriers.

Figure 19 shows aggregated type check removal across macro benchmarks, which are more representative of real-world programs. With the `arithmetic` heuristic, approximately half of all type checks are eliminated with a limit of 3 versions, beyond which increasing the limit yields little benefit. At this version limit, most remaining type checks are those applied on values read from data structures such as lists and vectors, which are not tracked in ΛV 's contexts. Benchmarks benefiting the most from ΛV are those making extensive use of

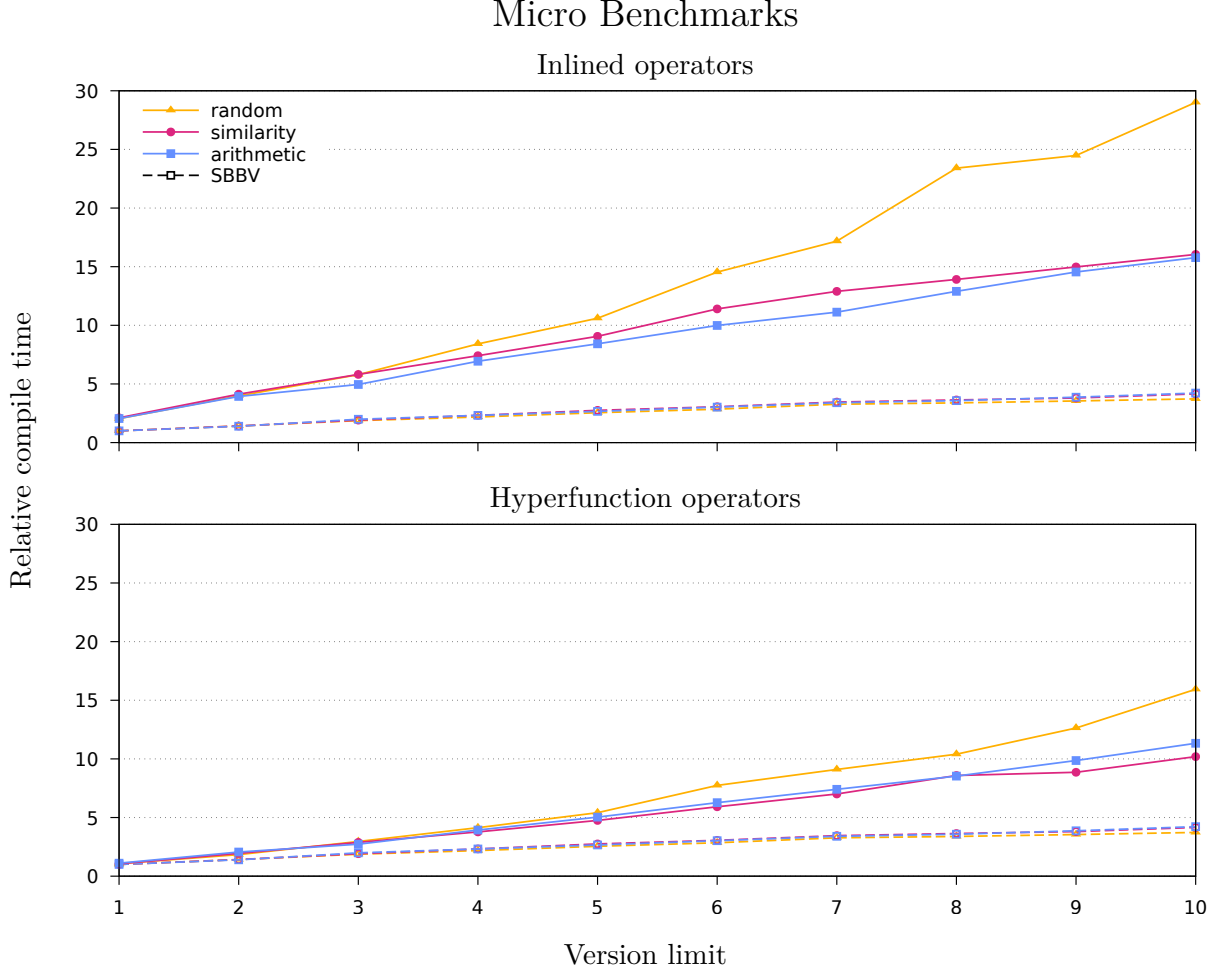


Figure 20 – Compile time of ΛV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom; SBBV included for reference despite using inlined operators), on micro benchmarks. The x-axis is the version limit. The y-axis is compile time relative to SBBV with a limit of one version (geometric mean). Lower is better.

numerical arithmetic (see `almabench`, `earley`, `maze`, `nucleic`, and `slatex` in Appendix C), precisely the class of computations for which `arithmetic` was designed.

Among the evaluated merge heuristics, `arithmetic` performs significantly better across all benchmarks, except SBBV micro benchmarks, where type checks rapidly plateau regardless of heuristic. The `random` heuristic serves as a worst case: it still permits type check removal at higher version limits, but substantially less than the other heuristics. The `similarity` heuristic outperforms `random` but only by a small margin.

Additionally, Figures 18 and 19 show that implementing built-in operators as hyperfunctions does not reduce the number of eliminated type checks compared to specialization with inlined operators. Yet, the hyperfunction representation allows reusing specialized operator code across call sites with the same calling context, rather than inlining and respecializing

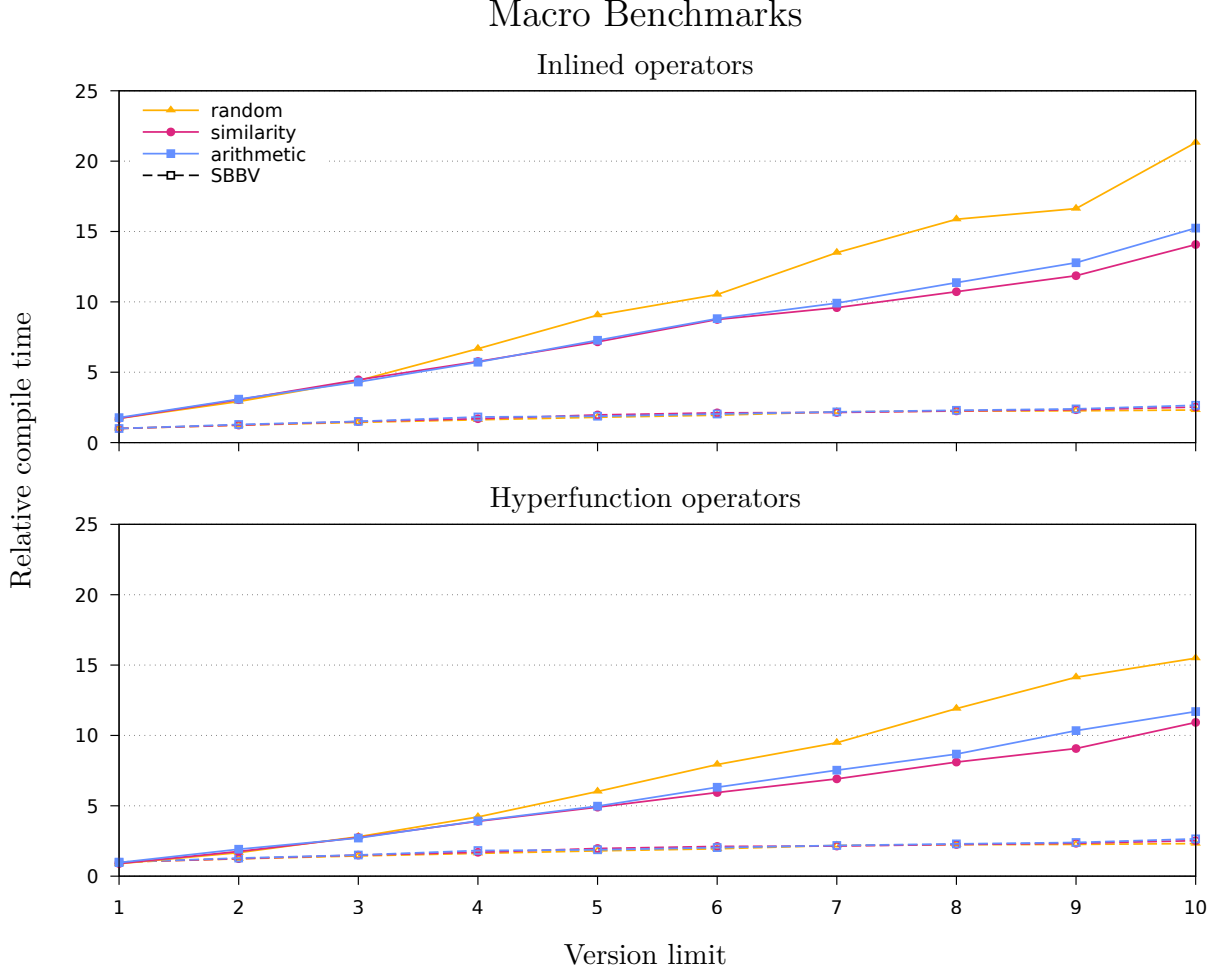


Figure 21 – Compile time of ΛV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom; SBBV included for reference despite using inlined operators), on macro benchmarks. The x-axis is the version limit. The y-axis is compile time relative to SBBV with a limit of one version (geometric mean). Lower is better.

operators for identical argument types multiple times. Operators can still be inlined *after* specialization, suggesting that ΛV enables function inlining that benefits from type analysis.

3.5.3. Compile Time

ΛV 's interprocedural context propagation eliminates significantly more run-time checks than SBBV. This, however, requires exploring more contexts and back-propagating a function's exit contexts to call sites so that specialized return points can be generated. This section evaluates the impact of this extra work on compile time. Here, *compile time* refers to the execution of the ΛV algorithm. Figure 20 shows aggregated ΛV compile time across micro benchmarks, both with inlined operators and with operators represented as hyperfunctions. Relative compile time is presented with increasing version limits, using SBBV compile

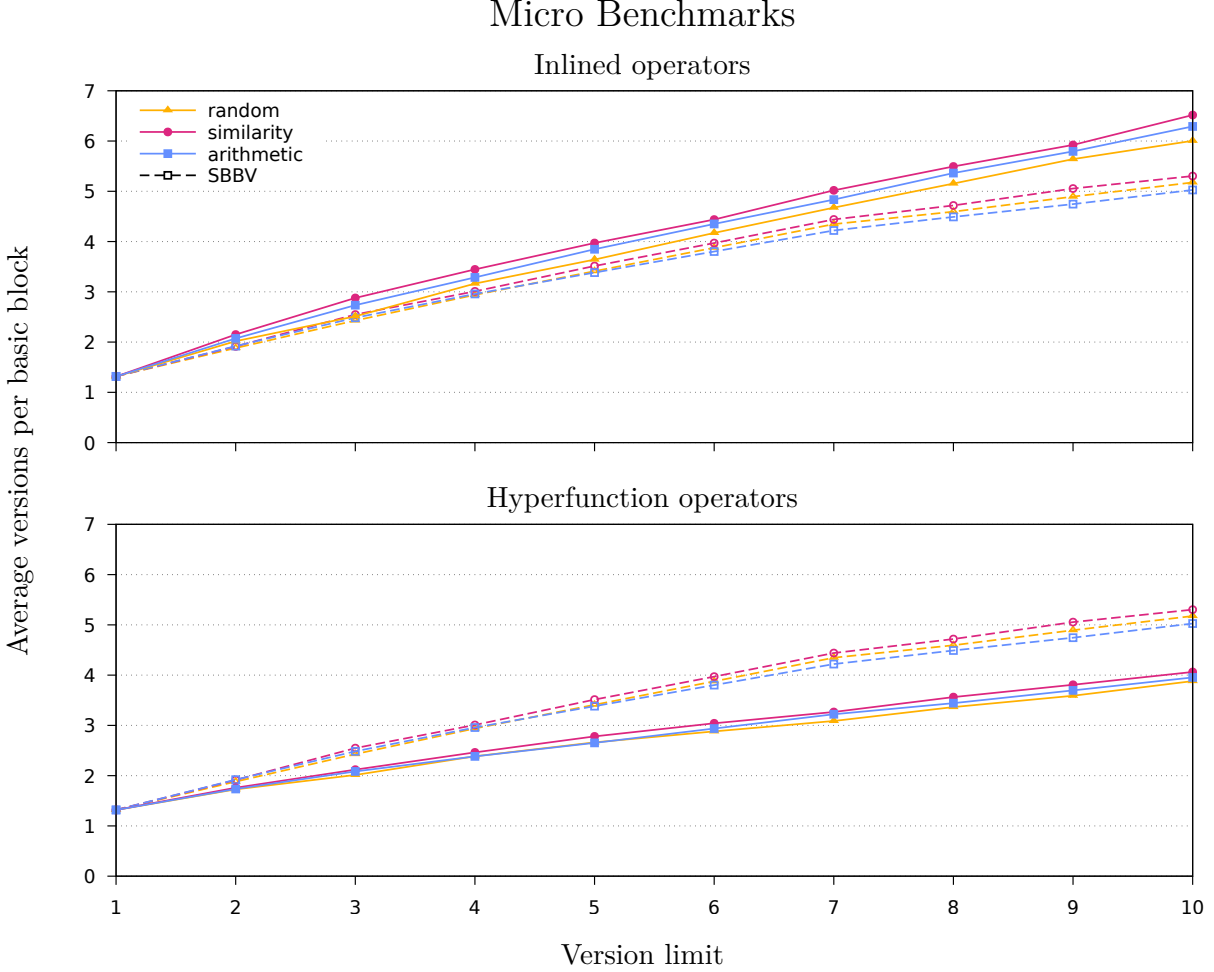


Figure 22 – Code duplication of AV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom; SBBV included for reference), on micro benchmarks. The x-axis is the version limit. The y-axis is the average number of versions per basic block in the source program (geometric mean). Lower is better.

time with a limit of one version as the baseline. Absolute compile time per benchmark can be found in Appendix C.

While the operator representation, inlined or as hyperfunctions, has little impact on type check elimination, it has a significant impact on compile time. In both cases, compile time increases roughly linearly with the version limit, but the increase is steeper with inlined operators: a slope of ~ 1.5 per version for inlined operators against ~ 1.1 for hyperfunctions, using the `arithmetic` and `similarity` merge heuristics. By contrast, this experiment shows an increase of ~ 0.35 per version with SBBV, consistent with the experiments from Chapter 2. Results for the `random` merge heuristic are also shown, displaying a steeper increase in compile time and demonstrating that the choice of merge heuristic also affects compile time. Figure 21 shows a similar pattern on macro benchmarks.

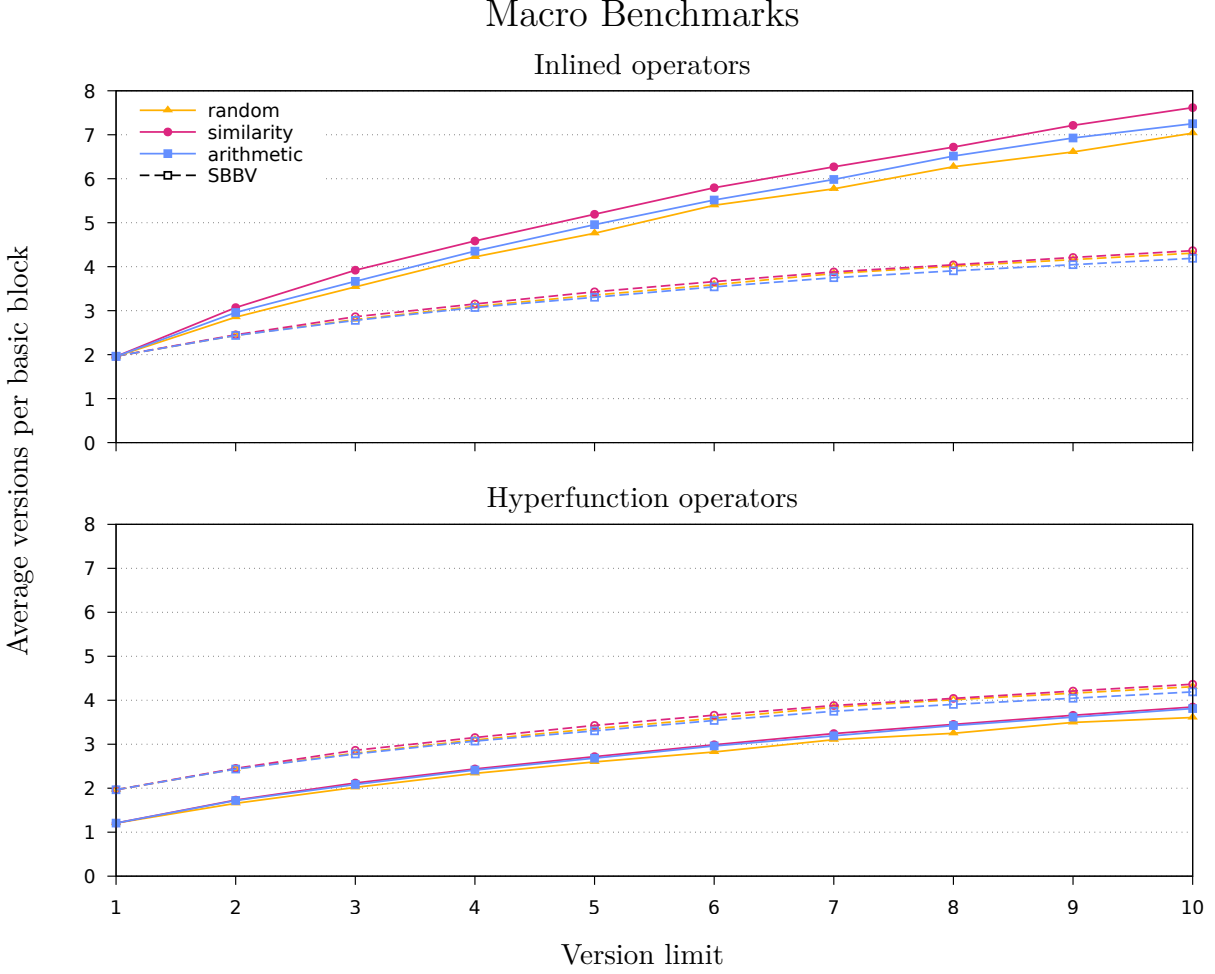


Figure 23 – Code duplication of AV (solid lines) and SBBV (dashed) with inlined operators (top) and hyperfunction operators (bottom; SBBV included for reference), on macro benchmarks. The x-axis is the version limit. The y-axis is the average number of versions per basic block in the source program (geometric mean). Lower is better.

This difference in compile time between operator representations is explained by hyperfunctions allowing the reuse of specializations across multiple call sites, whereas inlined operators must be respecialized, often for redundant call contexts, at each call site. This is confirmed by Figures 22 and 23, which show the number of versions in the specialized program per basic block in the source program before inlining. When operators are inlined, there is a double duplication cost: one from inlining itself and one from AV. Representing operators as hyperfunctions avoids this double cost and yields fewer versions. For very small programs, such as micro benchmarks where some operators are used only once, this effect is less pronounced at low version limits but becomes apparent as the limit increases. On macro benchmarks, however, the number of versions generated by AV with hyperfunction operators is always lower than with inlined operators, even at low version limits.

This reuse of operator specializations suggests a possible improvement to AV. Using function *outlining* [35], as opposed to inlining, it may be possible to further reduce compile time by extracting and representing similar code segments as a single hyperfunction, specializing them only once. The extracted segments could then be inlined back after AV, avoiding the cost of jumps across abstraction barriers while still accelerating AV. Exploring this optimization is left as future work.

3.6. Hyperfunctions Implementation Related Work

This section revisits past work on implementing multi-entry/multi-return subroutines, named *hyperfunctions* in this chapter, and on which a fair amount of research has taken place. The idea rarely gained significant traction, arguably because multiple return points require a graph-like representation, which complicates code maintainability and readability. It effectively reintroduces a form of jumps and labels that modern programming languages have largely moved away from. Lambda Versioning sidesteps this issue by providing a technique to generate useful hyperfunctions from single-entry/single-return source code.

3.6.1. Fortran 77

The idea of functions with multiple entry points and multiple return points dates back at least to Fortran 77, whose subroutines supported multiple entry points and *alternate returns* [11]. Below is an example of a comparison subroutine in Fortran 77 that uses alternate returns to distinguish between the three possible outcomes of a comparison.

```
1  CALL COMPARE(P, Q, *10, *20, *30)
2 10 WRITE(*,*) 'P < Q'
3     GOTO 99
4 20 WRITE(*,*) 'P = Q'
5     GOTO 99
6 30 WRITE(*,*) 'P > Q'
7 99 ...
8
9  SUBROUTINE COMPARE(A, B, *, *, *)
10 REAL A, B
11 IF (A .LT. B) RETURN 1
12 IF (A .EQ. B) RETURN 2
13 RETURN 3
14 END
```

However, this early implementation had shortcomings: it lacked a mechanism for passing return points to subsequent calls, precluding tail call optimization, and suffered from inefficiency as alternate returns were typically implemented as a switch at the return point. This feature has since been deprecated in later Fortran versions.

3.6.2. Multi-Return Function Calls

A typed lambda calculus supporting *multi-returns* has been developed in [56]. It allows functions to receive different return continuations, each potentially accepting values of different types. One of the motivating examples in that work is a “parsimonious filter” recursive function that allocates as few new pairs as possible [56, Figure 1]. This example can be translated into the hyperfunction below, illustrating the connection.⁷

```

1 (define parsimonious-filter
2   (hyperfunction
3     entries:
4       (any × any: A
5         K: (any × pair | null → pair | null
6            any × pair | null → arg[1]))
7     blocks:
8       (A: (lambda (K pred lst)
9            (if (null? lst)
10              (K[1] lst)
11              (B K pred lst))))
12      (B: (lambda (K pred lst)
13           (if (pred (car lst))
14             (C K pred lst)
15             (D K pred lst))))
16      (C: (lambda (K pred lst)
17           (call filter: any × any
18             any × pair | null → pair | null: (lambda (r) (E K lst r))
19             any × pair | null → arg[1]: K[1]
20             pred (cdr lst))))
21      (D: (lambda (K pred lst)
22           (call filter: any × any
23             any × pair | null → pair | null: K[0]
24             any × pair | null → arg[1]: K[0]
25             pred (cdr lst))))
26      (E: (lambda (K lst r) (K[0] (cons (car lst) r))))))

```

The authors also identify a crucial asymmetry between function calls and returns: calls are strict in the function, meaning information about *where* the call is going is required, while returns are strict in the returned value, meaning it is *what* is returned that matters. This is mirrored in ΛV , where the identity of a hyperfunction is required to invoke its entry points, but it is the type of the returned value, not the identity of the return point, that must be known to select a return continuation. They further note that multi-return introduces a hierarchy of tail calls, which has indeed been observed in Section 3.2:

- **semi-tail call**: some return points are fresh continuations, others are the caller’s.
- **tail call**: the caller’s return points are passed to the callee, possibly reordered.
- **super-tail call**: a subset of the caller’s return points are passed, shrinking the stack.

7. The $\text{any} \times \text{pair} | \text{null} \rightarrow \text{arg}[1]$ represents an exit context with an equivalence class equating the second argument ($\text{arg}[1]$) and the returned value, invoked when the tail of the list is left unchanged. Note that ΛV would not apply this optimization as it is not semantics-preserving.

3.6.3. Contextual Dispatch

The multi-entry nature of hyperfunctions is reminiscent of *contextual dispatch* [28], where multiple specialized versions of a function are generated. ΛV expands on this by making basic blocks, rather than functions, the atomic unit of duplication, and by introducing contextual returns. Relevant to this thesis is the dynamic dispatch mechanism described in [28]. In ΛV , calls to unknown functions fall back to a generic call operation that propagates no type information, as in SBBV. This is limiting, since a specialized entry point may well exist for a given calling context, yet its existence cannot be determined at compile-time. [28] addresses this with a run-time dispatch mechanism that extends the partial order of contexts to a total order, enabling selection of the most specific entry point for a given call site at run-time, with caching for efficiency. This was notably implemented in R, a highly dynamic language.

3.6.4. Extended Flat Closures

Extended flat closures [50] are a generalization of flat lexical closures that support multiple entry points. They extend the representation of a closure by replacing its entry point with a pointer to an entry point table, which stores indexed entry point addresses. Entry point tables are pre-allocated as permanent objects, and thus do not affect garbage collection and add no overhead to closure allocation. Multi-return addresses are implemented likewise, using return point tables: pre-allocated tables of indexed return addresses. On a call, rather than pushing multiple return addresses, a pointer to the return point table associated with the call site is pushed onto the stack. By allocating one return point table per call site, multi-return functions are implemented without memory overhead beyond these static tables, and the performance overhead is low, adding only a single indirection to call and return instructions. One downside, however, is that the fixed order of pre-allocated tables would obstruct the return-point reordering that tail-call optimization requires, were this representation used for the hyperfunctions generated by ΛV .

3.6.5. Call Site Splitting and Call Interface

The interprocedural analysis applied by ΛV can be seen as defining a specialized call interface for each hyperfunction, alongside a generic fallback. This is similar to the notion of *webs* defined in [49], a framework for interprocedural optimization that groups functions requiring similar call interfaces based on the program’s call graph. Under that framework, ΛV essentially monomorphizes call sites by duplicating entry points, referred to as *web splitting*, yielding specialized call interfaces for small groups of related entry points. [49] does not propose an implementation strategy, but highlights the important observation that

each entry point can be given its own specialized call interface, since invoking a specialized entry point requires knowing the identity of the called hyperfunction.

3.6.6. Second-Class Continuations and Contification

A frequent performance issue when compiling CPS code is that lambdas can represent both continuations and first-class functions, which are difficult for a compiler to distinguish. In this AV implementation, however, all lambdas in the source program have been converted to hyperfunctions, so lambdas in the resulting transformation can all be treated as second-class continuations. Importantly, this allows the latter to be compiled as jumps [34].

A notable remark, inspired by [34, 49], is that AV applies call site splitting in the process of specializing entry points, which may enable *contification* [29], a process by which (hyper)functions are turned into continuations. In other words, hyperfunction inlining can be applied *after* AV to benefit from its type analysis and call site splitting. This is a significant difference from SBBV, where inlining had to take place *before* specialization, potentially misallocating inlining resources or missing inlining opportunities altogether.

3.6.7. Hardware-Level Implementation

Modern hardware has co-evolved with programming languages in which single-entry/single-return subroutines are ubiquitous. In that paradigm, information is passed as values rather than encoded in control flow, and hardware optimizations have accordingly focused on value-passing, such as value prediction and branch prediction. It is worth considering whether specialized hardware support could benefit compilation paradigms such as AV, where control flow encodes rich contextual information. For instance, lowering CPS to hardware has been shown to reduce the run-time cost of exceptions when programs are transformed to *double-barrelled CPS* [58], where each function receives two continuations: one for returning and one for raising an exception. A separate but relevant line of research concerns domain-specific language (DSL) implementation and optimization on FPGAs [19]. Since the hyperfunction representation constitutes a form of DSL, such techniques may be applicable to efficient native hyperfunction compilation on FPGAs.

3.7. Limitations

This section highlights limitations of the work presented in this chapter.

The most significant limitation is the absence of a native hyperfunction implementation, which prevented an evaluation of AV’s effect on execution time. As it stands, AV is a program analysis technique that has yet to be elevated to a full compilation technique. The related

work surveyed in Section 3.6 suggests no fundamental obstacle to implementing efficient hyperfunctions, but this remains to be demonstrated empirically.

A further limitation is that the current AV algorithm requires the identity of a hyperfunction to be known exactly before any of its entry points can be invoked. This is partially mitigated by the fact that the algorithm naturally duplicates call sites, frequently producing monomorphic ones, though this mitigation is bounded by the version limit. A possible direction is to relax this requirement by allowing call sites to be specialized for a set of hyperfunctions, subject to a compatibility constraint on their entry points. This still raises the issue of specialization under separate compilation. Since specialized entry points are labeled with their expected call context, calls to specialized entry points of dynamically loaded functions remain possible, but would require run-time contextual dispatch to an existing entry point of the function. A logical next step beyond the intraprocedural and interprocedural techniques presented in this thesis would therefore be a mechanism for cross-module AV.

Additionally, the present AV implementation is restricted to type analysis. There is, however, no fundamental reason to limit context specialization to variable types. Other program properties worth tracking include value intervals, global variables, structure contents, and interprocedural common subexpressions. Furthermore, since AV precisely tracks types across abstraction barriers, it opens the door to passing unboxed values across them.

As discussed in Section 3.4, the merge heuristic used in the evaluation of AV is deliberately simple. It is likely overly specific to arithmetic computations, making it ill-suited for other kinds of programs and possibly other programming languages. The space of merge heuristics remains largely unexplored, and a more thorough investigation is likely to improve the effectiveness of AV and related BBV techniques.

3.8. Conclusion

This chapter introduced Lambda Versioning, an interprocedural extension of SBBV. AV explores the whole-program optimization made possible by Basic Block Versioning when the calling scheme permits call sites to pass multiple return points to a callee. Just as SBBV enhanced abstract interpretation with code duplication to account for the wide variety of operator behavior in dynamic languages, AV overcomes abstraction barriers by duplicating and specializing the return points of functions according to their program-wide usage.

In practice, AV was shown to eliminate more run-time checks than SBBV, since return point specialization propagates to the caller not only the returned type but also argument types discovered within the callee’s body. The most striking examples are recursive functions such as `fact` and `fib`, which can be optimized to check the type of their argument only

once, then switch to a specialized entry point for recursive calls that performs no further type checks. On larger programs, AV also eliminated more checks than SBBV alone.

Another major improvement presented in this chapter is the **arithmetic** merge heuristic, which addresses the shortcomings of the less performant **similarity** heuristic from Chapter 2. This simple heuristic, whose implementation fits within a single page of Appendix A, demonstrates that AV can reduce whole-program optimization to a sequence of local choices (merges). Only a handful of heuristics have been studied, however, and the space of possible heuristics likely holds more performant and more general candidates.

This concludes the contributions of this thesis. The remaining chapters discuss future work and offer closing remarks.

Chapter 4

Future Work

This chapter presents potential research avenues for AOT Basic Block Versioning that were not developed sufficiently to be included in the body of this thesis. Since Lambda Versioning currently treats data structures, and closures in particular, as opaque, extending it to reason about the mutability and escapability of data structures could enable more powerful type analysis, immediate representations for known-type values, whole-program analysis of highly dynamic languages such as Python and JavaScript, and, more speculatively, the generation of optimizing compilers.

4.1. Data Structures, Mutability, and Escapability

AV treats data structures as completely opaque. Every value extracted from a pair, vector, box, closure, or mutable global variable has its type initialized to **any**. This ought to be solved by permitting fine-grained data structure types that include information about their contents. For instance, consider the following program.

```
1 (define (adder x) (lambda (y) (+ x y)))
2
3 (let ((incr (adder (read))))
4   (incr 0)
5   (incr 1))
```

The function `incr` is a closure over an initially unknown value `x`. After the first call to `incr`, however, the type of `x` is tested, and a specialized return point that narrows the type of `incr`'s free variable can be generated. In the specialized code below, this is shown by the type of `incr`, initially $\lambda 1[x: \text{any}]$, being narrowed to $\lambda 1[x: \text{fx}]$. The second call to `incr` can then jump to an entry point specialized for this known type, here denoted by the context $\text{fx} \times [x: \text{fx}]$, which indicates `fx` first argument and `fx` free variable `x`.

```

1 (define λ1
2   (hyperfunction
3     closure: (x)
4     entries:
5       (fx × [x: any]: A1
6         K: (fx × [x: fx] → fx   fx × [x: fx | fl | bg] → fx | fl | bg)
7         fx × [x: fx]: A2
8         K: (fx × [x: fx] → fx   fx × [x: fx] → fx | bg)
9     blocks:
10      A1: (lambda (K y x) ;; y: fx / x: any
11            (call +.fx × any
12              fx × fx → fx: K[0]
13              fx × fx | fl | bg → fx | fl | bg: K[1]))
14      A2: (lambda (K y x) ;; y: fx / x: fx
15            (call +.fx × any
16              fx × fx → fx: K[0]
17              fx × fx → fx | bg: K[1])))))
18
19 (define adder
20   (hyperfunction
21     entries: (any: A   K: (any → λ1[x: any]))
22     blocks:
23       A: (lambda (K x) (K[0] (instantiate λ1 x)))) ;; x: any
24
25 (define top-level
26   (hyperfunction
27     entries: (∅: A   K: (∅ → ∅))
28     blocks:
29       A: (lambda () ;; ∅
30            (call adder.any
31              any → λ1[x: any]: (lambda (incr) (B incr))
32              (read)))
33       B: (lambda (incr) ;; incr: λ1[x: any]
34            (call incr.fx × [x: any]
35              fx × [x: fx] → fx: (lambda (r) (C1 incr))
36              fx × [x: fx | fl | bg] → fx | fl | bg: (lambda (r) (C2 incr))
37              0))
38       C1: (lambda (incr) ;; incr: λ1[x: fx]
39            (call incr.fx × [x: fx]
40              fx × [x: fx] → fx: (lambda (r) (D1 incr))
41              fx × [x: fx] → fx | bg: (lambda (r) (D2 incr))
42              1))
43       C2: (lambda (incr) ;; incr: λ1[x: fx | fl | bg]
44            (call incr.fx × [x: any]
45              fx × [x: fx] → fx: (lambda (r) (exit))
46              fx × [x: fx | fl | bg] → fx | fl | bg: (lambda (r) (exit))
47              1))))

```

This last example is simple because the variable `x` is immutable. One obstacle to analyzing the shape and type of values within mutable structures is that the compiler must ensure that these structures do not escape the scope of analysis; otherwise, it cannot guarantee that no other part of the program mutates them. The “parsimonious filter” from [56, Figure 1] is an example of a multi-return function that informs the caller about the properties of a structure: a specialized return point indicates that the returned list is the one given as argument. This idea can be extended to provide guarantees about the mutability and escapability of structures. It is achieved by attaching information to variables in contexts, indicating that

an object has not been mutated. In particular, this information can be propagated at return points to guarantee to the caller that arguments were not mutated. As an example, consider the following program.

```

1 (define (map f lst)
2   (if (null? lst)
3       '()
4       (cons (f (car lst)) (map f (cdr lst)))))
5
6 (map (lambda (x) (+ x 1)) '(0 1 2))

```

It should be possible to statically deduce that the call to `map` returns a proper list of fixnums. This is complicated, however, by the fact that lists are mutable and that `map` loses track of control flow when calling `f` (typed as $\lambda 1$). For all the compiler knows, `f` could hold a reference to the list and mutate it. A workaround is to create a specialized entry point where `lst` is marked as $\neg\text{escaped}$, indicating that `map` can consider itself the sole owner of `lst`. This permits a specialized return point in which `map` guarantees to its caller that the list passed as argument was not mutated, denoted by $\neg\text{mutated}$, and that the caller is the sole owner of the returned list, which is a proper list of fixnums.

```

1 (define map
2   (hyperfunction
3     entries:
4       ( $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped})$ ): A
5         K: ( $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx] \cap \neg\text{escaped}$ 
6            $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx|bg] \cap \neg\text{escaped}$ )
7     blocks:
8       A: (lambda (K f lst) ;; f:  $\lambda 1$  / lst:  $\text{list}[fx] \cap \neg\text{escaped}$ 
9         (if (null? lst)
10            (K[0] lst)
11            (B K f lst)))
12       B: (lambda (K f lst) ;; f:  $\lambda 1$  / lst:  $\text{list}[fx] \cap \neg\text{escaped}$ 
13         (call f.fx
14              fx  $\rightarrow$  fx: (lambda (head) (C1 K head f lst))
15              fx  $\rightarrow$  bg: (lambda (head) (C2 K head f lst))
16              (car lst)))
17       C1: (lambda (K head f lst) ;; head: fx / f:  $\lambda 1$  / lst:  $\text{list}[fx] \cap \neg\text{escaped}$ 
18         (call map. $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped})$ 
19               $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx]$ :
20                (lambda (tail) (D1 K tail head))
21               $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx|bg]$ :
22                (lambda (tail) (D2 K tail head))
23              f (cdr lst)))
24       C2: (lambda (K head f lst) ;; head: bg / f:  $\lambda 1$  / lst:  $\text{list}[fx] \cap \neg\text{escaped}$ 
25         (call map. $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped})$ 
26               $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx] \cap \neg\text{escaped}$ :
27                (lambda (tail) (D2 K tail head))
28               $\lambda 1 \times (\text{list}[fx] \cap \neg\text{escaped} \neg\text{mutated}) \rightarrow \text{list}[fx|bg] \cap \neg\text{escaped}$ :
29                (lambda (tail) (D2 K tail head))
30              f (cdr lst)))
31       D1: (lambda (K tail head) ;; tail:  $\text{list}[fx] \cap \neg\text{escaped}$  / head: fx
32         (K[0] (cons head tail)))
33       D2: (lambda (K tail head) ;; tail:  $\text{list}[fx|bg] \cap \neg\text{escaped}$  / head: fx|bg
34         (K[1] (cons head tail))))

```

Including the `escaped` and `mutated` properties in the abstract interpretation of ΔV is not particularly complex, since they both form simple lattices (the bottom layer consisting of the states `escaped/mutated` and `¬escaped/¬mutated`, and the top layer being the union of these, i.e., unknown escapability/mutability). The main challenge is to define a merge heuristic that selects the versions whose combinations of properties should be preserved, a task that can be expected to become increasingly difficult as the number of tracked properties increases. Another challenge is that the inclusion of data structure shapes, coupled with equivalence classes, transforms contexts from variable-type mappings into approximations of reference graphs, further complicating merging.

4.2. Context Memoization and Constant Folding

One possible solution to the increasing pressure on the merge heuristic that arises as more properties are tracked is a still embryonic technique called *context memoization*. It consists in pausing whole-program ΔV upon reaching a call site to analyze the target of that call site specifically, possibly with a higher version limit, in order to identify precise return contexts. The union of these precise return contexts is then memoized by the call site, and ΔV resumes normally. The precise versions generated for this entry point are likely to be merged later, but the return context can then always be narrowed back using the memoized context. An interesting consequence of this technique is the emergence of interprocedural constant folding at some call sites. As an example, consider the following program.

```
1 (define (polynomial x) (+ (* x x) (* 3 x)))
2 (display (polynomial 5))
```

Allowing ΔV to specialize the `polynomial` call site for the exact argument 5 instead of `fx` yields the following hyperfunction, with a precise return point identifying that this call always returns exactly 40. Furthermore, the property `¬side-effect` indicates that the call has no side effects, such as mutations, I/O, or potential exceptions.

```
1 (define polynomial
2   (hyperfunction
3     entries: (5: A K(5→40 ∧ ¬side-effect))
4     blocks:
5       A: (lambda (K x) ;; x: 5
6           (call *.5 × 5
7               5 × 5→25 ∧ ¬side-effect: (lambda (r1) (B K r1 x))
8               x x))
9       B: (lambda (K r1 x) ;; x: 25 / r1: 25
10          (call *.3 × 5
11              3 × 5→15 ∧ ¬side-effect: (lambda (r2) (C K r2 r1 x))
12              3 x))
13       C: (lambda (K r2 r1) ;; r1: 25 / r2: 15
14          (call +.25 × 15
15              25 × 15→40 ∧ ¬side-effect: K[0]
16              r1 r2))))
```

On larger programs, the versions generated for the blocks of `polynomial` are likely to be merged away as AV progresses. In this intermediate state, however, the return point `5→40∩¬side-effect` indicates that the call site to `polynomial` can safely be replaced by the value `40`, eliminating the call altogether. This information remains valid regardless of any merges later applied within `polynomial`; the overly specific versions generated by this analysis can therefore be discarded, which frees the versions used to apply this optimization, effectively resulting in inlining and constant folding at the call site.

4.3. Untagged Immediate Values

Another application of escape analysis by AV is the possibility of using untagged values. In dynamic languages, values must carry typing information, often in the form of tag bits. If a value has a known type and is guaranteed not to escape, this tag can be eliminated altogether, allowing direct machine operations on values without having to strip typing information beforehand. Furthermore, information about the types and shapes of structures could enable allocation removal, replacing heap-allocated values with immediate, stack-allocated values, which yields significant performance improvements in dynamic languages [9].

One expected obstacle to tag and allocation removal, however, is the interaction with the garbage collector, which requires information about the type and shape of values. Fortunately, while this information would no longer appear on the values themselves, it could be recovered by allowing the garbage collector to access the context of each version at run-time, associating instruction pointers and return points with their corresponding context [4, 32]. This could also enable a compact memory representation of stack frames, where values are assigned both a source type and a memory type that dictates how they are encoded in different contexts [7]. Another way to frame this is that code duplication generates regions of the code where values are statically typed. In these regions, type information can be encoded in the program rather than replicated on each value, allowing the use of memory management techniques from static languages.

4.4. Application to Highly Dynamic Languages

The previous sections present techniques expected to be on the critical path to applying AV to highly dynamic languages such as JavaScript or Python. Interprocedural analysis is necessary, since the possibility of meta-dynamism in these languages otherwise forces any analysis to drop almost all guarantees on a call, but it is not sufficient. The semantics of these languages revolve in part around the state of mutable objects that dictate the behavior of built-in functions and operators, such as the `builtins` module in Python or mutable prototypes in JavaScript. Previous work demonstrated that an AOT Python compiler can

attain peak performance by aggressively speculating that these are not mutated, keeping slow paths as backups. For instance, the following Python function can be optimized under the assumption that the `range` built-in class is not mutated and that `s` remains a small integer, yielding nearly C-like performance [41].

```
1 def sum_to_n(n):
2     s = 0
3     for i in range(n + 1):
4         s += i
5     return s
```

Unless speculation is exactly correct, however, execution falls back on a slow, suboptimal path. Furthermore, checking that assumptions are valid at run-time requires evaluating guards. These must be used parsimoniously to avoid trading the cost of type checks for the cost of guards, which so far has been done only by hand-optimized operators, and only locally. One solution is to add such guards more aggressively, disregarding their potential run-time cost, and to apply AV to eliminate the redundant ones. Another, more general solution is to apply AV with mutability and escapability analysis to generate optimized versions in which the state of some global objects, such as Python’s `builtins` module, is known, enabling whole-program analysis, and thus highly optimized code generation, for highly dynamic languages.

4.5. Float Self-Tagging

Floating-point numbers are typically encoded using the IEEE 754 standard, which requires the full 64 bits of a machine word. This leaves no room for a type tag, so flonums in dynamic languages are often represented as tagged pointers to heap-allocated floats. This heap allocation is inefficient: every operation on a flonum requires extra memory reads and writes, and creates many short-lived intermediate objects that strain the garbage collector.

Developed in parallel with this thesis, *float self-tagging* is an object tagging scheme that uses an invertible bitwise transformation to map some flonums to immediate 64-bit floating-point numbers, requiring no heap allocation. It exploits the fact that some floats are far more likely to occur in practice, such as zeros and numbers whose magnitude is centered around 1.0. By using a different encoding to represent this subset, bits can be freed for a type tag. Float self-tagging was published at the *International Conference on Object-Oriented Programming Systems, Languages, and Applications 2025* (OOPSLA’25) [43]; the full paper appears in Appendix E.

One relevant consequence of float self-tagging is that it distinguishes immediate from allocated floats, both subsets of the flonum type. Operations on flonums must therefore check not only the source type of values but also their memory type. This is a downside:

when strain on the garbage collector is low, the extra bit manipulation these checks require can make the encoding slightly slower than efficiently allocated floats, for instance with bump allocation. BBV may offer a solution to this issue. By rewriting arithmetic operators to distinguish flonums from immediate flonums, as in the code below, they can be specialized for immediate floating-point arithmetic.

```

1 (define (* a b)
2   (cond
3     ((and (fixnum? a) (fixnum? b))
4      (or (fx*? a b) (##* a b)))
5     ((and (iflonum? a) (iflonum? b))
6      (or (ifl*? a b) (fl* a b)))
7     (else
8      (##* a b))))

```

The `*` Scheme operator is rewritten to check whether two values are immediate flonums (`iflonum?` checks), in which case they are multiplied with `ifl*?`, which returns either an immediate flonum or `#f` if the operation *overflows* outside the immediate flonum range. This is analogous to small integers (fixnums) and big integers (bignums), which BBV distinguishes to generate fixnum-specialized versions. The same technique should remove the overhead of float self-tagging.

4.6. Compiler Generation

Once an optimizer is powerful enough to specialize a program with respect to part of its input, it acquires an application first identified by Futamura [30]: specializing an interpreter with respect to a fixed source program eliminates the overhead of instruction dispatch and operand decoding, yielding residual code equivalent to having compiled that program. A sufficiently powerful optimizer can even apply optimizations to the interpreter that carry through to the semantics of the source program. Carried to its limit, this specialization process yields not just compiled code but an optimizing compiler. Whole-program optimization is thus not merely a means of making a given program faster, but also a mechanism for generating optimizing compilers. Since interpreters are notoriously simpler to implement than optimizing compilers, this idea has been applied in several lines of research to generate optimizing JIT [10, 62] and AOT compilers [3, 26, 37, 48].

Lambda Versioning could be extended for such compiler generation. For instance, consider the following example of an interpreter for a rudimentary instruction set. The example is written so that the instruction pointer (`ip`) is live at every point in the code. Hence, by fixing the input `code` and modifying the main loop of the ΛV algorithm, ΛV can specialize per instruction in the source program rather than per block in the interpreter's code.

```

1 (define (interpret code ip stack)
2   (match (vector-ref code ip)
3     ((jumpif ,iftrue) (interpret-jumpif iftrue code ip stack))
4     ((push ,x) (interpret-push x code ip stack))
5     ((dup) (interpret-dup code ip stack))
6     ...))
7
8 (define (interpret-jumpif iftrue code ip stack)
9   (if (car stack)
10      (interpret code iftrue (cdr stack))
11      (interpret code (+ ip 1) (cdr stack))))
12
13 (define (interpret-push x code ip stack)
14   (interpret code (+ ip 1) (cons x stack)))
15
16 (define (interpret-dup code ip stack)
17   (interpret code (+ ip 1) (cons (car stack) stack)))
18
19 (let ((ip 0))
20   (interpret code ip '()))

```

Algorithm 3 presents this modified version of ΛV 's main loop for specializing such an interpreter. The duplication limit is applied across versions whose contexts share both the same block and instruction pointer, effectively imposing the limit per instruction of the source program rather than per block of the interpreter's code. The segment highlighted in green is the change applied to the original algorithm. Since only versions sharing the same instruction pointer are merged, the instruction executed by a given version is never merged away and always statically known, eliminating the overhead of the instruction dispatch loop. In its current state, however, ΛV cannot achieve much more than that. For instance, opaque structures currently prevent tracking values on the stack, which precludes eliminating type checks at the source-program level. Nonetheless, with improvements to ΛV such as those discussed in this chapter, this research avenue is worth pursuing.

Input: (<i>entry</i> : block, <i>limit</i> : integer)	Output: —
<pre> 1 define lambdaVersioning (<i>entry</i>, <i>limit</i>): 2 <i>entryVersion</i> $\leftarrow$ getOrCreateVersion (<i>entry</i>, $\top$) 3 markEntry (<i>entryVersion</i>) 4 while <i>version</i> $\leftarrow$ <i>queue.dequeue</i>() : 5 if <i>version</i> is not reachable or <i>version</i> is done : continue 6 <i>instructionPointer</i> $\leftarrow$ <i>version.context</i>["ip"] 7 <i>versions</i> $\leftarrow$ {<i>v</i> $\in$ <i>version.block.reachableVersions</i> : <i>v.context</i>["ip"] = <i>instructionPointer</i>} 8 if <i>versions</i> > <i>limit</i> : mergeSome (<i>versions</i>) 9 if <i>version</i> is reachable : specialize (<i>version</i>) </pre>	

Algorithm 3: Modified version of ΛV 's main loop for specializing an interpreter.

Chapter 5

Conclusion

This thesis has been an exploration of ahead-of-time Basic Block Versioning (BBV), a novel analysis and optimization technique for ahead-of-time compilers. It builds upon the theory of abstract interpretation, a framework for finding a sound, and ideally precise, approximation of a program’s semantics. This approximation can then be used by a compiler for efficient code generation. Unfortunately, abstract interpretation frequently falls short when analyzing dynamic languages. In many cases, this is not a failure of abstract interpretation, but rather a consequence of dynamic languages, whose semantics range over a wide set of possible behaviors depending on the exact inputs, for instance in the presence of polymorphism. In other words, the approximation found by abstract interpretation on dynamic languages is broad because the range of possible behaviors of such programs is broad.

AOT BBV addresses this issue by leveraging code duplication to generate specialized paths that optimize for likely run-time execution contexts while still covering all possible run-time behaviors. This is achieved by applying abstract interpretation while eagerly duplicating basic blocks whenever a new execution context is uncovered, thereby delaying the union and widening operations usual to abstract interpretation. Only when a duplication limit is reached are some basic blocks selected for merging, allowing precise abstract interpretation while curbing code bloat.

Chapter 2 describes the first instance of AOT BBV: Static Basic Block Versioning (SBBV). It provides a complete description of the SBBV algorithm, along with empirical experiments demonstrating the efficacy of the technique at eliminating dynamic checks such as type checks, overflow checks, and bounds checks. In these experiments, SBBV is shown to accelerate the execution time of optimized programs by about 10%.

Chapter 3 presents Lambda Versioning (LV), an interprocedural extension of SBBV. The main limitation of SBBV is its inability to propagate information across abstraction barriers during abstract interpretation. Again, this is not a failure of SBBV itself, but rather an

inevitable consequence of dynamic languages, whose polymorphic functions must be given a single return point that accounts for all possible return types. To solve this issue, this chapter revisits multi-entry/multi-return functions, called *hyperfunctions* within this thesis, and shows how AV can transform a program using single-entry/single-return functions into one using hyperfunctions. Leveraging multiple returns, AV propagates contextual information across call sites and return points to generate highly specialized code. This often results in programs where the type of an input is tested only once, after which execution defers to a specialized branch that requires no further type checks.

Finally, Chapter 4 presents future work, with the goal of demonstrating that AOT BBV can be used for a wide range of specializations. For instance, the main limitation of AV, in its current state, is that data structures, and closures in particular, are treated as completely opaque. Yet it is shown that, in some cases, AV could apply an interprocedural escape and mutability analysis that would uncover type information about the contents of these structures. This chapter suggests applications that could result from such improved analysis: more powerful type analysis, immediate representations of some structures, whole-program analysis of highly dynamic languages, and generation of optimizing compilers.

Over the last decades, highly optimizing compilers have been built on the premise that efficient compilation of dynamic languages requires information that can be uncovered only at run-time, leading to increasingly complex, multi-tiered, just-in-time compilers that no single programmer could hope to construct within a lifetime. With ahead-of-time Basic Block Versioning, this thesis demonstrates that this premise is worth reconsidering, and suggests that simpler AOT compilation frameworks may serve as the backbone of compilers for dynamic languages, both existing and future.

References

- [1] 2026. R7RS Benchmarks. <https://github.com/ecraven/r7rs-benchmarks>. (2026).
- [2] Muteb Alshammari and Abdelmounaam Rezgui. 2020. A single-source shortest path algorithm for dynamic graphs. *AKCE International Journal of Graphs and Combinatorics* 17, 3 (2020), 1063–1068. <https://doi.org/10.1016/j.akcej.2020.01.002>
- [3] Lars Ole Andersen. 1992. Partial evaluation of C and automatic compiler generation. In *Compiler Construction*, Uwe Kastens and Peter Pfahler (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 251–257.
- [4] Andrew W. Appel. 1989. Runtime Tags Aren’t Necessary. *LISP Symb. Comput.* 2, 2 (1989), 153–162.
- [5] Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. 2000. Dynamo: a transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI ’00)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/349299.349303>
- [6] Edd Barrett, Carl Friedrich Bolz-Tereick, Rebecca Killick, Sarah Mount, and Laurence Tratt. 2017. Virtual machine warmup blows hot and cold. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 52 (Oct. 2017), 27 pages. <https://doi.org/10.1145/3133876>
- [7] Thaïs Baudon, Gabriel Radanne, and Laure Gonnord. 2023. Bit-Stealing Made Legal: Compilation for Custom Memory Representations of Algebraic Data Types. *Proc. ACM Program. Lang.* 7, ICFP, Article 216 (Aug. 2023), 34 pages. <https://doi.org/10.1145/3607858>
- [8] Felix Berlakovich and Stefan Brunthaler. 2024. Cross Module Quickening - The Curious Case of C Extensions. In *38th European Conference on Object-Oriented Programming, ECOOP 2024, Vienna, Austria, September 16-20, 2024 (LIPICs)*, Jonathan Aldrich and Guido Salvaneschi (Eds.), Vol. 313. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 6:1–6:29. <https://doi.org/10.4230/LIPICS.ECOOP.2024.6>
- [9] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, Michael Leuschel, Samuele Pedroni, and Armin Rigo. 2011. Allocation removal by partial evaluation in a tracing

- JIT. In *Proceedings of the 20th ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation (PEPM '11)*. Association for Computing Machinery, New York, NY, USA, 43–52. <https://doi.org/10.1145/1929501.1929508>
- [10] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, and Armin Rigo. 2009. Tracing the meta-level: PyPy’s tracing JIT compiler. In *Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS '09)*. Association for Computing Machinery, New York, NY, USA, 18–25. <https://doi.org/10.1145/1565824.1565827>
- [11] Walt Brainerd. 1978. Fortran 77. *Commun. ACM* 21, 10 (Oct. 1978), 806–820. <https://doi.org/10.1145/359619.359621>
- [12] Craig Chambers. 1992. *The Design and Implementation of the SELF Compiler, an Optimizing Compiler for Object-Oriented Programming Languages*. Ph.D. Dissertation. Stanford University, Department of Computer Science. <https://searchworks.stanford.edu/view/2354991>
- [13] Maxime Chevalier-Boisvert. 2015. *On the Fly Type Specialization without Type Analysis*. Ph.D. Dissertation. Université de Montréal. <https://doi.org/10.71781/10173>
- [14] Maxime Chevalier-Boisvert and Marc Feeley. 2015. Simple and Effective Type Check Removal through Lazy Basic Block Versioning. In *29th European Conference on Object-Oriented Programming, ECOOP 2015, Prague, Czech Republic, July 5-10, 2015 (LIPIcs)*, John Tang Boyland (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 101–123. <https://doi.org/10.4230/LIPICS.ECOOP.2015.101>
- [15] Maxime Chevalier-Boisvert and Marc Feeley. 2016. Interprocedural Type Specialization of JavaScript Programs Without Type Analysis. In *30th European Conference on Object-Oriented Programming, ECOOP 2016, Rome, Italy, July 18-22, 2016 (LIPIcs)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 7:1–7:24. <https://doi.org/10.4230/LIPICS.ECOOP.2016.7>
- [16] Maxime Chevalier-Boisvert, Noah Gibbs, Jean Boussier, Si Xing (Alan) Wu, Aaron Patterson, Kevin Newton, and John Hawthorn. 2021. YJIT: a basic block versioning JIT compiler for CRuby. In *VMIL 2021: Proceedings of the 13th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, Virtual Event / Chicago, IL, USA, 19 October 2021*, Gregor Richards and Manuel Rigger (Eds.). ACM, 25–32. <https://doi.org/10.1145/3486606.3486781>
- [17] Maxime Chevalier-Boisvert, Takashi Kokubun, Noah Gibbs, Si Xing (Alan) Wu, Aaron Patterson, and Jemma Issroff. 2023. Evaluating YJIT’s Performance in a Production Context: A Pragmatic Approach. In *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR*

- 2023). Association for Computing Machinery, New York, NY, USA, 20–33. <https://doi.org/10.1145/3617651.3622982>
- [18] Jiho Choi, Thomas Shull, and Josep Torrellas. 2019. Reusable inline caching for JavaScript performance. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, Kathryn S. McKinley and Kathleen Fisher (Eds.). ACM, 889–901. <https://doi.org/10.1145/3314221.3314587>
- [19] Jason Cong, Jason Lau, Gai Liu, Stephen Neuendorffer, Peichen Pan, Kees Vissers, and Zhiru Zhang. 2022. FPGA HLS Today: Successes, Challenges, and Opportunities. *ACM Trans. Reconfigurable Technol. Syst.* 15, 4, Article 51 (Aug. 2022), 42 pages. <https://doi.org/10.1145/3530775>
- [20] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*, Robert M. Graham, Michael A. Harrison, and Ravi Sethi (Eds.). ACM, 238–252. <https://doi.org/10.1145/512950.512973>
- [21] Christopher Edward Cummins. 2020. *Deep learning for compilers*. Ph.D. Dissertation. University of Edinburgh, UK. <https://doi.org/10.7488/ERA/168> See Table 1.1: Development history, logical lines of code, and estimated cost of 10 popular open-source compiler projects.
- [22] Jack W. Davidson and Sanjay Jinturkar. 1996. Aggressive Loop Unrolling in a Retargetable Optimizing Compiler. In *Compiler Construction, 6th International Conference, CC’96, Linköping, Sweden, April 24-26, 1996, Proceedings (Lecture Notes in Computer Science)*, Tibor Gyimóthy (Ed.). Springer, 59–73. https://doi.org/10.1007/3-540-61053-7_53
- [23] L. Peter Deutsch and Allan M. Schiffman. 1984. Efficient Implementation of the Smalltalk-80 System. In *Conference Record of the Eleventh Annual ACM Symposium on Principles of Programming Languages, Salt Lake City, Utah, USA, January 1984*, Ken Kennedy, Mary S. Van Deusen, and Larry Landweber (Eds.). ACM Press, 297–302. <https://doi.org/10.1145/800017.800542>
- [24] Danny Dubé and Marc Feeley. 2002. A demand-driven adaptive type analysis. In *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP ’02), Pittsburgh, Pennsylvania, USA, October 4-6, 2002*, Mitchell Wand and Simon L. Peyton Jones (Eds.). ACM, 84–97. <https://doi.org/10.1145/581478.581487>

- [25] Shimon Even and Yossi Shiloach. 1981. An On-Line Edge-Deletion Problem. *J. ACM* 28, 1 (1981), 1–4. <https://doi.org/10.1145/322234.322235>
- [26] Chris Fallin and Maxwell Bernstein. 2025. Partial Evaluation, Whole-Program Compilation. *Proc. ACM Program. Lang.* 9, PLDI (2025), 324–347. <https://doi.org/10.1145/3729259>
- [27] Marc Feeley and contributors. 2026. Gambit. <https://gambitscheme.org>. (2026).
- [28] Olivier Flückiger, Guido Chari, Ming-Ho Yee, Jan Ječmen, Jakob Hain, and Jan Vitek. 2020. Contextual dispatch for function specialization. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 220 (Nov. 2020), 24 pages. <https://doi.org/10.1145/3428288>
- [29] Matthew Fluet and Stephen Weeks. 2001. Contification using dominators. *SIGPLAN Not.* 36, 10 (Oct. 2001), 2–13. <https://doi.org/10.1145/507669.507639>
- [30] Yoshihiko Futamura. 1999. Partial Evaluation of Computation Process - An Approach to a Compiler-Compiler. *High. Order Symb. Comput.* 12, 4 (1999), 381–391. <https://doi.org/10.1023/A:1010095604496>
- [31] Kimball Germane, Jay McCarthy, Michael D. Adams, and Matthew Might. 2019. Demand Control-Flow Analysis. In *Verification, Model Checking, and Abstract Interpretation - 20th International Conference, VMCAI 2019, Cascais, Portugal, January 13-15, 2019, Proceedings (Lecture Notes in Computer Science)*, Constantin Enea and Ruzica Piskac (Eds.). Springer, 226–246. https://doi.org/10.1007/978-3-030-11245-5_11
- [32] Benjamin Goldberg and Michael Gloger. 1992. Polymorphic Type Reconstruction for Garbage Collection Without Tags. In *Proceedings of the Conference on Lisp and Functional Programming, LFP 1992, San Francisco, California, USA, 22-24 June 1992*, Jon L. White (Ed.). ACM, 53–65. <https://doi.org/10.1145/141471.141504>
- [33] Urs Hölzle, Craig Chambers, and David M. Ungar. 1991. Optimizing Dynamically-Typed Object-Oriented Languages With Polymorphic Inline Caches. In *ECOOP’91 European Conference on Object-Oriented Programming, Geneva, Switzerland, July 15-19, 1991, Proceedings (Lecture Notes in Computer Science)*, Pierre America (Ed.). Springer, 21–38. <https://doi.org/10.1007/BFB0057013>
- [34] Andrew Kennedy. 2007. Compiling with continuations, continued. *SIGPLAN Not.* 42, 9 (Oct. 2007), 177–190. <https://doi.org/10.1145/1291220.1291179>
- [35] Raghavan Komondoor and Susan Horwitz. 2000. Semantics-Preserving Procedure Extraction. In *POPL 2000, Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Boston, Massachusetts, USA, January 19-21, 2000*, Mark N. Wegman and Thomas W. Reps (Eds.). ACM, 155–169. <https://doi.org/10.1145/325694.325713>

- [36] Frédéric Lahaie-Bertrand, Léonard Oest O’Leary, Olivier Melançon, Marc Feeley, and Stefan Monnier. 2025. Arborescent Garbage Collection: A Dynamic Graph Approach to Immediate Cycle Collection. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on Memory Management, ISMM 2025, Seoul, Republic of Korea, 17 June 2025*, Martin Maas, Tim Harris, and Onur Mutlu (Eds.). ACM, 14–26. <https://doi.org/10.1145/3735950.3735953>
- [37] Florian Latifi, David Leopoldseider, Christian Wimmer, and Hanspeter Mössenböck. 2021. CompGen: generation of fast JIT compilers in a multi-language VM. In *DLS 2021: Proceedings of the 17th ACM SIGPLAN International Symposium on Dynamic Languages, Virtual Event / Chicago, IL, USA, October 19, 2021*, Arjun Guha (Ed.). ACM, 35–47. <https://doi.org/10.1145/3486602.3486930>
- [38] Daan Leijen. 2022. *What About the Integer Numbers? Fast Arithmetic with Tagged Integers - A Plea for Hardware Support*. Technical Report MSR-TR-2022-17. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/what-about-the-integer-numbers-fast-arithmetic-with-tagged-integers-a-plea-for-hardware-support/>
- [39] David Leopoldseider, Lukas Stadler, Thomas Würthinger, Josef Eisl, Doug Simon, and Hanspeter Mössenböck. 2018. Dominance-based duplication simulation (DBDS): code duplication to enable compiler optimizations. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization, CGO 2018, Vösendorf / Vienna, Austria, February 24-28, 2018*, Jens Knoop, Markus Schordan, Teresa Johnson, and Michael F. P. O’Boyle (Eds.). ACM, 126–137. <https://doi.org/10.1145/3168811>
- [40] John McCarthy. 1960. Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Commun. ACM* 3, 4 (1960), 184–195. <https://doi.org/10.1145/367177.367199>
- [41] Olivier Melançon, Marc Feeley, and Manuel Serrano. 2023. An Executable Semantics for Faster Development of Optimizing Python Compilers. In *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering, SLE 2023, Cascais, Portugal, October 23-24, 2023*, João Saraiva, Thomas Degueule, and Elizabeth Scott (Eds.). ACM, 15–28. <https://doi.org/10.1145/3623476.3623529>
- [42] Olivier Melançon, Marc Feeley, and Manuel Serrano. 2024. Static Basic Block Versioning. In *38th European Conference on Object-Oriented Programming, ECOOP 2024, Vienna, Austria, September 16-20, 2024 (LIPIcs)*, Jonathan Aldrich and Guido Salvaneschi (Eds.), Vol. 313. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 28:1–28:27. <https://doi.org/10.4230/LIPICS.ECOOP.2024.28>

- [43] Olivier Melançon, Manuel Serrano, and Marc Feeley. 2025. Float Self-Tagging. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 1620–1646. <https://doi.org/10.1145/3763108>
- [44] Kevin Menard, Chris Seaton, and Benoit Dalozé. 2018. Specializing Ropes for Ruby. In *Proceedings of the 15th International Conference on Managed Languages & Runtimes (ManLang '18)*. Association for Computing Machinery, New York, NY, USA, Article 10, 7 pages. <https://doi.org/10.1145/3237009.3237026>
- [45] Matthew Might, Yannis Smaragdakis, and David Van Horn. 2010. Resolving and exploiting the k -CFA paradox: illuminating functional vs. object-oriented program analysis. In *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010, Toronto, Ontario, Canada, June 5-10, 2010*, Benjamin G. Zorn and Alex Aiken (Eds.). ACM, 305–315. <https://doi.org/10.1145/1806596.1806631>
- [46] MITRE Corporation. 2024. CVE-2024-4761: Out-of-Bounds Write in V8 in Google Chrome. Common Vulnerabilities and Exposures. (May 2024). <https://www.cve.org/CVERecord?id=CVE-2024-4761>
- [47] Frank Mueller and David B. Whalley. 1995. Avoiding Conditional Branches by Code Replication. In *Proceedings of the ACM SIGPLAN'95 Conference on Programming Language Design and Implementation (PLDI), La Jolla, California, USA, June 18-21, 1995*, David W. Wall (Ed.). ACM, 56–66. <https://doi.org/10.1145/207110.207116>
- [48] Nahuel Palumbo, Guillermo Polito, Stéphane Ducasse, and Pablo Tesone. 2025. Metacompilation of Baseline JIT Compilers with Druid. *Art Sci. Eng. Program.* 10, 1 (2025). <https://doi.org/10.22152/PROGRAMMING-JOURNAL.ORG/2025/10/9>
- [49] Benjamin Quiring, David Van Horn, John Reppy, and Olin Shivers. 2025. Webs and Flow-Directed Well-Typedness Preserving Program Transformations. *Proc. ACM Program. Lang.* 9, PLDI, Article 177 (June 2025), 25 pages. <https://doi.org/10.1145/3729280>
- [50] Baptiste Saleil and Marc Feeley. 2017. Interprocedural Specialization of Higher-Order Dynamic Languages Without Static Analysis. In *31st European Conference on Object-Oriented Programming, ECOOP 2017, Barcelona, Spain, June 19-23, 2017 (LIPIcs)*, Peter Müller (Ed.), Vol. 74. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 23:1–23:23. <https://doi.org/10.4230/LIPICS.ECOOP.2017.23>
- [51] Robert Scheifler. 1977. An Analysis of Inline Substitution for a Structured Programming Language. *Commun. ACM* 20, 9 (1977), 647–654. <https://doi.org/10.1145/359810.359830>

- [52] Manuel Serrano. 2021. Of JavaScript AOT compilation performance. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–30. <https://doi.org/10.1145/3473575>
- [53] Manuel Serrano. 2026. Bigloo. <https://www-sop.inria.fr/index/fp/Bigloo>. (2026).
- [54] Manuel Serrano and Robert Bruce Findler. 2020. Dynamic property caches: a step towards faster JavaScript proxy objects. In *CC '20: 29th International Conference on Compiler Construction, San Diego, CA, USA, February 22-23, 2020*, Louis-Noël Pouchet and Alexandra Jimborean (Eds.). ACM, 108–118. <https://doi.org/10.1145/3377555.3377888>
- [55] Olin Shivers. 1988. Control-Flow Analysis in Scheme. In *Proceedings of the ACM SIGPLAN'88 Conference on Programming Language Design and Implementation (PLDI), Atlanta, Georgia, USA, June 22-24, 1988*, Richard L. Wexelblat (Ed.). ACM, 164–174. <https://doi.org/10.1145/53990.54007>
- [56] Olin Shivers and David Fisher. 2004. Multi-return function call. *SIGPLAN Not.* 39, 9 (Sept. 2004), 79–89. <https://doi.org/10.1145/1016848.1016864>
- [57] Litong Song and K.M. Kavi. 2002. A technique for variable dependence driven loop peeling. In *Fifth International Conference on Algorithms and Architectures for Parallel Processing, 2002. Proceedings.* 390–395. <https://doi.org/10.1109/ICAPP.2002.1173607>
- [58] Paul Teng and Christophe Dubach. 2025. Hardware Synthesizable Exceptions using Continuations. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC '25)*. Association for Computing Machinery, New York, NY, USA, 1104–1111. <https://doi.org/10.1145/3658617.3697719>
- [59] Tim Whiting and Kimball Germane. 2025. Context-Sensitive Demand-Driven Control-Flow Analysis. In *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part II (Lecture Notes in Computer Science)*, Viktor Vafeiadis (Ed.). Springer, 374–401. https://doi.org/10.1007/978-3-031-91121-7_15
- [60] Christian Wimmer and Stefan Brunthaler. 2013. ZipPy on Truffle: A Fast and Simple Implementation of Python. In *Proceedings of the 2013 Companion Publication for Conference on Systems, Programming, & Applications: Software for Humanity (SPLASH '13)*. Association for Computing Machinery, New York, NY, USA, 17–18. <https://doi.org/10.1145/2508075.2514572>
- [61] Christian Wimmer and Thomas Würthinger. 2012. Truffle: a self-optimizing runtime system. In *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity (SPLASH '12)*. Association for Computing Machinery, New York, NY, USA, 13–14. <https://doi.org/10.1145/2384716.2384723>

- [62] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. 2017. Practical partial evaluation for high-performance dynamic language runtimes. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 662–676. <https://doi.org/10.1145/3062341.3062381>

Appendix A

Merge Heuristics

This appendix presents implementation details of the **arithmetic** and **similarity** merge heuristics evaluated in this thesis. Both are *distance-based*: they can be described as picking the closest pair of contexts under a given distance function. This appendix therefore describes them in terms of their distance functions. Such distance-based heuristics are strictly local: their inputs are limited to the merge candidates and their contexts, precluding whole-program analysis and thus bounding the cost of selecting contexts to merge. The **random** merge heuristic, used as a worst-case comparison in the evaluation of Lambda Versioning (AV) in Chapter 3, is also briefly discussed.

One key finding of Chapter 3 is that the **similarity** heuristic does not fare significantly better than the **random** heuristic for type check elimination by Static Basic Block Versioning (SBBV). The earliest implementation of SBBV therefore worked *despite* its heuristic. This is explained by SBBV exploring fewer contexts, due to the lack of interprocedural information propagation, and thus being less sensitive to the choice of heuristic. For AV, however, **similarity** fares significantly better than **random**, eliminating more run-time type checks at lower compile time, though not as well as **arithmetic**, which eliminates more run-time checks at similar compile time.

A.1. Arithmetic Merge Heuristic

The **arithmetic** merge heuristic is distance-based and relies on two main assumptions to select the contexts to merge: pure arithmetic (operands of the same type) is more frequent than mixed arithmetic (operands of differing types), and overflows are rare. Contexts containing at least one variable that may be a bignum are therefore merged first, followed by contexts containing variables of differing numeric types. When pairs of contexts share the same priority, their Hamming distance is used as a tiebreaker.

The overall effect of this heuristic is to preferentially preserve contexts in which all variables are fixnums or all variables are flonums. Mixed-arithmetic contexts can still be generated, but only if the version limit permits it. Contexts specialized for bignum arithmetic are typically not generated, which is desirable since the relative cost of type checks is significantly lower in bignum arithmetic.

The implementation of the `arithmetic` distance function is shown below. Some implementation details were removed for clarity.

```

1 (define (context-all-number-types=? ctx number-type)
2   (context-forall? ctx
3     (lambda (variable type)
4       (or (type=? type number-type) (not (type<=? type '(fx fl bg)))))))
5
6 (define (context-has-bignum-type? ctx)
7   (context-any? ctx
8     (lambda (variable type)
9       (and (type>=? type '(bg)) (type<=? type '(fx fl bg))))))
10
11 (define (context-has-mixed-arithmetic? ctx)
12   (or (context-contains ctx '(fx fl))
13       (and (context-contains ctx '(fx)) (context-contains ctx '(fl)))))
14
15 (define (context-distance-arithmetic ctx1 ctx2)
16   (define bias
17     (cond
18       ;; merge contexts following arithmetic overflow
19       ((and (context-has-bignum-type? ctx1) (context-has-bignum-type? ctx2))
20        0.1)
21       ((and ;; bias against merging pure arithmetic with impure arithmetic
22             (or (context-contains ctx1 '(fx)) (context-contains ctx2 '(fx)))
23             (not (eq? (context-all-number-types=? ctx1 '(fx))
24                       (context-all-number-types=? ctx2 '(fx)))))
24        100)
25       ((and ;; same as above for flonums
26             (or (context-contains ctx1 '(fl)) (context-contains ctx2 '(fl)))
27             (not (eq? (context-all-number-types=? ctx1 '(fl))
28                       (context-all-number-types=? ctx2 '(fl)))))
29        100)
30       ((context-any? ctx1 ;; would '(fx) or '(fl) type be widened?
31         (lambda (variable type1)
32           (let ((type2 (context-ref ctx2 variable)))
33             (or (and (or (type=? type1 '(fx)) (type=? type2 '(fx)))
34                     (not (type=? (type-union type1 type2) '(fx))))
35                 (and (or (type=? type1 '(fl)) (type=? type2 '(fl)))
36                     (not (type=? (type-union type1 type2) '(fl))))))))
37         (if (and (not (context-has-mixed-arithmetic? ctx1))
38                 (not (context-has-mixed-arithmetic? ctx2)))
39             10
40             2))
41       (else 1)))
42
43   (* (expt bias (context-length ctx1)) (hamming-distance ctx1 ctx2)))

```

A.2. Similarity Merge Heuristic

The `similarity` heuristic is distance-based and relies mostly on the Hamming distance between contexts. This Hamming distance is computed per variable, as the number of

concrete types on which the variable’s abstract types disagree between the two contexts, and summed over all variables. For instance, a variable of abstract type `fx|fl` in one context and `fx|bg` in the other contributes a distance of two. A specificity bias is added to the resulting distance to steer merges toward contexts that are either too broad or, when interval arithmetic is tracked, too specific.

The implementation of the `similarity` distance function is shown below. Some implementation details were removed for clarity.

```

1 (define (context-distance-similarity ctx1 ctx2)
2   (define specificity-bias 31)
3   (define (type-specificity type)
4     ;; Return an integer representing how specific the type is.
5     ;; A lower score means a more specific type:
6     ;; - constants have specificity 0
7     ;; - top has specificity specificity-bias x (# of tracked types)
8     (if (constant? type)
9         0
10        (let ((bits (type-motley-bits type))
11              (lo (type-fixnum-range-lo type))
12              (hi (type-fixnum-range-hi type)))
13          (+ (integer-length (- hi lo))
14             (* specificity-bias (bit-count bits))))))
15
16   (define (average-specificity ctx)
17     ;; Compute the average specificity of all types in a given context.
18     (let loop ((i 0) (acc 0))
19       (if (< i (context-size ctx))
20         (let ((type (context-ref ctx i)))
21           (loop (+ i 1) (+ acc (type-specificity type))))
22         (quotient acc (context-size ctx))))
23
24   (define (usefulness ctx)
25     ;; Return a number representing the usefulness of a context.
26     ;; A lower score means a more useful context. A bias was chosen
27     ;; such that contexts whose average specificity corresponds to a
28     ;; single type have lowest score.
29     (abs (- (average-specificity ctx) specificity-bias)))
30
31   ;; Return the sum of the Hamming distances of all types in
32   ;; contexts plus a bias to discourage merging useful types.
33   (let loop ((i 0) (acc 0))
34     (if (< i (context-size ctx1))
35       (let ((type1 (context-ref ctx1 i))
36             (type2 (context-ref ctx2 i)))
37         (loop (+ i 1) (+ acc (hamming-distance type1 type2))))
38       (+ (* acc 100000)
39          (/ 99999 (+ 1 (usefulness ctx1) (usefulness ctx2))))))

```

A.3. Random Merge Heuristic

The `random` merge heuristic is not distance-based; it picks contexts to merge at random. It serves as a worst-case baseline against which to compare the other merge heuristics. As expected, it performs poorly at eliminating run-time type checks. It was also shown to significantly increase the compile time of AV, demonstrating that a good choice of heuristic can also accelerate convergence.

Appendix B

Hyperfunction Macro

This appendix presents the Gambit Scheme implementation of the `hyperfunction` macro. This implementation is not especially performant, but demonstrates that a functioning hyperfunction interface can be implemented with minimal effort.

```
1 (define-type ***hyperfunction***
2   constructor: ***make-hyperfunction***
3   id closure slots)
4
5 (define-type ***continuation***
6   constructor: ***make-continuation***
7   slots)
8
9 (define-macro (procedure? arg)
10  '(**hyperfunction***? ,arg))
11
12 (define-macro ($make-hyperfunction id closure . versions)
13  '(**make-hyperfunction*** ,id ,closure (##vector ,@versions)))
14
15 (define-macro ($hyperfunction-closure m)
16  '(**hyperfunction***-closure ,m))
17
18 (define-macro ($hyperfunction-ref m i)
19  '(**vector-ref (**hyperfunction***-slots ,m) ,i))
20
21 (define-macro ($make-continuation . versions)
22  '(**make-continuation*** (##vector ,@versions)))
23
24 (define-macro ($continuation-ref c i)
25  '(**vector-ref (**continuation***-slots ,c) ,i))
26
27 (define-macro ($hyperfunction . body)
28  (import _match)
29
30  (define (keyword->symbol x)
31    (string->symbol (keyword->string x)))
32
33  (define (section body kw #!optional (tail #f))
34    (let loop ((body body))
35      (cond
36        ((and (pair? body) (equal? (car body) kw))
37         (cond
38           (tail (cdr body))
```

```

39      ((pair? (cdr body)) (cadr body))
40      (else (error "$hyperfunction empty" kw))))
41      ((pair? body) (loop (cdr body)))
42      (else (error "$hyperfunction missing" kw))))))
43
44 (define (parse-block block)
45   (match block
46     ((,hd . ,tl) (cons (parse-block hd) (parse-block tl)))
47     (,x (K:n->lookup x))))
48
49 (define (parse-blocks blocks)
50   (cond
51     ((null? blocks) '())
52     ((and (pair? blocks) (keyword? (car blocks)) (pair? (cdr blocks)))
53      (cons
54        (cons (keyword->symbol (car blocks)) (parse-block (cadr blocks)))
55        (parse-blocks (cddr blocks))))
56     ((and (pair? blocks) (list? (car blocks)))
57      (append
58        (parse-blocks (car blocks))
59        (parse-blocks (cdr blocks))))
60     (else
61      (error "$hyperfunction ill-formed blocks" (car blocks)))))
62
63 (define (parse-symbol-list lst label #!optional (allow-false #f))
64   (define test?
65     (if allow-false
66         (lambda (x) (or (equal? x #f) (symbol? x)))
67         symbol?))
68
69   (if (not (every test? lst)) (error "$hyperfunction ill-formed" label lst))
70   lst)
71
72 (define (parse-id id)
73   (if (not (symbol? id)) (error "$hyperfunction ill-formed id:" id))
74   id)
75
76 (define (parse-K K)
77   (if (not (symbol? K)) (error "$hyperfunction ill-formed continuation:" K))
78   K)
79
80 (define (parse-args args) (parse-symbol-list args args))
81 (define (parse-closure closure) (parse-symbol-list closure closure))
82 (define (parse-entry entry)
83   (match entry
84     ((,name . ,body)
85      (let ((entry-label (section entry entry:))
86            (Ks (section entry K:)))
87        (if (not (symbol? entry-label))
88            (error "$hyperfunction ill-formed entry:" name))
89            (if (not (member entry-label (map car blocks)))
90                (error "$hyperfunction no entry block named" entry-label))
91                (list name entry-label (parse-symbol-list Ks K: #t))))
92      (, _ (error "$hyperfunction ill-formed entries:" entry))))
93
94 (define (entry-name e) (list-ref e 0))
95 (define (entry-label e) (list-ref e 1))
96 (define (entry-Ks e) (list-ref e 2))
97
98 (define (parse-entries entries)
99   (if (null? entries) (error "$hyperfunction empty entries"))
100   (map parse-entry entries))
101
102 (define (parse-generic generic)
103   (define generic-entry (list-ref entries generic))

```

```

103 (match generic-entry
104   ((,name ,label ,Ks)
105    (let ((Ks-length (length Ks)))
106      (define generic-entry-label (gensym '***generic-entry***)
107      (define generic-entry
108        (cons
109          generic-entry-label
110          '(lambda (,K ,@args ,@closure)
111            (,label
112              ($make-continuation ,@(map (lambda _ K) (iota Ks-length)))
113              ,@args
114              ,@closure))))))
115
116      (set! blocks (cons generic-entry blocks))
117      generic-entry-label))))
118
119 (define (K:n->lookup* sym)
120   (define K-string (symbol->string K))
121   (define sym-string (symbol->string sym))
122   (define K-length (string-length K-string))
123   (define sym-length (string-length sym-string))
124   (and (string-prefix? K-string sym-string)
125        (let* ((tail (substring sym-string K-length sym-length)))
126          (and (string-prefix? ":" tail)
127               (let ((n (string->number (substring tail 1 (string-length tail)))))
128                 (and n (>= n 0) '($continuation-ref ,K ,n)))))))
129
130 (define (K:n->lookup sym)
131   (if (symbol? sym) (or (K:n->lookup* sym) sym) sym))
132
133 (define K (parse-K (section body continuation:)))
134 (define id (parse-id (section body id:)))
135 (define args (parse-args (section body args:)))
136 (define closure (parse-closure (section body closure:)))
137 (define blocks (parse-blocks (section body blocks: #t)))
138 (define entries (parse-entries (section body entries:)))
139 (define generic-entry-label (parse-generic (section body generic:)))
140
141 '(lambda ,closure
142   ,@(map (lambda (block) '(define ,(car block) ,(cdr block))) blocks)
143   ($make-hyperfunction ',id (##list ,@closure) ,generic-entry-label
144     ,@(map entry-label entries))))
145
146 (define-macro ($CALL f i Ks . args)
147   (import _match)
148   (define K
149     (match Ks
150      ((($K . ,lambdas) '($make-continuation ,@lambdas))
151       (, _ (error "$CALL expected a $K" Ks))))
152   (define (call $f)
153     '##apply ($hyperfunction-ref , $f , (##+ i 1)) ,K ,@args (
154       $hyperfunction-closure , $f)))
154   (if (symbol? f)
155       (call f)
156       (let (( $f (gensym 'f))) '(let ((, $f ,f)) ,(call $f)))))
157
158 (define-macro ($CALL-GENERIC f K . args)
159   (define (call $f)
160     '##apply ($hyperfunction-ref , $f 0) ,K ,@args ($hyperfunction-closure , $f)))
161   (if (symbol? f)
162       (call f)
163       (let (( $f (gensym 'f))) '(let ((, $f ,f)) ,(call $f)))))

```


Appendix C

Lambda Versioning Experimental Results

This appendix contains the detailed setup and results of the experiments from Chapter 3.

A subset of the R7RS benchmark suite [1] is used. This is the same subset used to evaluate SBBV in Chapter 2, with the addition of the `fact20` and `fact100` benchmarks. Benchmarks are split into two categories: micro benchmarks (fewer than 400 lines of code) and macro benchmarks (more than 400 lines of code). A description of each benchmark is provided in Figure 24, on the next page.

Figures 25 to 30 show the number of type checks executed at run-time by micro benchmarks, when compiling without AV and with AV at version limits ranging from 1 to 10. Figures 31 to 36 show the same for macro benchmarks.

Figures 37 to 42 show the wall-clock compile time of micro benchmarks by AV at version limits ranging from 1 to 10. Figures 43 to 48 show the same for macro benchmarks. Compile times were measured on a machine with an Intel Core i7-7700K and 48 GB of RAM, running Debian 10.13 with kernel version SMP Debian 4.19.269-1.

Benchmarks are grouped by merge heuristic (`arithmetic` , `similarity` , and `random` ) and by operator representation (inlined or as hyperfunctions).

The results in this appendix are used to compute the geometric means shown in Chapter 3. The only exception is the `compiler` benchmark, excluded from the geometric means because data are not consistently available across all version limits. This benchmark is a full Scheme compiler that generates M68000 code, comprising about 12,000 lines of code and 1,500 functions; for comparison, the next largest benchmark (`nucleic`) has about 3,500 lines and 200 functions. Its compile time exceeds several hours with high version limits, against a few minutes for the other macro benchmarks, and was timed out at 2.5 hours. Timeouts are marked by . Figure 47 contains compile times that exceed this timeout, as it was run before the timeout was imposed.

Micro benchmarks

<code>ack</code>	Ackermann function. Uses small integers and recursion.
<code>array1</code>	Read/write operations on arrays. Uses small integers and vectors.
<code>bague</code>	Solver of the baguenaudier puzzle. Uses small integers and vectors.
<code>cpstak</code>	CPS version of Takeuchi function. Uses small integers and recursion.
<code>fact20</code>	Factorial of 20. Uses small integers and recursion.
<code>fact100</code>	Factorial of 100. Uses small integers, big integers and recursion.
<code>fft</code>	Fast Fourier Transform. Uses floats and vectors.
<code>fib</code>	Fibonacci function. Uses small integers and recursion.
<code>fibfp</code>	Fibonacci function. Uses floats and recursion.
<code>nqueens</code>	Solver of the N-queens puzzle. Uses lists, small integers, and recursion.
<code>primes</code>	Sieve algorithm for finding primes. Uses lists and small integers.
<code>quicksortfl</code>	Quicksort on a float array. Uses floats and <code>f64</code> vectors.
<code>sum</code>	Sum of integers from 1 to 10000. Uses small integers.
<code>sumfp</code>	Sum of floats from 1 to 10^6 . Uses floats.
<code>tak</code>	Takeuchi function. Uses small integers and recursion.

Macro benchmarks

<code>almabench</code>	Compute the celestial coordinates of the sun at noon. Uses floats and vectors.
<code>boyer</code>	Prolog-like rule-directed rewriting engine. Uses lists and symbols.
<code>conform</code>	Graph type checker using equivalence classes. Uses lists and strings.
<code>compiler</code>	Scheme compiler generating M68000 code. Uses pairs, symbols, vectors, and strings.
<code>dynamic</code>	Type inference for Scheme. Uses lists, symbols, and higher-order functions.
<code>earley</code>	Earley parser. Uses vectors, lists, small integers and symbols.
<code>leval</code>	Scheme interpreter based on closures. Uses lists, symbols, and higher-order functions.
<code>maze</code>	Hexagonal grid maze generator. Uses vectors and small integers.
<code>nucleic</code>	3D structure determination of a nucleic acid. Uses vectors and floats.
<code>peval</code>	Partial evaluator for Scheme. Uses lists, symbols, and higher-order functions.
<code>scheme</code>	Scheme interpreter based on closures. Uses lists, symbols, and higher-order functions.
<code>slatex</code>	LaTeX processor. Uses characters, strings, lists, vectors, and small integers.

Figure 24 – Benchmark descriptions.

Type Checks Micro Benchmarks: Arithmetic/Inlined

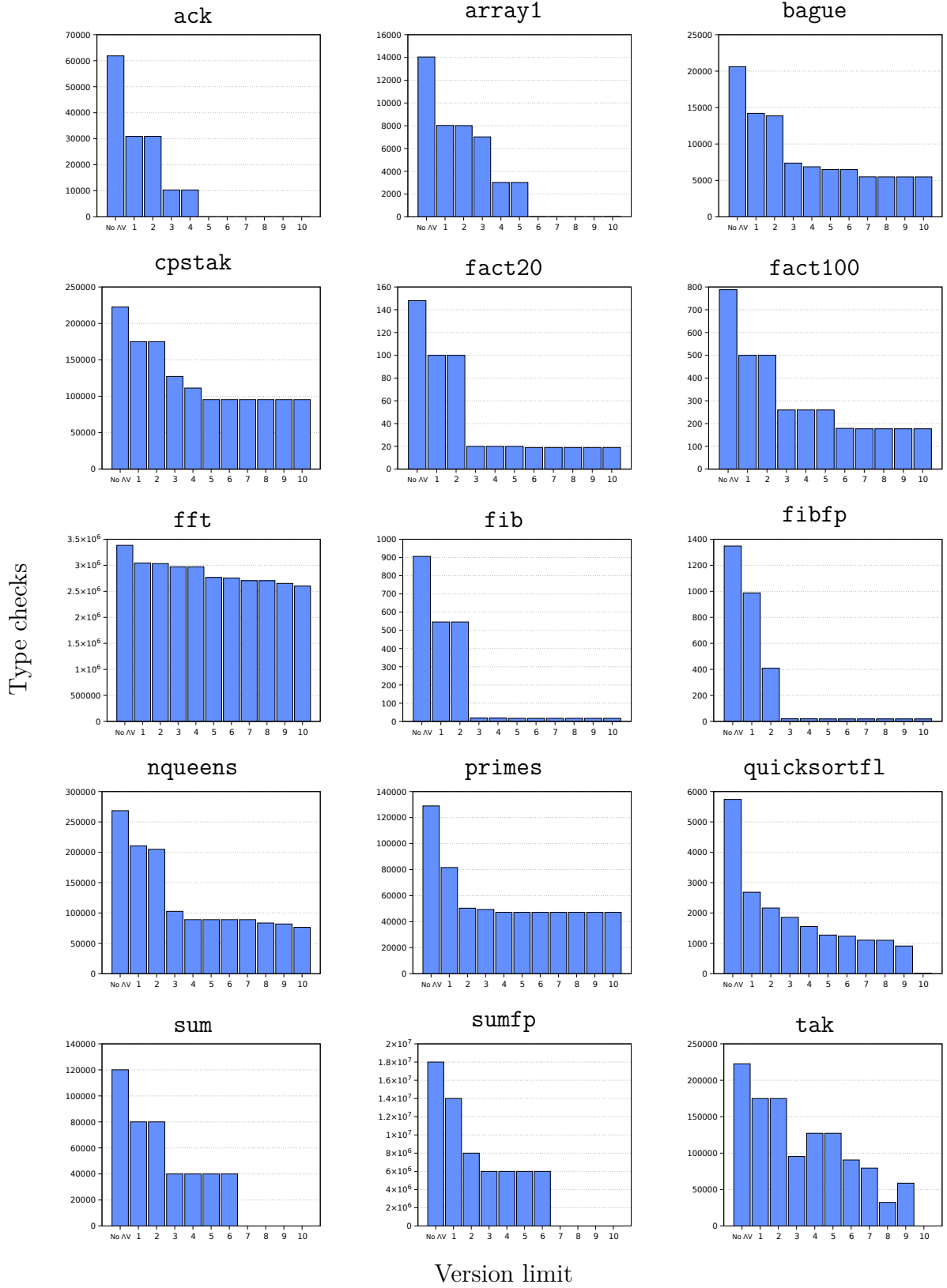


Figure 25 – Type checks micro benchmarks: arithmetic and inlined operators.

Type Checks Micro Benchmarks: Similarity/Inlined



Figure 26 – Type checks micro benchmarks: similarity and inlined operators.

Type Checks Micro Benchmarks: Random/Inlined

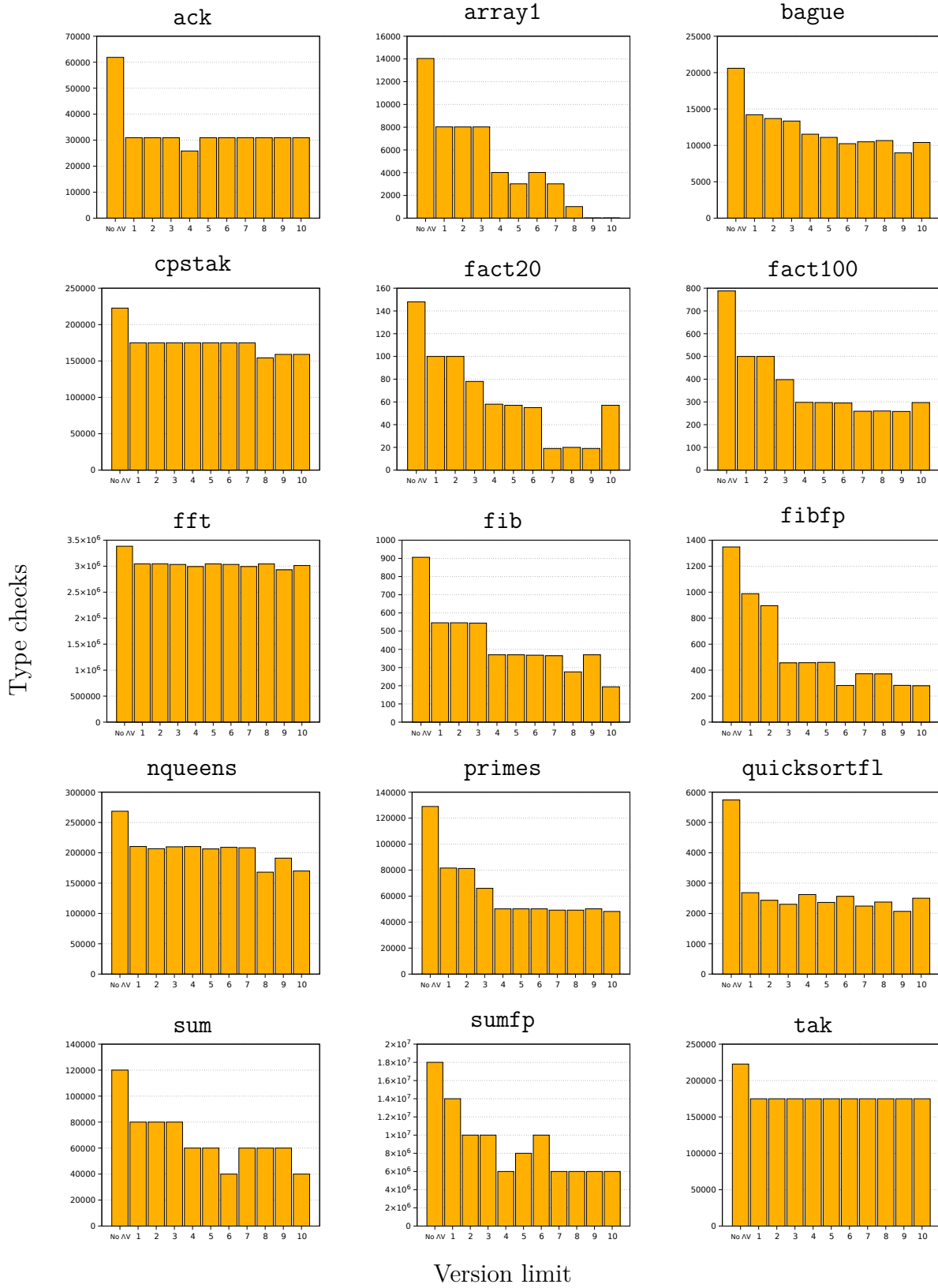


Figure 27 – Type checks micro benchmarks: random and inlined operators.

Type Checks Micro Benchmarks: Arithmetic/Hyperfunction

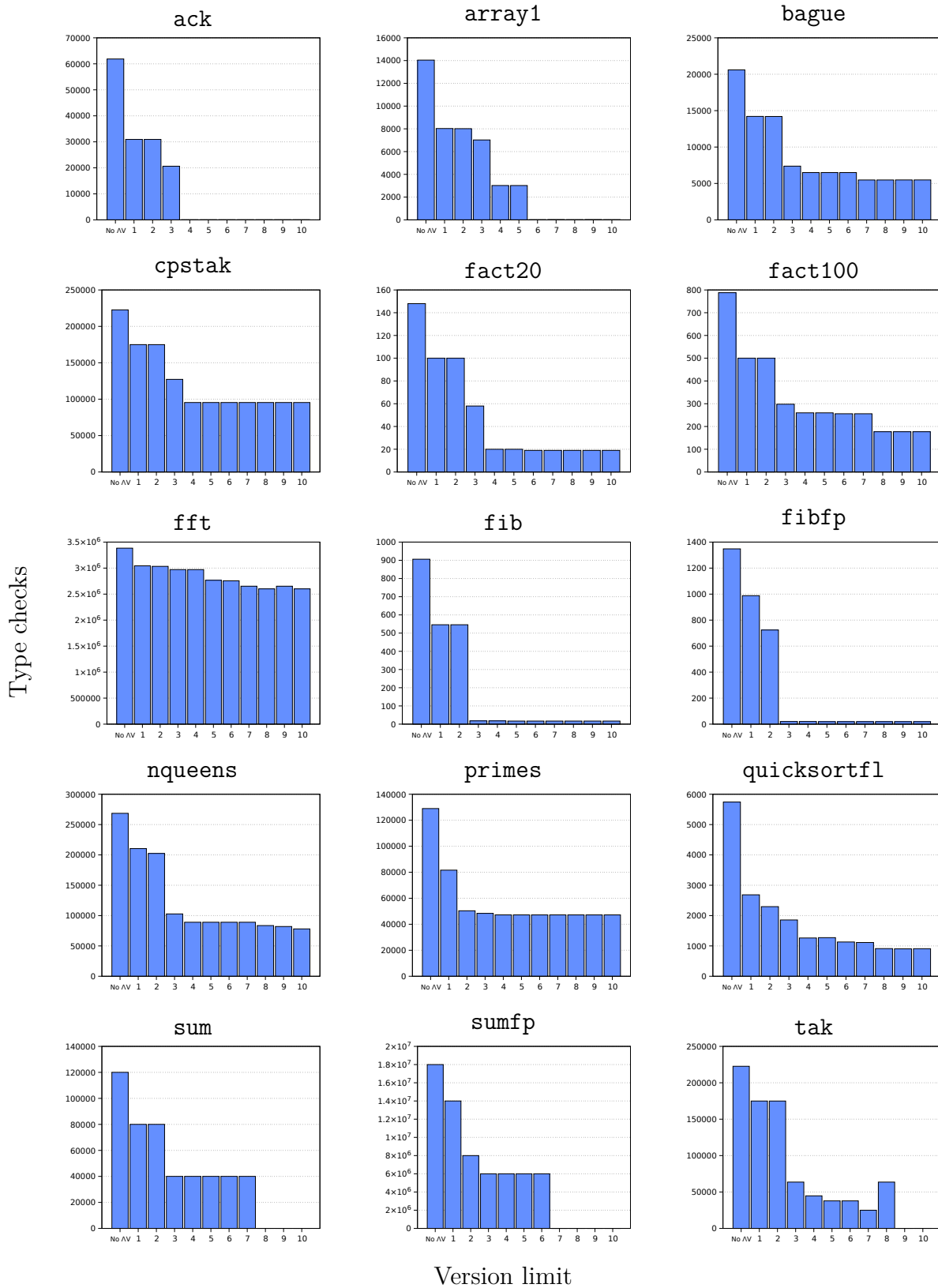


Figure 28 – Type checks micro benchmarks: arithmetic and hyperfunction operators.

Type Checks Micro Benchmarks: Similarity/Hyperfunction



Figure 29 – Type checks micro benchmarks: similarity and hyperfunction operators.

Type Checks Micro Benchmarks: Random/Hyperfunction

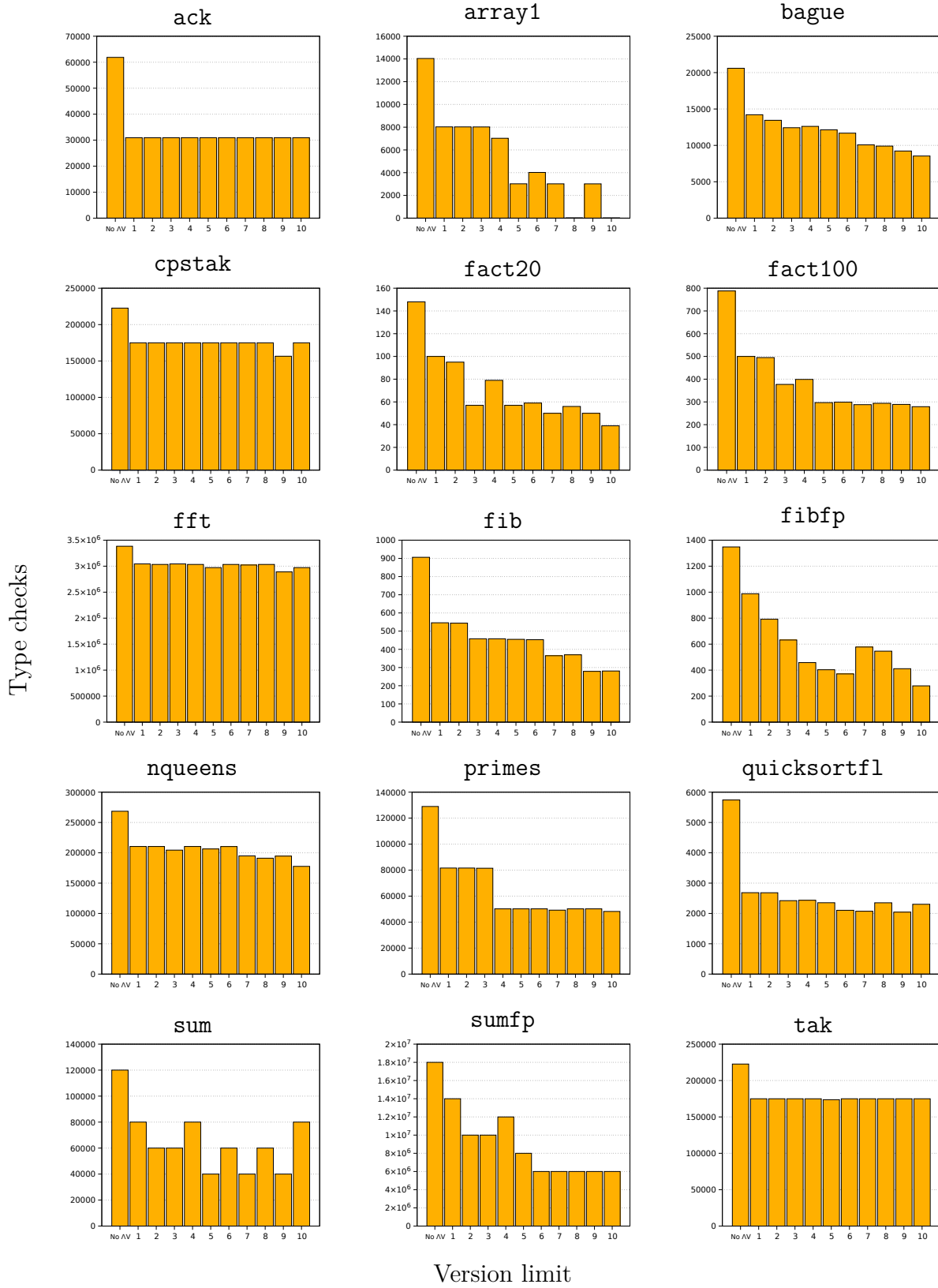


Figure 30 – Type checks micro benchmarks: random and hyperfunction operators.

Type Checks Macro Benchmarks: Arithmetic/Inlined

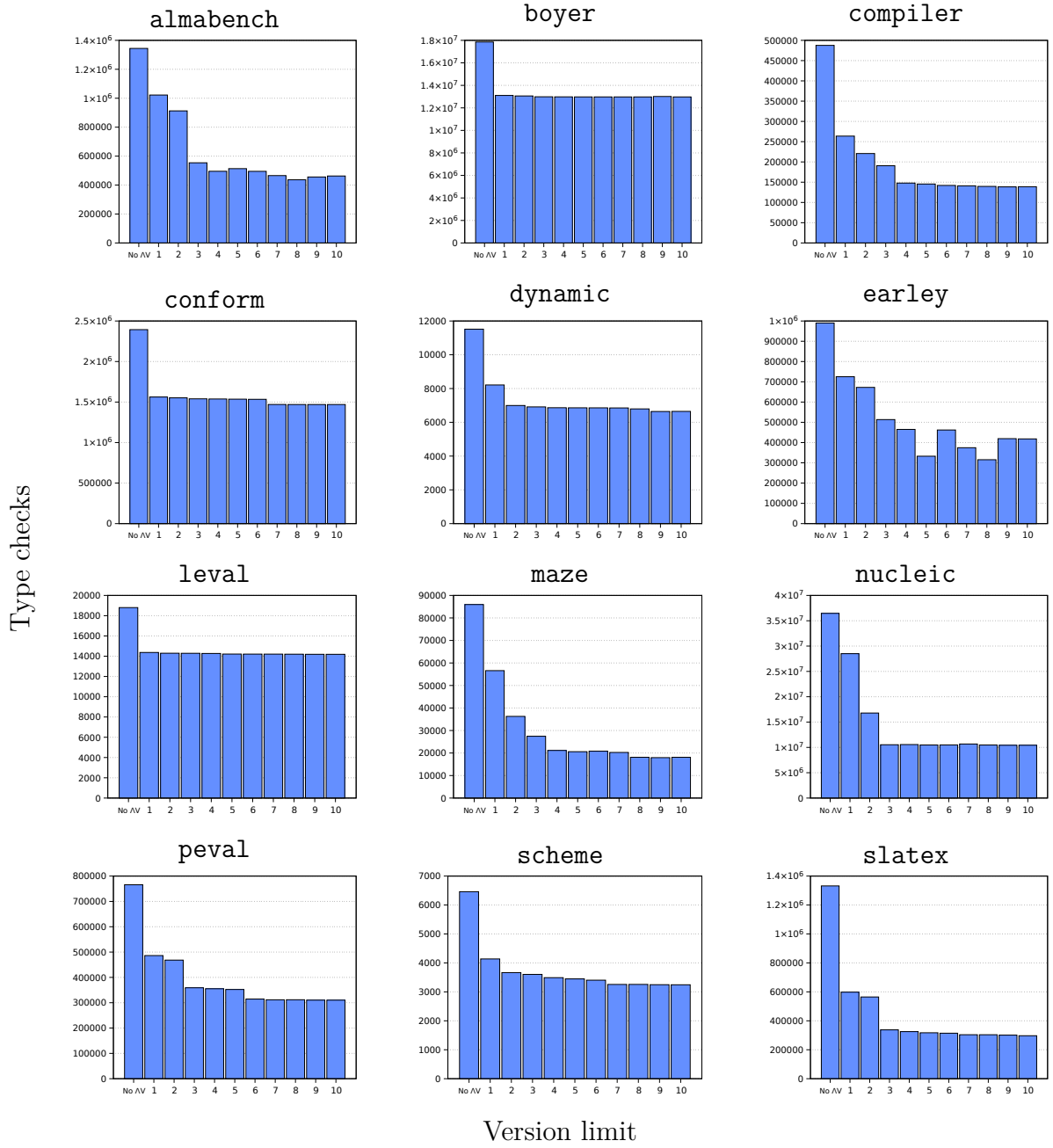


Figure 31 – Type checks macro benchmarks: arithmetic and inlined operators.

Type Checks Macro Benchmarks: Similarity/Inlined

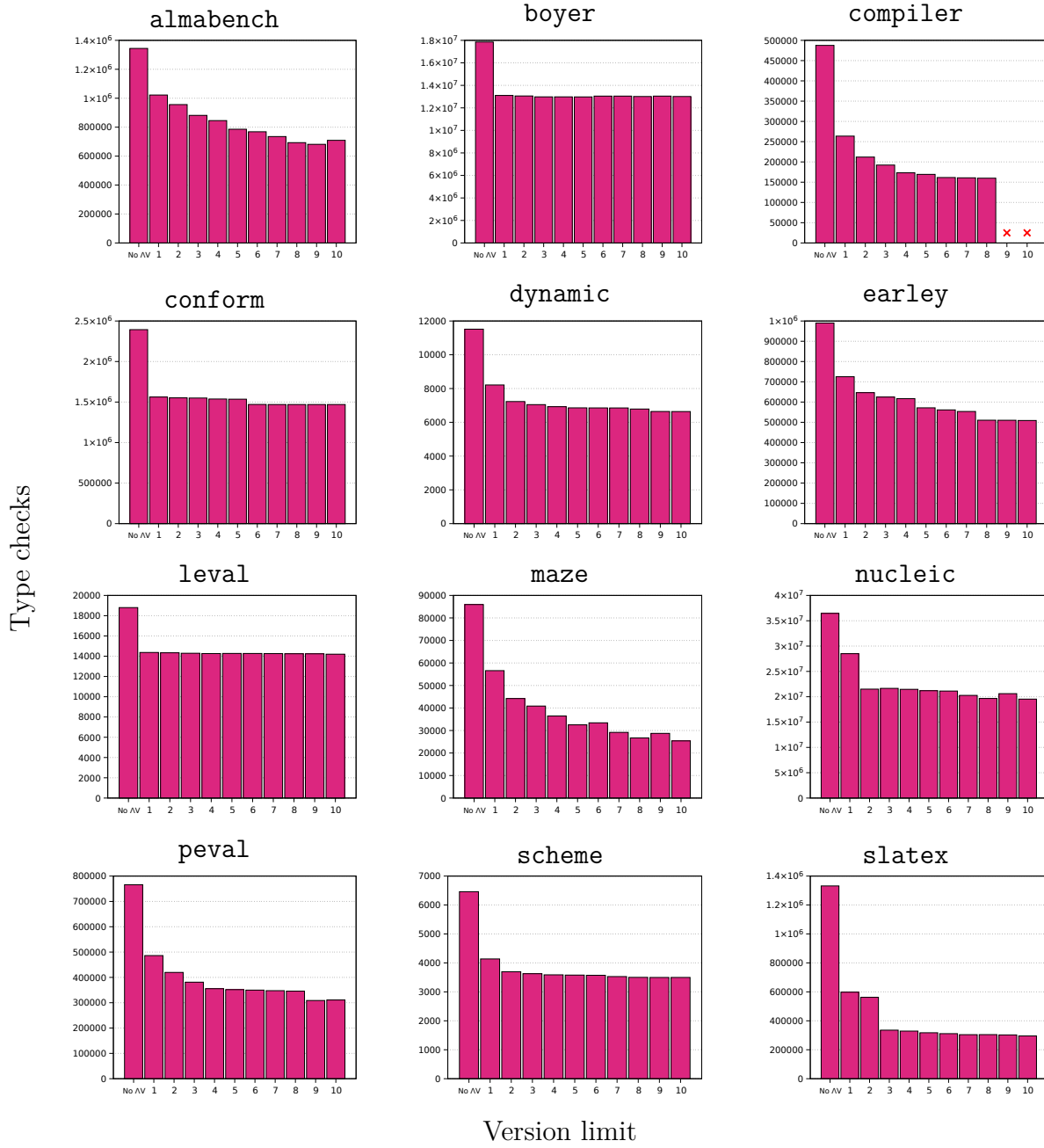


Figure 32 – Type checks macro benchmarks: similarity and inlined operators.

Type Checks Macro Benchmarks: Random/Inlined

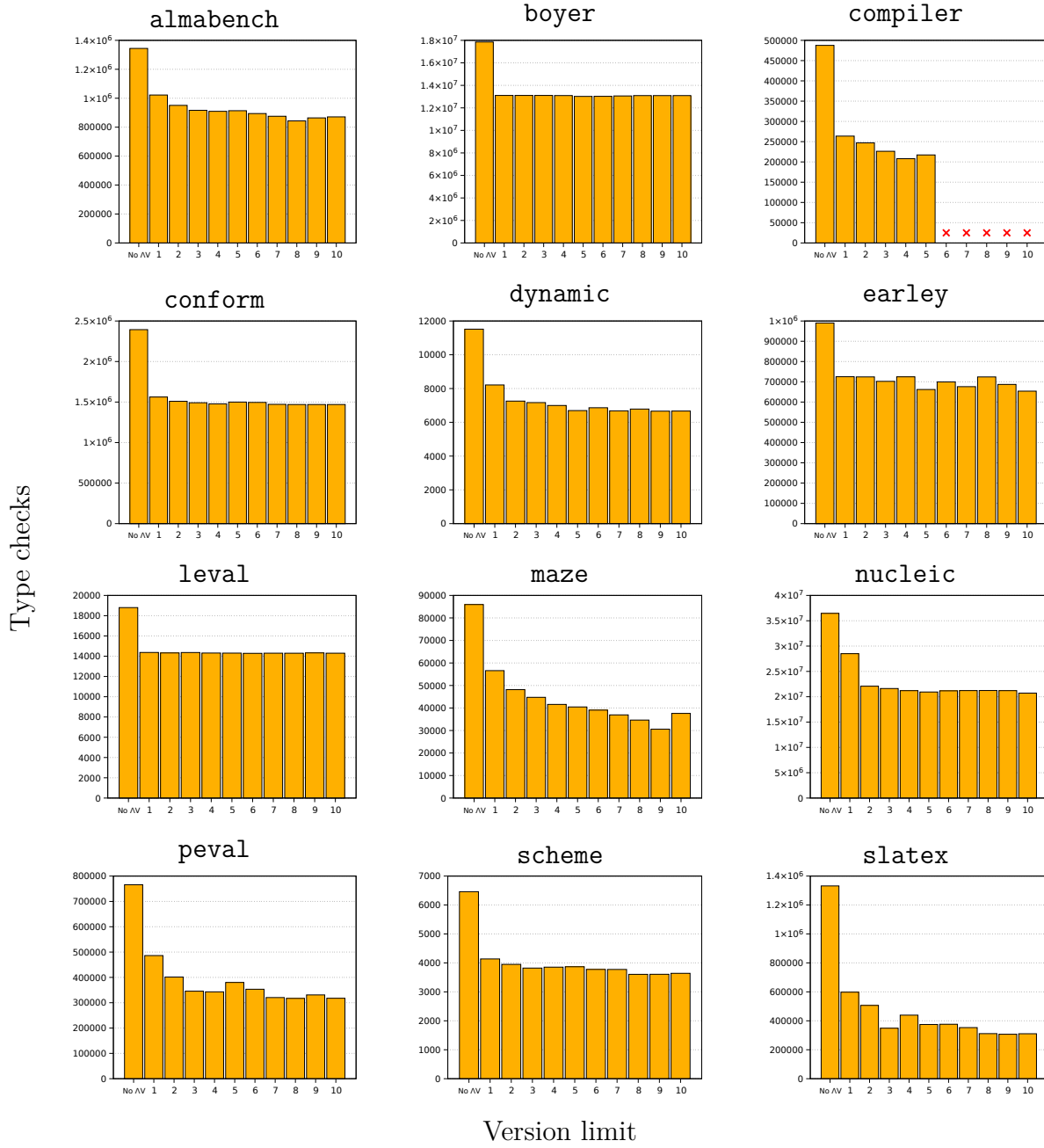


Figure 33 – Type checks macro benchmarks: random and inlined operators.

Type Checks Macro Benchmarks: Arithmetic/Hyperfunction

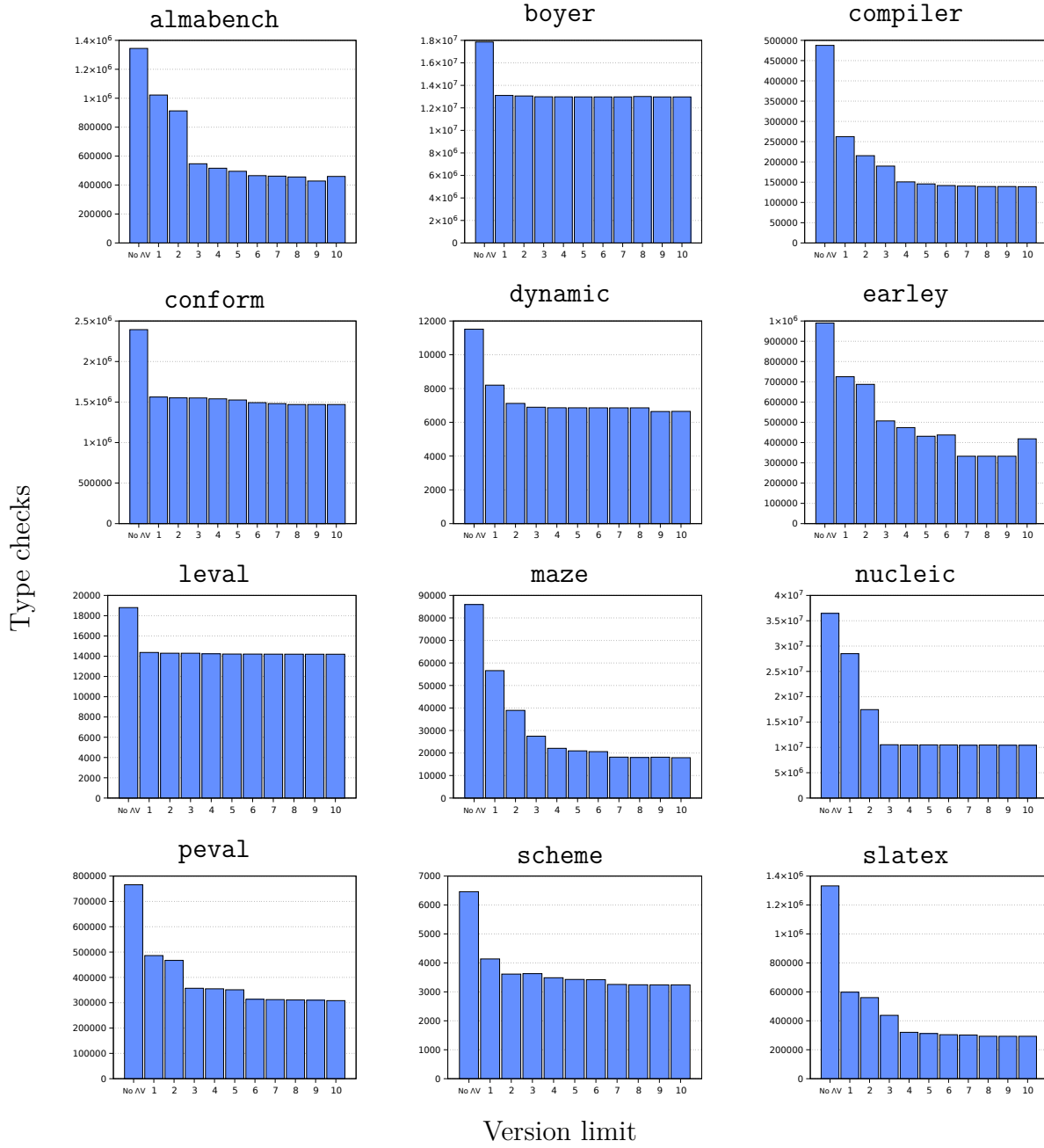


Figure 34 – Type checks macro benchmarks: arithmetic and hyperfunction operators.

Type Checks Macro Benchmarks: Similarity/Hyperfunction

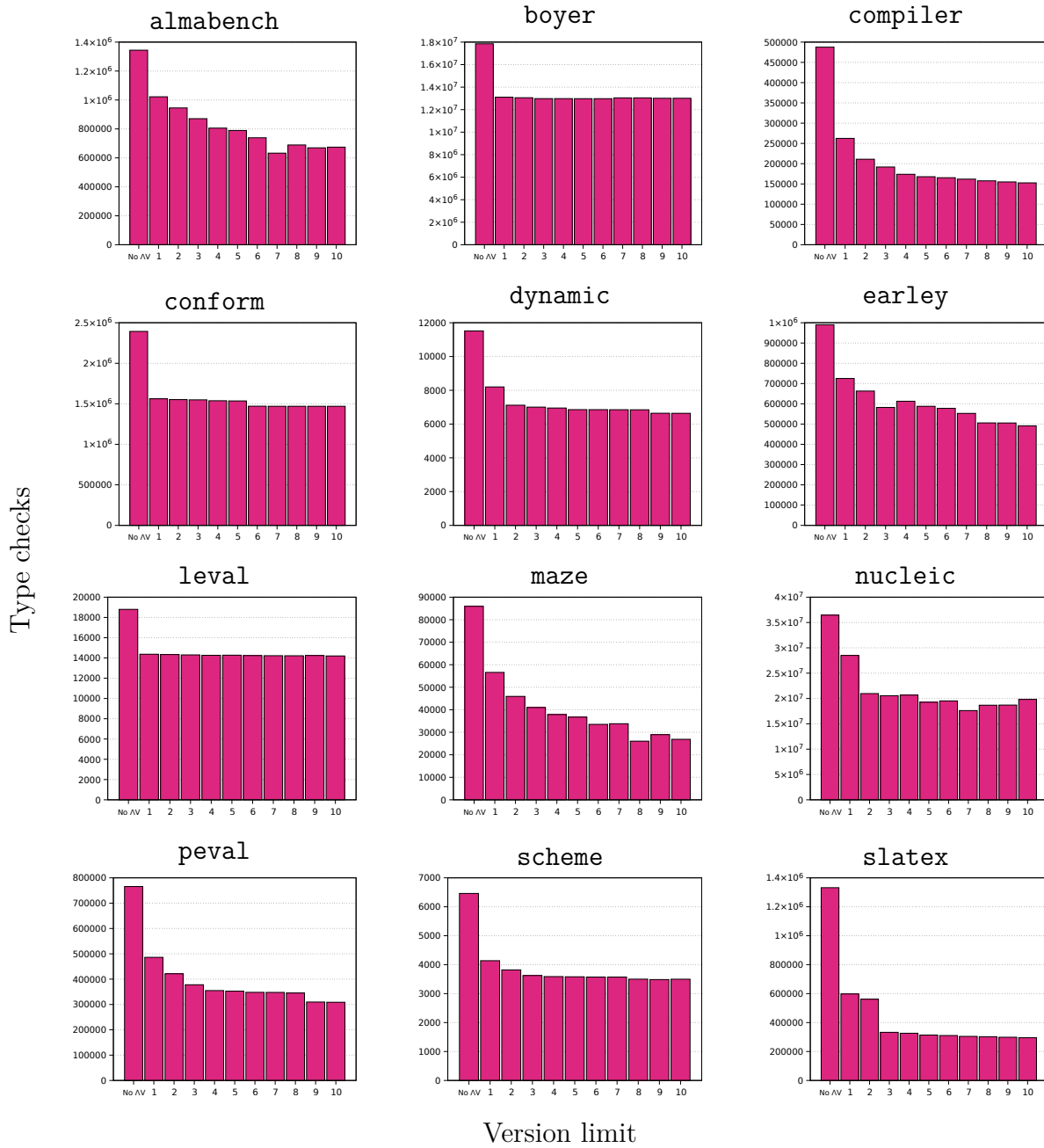


Figure 35 – Type checks macro benchmarks: similarity and hyperfunction operators.

Type Checks Macro Benchmarks: Random/Hyperfunction

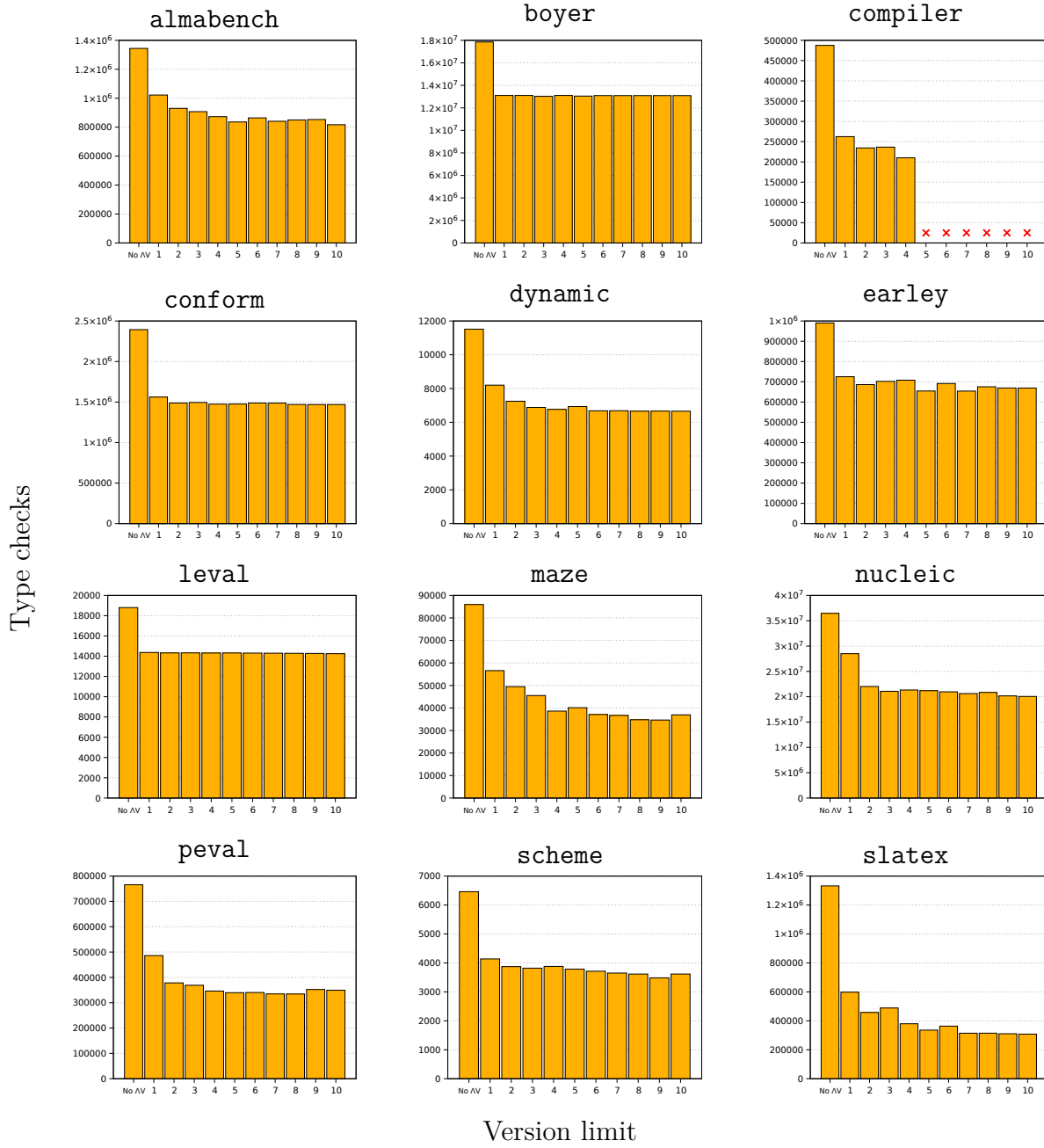


Figure 36 – Type checks macro benchmarks: random and hyperfunction operators.

Compile Time Micro Benchmarks: Arithmetic/Inlined

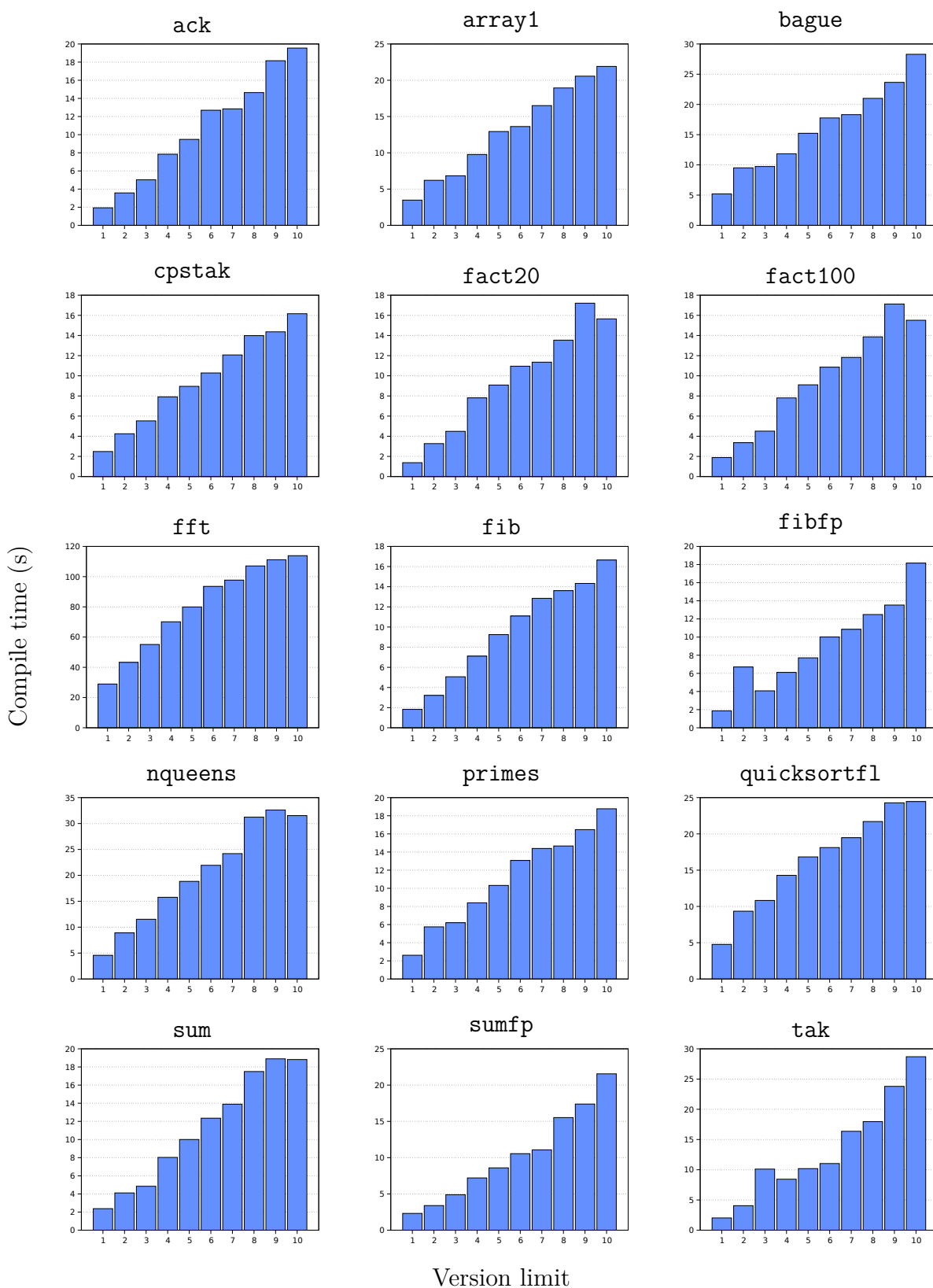


Figure 37 – Compile time micro benchmarks: arithmetic and inlined operators.

Compile Time Micro Benchmarks: Similarity/Inlined



Figure 38 – Compile time micro benchmarks: similarity and inlined operators.

Compile Time Micro Benchmarks: Random/Inlined

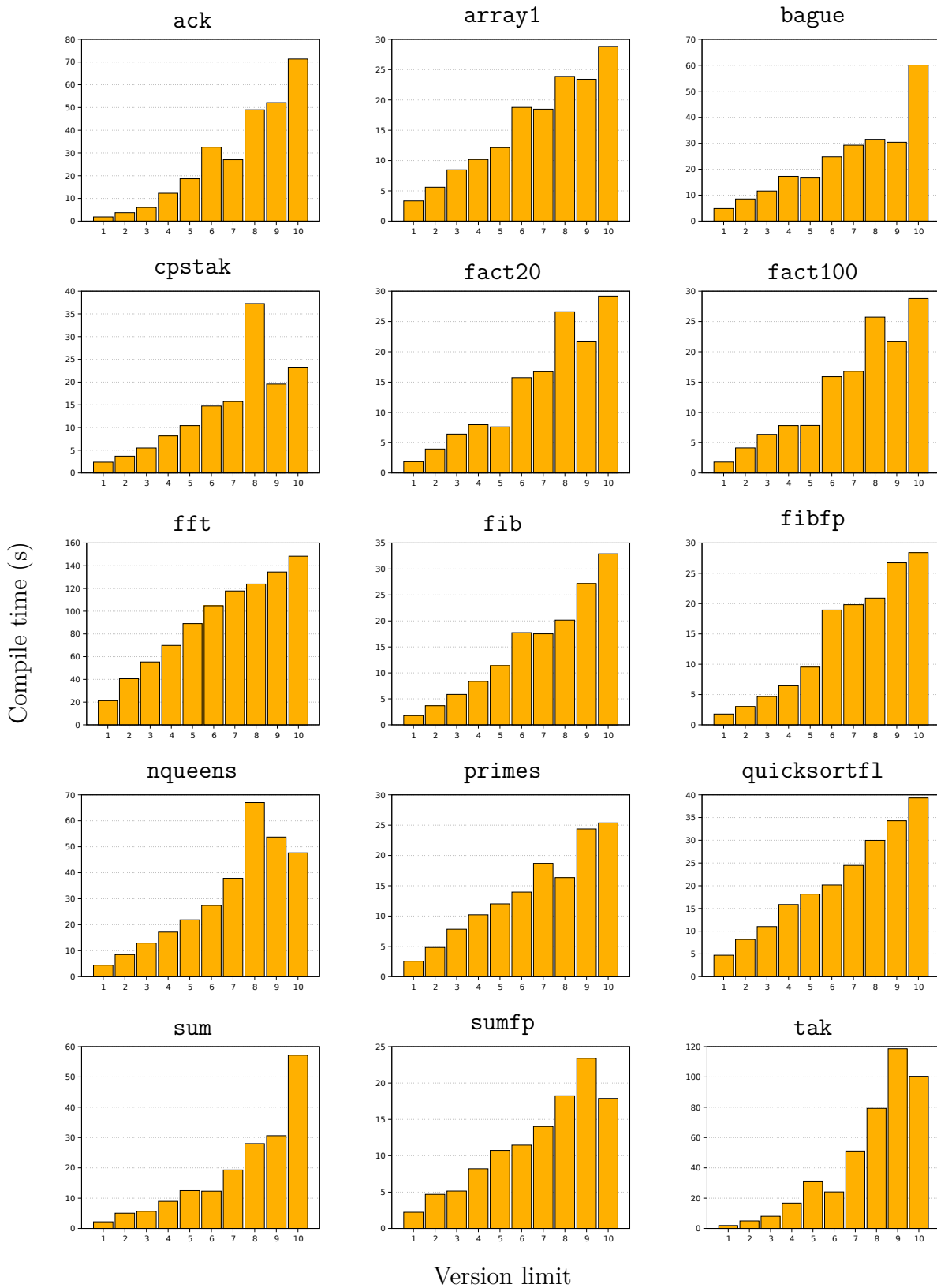


Figure 39 – Compile time micro benchmarks: random and inlined operators.

Compile Time Micro Benchmarks: Arithmetic/Hyperfunction

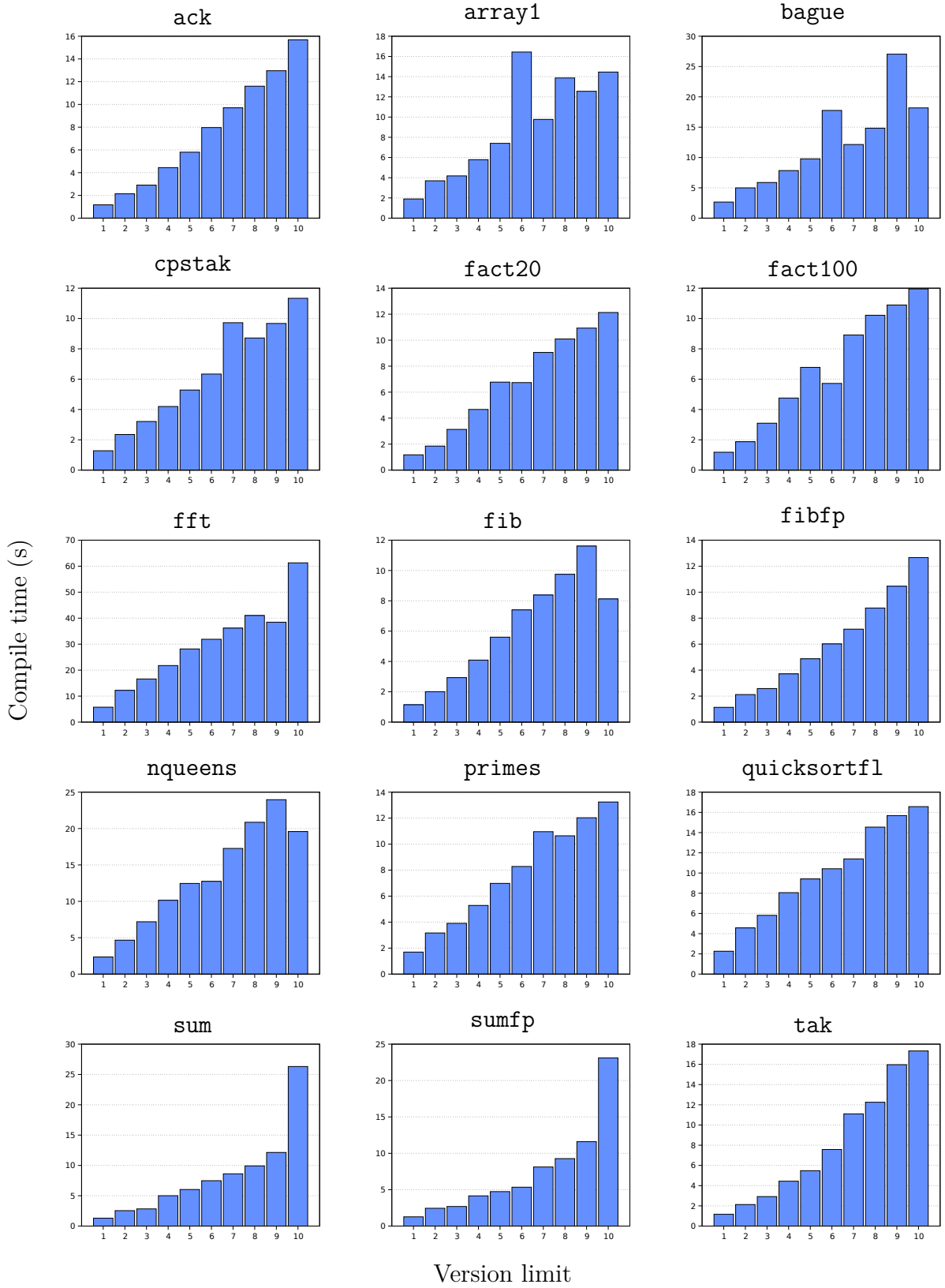


Figure 40 – Compile time micro benchmarks: arithmetic and hyperfunction operators.

Compile Time Micro Benchmarks: Similarity/Hyperfunction

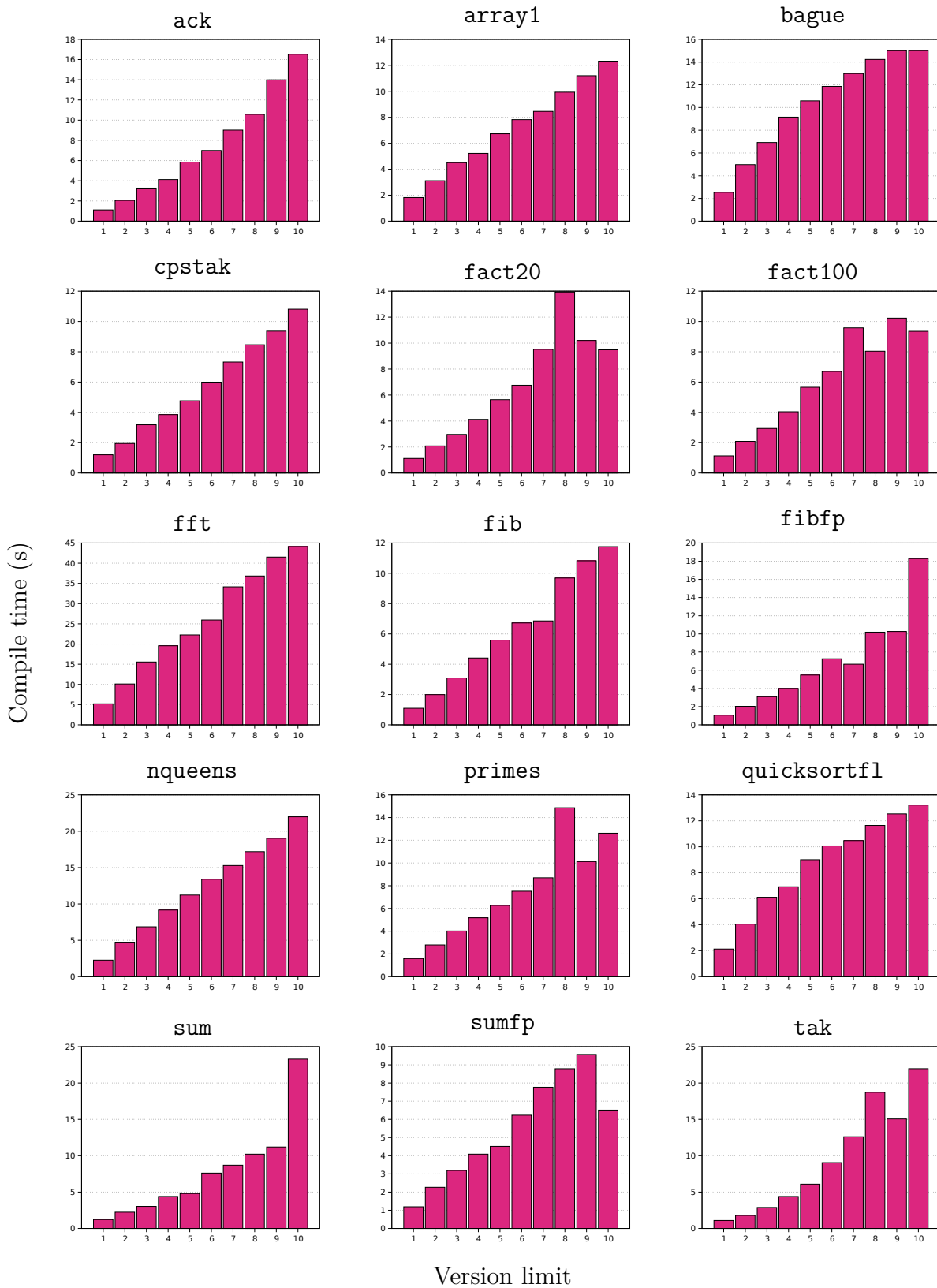


Figure 41 – Compile time micro benchmarks: similarity and hyperfunction operators.

Compile Time Micro Benchmarks: Random/Hyperfunction

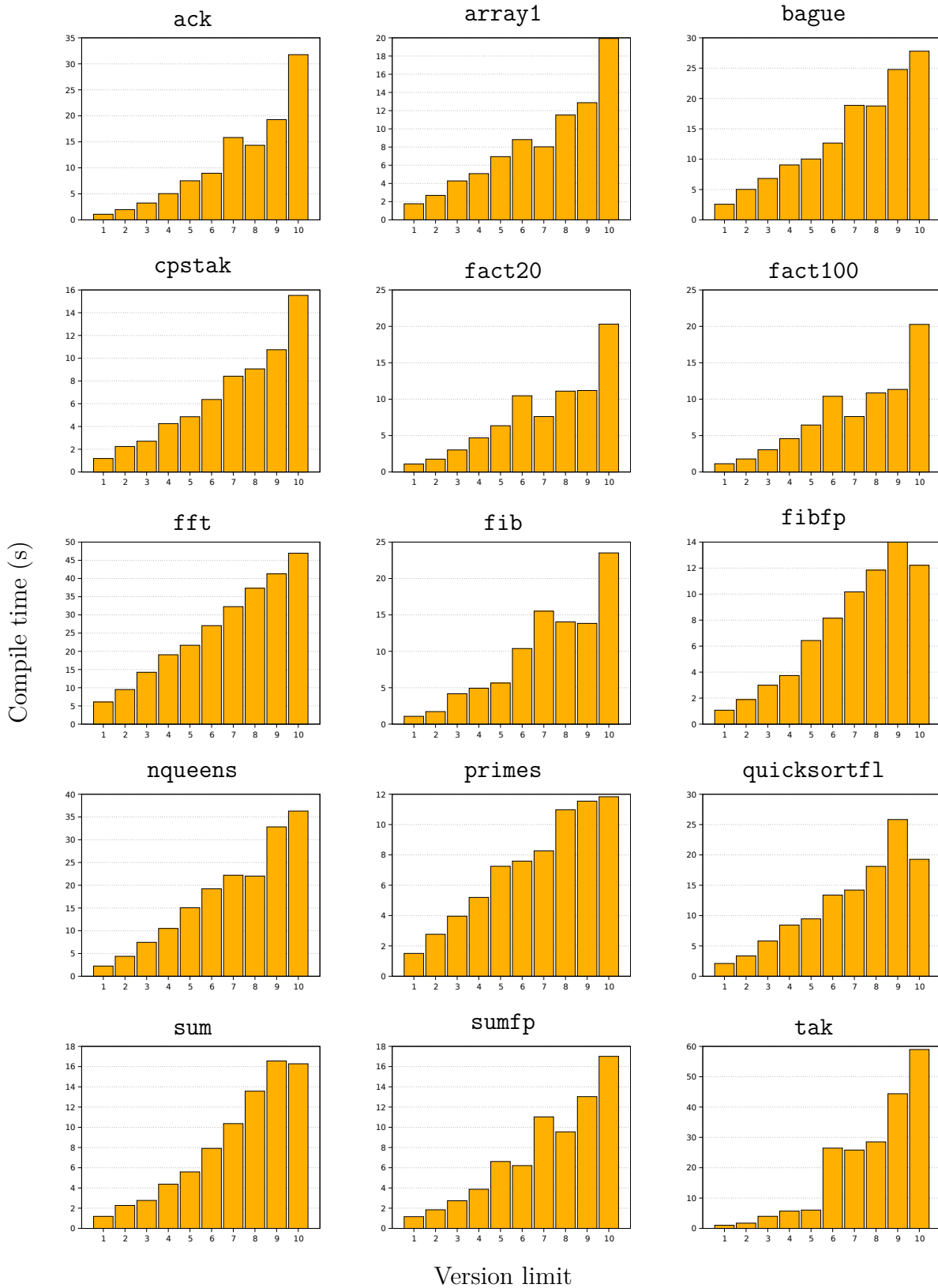


Figure 42 – Compile time micro benchmarks: random and hyperfunction operators.

Compile Time Macro Benchmarks: Arithmetic/Inlined

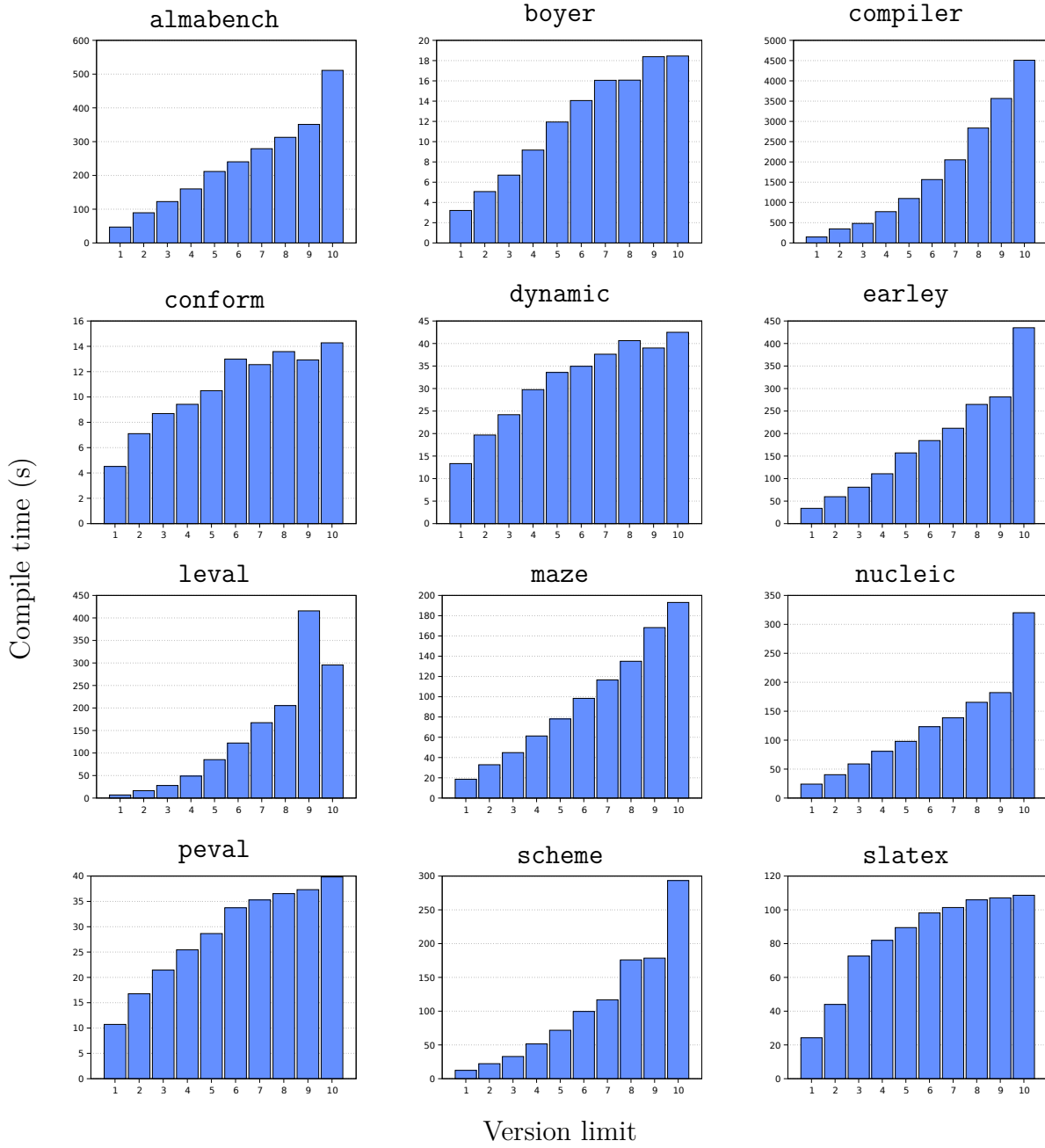


Figure 43 – Compile time macro benchmarks: arithmetic and inlined operators.

Compile Time Macro Benchmarks: Similarity/Inlined

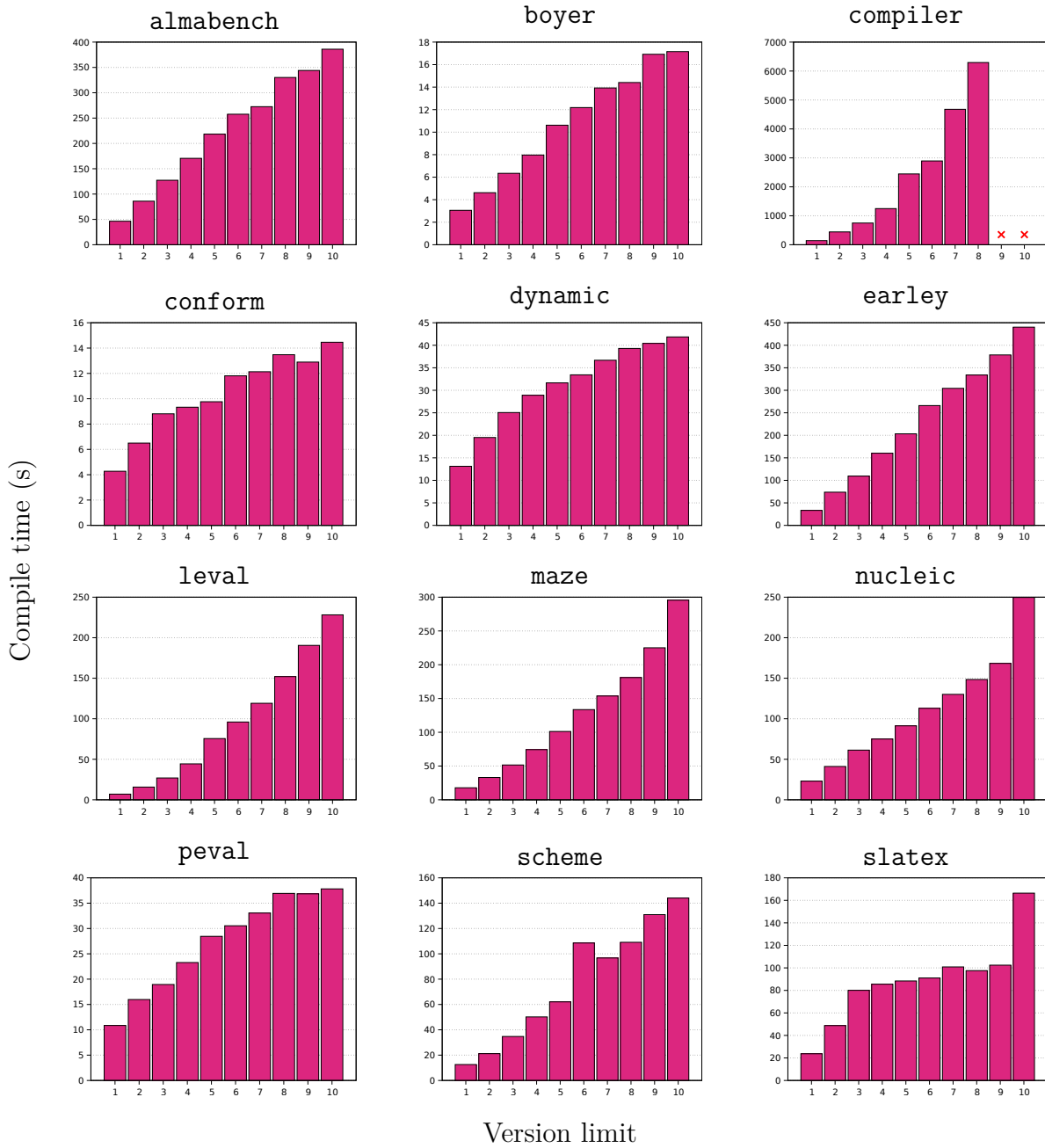


Figure 44 – Compile time macro benchmarks: similarity and inlined operators.

Compile Time Macro Benchmarks: Random/Inlined

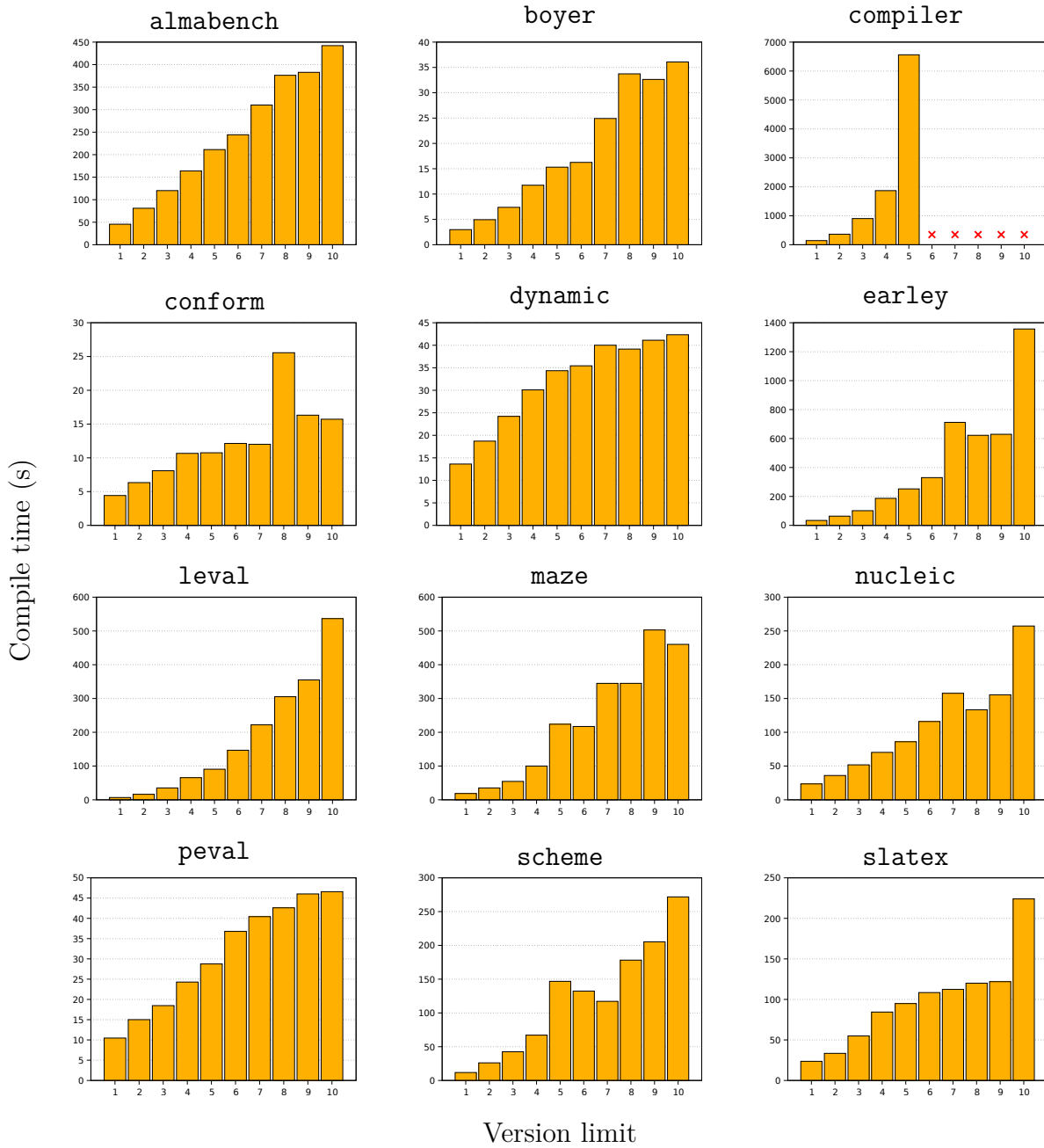


Figure 45 – Compile time macro benchmarks: `random` and `inlined` operators.

Compile Time Macro Benchmarks: Arithmetic/Hyperfunction

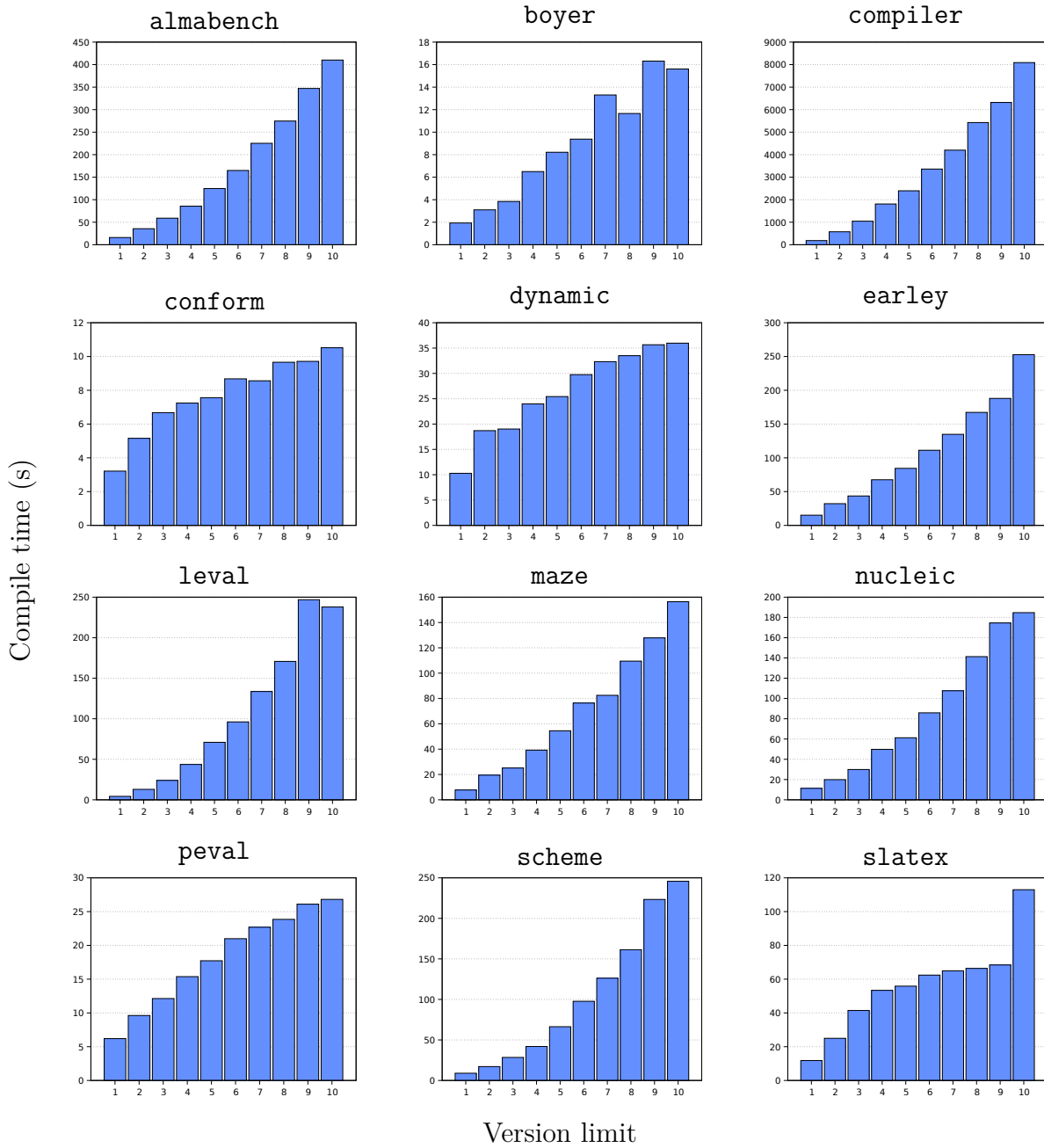


Figure 46 – Compile time macro benchmarks: arithmetic and hyperfunction operators.

Compile Time Macro Benchmarks: Similarity/Hyperfunction

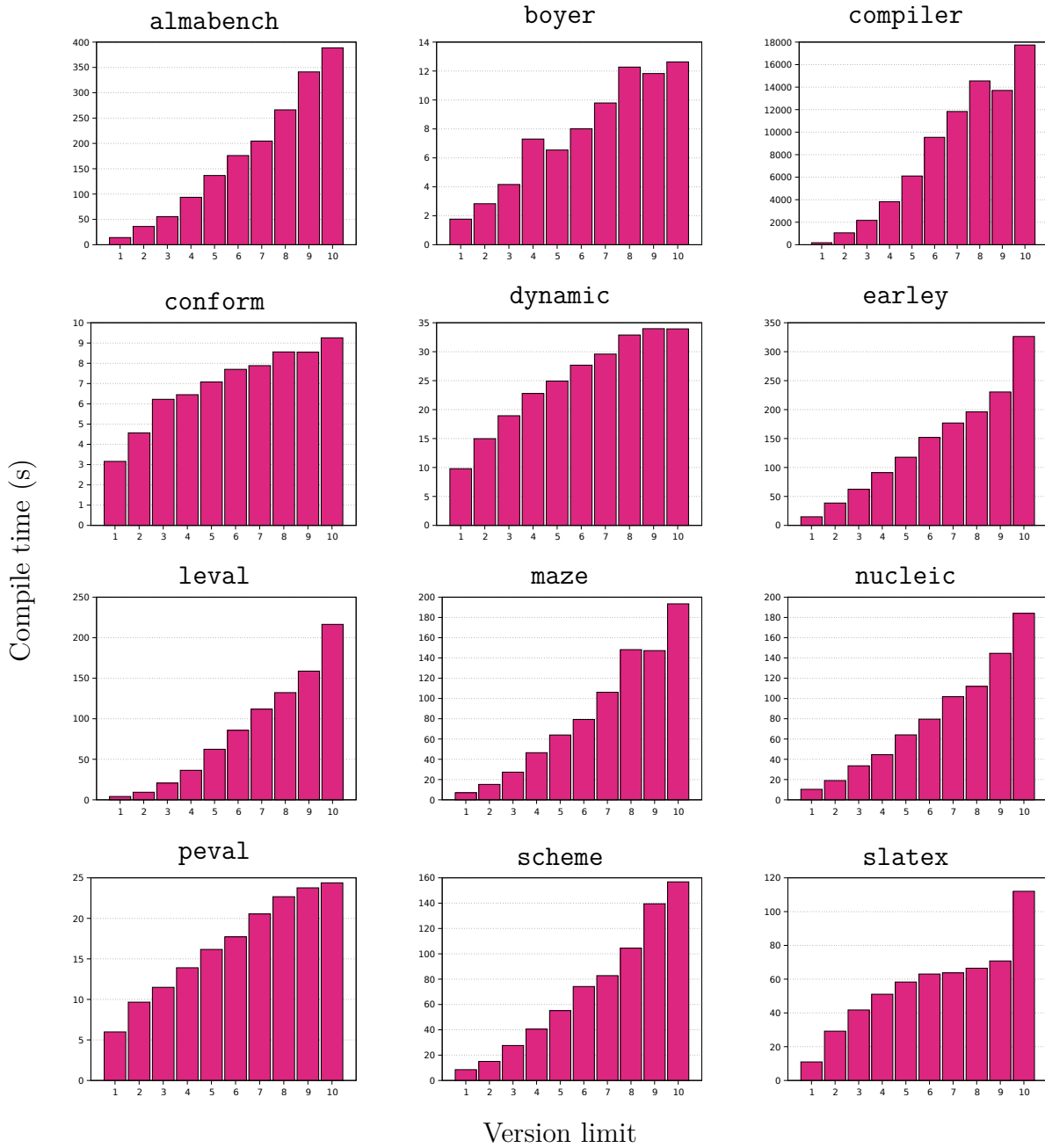


Figure 47 – Compile time macro benchmarks: similarity and hyperfunction operators.

Compile Time Macro Benchmarks: Random/Hyperfunction

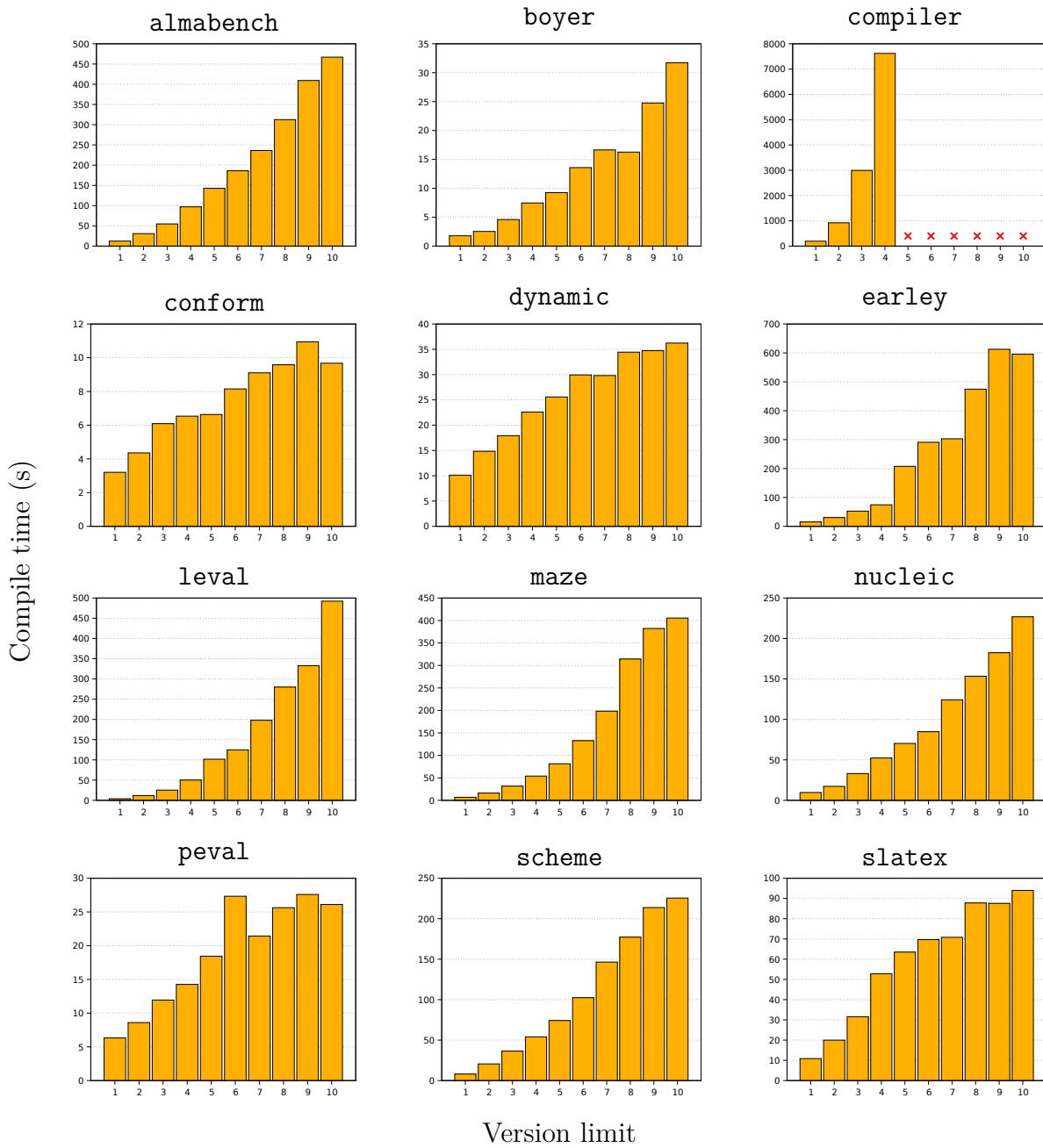


Figure 48 – Compile time macro benchmarks: **random** and **hyperfunction** operators.

Appendix D

Static Basic Block Versioning Paper

This appendix contains the *Static Basic Block Versioning* paper published at the *European Conference on Object-Oriented Programming 2024* (ECOOP'24) [42], the first published work on Static Basic Block Versioning (SBBV), discussed in Chapter 2.

The paper contains discussions of variable aliasing, integer interval analysis, elimination of vector bounds checks, and implementation details of SBBV in the Gambit and Bigloo Scheme compilers, which were not included in the main body of this thesis. As the first work on SBBV, however, it uses a suboptimal merge heuristic, the `similarity` heuristic, later found to be poor (see Chapter 3), and leaves the exploration of heuristics as future work, addressed in this thesis. It also lacks a thorough discussion of the single-source reachability algorithm underlying SBBV. As such, this paper is better read as a preface to Chapter 2.

Contribution Statement

The candidate is the primary author of this paper, implemented the Static Basic Block Versioning algorithm in the Gambit Scheme compiler, and conducted the experimental evaluation. The algorithm design and writing of the paper were shared equally among all authors. The implementation in the Bigloo Scheme compiler was carried out by the third author.

Static Basic Block Versioning

Olivier Melançon  

Université de Montréal, Canada

Marc Feeley   

Université de Montréal, Canada

Manuel Serrano   

Inria/UCA, Inria Sophia Méditerranée, Sophia Antipolis, France

Abstract

Basic Block Versioning (BBV) is a compilation technique for optimizing program execution. It consists in duplicating and specializing basic blocks of code according to the execution contexts of the blocks, up to a version limit. BBV has been used in Just-In-Time (JIT) compilers for reducing the dynamic type checks of dynamic languages. Our work revisits the BBV technique to adapt it to Ahead-of-Time (AOT) compilation. This Static BBV (SBBV) raises new challenges, most importantly *how to ensure the convergence of the algorithm* when the specializations of the basic blocks are not based on profiled variable values and *how to select the good specialization contexts*. SBBV opens new opportunities for more precise optimizations as the compiler can explore multiple versions and only keep those within the version limit that yield better generated code.

In this paper, we present the main SBBV algorithm and its use to optimize the dynamic type checks, array bound checks, and mixed-type arithmetic operators often found in dynamic languages. We have implemented SBBV in two AOT compilers for the Scheme programming language that we have used to evaluate the technique's effectiveness. On a suite of benchmarks, we have observed that even with a low limit of 2 versions, SBBV greatly reduces the number of dynamic type tests (by 54% and 62% on average) and accelerates the execution time (by about 10% on average). Previous work has needed a higher version limit to achieve a similar level of optimization. We also observe a small impact on compilation time and code size (a decrease in some cases).

2012 ACM Subject Classification Software and its engineering → Just-in-time compilers; Software and its engineering → Source code generation; Software and its engineering → Object oriented languages; Software and its engineering → Functional languages

Keywords and phrases Compiler, Ahead-of-Time Compilation, Optimization, Dynamic Languages

Digital Object Identifier 10.4230/LIPIcs.ECOOP.2024.28

1 Introduction

Optimizing compilers perform various analyses to discover properties of the program that are preconditions for performing optimizations. In a Just-In-Time (JIT) compiler, the cost of these analyses and optimizations is a critical issue as the time they take becomes part of the program's execution time. The use of expensive analyses and optimizations incur a long *warm-up* where the first part of a program's execution is sluggish and the program may even terminate before it has reached an optimization steady state.

Basic Block Versioning (BBV) is an optimization approach that strikes a balance between the optimization cost and the speed of the generated code to achieve a fast warm-up time and reasonably good execution speed. BBV has been used in JIT compilers for dynamically typed programming languages; in research compilers for JavaScript [5, 6] and Scheme [27, 28], and it is now used successfully in production in the official Ruby implementation [7, 8].

BBV uses the program's Control Flow Graph (CFG) created by the compiler as a template for creating a specialized CFG. For this, BBV traverses the CFG starting at its entry point while keeping track of the *context* that contains program properties of relevance



© Olivier Melançon, Marc Feeley, and Manuel Serrano;
licensed under Creative Commons License CC-BY 4.0

38th European Conference on Object-Oriented Programming (ECOOP 2024).

Editors: Jonathan Aldrich and Guido Salvaneschi; Article No. 28; pp. 28:1–28:27



Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

for specialization, such as the type of the values contained in the live variables. Each basic block has a set of contexts. The information contained in this set is *conservative*: for each possible program state when that basic block is reached during a program execution, there must be at least one context consistent with that state. Due to its conservative nature, it is allowed to have unreachable contexts in the set. In principle, each basic block of the original CFG could be specialized to all contexts in its context set, including unreachable contexts. The specialized versions of a basic block may contain optimizations that are valid in the corresponding context, such as the elimination of type checks when the type of a value has been determined at an earlier point in the execution.

An important concern is that multiple specialized copies of each basic block may be created, leading to a larger amount of code (*bloat*) and a longer compile time, load time, and execution time (due to the reduced performance of the instruction cache, among other reasons). In theory the bloat can be exponential in the size of the program.

Previous works use the same approach to this issue: a cap is placed on the number of versions for each basic block (*i.e.*, the number of versions is no greater than N , typically a small number like 5 or 10). In a JIT compiler the versions of a basic block are generated as the program's execution advances and reaches a basic block with a new context. This variant of BBV is called *lazy* BBV. When a new context is encountered and this would cause the version limit to be reached, a version specialized to that context must not be created because it would prevent another specialization if one was needed later in the execution. Instead, a *fully generic* version that covers all possible contexts is created as the last version and is used whenever a new version would be needed. Lazy BBV is relatively simple to implement but it has some important limitations:

- **Lazy BBV is a greedy algorithm.** The versions that are generated before the generic version, which are the first ones encountered at execution time, may not be the versions that are part of hot code. For example, if some function is used both during the initialization phase and in the main part of the program, then the specializations will be focused on what happens in the initialization phase. This function may be hot code when called from the main part of the program in a new context and, because the version limit is reached, it will be using the generic version (likely the slowest of them all).
- **High specialization is hard to achieve reliably.** The precision of the versioning context, *i.e.*, the number and information content of the program properties it tracks, has a direct impact on how quickly the slow generic version is used. For example, a precise versioning context that tracks not only the type but the range of values of an integer loop iteration variable starting at 1 and incremented at each iteration, will be able to create specialized versions of the first few iterations of the loop body (one version for each specific value of the iteration variable below N). This will not work well for loops that have a large number of iterations, because the iterations N and above will be handled by a slow generic version of the loop body. On the other hand, a context of this precision will work very well for programs where the loops have fewer than N iterations because BBV will completely unroll these loops. The BBV implementer will have to choose a moderate precision of the versioning context to avoid using the generic version too quickly, and consequently this will miss optimizations in some cases, such as total loop unrolling.
- **It requires JIT compilation.** The nature of a JIT compiler makes it easy to ensure that versions for unreachable contexts are never created. Unfortunately, this entails a warm-up time at execution, and in some use cases JIT compilation is not an option. In [5], an *eager* variant of BBV suitable for an Ahead-of-Time (AOT) compiler was described and compared to lazy BBV. That implementation of eager BBV yielded comparatively

poor speed and bloat because specialization is not guided by the actual need of a program execution and parts of the CFG that are explored are not typically executed, such as error cases and out-of-line handlers. Consequently, the specialized versions created before the limit is reached are more likely to be irrelevant at improving execution speed.

In this paper we describe a new design for a BBV algorithm suitable for an AOT compiler that mitigates these limitations. Our algorithm also traverses the CFG to determine which contexts reach each basic block. The first main difference with previous work is the handling of the version limit. When a new context is encountered and this would cause the version limit to be exceeded, the algorithm heuristically chooses a pair of contexts reached for that basic block and replaces them by a *merged* context that is more conservative than the contexts in the pair (in other words, a more general context). The algorithm continues traversing the CFG until a fixed-point is reached, *i.e.*, no new versions need to be created. The use of context merging allows contexts to be very precise at first, and it is the algorithm that reduces the precision as needed to keep the number of versions within the limit. The second main difference with previous work is the refinement of the notion of types to integer intervals to allow the BBV optimization to remove integer arithmetic overflow checks and array indexing bound checks.

In the next section, we present our algorithm and discuss its termination. In Section 3 we extend the algorithm with more precise contexts. The implementation in two mature compilers is explained and evaluated in Section 4. Related work is given in Section 5.

2 The Static BBV Algorithm

In this section we present the *Static* BBV (SBBV) algorithm. We will start with an overview by illustrating the algorithm’s behavior using the traditional `find` function that many dynamic and functional languages provide.

2.1 SBBV by Example

The `find` function takes a predicate and a list of values and it returns the first element of the list that satisfies the predicate or false if no such element is found. In the Scheme programming language [17], which we use throughout this paper, it can be defined as:

```

1 (define (find p x)           ;; p is the predicate and x is the list to search
2   (if (pair? x)              ;; is the list non-empty?
3     (if (p (car x))           ;; call predicate on the first element
4       (car x)                 ;; return it if it satisfies the predicate
5       (find p (cdr x)))       ;; otherwise, continue searching the rest of the list
6     #f))                     ;; return false when no element in the list satisfies the predicate

```

The safety of this code is guaranteed by verifying the validity of the arguments of the primitive operations at run time. In this example, the primitive operations for which a type verification is needed are the `car` and `cdr` accessors (lines 3-5) and the function invocation of the predicate (line 3). In safe mode, a Scheme compiler adds the required dynamic checks to the code, making the possible points of failed safety checks explicit:

```

1 (define (find p x)
2   (if (pair? x)
3     (if ((if (procedure? p) p (fail)) (if (pair? x) (car x) (fail)))
4       (if (pair? x) (car x) (fail))
5       (find p (if (pair? x) (cdr x) (fail))))
6     #f))

```

Here we use calls to the `fail` function to indicate cases where execution cannot continue due to a failed verification. The `fail` function is special in that it never returns.

One might expect a smart compiler, such as one implementing *occurrence typing* [32], to discover that the type tests on x are redundant, but let us assume that no such optimization is applied. This is done for illustrative purpose and because one of our objectives is to show that SBBV subsumes other optimization techniques, such as occurrence typing.

The unoptimized CFG of `find` is displayed in Figure 1a. SBBV will produce an optimized version of that CFG with fewer dynamic checks. For that, it propagates the information about variables in order to produce *specialized* versions of the basic blocks. For instance, block #12 (Figure 1a) checks that the end of the list is not yet reached by testing if x is a pair. In the positive branch, that is the path starting at block #4, it is known to be a pair and no further type tests are needed to ensure the correct execution until an assignment to x occurs (*i.e.*, the other tests that x is a pair are redundant).

The CFG produced by SBBV is shown in Figure 1b. We observe that SBBV has isolated the first iteration of the loop of the `find` function and the other iterations are handled by the loop formed by the subgraph $\{ \#23, \#25, \#27, \#29, \#31, \#32 \}$. In that loop the type of p is never tested because it has been tested in the first iteration before entering the loop handling the other iterations, and the argument x is only tested once per iteration, which is a necessary part of the loop termination logic. We also observe that for this simple example, SBBV has produced an optimal CFG in the sense that in a dynamic context with no global knowledge about the variable types and the data structure types, it executes the minimum number of dynamic checks required to ensure a safe execution. In Section 4.2 we evaluate the number of type tests SBBV is able to remove on more realistic programs.

In the rest of this section, we present and explain the algorithm that transforms the graph of Figure 1a into that of Figure 1b.

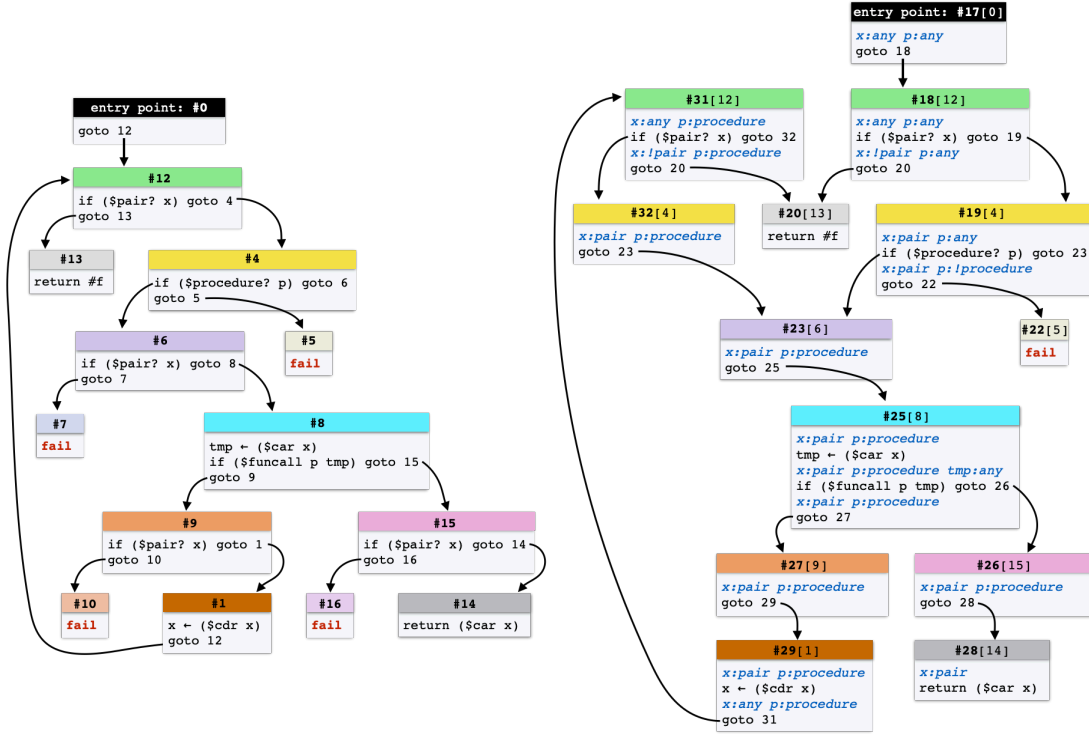
2.2 The Algorithm

SBBV specializes a CFG, which might either denote the whole program to be compiled or only a fragment of it. For instance, it can be decided to use SBBV for specializing each function in isolation or to specialize the whole program at a time. The main function of the algorithm (Algorithm 1) takes a basic block to specialize and the initial context used for that specialization as parameters. The data structures it uses are presented in Figure 2. A context is a mapping of variable names to value information. In this section these mappings associate variables to types. We will see in Section 3.2 that contexts can contain more precise type information. The algorithm specializes each instruction of the basic block and it recursively specializes the blocks that follow the basic block currently under specialization.

The algorithm performs a breadth-first traversal of the CFG. For that, it uses a work queue where it pushes the basic blocks and contexts that need further specializations. Specializing a block may cause the algorithm to specialize new blocks. For instance, when the algorithm scans the block #12 of Figure 1a, it discovers that in the positive branch the variable x is known to be a pair and then, it pushes onto the work queue the demand of a new specialization of the block #4 for the context $\{ x \mapsto \text{pair} \}$.

If the block extracted from the queue has not been merged (we explain in a moment what it means for a block to be merged), then it is specialized (line 10). This, in turn, can add new pending specializations to the queue. The algorithm proceeds until the queue is empty.

To ensure the convergence of the algorithm, it is enough to ensure that the function `BLOCKNEWVERSION` (Algorithm 2) pushes a new block onto the queue if and only if no such block has already been specialized for the requested context as the contexts contain a finite number of variable to value mappings. However, this convergence criteria is not enough in practice, the code size expansion of the specialization should also be controlled. This is the



(a) Original unspecialized CFG.

(b) Specialized CFG.

Figure 1 The original and specialized CFGs of the `find` function. Basic blocks have a numeric label for easy reference. In the specialized CFG, basic blocks have the same color as the original block they were specialized from (whose label is in square brackets). For instance, block #19 is a specialized version of block #4 of the original CFG. In the specialized CFG, the contexts of the specializations are displayed in blue. It can be observed that a given block can be specialized multiple times. For instance, block #12 has been specialized twice (blocks #18 and #31) and block #4 has been specialized twice (blocks #19 and #32). Blocks #7, #10, and #16 have no version in the specialized CFG because they do not have a reachable specialized context. Blocks #9 and #15 have been specialized to an unconditional jump because the `($pair? x)` test is known to be true.

context: $ctx, ctx_0, \dots$

- *types*: a list of mappings of variable to type information

basic block: $bb, bb_0, \dots$

- *instrs*: a list of instructions
- *versions*: a list of specialized basic blocks

specialized basic block: $bs, bs_0, \dots$

- *bb*: the corresponding basic block
- *merge*: a specialized basic block into which it has been merged or false
- *ctx*: a context

Figure 2 The data structures used in the SBBV algorithm.

■ **Algorithm 1** Main algorithm.

```

1: function SBBV( $bb_0, ctx_0$ )  $\triangleright$  specialize a CFG starting with the block  $bb_0$ 
2:    $wq \leftarrow \text{empty\_queue}$   $\triangleright$  create a fresh queue used for this specialization
3:    $bs_0 \leftarrow \text{BLOCKNEWVERSION}(bb_0, ctx_0, wq)$   $\triangleright$  push the request for specialization of  $bb_0$ 
4:   while  $\neg wq.\text{isempty}()$  do
5:      $bs \leftarrow wq.\text{pop}()$   $\triangleright$  fetch the first bb of the queue
6:      $bb \leftarrow bs.bb$   $\triangleright$  get the original unspecialized bb
7:     if  $|\{\forall b \in bb.\text{versions}, \neg b.\text{merge}\}| > \text{VERSION\_LIMIT}(bb)$  then
8:        $\text{BLOCKMERGESOME}(bb, wq)$   $\triangleright$  too many specializations, merge
9:     if  $\neg bs.\text{merge}$  then
10:       $\text{BLOCKSPECIALIZE}(bs, wq)$   $\triangleright$  specialize the block only if not already merged
11:   return  $bs_0$   $\triangleright$  return the specialization of the initial block

```

purpose of the test at line 7 of Algorithm 1. If the number of specialized versions of a single block exceeds a threshold, which is a parameter of the algorithm, some specializations of that block must be merged (see line 8).

The ancillary function `BLOCKNEWVERSION` is responsible for creating new blocks to be specialized. First, it checks if the requested block already exists, in which case it returns it. Otherwise, it creates a fresh version and pushes it onto the queue (line 9). Note that at this stage the instructions of the block are not scanned nor specialized. Pushing the block onto the queue is a mere request for specialization. It might be the case that at the moment where the block will be popped from the queue that this block has been merged into a less specialized version. This happens to prevent code size explosion.

■ **Algorithm 2** Create or fetch a specialized version.

```

1: function BLOCKNEWVERSION( $bb, ctx, wq$ )
2:   if  $ctx \in bb.\text{versions}$  then
3:     return  $\text{BLOCKLIVE}(bb.\text{versions}[ctx])$   $\triangleright$  return the already specialized block
4:   else
5:      $bs \leftarrow \text{new basic block}$   $\triangleright$  create a fresh empty basic block
6:      $bb.\text{versions}[ctx] = bs$   $\triangleright$  connect the new block and the parent block
7:      $bs.bb = bb$   $\triangleright$  initialize the new block
8:      $bs.ctx = ctx$ 
9:      $wq.\text{push}(bs)$   $\triangleright$  push it onto the queue for future specialization
10:   return  $bs$ 

```

The utility function `BLOCKLIVE` returns the first specialized version of a block that has not been merged into a more general version.

■ **Algorithm 3** Follow a chain of merged blocks.

```

1: function BLOCKLIVE( $bs$ )
2:   if  $bs.\text{merge}$  then
3:     return  $\text{BLOCKLIVE}(bs.\text{merge})$ 
4:   else
5:     return  $bs$ 

```

When the number of specializations of a basic block bb exceeds `VERSION_LIMIT( $bb$ )`, some contexts need to be merged. Note that we express the version limit as a function of the basic block to allow the algorithm to adapt the limit to different types of basic blocks, such as

those marked by the compiler front-end as probably benefiting from more specialization. This function could simply return a constant value, as we have done in our experiments. Context merging is done by the function `BLOCKMERGESOME` (Algorithm 4). It selects two versions not already merged (line 2), computes the union of the two corresponding contexts (line 3), and then replaces the merged blocks in the CFG (line 14). Merging is only triggered when a version is removed from the work queue (Algorithm 1, line 5). This allows the number of versions to temporarily exceed the version limit. This *delayed merging* increases the choices available to the selection heuristic and possibly leads to better merges.

Merging a block in the CFG entails deleting all incoming edges of the merged blocks to redirect them to the block resulting from the merge. Any deletion of an edge may render some specialized block unreachable from the CFG's entry point. Similarly, an added edge can make some previously unreachable block reachable anew. Keeping track of unreachable blocks is required since those no longer have to be traversed and must not be considered when selecting versions to merge in `BLOCKMERGESOME` (line 2). This can be done efficiently by maintaining an Even-Shiloach tree [14] of the CFG to help the SBBV algorithm detect whenever the reachability of a specialized block changes. Any block made unreachable by a merge is marked so that it is no longer considered in the merge selection and CFG traversal.

■ **Algorithm 4** Merge blocks.

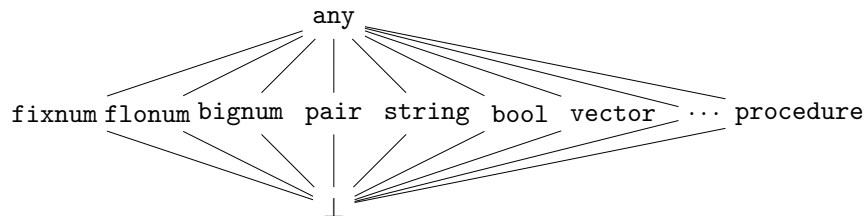
```

1: function BLOCKMERGESOME( $bb, wq$ )
2:    $bs_1, bs_2 \leftarrow \Theta^2 \{ \forall b \in bb.\text{versions}, \neg b.\text{merge} \}$            ▷ select two versions to merge
3:    $ctx \leftarrow bs_1.\text{ctx} \sqcup bs_2.\text{ctx}$                                    ▷ merge the two corresponding contexts
4:    $bs \leftarrow \text{BLOCKNEWVERSION}(bb, ctx, wq)$                            ▷ create a new block for the merge
5:   if  $bs_1 \equiv bs$  then                                                 ▷ replace the merged blocks
6:   |   BLOCKMERGEANDREPLACE( $bs_2, bs$ )
7:   else if  $bs_2 \equiv bs$  then
8:   |   BLOCKMERGEANDREPLACE( $bs_1, bs$ )
9:   else
10:  |   BLOCKMERGEANDREPLACE( $bs_1, bs$ )
11:  |   BLOCKMERGEANDREPLACE( $bs_2, bs$ )
12: function BLOCKMERGEANDREPLACE( $obs, mbs$ )
13:  |    $obs.\text{merge} \leftarrow mbs$                                          ▷ mark that obs is merged into mbs
14:  |   replace  $obs$  with  $mbs$  in the CFG                                   ▷ patch the CFG

```

The operator Θ^2 selects two unmerged specializations for the block bb . For the sake of the correctness of the algorithm, this operator might select any two versions. For instance, it could select two random versions. Of course, a better operator would positively impact the result of the compilation. In Section 4.3 we present the operator we have used so far.

The behavior of the merge operator $\sqcup$ is independent of the SBBV algorithm but it must produce a new context that at least encompasses the contexts of the merged blocks. The natural solution is to use a lattice for organizing the information associated with variables and to go up some level for each merge. We will use the following lattice:



Note that a number can be a **flonum** (floating point numbers), a **fixnum** (small integers that fit in a machine word), or a **bignum** (integers that don't fit in a machine word). While this would be a good match for JavaScript numerical types, for a full Scheme, or Python, implementation there would also be a representation for rational and complex numbers, but we will ignore them to simplify the discussion.

Consider the merge of two contexts mapping a variable v respectively to the types **fixnum** and **flonum**. The merge operation should produce a context mapping v to **any**. In Section 3.2 we show the use of a more precise lattice for representing properties.

The last part of the SBBV algorithm is in charge of specializing blocks. It merely creates a new block where the instructions have been specialized one by one.

■ **Algorithm 5** Specialize a block and its instructions.

```

1: procedure BLOCKSPECIALIZE( $bs, wq$ )
2:    $bb \leftarrow bs.bb$ 
3:    $ctx \leftarrow bs.ctx$ 
4:   for all  $i \in bb.instrs$  do
5:      $(ni, nctx) \leftarrow \text{INSSPECIALIZE}(i, ctx, wq)$ 
6:      $bs.instrs \leftarrow bs.instrs + ni$ 
7:    $ctx \leftarrow nctx$ 

```

Specializing an instruction that implements a type test produces a new context. For instance, when specializing the block #12 of Figure 1a, in the positive branch, the argument x is known to be a pair. The block #4 is then specialized with a context that reflects that information, which is then propagated to following blocks. Conversely, on the negative branch, the argument x is known not to be a pair. This information too, is propagated to following blocks. To handle these evolutions of the contexts, the procedure BLOCKSPECIALIZE updates the context it uses for specializing the instructions after each iteration (Algorithm 5, line 7).

The function INSSPECIALIZE selects a specialized appropriate for the instruction.

■ **Algorithm 6** Specialization of an instruction.

```

1: function INSSPECIALIZE( $i, ctx, wq$ )
2:   if  $i.kind$  is “goto” then return INSSPECIALIZEGOTO( $i, ctx, wq$ )
3:   else if  $i.kind$  is “if” then return INSSPECIALIZEIF( $i, ctx, wq$ )
4:   else if  $i.kind$  is ... then ...
5:   else return ( $i, ctx$ )

```

The specialization of a **goto** instruction mostly consists in forwarding the specialization context ctx to the target of the instruction. For instance, when specializing the **goto** in block #1 of Figure 1a with the context $\{x \mapsto \text{pair}, p \mapsto \text{procedure}\}$ the **goto** instruction triggers the specialization of the block #12 with the same context. The instruction brings no new knowledge about the variables types so it returns an unmodified context.

■ **Algorithm 7** Specialization of **goto** instructions.

```

1: function INSSPECIALIZEGOTO( $i, ctx, wq$ )
2:    $ni \leftarrow i.dup()$ 
3:    $ni.target \leftarrow \text{BLOCKNEWVERSION}(i.target, ctx, wq)$ 
4:   return ( $ni, ctx$ )

```

The specialization of an `if` is more involved because this is where new knowledge is acquired and where requests for new specializations are emitted. If the test of the expression is not a type test, the specialization behaves as the specialization of a `goto` instruction (Algorithm 8, line 2). If the instruction implements a type check and if the context is such that the test always succeeds, then the instruction is replaced with a `goto` instruction which directly branches to the positive block (line 6). This is illustrated by the specialization of the block #6 of Figure 1a into the block #23 of Figure 1b. Conversely, if the test is known to evaluate to false, the instruction is replaced with a `nop` instruction.

■ **Algorithm 8** Specialization of `if` instructions.

```

1: function INSPECIALIZEIF( $i, ctx, wq$ )
2:   if  $\neg i.test$  is a typecheck then
3:      $ni \leftarrow i.dup()$ 
4:      $ni.target \leftarrow BLOCKNEWVERSION(i.target, ctx, wq)$ 
5:     return ( $ni, ctx$ )
6:   else if  $ctx.types.isTrue(i.test)$  then
7:      $ni \leftarrow new\ insGoto(i.target)$ 
8:     return INSPECIALIZE( $ni, ctx, wq$ )
9:   else if  $ctx.types.isFalse(i.test)$  then
10:     $ni \leftarrow new\ insNop()$ 
11:    return ( $ni, ctx$ )
12:   else
13:     $ni \leftarrow i.dup()$ 
14:     $ctx^+ \leftarrow ctx \cup \{ i.test.var \mapsto i.test.type \}$ 
15:     $ctx^- \leftarrow ctx \cup \{ i.test.var \mapsto \neg i.test.type \}$ 
16:     $ni.target \leftarrow BLOCKNEWVERSION(i.target, ctx^+, wq)$ 
17:    return ( $ni, ctx^-$ )

```

The most interesting situation is when the result of the type test cannot be inferred from the current context (line 12). In that case, two new contexts are created, in accordance to the narrowing rules for that test. The positive branch of the test will be specialized with a context reflecting the success of the type test and the context reflecting a negative result is returned to the procedure BLOCKSPECIALIZE (for instance, see the block #12 of Figure 1a that creates the context of the block #19 of Figure 1b).

3 Improved Specializations

In Section 2 we have presented the general SBBV algorithm. We have exposed its principles that we have illustrated with a simple type analysis that maps variable usages to types. In this section, we show how the algorithm can be extended to specialize the blocks according to more fine-grained information.

3.1 Variable Aliasing

The contexts used in the example in Figure 2 enables SBBV to specialize blocks according to the type of the variables. However, it does not keep track of variable aliases, which jeopardizes the benefit of the optimization. Compilers tend to introduce many temporaries for evaluating expressions and if these aliases are not handled efficiently by the SBBV specialization, what is learned about a variable's value will not be propagated to the other

context: $ctx, ctx_0, \dots$

- **types:** a list of mappings of variable to type information
- **equiv:** a list of equivalence classes

■ **Figure 3** Extended specialization contexts with variable class equivalence.

variables containing the same value. Thankfully, handling aliases merely requires extending the definition of the contexts and to handle the specialization of the `mov` instruction, which assigns a value to a variable. The new definition of the contexts is extended into that of Figure 3. The specialization of the `mov` instruction, that copies a variable into another and that is represented by the $\leftarrow$ operator in the CFGs, is given in Algorithm 9. For the sake of simplicity, this extension keeps track of aliasing of read-only variables only. Assigned variables are never treated as aliases of other variables. Also, not presented here, the specialization of the `if` instruction (Algorithm 8) is modified so that it also propagates the gathered type information to the variable’s aliases.

■ **Algorithm 9** Specialization of `mov` instructions.

```

1: function INSPECIALIZEMOV( $i, ctx, wq$ )
2:    $nctx \leftarrow ctx$ 
3:    $nctx.equiv[i.target] \leftarrow \emptyset$ 
4:   if  $i.source$  is a read-only variable then
5:      $nctx.equiv[i.target] \leftarrow \{i.source\}$ 
6:   return ( $i, nctx$ )

```

3.2 Specialization of Arithmetic Operations

The SBBV algorithm is general-purpose and it can be applied to other properties of the variables and values to go beyond type check removal. In this section we show how to leverage this flexibility to also specialize the basic blocks according to fixnum integer intervals $LO..HI$, where LO and HI are values in the fixnum range. This will allow the compiler to generate code using fixnum arithmetic operators that are fast (because they directly map to machine instructions) and removing overflow checks and bound checks.

The benefits of this extension can be illustrated on the expression $(+ \ x \ 1)$, which adds 1 to x , a very common operation in most programs. The specific operation executed depends on the type of x . The result could be a fixnum, a flonum, a bignum, or the operation could raise an exception if x is not a numerical type. Moreover, the result of $(+ \ x \ 1)$ will be a bignum if x is `maxfix`, the largest fixnum value. A similar dispatch is part of the semantics of most arithmetic operators $(-, *, \dots)$ and comparison operators $(=, <, \dots)$, and in the general case, such as $(+ \ x \ y)$, the dispatch is on the combination of types of x and y .

Optimizing compilers usually inline the handling of the most common cases, such as all operands being fixnums, and all operands being flonums, and defer the handling of the remaining cases to an out-of-line function. For example, the expression $(+ \ x \ 1)$ could be expanded by the compiler to this code:

```

(if ($fixnum? x)
  (or ($fx+? x 1)    ;; fixnum add 1 with overflow check (#f returned on overflow)
    ($+ x 1))        ;; call $+ function to handle bignum result case
  (if ($flonum? x)
    ($fl+ x 1.0)      ;; flonum add 1
    ($+ x 1)))        ;; call $+ function to handle other cases including errors

```

Here we use names prefixed with ‘\$’ to indicate internal operations of the system:

- $(\$fixnum? \ x)$ and $(\$flonum? \ x)$ test to see if x is a fixnum, or a flonum respectively.
- $(\$+ \ x \ y)$ is an addition function in the runtime system that handles all possible type combinations for x and y , including those that raise an exception.
- $(\$fl+ \ x \ y)$ adds two flonums to give a flonum result.
- $(\$fx+? \ x \ y)$ adds two fixnums to give a fixnum result or false in the case of an overflow. This operation includes an overflow check that makes it somewhat more expensive than $(\$fx+ \ x \ y)$ that does not check for overflow (note the absence of the trailing ‘?’).

When the CFG of the above expansion of $(+ \ x \ 1)$ is processed by the SBBV algorithm various optimizations can happen. If the context indicates that x is a fixnum then the $(\$fixnum? \ x)$ test in the specialized basic block is an unconditional jump to the CFG of $(\text{or } (\$fx+? \ x \ 1) (\$+ \ x \ 1))$, effectively removing the code that handles flonums, bignums, and other types. Moreover, if it is known that x is in the interval $LO..HI$ where $HI < \text{maxfix}$, then the result of $(\$fx+? \ x \ 1)$ is in the fixnum interval $LO + 1..HI + 1$, so an overflow is impossible, *i.e.*, $(\$fx+? \ x \ 1)$ is necessarily a fixnum. Consequently the CFG can be specialized to $(\$fx+ \ x \ 1)$, which is a machine integer addition with no overflow check.

Other languages, such as Ruby, support similar generic arithmetic, but more importantly, languages such as JavaScript [16] and Python that do not expose small integers require that the compiler be able to detect when operations can be implemented as fixnum operations. Hence, these fast implementations must be able to detect when an operation overflows and, exactly as Scheme does, promote the number in such a case (JavaScript promotes them to IEEE floating point numbers, Python to bignums).

To extend the analysis, we refine the definition of specialization contexts (Figure 4) and we refine the lattice of values (Figure 5). Fixnum values are now represented with intervals.

Handling numerical values requires us to modify the specialization of the `if` and the *fixnum arithmetic with overflow* instructions, such as $\$fx+?$. The first one must compute new interval approximations to be propagated in the positive and negative branches by using interval narrowing techniques [11]. The second one must implement arithmetic operations over intervals [23], such as $x_{LO}..x_{HI} + y_{LO}..y_{HI} = (x_{LO} + y_{LO})..(x_{HI} + y_{HI})$.

■ **Algorithm 10** Specialization of `if` instructions with numerical values.

```

1: function INSSPECIALIZEIF( $i, ctx, wq$ )
2:   if  $i$  is an integer comparison then
3:      $(ctx^+, ctx^-) \leftarrow \text{intervalNarrowing}(i, ctx)$ 
4:      $ni \leftarrow i.\text{dup}()$ 
5:      $ni.\text{target} \leftarrow \text{BLOCKNEWVERSION}(i.\text{target}, ctx^+, wq)$ 
6:     return  $(ni, ctx^-)$ 
7:   else
8:     as in algorithm 8

```

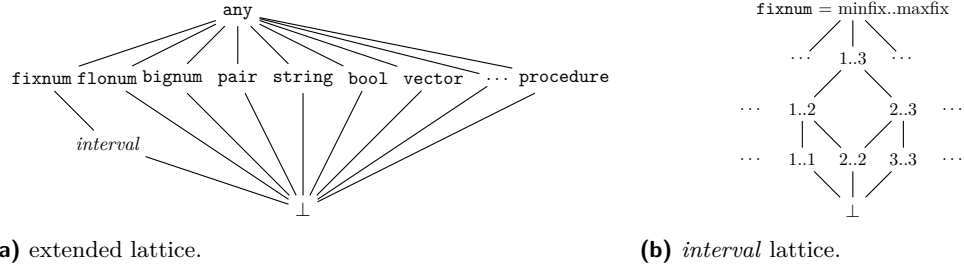
For instance, let us assume the specialization of the instruction “`if ($fx > i 3)`” in a context $\{i \mapsto \text{minfix}..10\}$. The new INSSPECIALIZEIF will generate the two new contexts $\{i \mapsto 4..10\}$ and $\{i \mapsto \text{minfix}..3\}$ for the positive and negative outcomes respectively.

The numerical operation specialization replaces a numerical operator, such as $\$fx+?$, whose result is known not to overflow with a faster operator that does not check for overflow, such as $\$fx+$. It must also implement some widening operation [10] in order to hasten the convergence of the algorithm. Without widening the algorithm would require a number of iterations proportional to the size of the interval representing numbers, which would be prohibitive. The widening is handled by the $\sqcup$ operator of the Algorithm 4 (see Section 2.2).

context: $ctx, ctx_0, \dots$

- types: a list of mappings of variable to type information
- equiv: a list of equivalence classes
- range: a list of mappings of variable to integer intervals

■ **Figure 4** Specialization contexts extended with integer intervals.



■ **Figure 5** Extending the lattice used for variable values to handle integer intervals.

3.3 Vector Support

The previous section showed how to extend SBBV to approximate integer values as intervals described with two integer bounds. While this enables the compiler to remove overflow checks, it is insufficient to remove bound checks of vector accesses as the length of a vector is generally unknown statically and cannot be represented with an exact integer value.

To handle vectors we introduce a refined representation of intervals. With this extension interval bounds can be represented with integers as before, but also with the symbolic value $\llbracket v \rrbracket - i$ that is equal to the length of vector v minus the integer offset $i \geq 0$. Here we assume that the length of a vector is always a nonnegative fixnum. For instance, the interval $0..\llbracket v \rrbracket - 1$ denotes all the nonnegative fixnums that are lower than the length of the vector v , in other words, the valid indexes of vector v . The upper bound of that interval, *i.e.*, $\llbracket v \rrbracket - 1$, itself denotes a fixnum in the interval $-1..\text{maxfix} - 1$.

We illustrate the benefit of this extension to SBBV with a variant of the `find` function presented in Section 2 that operates on vectors and lists:

```

1 (define (findv p x)
2   (if (vector? x)
3     (let ((len (vector-length x)))
4       (let loop ((i 0))
5         (if (< i len)
6             (let ((e (vector-ref x i)))
7               (if (p e)
8                   e
9                   (loop (+ i 1))))
10            #f)))
11   (find p x))

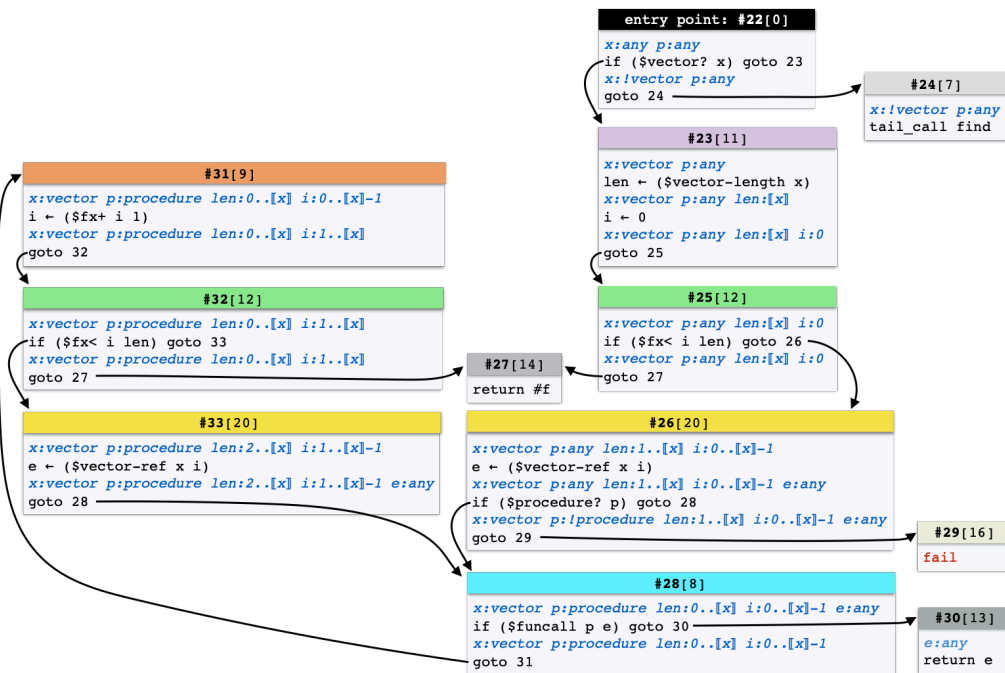
```

```

1 (define (findv p x)
2   (if ($vector? x)
3     (let ((len (if ($vector? x) ($vector-length x) ($fail))))
4       (let loop ((i 0))
5         (if (if (and ($fixnum? i) ($fixnum? len))
6             ($fx< i len)
7             (if (and ($flonum? i) ($flonum? len))
8                 ($fl< i len)
9                 ($< i len))))
10        (let ((e (if (and ($vector? x) ($fixnum? i)
11            ($fx>= i 0) ($fx< i ($vector-length x)))
12            ($vector-ref x i)
13            ($fail))))
14          (if ((if ($procedure? p) p ($fail)) e)
15              e
16              (loop (if (and ($fixnum? i) ($fixnum? 1))
17                  (or ($fx+? i 1) ($+ i 1))
18                  (if (and ($flonum? i) ($flonum? 1))
19                      ($fl+ i 1)
20                      ($+ i 1)))))))
21     ($fail)))
22 (find p x))

```

■ **Figure 6** The code of the `findv` function where all dynamic checks are explicit.



■ **Figure 7** The specialized CFG of the `findv` function showing that the index calculations are done entirely with fixnums with no overflow checks and no vector bound checks. For brevity, we write `[x]` instead of `[x] - 0`, and the interval `[x]..[x]` is abbreviated to `[x]`.

Figure 6 shows the code after the compiler has blindly expanded each operation to include all required dynamic checks. The refinements of SBBV presented in this section enables the compiler to create the specialized CFG shown in Figure 7, which is optimal (all bound checks and overflow checks have been removed). There is only a procedure check for parameter $\mathbf{p}$ in the first iteration of the loop, and only if parameter $\mathbf{x}$ is a non-empty vector.

This refinement does not impact the SBBV algorithm nor does it demand to change the specialization of the arithmetic instructions but it requires us to extend the interval operators to treat cases where at least one bound is a symbolic value. For example, in the case of the interval addition $x_{\text{lo}}..x_{\text{hi}} + y_{\text{lo}}..y_{\text{hi}} = (x_{\text{lo}} + y_{\text{lo}})..(x_{\text{hi}} + y_{\text{hi}})$, the addition of the lower bounds $(x_{\text{lo}} + y_{\text{lo}})$ is computed from the following rules, where i and j denote integer values, and v and w denote vector identifiers:

$$\begin{aligned} i +_{\text{lo}} j &\mapsto i + j \\ ([v] - i) +_{\text{lo}} j &\mapsto j - i \\ j +_{\text{lo}} ([v] - i) &\mapsto ([v] - i) +_{\text{lo}} j \\ ([v] - i) +_{\text{lo}} ([w] - j) &\mapsto -i - j \end{aligned}$$

and the addition of the upper bounds $(x_{\text{hi}} + y_{\text{hi}})$ is computed from the following rules, where *overflow* denotes an upper bound that is not a fixnum:

$$\begin{aligned} i +_{\text{hi}} j &\mapsto i + j \\ ([v] - i) +_{\text{hi}} j &\mapsto [v] - (i - j) && \text{if } i \geq j \\ ([v] - i) +_{\text{hi}} j &\mapsto \text{overflow} && \text{if } i < j \\ j +_{\text{hi}} ([v] - i) &\mapsto ([v] - i) +_{\text{hi}} j \\ ([v] - i) +_{\text{hi}} ([w] - j) &\mapsto \text{overflow} \end{aligned}$$

The narrowing operations for comparisons are similar to that of regular intervals, considering that vector lengths are themselves modelled as intervals from 0..maxfix. Below are the rules for the narrowing of $x < y$ from which the rules for the other comparisons can be derived:

Narrowing rule for: $x < y$ with $\{x \mapsto x_{\text{lo}}..x_{\text{hi}}, y \mapsto y_{\text{lo}}..y_{\text{hi}}\}$

Positive outcome: $\{x \mapsto x_{\text{lo}}.. \min_{\text{hi}}(x_{\text{hi}}, y_{\text{hi}} - 1), y \mapsto \max_{\text{lo}}(x_{\text{lo}} + 1, y_{\text{lo}})..y_{\text{hi}}\}$

Negative outcome: $\{x \mapsto \max_{\text{lo}}(x_{\text{lo}}, y_{\text{lo}})..x_{\text{hi}}, y \mapsto y_{\text{lo}}.. \min_{\text{hi}}(x_{\text{hi}}, y_{\text{hi}})\}$

$$\begin{aligned} \max_{\text{lo}}(x, y) &\mapsto x \text{ if } \text{val}_{\text{lo}}(x) > \text{val}_{\text{lo}}(y) \text{ else } y \\ \min_{\text{hi}}(x, y) &\mapsto x \text{ if } \text{val}_{\text{hi}}(x) < \text{val}_{\text{hi}}(y) \text{ else } y \end{aligned}$$

$$\begin{aligned} \text{val}_{\text{lo}}(i) &\mapsto i \\ \text{val}_{\text{lo}}([v] - i) &\mapsto i \\ \text{val}_{\text{hi}}(i) &\mapsto i \\ \text{val}_{\text{hi}}([v] - i) &\mapsto \text{maxfix} - i \end{aligned}$$

It is noteworthy that, as a result of the interval widening, some of the inferred intervals are somewhat conservative (for example at block #28 the interval for `len` could have been $1..[v]$). The widening loses some information but makes computing a fix-point faster.

4 Experiments

In this section we demonstrate the practicality of SBBV through experiments. To ensure that our results are not overly system specific, we have integrated an SBBV pass in the compilation pipeline of two existing Scheme compilers, Bigloo [19] and Gambit [20]. Both

of these are independently developed mature optimizing AOT Scheme to C compilers that use a CFG representation of the compiled program. Moreover these compilers are used as back-ends of optimizing compilers for JavaScript [31, 30] and Python [22]. Bigloo and Gambit implement a slew of features and classical optimizations such as constant-folding, function inlining, flat closures, and lambda-lifting. Any performance improvements would constitute a notable achievement given the many years of fine-tuning that went into their development. We put this in perspective in Section 4.5.

Adding SBBV to these compilers allows them to use the type, range and value of variables to perform flow sensitive code specialization that is tailored to the program logic, with low code bloat. The experiments have been designed to demonstrate further performance improvements by SBBV than by other optimization techniques implemented by these compilers.

We evaluate the impact of SBBV by applying it to a suite of benchmarks. Each benchmark is compiled with and without SBBV to measure its impact on the number of dynamic checks, program size, execution time, and compilation time. Section 4.1 provides a brief description of the benchmark suite. Benchmarks were executed on a machine with an Intel Core i7-7700K, 48 GB of RAM, and under Debian 10.13 with kernel version SMP Debian 4.19.269-1.

In order to measure SBBV’s impact on the number of dynamic checks, both compilers have been instrumented to count the number of dynamic checks during a program execution. Executions for measuring time and dynamic checks are done separately to ensure that counting checks does not affect the measured execution time. Execution time is measured by profiling each executable with “`perf stat`” to measure its execution real-time. Each benchmark is parameterized such that its execution lasts at least five seconds on our machine, and is repeated 50 times, removing the top and bottom 5 outliers. The parameters of each benchmark are provided as command-line arguments to ensure that the compiler does not optimize for specific values or types. All timing results in this section are the average execution time of each benchmark. The relative standard deviation of the execution time never exceeds 0.24% on macrobenchmarks, and 2.20% on microbenchmarks; consequently we omit standard deviations in figures to improve readability.

4.1 Benchmark Programs

Our benchmark suite combines programs from two sources: the R7RS benchmark suite [1] that is commonly used for evaluating the performance of Scheme systems, and benchmark programs used in [30] that have Scheme and JavaScript versions.

We use both macrobenchmarks and microbenchmarks, which we classify according to their size (fewer than 150 lines of code is a microbenchmark). These two classes are distinguished because microbenchmarks stress a narrow set of features and consequently are poor predictors of the overall performance of a system. We only use microbenchmarks as instruments for shedding light on specific behaviors of the SBBV algorithm.

Here is a brief description of the benchmark programs:

Macrobenchmarks:

- **almabench** (430 LOC): Compute the celestial coordinates of the sun at noon. Uses floating point numbers, vectors, and assignments.
- **boyer** (610 LOC): Prolog-like rule-directed rewriting engine. Uses pairs and symbols.
- **compiler** (11,740 LOC): Old version of the Gambit Scheme compiler generating M68000 code. Uses pairs, symbols, vectors, and strings.
- **conform** (490 LOC): Graph type checker using equivalence classes. Uses lists and strings.

- **dynamic** (2,350 LOC): Dynamic type inference for Scheme. Uses lists, symbols, and higher-order functions.
- **earley** (660 LOC): Earley parser parsing an ambiguous grammar. Uses vectors, lists, small integers and symbols.
- **leval** (560 LOC): Scheme interpreter based on closures. Uses lists, symbols, and higher-order functions.
- **maze** (740 LOC): Hexagonal grid maze generator. Uses vectors and small integers.
- **nucleic** (3,510 LOC): 3D structure determination of a nucleic acid. Uses vectors and floating point numbers.
- **peval** (630 LOC): Partial evaluator for Scheme. Uses lists, symbols, and higher-order functions.
- **scheme** (1,090 LOC): Other Scheme interpreter based on closures. Uses lists, symbols, and higher-order functions.
- **slatex** (2,470 LOC): Scheme to Latex processor. Uses characters, strings, lists, vectors, and small integers.

Microbenchmarks:

- **ack** (10 LOC): Ackermann function. Uses small integers and recursion.
- **bague** (110 LOC): Solver of the *baguenaudier* puzzle. Uses small integers and vectors.
- **fib** (20 LOC): Fibonacci function. Uses small integers and recursion.
- **fibfp** (20 LOC): Fibonacci function. Uses floating point numbers and recursion.
- **nqueens** (40 LOC): Solver of the N-queens puzzle. Uses lists, small integers, and recursion.
- **primes** (40 LOC): Sieve algorithm for finding primes. Uses lists and small integers.
- **tak** (20 LOC): Takeuchi function. Uses small integers and recursion.

4.2 Counting Dynamic Checks

In the Bigloo and Gambit implementations, many built-in procedures implicitly check the type of their arguments and signal an error if they are invalid, such as arithmetic on non-number types or index out of bound when indexing a vector. Polymorphic operators also use implicit dynamic type checks to dispatch computation to specialized primitives.

To measure dynamic checks, all built-in procedures used in our benchmarks are redefined with macros that use inline checks. This is semantically equivalent to operations that apply checks and dispatch to specialized primitives implicitly. For instance, the `BBVvector-ref` and `BBV+` macros implement the `vector-ref` and `+` operations respectively:

```

1 (define-macro (BBVvector-ref v i)
2   '(let ((v ,v) (i ,i))
3     (if (and ($vector? v) ($fixnum? i)
4         ($fx>= i 0) ($fx< i ($vector-length v)))
5         ($vector-ref v i)
6         (error "vector-ref error")))))
7
8 (define-macro (BBV+ x y)
9   '(let ((x ,x) (y ,y))
10     (if (and ($fixnum? x) ($fixnum? y))
11         (or ($fx+? x y) ($+ x y))
12         (if (and ($flonum? x) ($flonum? y))
13             ($fl+ x y)
14             ($+ x y)))))

```

The macros use specialized operators (prefixed with `$` in the example), making all checks explicit (type checks, array bound checks, and integer overflow checks) and ensuring that both compilers perform the same set of checks and in the same order (see Section 3.2 for the definitions of `$fx+?`, `$+`, and the other primitive operations).

When SBBV has determined the type of a value, the type tests that are in the expansion of these macros (`$fixnum?`, `$flonum?`, etc) are effectively removed. Similarly, SBBV’s interval analysis may determine the range of possible integer values, allowing comparisons such as (`$fx>= i 0`) to be removed, and calls to overflow checking operators such as (`$fx+? x y`) to be replaced by the non-overflow checking (`$fx+ x y`) when the result cannot overflow.

However, not all dynamic checks originate from safe operators since programmers can add type tests and bound checks as part of their program’s logic. For this reason, we distinguish between checks introduced by safe operators, which we call *safety* checks in the context of this experiment, and those introduced by the programmer. The number of safety checks is computed by replacing all operators, such as `BBVvector-ref` and `BBV+`, by unsafe ones that execute no type tests, overflow checks or array bound checks. By subtracting the number of checks executed by the unsafe version of a program from the number of checks of its safe version, we obtain the number of safety checks.

4.3 Merge Selection

Applying SBBV requires a heuristic for selecting which versions of a block to merge when that block’s version limit is exceeded. This corresponds to choosing a concrete implementation for the Θ^2 operator from Algorithm 4. The quality of the selection function impacts the general performance of the SBBV algorithm. The current Bigloo and Gambit implementations use a merge heuristic that is rudimentary but still sufficient to establish the benefit of the approach.

In our implementations, when the version limit of a basic block is exceeded, versions that are the most similar are merged first. The similarity of two versions is computed by counting how many variables have the same type when entering the block. While both Bigloo and Gambit implement a *selection by similarity*, the exact implementations of their selection functions differ slightly due to how they represent types internally.

Given that the versions selected depend solely on the contexts when entering a block, this is a *local* merge heuristic. It requires no usage analysis of each variable in the block and its successors. A nonlocal merge heuristic could lead to better results if it prioritizes some versions over others with the objective of maximizing the benefits for the whole program. The space of possible merge heuristics is large and we intend to explore it in future work.

4.4 Results

We first present broad results before diving into a deeper analysis in the subsequent sections.

SBBV permits a trade-off between code size and dynamic checks removal. As the version limit increases, more blocks are duplicated and more dynamic checks are removed. This comes at the cost of increased executable size and compilation time. We found a limit of 2 versions to offer a good trade-off between checks removal, size, and compilation time.

Figure 8 shows the proportion of safety checks removed by SBBV, such as type, overflow, and array bound checks. In all benchmarks, the number of checks decreases when compared to the executable without SBBV. This indicates that SBBV can remove dynamic checks that the existing optimizations of Bigloo and Gambit could not remove.

The proportion of checks removed varies between Bigloo and Gambit, despite both compilers applying the same SBBV algorithm. Bigloo removes more checks without applying SBBV, thus leaving fewer checks to be removed by SBBV. However, the absolute number of remaining checks is similar between both implementations. To a lesser extent, differences in the implementations of the heuristic for selecting versions to merge also influence the number of removed checks by each compiler.

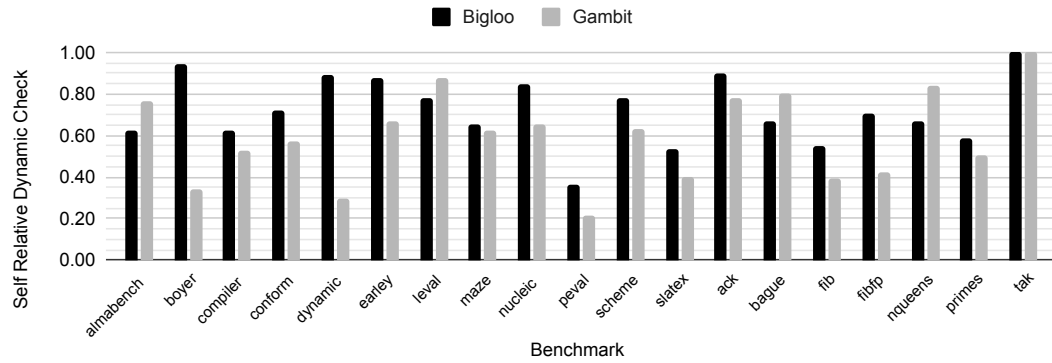


Figure 8 Relative number of safety checks executed for benchmarks compiled with and without SBBV (limit of 2 versions), separately for Bigloo and Gambit. 1.0 corresponds to compilation without SBBV. Lower values indicate fewer dynamic checks with SBBV.

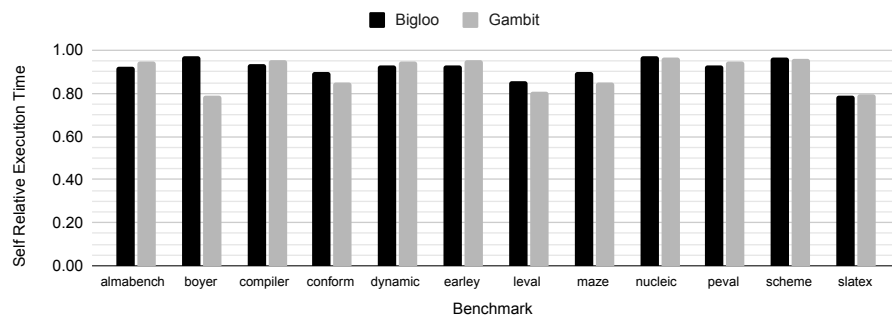


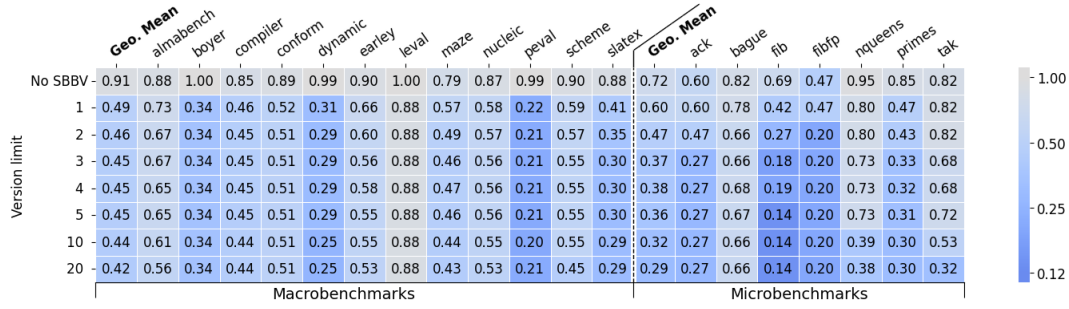
Figure 9 Relative execution time of macrobenchmarks compiled with and without SBBV (limit of 2 versions), separately for Bigloo and Gambit. 1.0 corresponds to compilation without SBBV. Lower values indicate better performance with SBBV.

Although the number of dynamic checks decreases with higher version limits, it does not always lead to a similar reduction of the execution time. Figure 9 shows that all macrobenchmarks execute faster with SBBV and a limit of 2 version (by 10% on average). However, no significant speedup is observed by further increasing the version limit. The relation between the version limit and execution time is discussed further in Section 4.4.3.

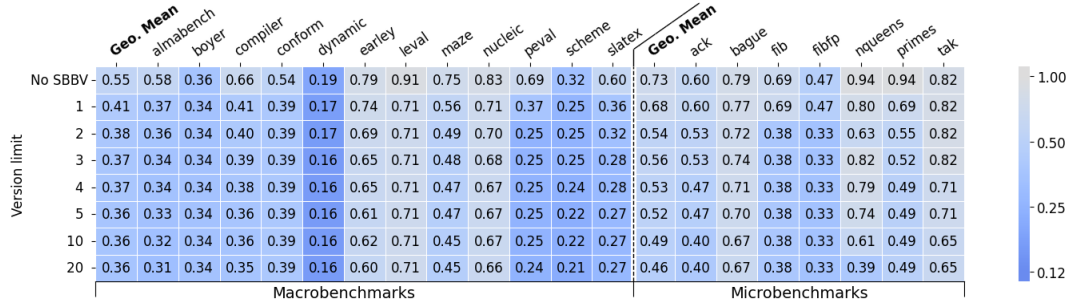
Increasing the version limit allows for more specialized versions. In the following sections, we take into account the impact of the version limit on the removal of dynamic checks, program size, execution speed, and compilation time. Each benchmark is compiled with version limits ranging from 1 to 5, as well as with limits of 10 and 20 versions, and it is compared to a compilation without SBBV. Limits higher than 5 are probably not very practical due to diminishing returns for the added compilation time. We tested with limits of 10 and 20 versions mostly to check the performance in extreme cases.

4.4.1 Dynamic Checks

Figure 10 shows the proportion of safety checks remaining after SBBV with increasing version limits. We estimate the number of remaining safety checks by subtracting checks in the unsafe version of a benchmark from those in the benchmark compiled with SBBV. To compute the total number of safety checks without SBBV, we apply the same formula to each benchmark compiled with no optimization, which effectively preserves all checks.



(a) Gambit.



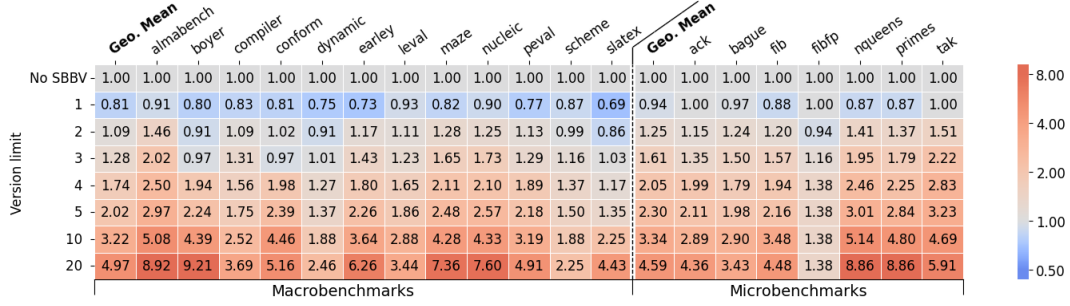
(b) Bigloo.

Figure 10 Effect of SBBV on the removal of dynamic checks (type, overflow, and array bound checks) with increasing version limits. Each cell shows the proportion of safety checks remaining for a given version limit and benchmark when compared to an unoptimized execution. The first row shows checks when SBBV is not applied and only existing optimization techniques are used. A ratio of 1 means that no dynamic checks were removed. Lower values indicate that more checks were removed.

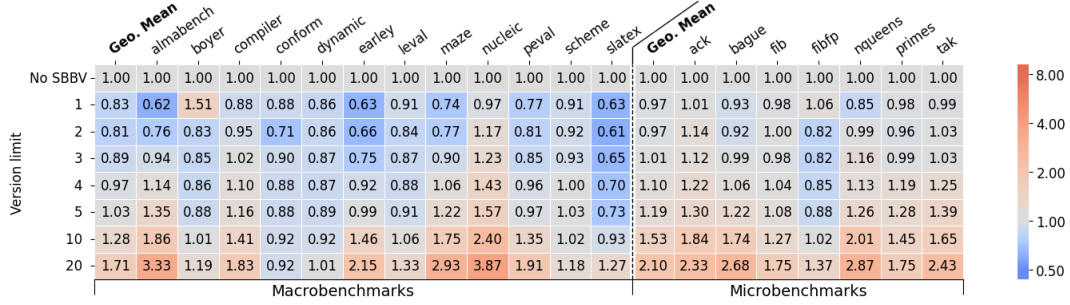
Figures 10a and 10b show, on the first row, the proportion of remaining checks after applying the standard Gambit and Bigloo optimization techniques (*No SBBV*). The following rows display results with SBBV and specific version limits. For all benchmarks, SBBV removes more checks than the standard optimizations. As the version limit increases, more specialized versions are generated, allowing removal of additional checks.

For low version limits, the number of dynamic checks steeply decreases as the limit increases. However, beyond a version limit of about 4, there is a diminishing return for almost all benchmarks. In some benchmarks, an upper bound is rapidly reached beyond which almost no more checks are removed (such as *boyer* at 1 version). In these cases, further increasing the limit contributes to the generation of relatively unimportant versions. Conversely, new useful versions are still discovered when increasing the version limit beyond 10 for some benchmarks (such as *tak* with Gambit).

Increasing the version limit sometimes increases the number of dynamic checks. For instance, in Figure 10a, the number of dynamic checks increases when incrementing the version limit from 2 to 3 in the *nqueens* benchmark with Bigloo. Increasing the version limit delays the merge of excess versions until more candidates are discovered. Given additional choices, the selection function may choose differently, merging a useful version that would have been kept otherwise. This highlights the room for improvement of our selection function.



(a) Gambit.



(b) Bigloo.

Figure 11 Program size with increasing version limits, relative to using the standard optimizations (*No SBBV*). Each cell shows the ratio between the program size with and without SBBV for a given version limit and benchmark. Lower values are better.

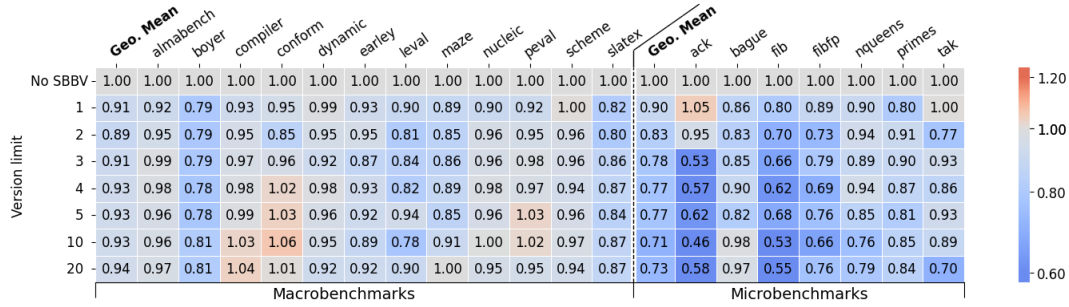
4.4.2 Program Size

We measured the program size of benchmarks compiled with SBBV. The size in bytes of each benchmark is obtained by disassembling its executable and subtracting the position of compiler specific labels. Hence, only the size of the code corresponding to each benchmark, excluding any runtime procedures, is considered.

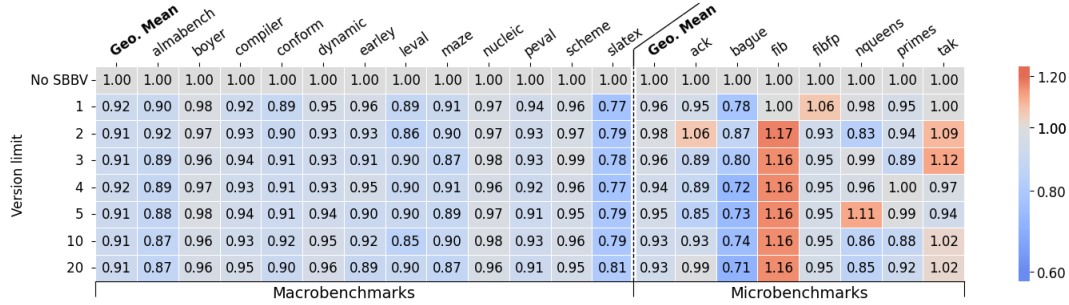
Since SBBV applies code duplication, higher version limits generally generate larger programs. Yet, low version limits may result in smaller executables. In the case of a limit of a single version, this is to be expected because SBBV becomes akin to a static type inference analysis without code duplication, which removes some unnecessary checks. However, for low enough version limits higher than one, SBBV can still reduce program size. In these cases, the removal of dynamic checks outweighs the duplication of basic blocks.

Figure 11b shows the relation between the size of a benchmark and the allotted version limit in Bigloo. On average, macrobenchmarks are smaller up to a limit of 5 versions. In general, the size increases with the version limit, but remains reasonably low with an average growth of about $1.7\times$ on macrobenchmarks with a limit as high as 20 versions.

Figure 11a shows a similar pattern in Gambit, but with a higher growth rate. In the worst case, a growth of about $10\times$ is observed (*boyer*, limit of 20 versions), highlighting the need to select a low enough version limit to curb code bloat. We found a version limit ranging from 2 to 4 to be a good compromise between removed dynamic checks and size.



(a) Gambit.



(b) Bigloo.

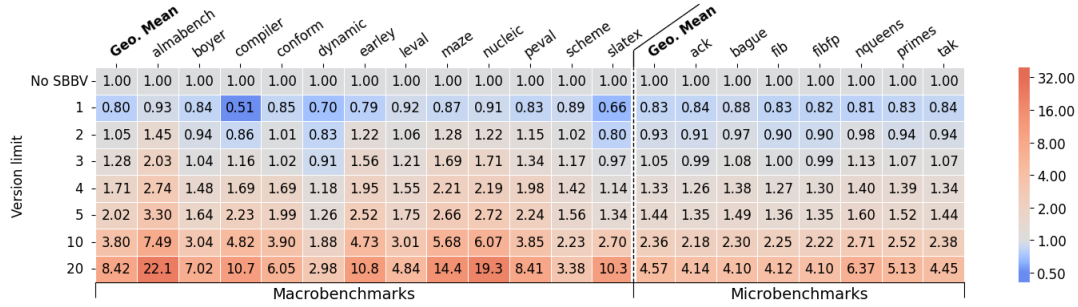
Figure 12 Execution time with SBBV and increasing version limits, relative to the standard optimizations (*No SBBV*). Each cell shows the ratio between the execution time with and without SBBV for a given version limit and benchmark. Lower values are better.

4.4.3 Execution Time

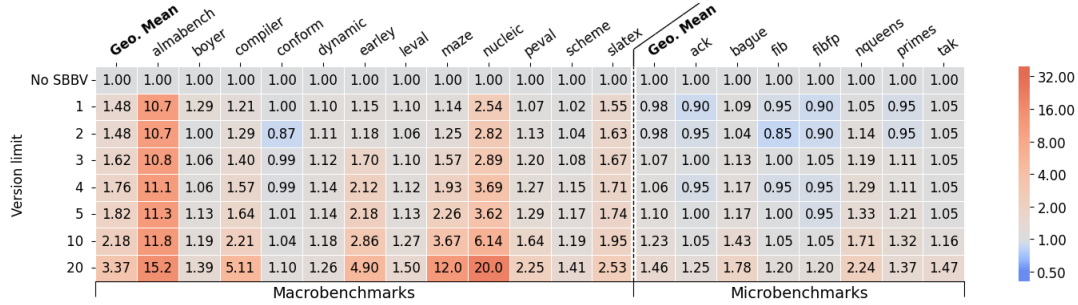
Figure 12 shows how the execution time of each benchmark varies with the version limit. Comparing performance to the number of dynamic checks from Figure 10 shows that a lower number of checks is not correlated to a faster execution in general. With Bigloo (Figure 12b), macrobenchmarks compiled with SBBV execute, on average, about 10% faster than without SBBV regardless of the version limit. With Gambit (Figure 12a), a similar speedup is observed for version limits below 4. Beyond this limit, execution speed still benefits, albeit to a lesser extent. We suspect that this discrepancy is caused by the increased code bloat observed with Gambit for high version limits, which reduces the performance of the instruction cache.

While some benchmarks benefit from a high version limit, the code speed and version limit is only vaguely correlated and contains noise. When optimizing for code speed, using a default version limit for all programs is suboptimal and it is good to give the programmer a manual control over the limit to explore the tradeoffs.

We explain this in part by hard-to-predict hardware optimizations by modern processor architectures. In particular, branch prediction makes dynamic type checking extremely cheap in typical code where a type check frequently returns the same result. Moreover, Bigloo and Gambit implement inexpensive type checks using pointer tagging. We hypothesize that SBBV would have a higher performance impact in implementations with more costly dynamic checks, such as NaN tagging, or object representations that need a memory access to check the type, such as BiBOP and object-oriented languages such as Java, Python, and Ruby.



(a) Gambit.



(b) Bigloo.

Figure 13 Compilation time with increasing version limits, relative to using the standard optimizations (*No SBBV*). Each cell shows the ratio between the compilation time with and without SBBV for a given version limit and benchmark. Lower values are better.

This highlights the need to refine the merge selection function. In the future, we intend to explore the space of possible merge heuristics, including nonlocal heuristics. We also wish to explore dynamic version limits, for instance by increasing the version limit of basic blocks that are likely to be in megamorphic code, as is done by YJIT [7, 8]

4.4.4 Compilation Time

We measured the compilation time for each benchmark and version limit and compared it to the compilation time without SBBV. Figure 13 shows the effect of the version limit on compilation time. In general, compilation time increases with the version limit. The reasons for this increase are twofold: firstly a higher version limit leads to more specialized versions of basic blocks within a control flow graph, secondly the increased size of the C code generated by Bigloo and Gambit leads to increased compilation time by the C compiler. Consequently, a lower program size is correlated to a shorter compilation time.

In the extreme case of a limit of 20 versions, compilation time increased by about $22\times$ in the worst case (**almbench** with Gambit). However, in Section 4.4.1 we showed that there is a diminishing return from increasing the limit beyond about 4 versions. Choosing a limit of 2 caps the worst observed compilation time increase to about $1.5\times$ with Gambit (**almbench**) while reaping most of the benefit of SBBV. On the same benchmark, Bigloo has a large compilation time increase starting at a limit of 1. This is due to the combined effect of a large function with multiple dispatch points and a live variable recalculation by the compiler that is not done by Gambit (register allocation is done before SBBV in the case of Gambit, and after SBBV in the case of Bigloo).

Program	Node.js 21.7.1	Bigloo 4.6a		Gambit 4.9.4-377		Chez 9.5.1	Racket 7.2
		No SBBV	SBBV	No SBBV	SBBV		
almabench	<u>6.31</u>	15.68	14.46	14.64	13.86	17.69	19.30
boyer	40.31	7.40	7.22	8.21	<u>6.49</u>	8.09	12.38
earley	56.75	14.90	13.83	10.87	10.34	<u>9.53</u>	24.73
leval	18.42	7.53	<u>6.47</u>	12.03	9.74	7.16	16.96
maze	10.07	7.47	6.70	6.75	<u>5.73</u>	12.01	12.12
bague	305.04	12.90	<u>11.19</u>	15.54	12.93	21.01	19.33

■ **Figure 14** Average execution times of the benchmarks from [30] in seconds. Bold numbers indicate the fastest execution time for each benchmark. Lower is better.

4.5 Putting the Results in Context

The significance of the above results can be best appreciated through a comparison with other systems whose performance is more widely known. Our goal is to show that both Gambit and Bigloo are competitive with some of the leading implementations of dynamically typed languages, and thus that using SBBV is attractive in the context of high-performance implementations to increase performance further.

The Node.js system is a good comparable given that the JavaScript V8 JIT compiler on which it is based has been extensively engineered and it executes a dynamically typed programming language with similar core constructs as Scheme, to which prototype object-oriented features are added. The Chez Scheme [26] and Racket [25] systems are also interesting as high-performance representatives of the Lisp/Scheme family.

In order to do a fair comparison of systems for different languages, we use benchmark programs that have been translated to both Scheme and JavaScript in previous work [30]. Those programs are a subset of those used in the previous sections. Because of the similarity of JavaScript and Scheme, a systematic translation of the constructs is possible while avoiding important stylistic changes that would compromise the validity of the comparison. We have used the latest versions of those systems available at the time of writing through the Debian package manager (Node.js 21.7.1, Chez Scheme 9.5.1, and Racket 7.2). We measure the program execution time and, for Gambit and Bigloo, we compile the program without SBBV and with SBBV and a limit of 2 versions. Figure 14 gives the execution times in seconds.

The execution time for Gambit and Bigloo without SBBV is faster than Node.js on all the benchmarks except **almabench**, up to 24× faster for **bague**. Node.js does better on **almabench** than any other system because V8 has special optimizations for floating point numbers and arrays that are used by **almabench**. Gambit and Bigloo without SBBV are in the same ballpark as Chez Scheme, faster on roughly half the benchmarks. Racket is typically slower than Chez Scheme, on which it is built internally. When SBBV is used, both Bigloo and Gambit are consistently faster than without SBBV, including on benchmarks on which they already outperformed other compilers. This reinforces our belief that SBBV allows some optimizing compilers for dynamically typed programming language to generate even better code.

5 Related Work

Basic Block Versioning (BBV) was introduced by Chevalier-Boisvert and Feeley [5] as a technique for type check removal in JavaScript. The *lazy* BBV variant, suitable for JIT compilers, only generates versions that are executed, so it limits code bloat. On the other

hand, when the version limit is reached a fully generic version must be used, which negatively impacts type check reduction. SBBV avoids falling back on a generic version by selecting a set of versions that cover all cases but that are at least somewhat specialized. As shown in their Figure 7, a version limit of 2, which is their setting *maxvers*=1, shows a modest reduction of type checks for many benchmarks when compared to *maxvers*=5 because the fallback on the fully generic version is reached too quickly. SBBV achieves good performance with lower version limits. A variant of lazy BBV is used in production in the YJIT compiler inside CRuby [7, 8], also falling back on a fully generic version upon reaching the version limit (which can be 4, 10, or 20 depending on the situation).

The *eager* BBV variant described in [5] is suitable for AOT compilers, like SBBV, but suffers from large code bloat so it was deemed impractical and not explored further [5]. In comparison, SBBV with a version limit of 2, which achieves a comparable level of dynamic check removal, causes an average code size increase of 9% for Gambit and a decrease of 19% for Bigloo.

SBBV can be explained by the theory of abstract interpretation introduced by Cousot and Cousot [10]. The authors presented a theoretical framework for building lattice-based fixed point algorithms. Their work ensures that the SBBV algorithm converges. Furthermore, their seminal paper introduced *union with widening*. Widening not only ensures that the algorithm converges, but also that it converges fast enough to be practical.

SBBV generalizes well-known optimization techniques such as loop-unrolling [2], constant-folding [2], and tail duplication [24]. Tail duplication replicates the code after conditional branches instead of merging the control flow to a single basic block. This permits propagating the information acquired from a condition beyond the body of each branch, allowing further optimizations.

Determining when to apply tail duplication remains a challenge. Leopoldseder *et al.* proposed a simulation-based approach to determine which duplications are the most promising in term of optimization opportunities while minimizing code size [18]. This is analogous to how the choice of a selection function impacts the efficiency of SBBV.

More recently, D’Souza *et al.* applied tail duplication in TASTyTruffle, a Scala JIT compiler using Truffle. TASTyTruffle performs tail duplication at the AST level to generate guarded versions of polymorphic functions that are then specialized by the Graal compiler [13].

Our work is related to occurrence typing that is in particular used in Racket [33]. Occurrence typing is a type system that allows a context-sensitive refinement of variable types during static analysis, for instance by typing a variable differently in each branch of a conditional statement. In the context of our work, occurrence typing synergizes well with tail duplication to further propagate context-sensitive type information.

Partial function inlining can also be done if SBBV is applied interprocedurally. We have not done this in the current work because it is tricky to extend the versioning contexts to track code pointers for function entry and return points. This will require future work.

Code optimization by duplication and specialization has been extensively studied and used for various programming languages [12, 3, 29, 4, 9]. Recently Flückiger *et al.* [15] optimized the compilation of R programs by function specialization. They show that this technique makes programs run 1.7× faster on average. Subsequently a study by Mehta *et al.* [21] has used a mechanism that keeps multiple versions of a given function specialized for contexts encountered at runtime by JIT compilers. Specialized versions of a function are stored in an external repository, allowing switching between versions when de-optimization occurs and to reuse versions of functions across executions and programs. SBBV is related to these techniques but the granularity of cloning is finer as it clones basic blocks while all

those previous works clone whole function bodies. This enables SBBV to better control the code expansion and to spend the cloning budget on relevant specializations without falling back on a fully generic version or using de-optimization, which is not an option in an AOT context. This sort of fine optimization tuning is out of reach for techniques that specialize at the level of whole function bodies.

6 Conclusion

Previous work has shown that the lazy variant of Basic Block Versioning (BBV) is effective in practice for optimizing dynamic checks in JIT compilers for dynamic languages [5, 27, 7, 8]. The *static* BBV (SBBV) approach that we have described in this paper is a variant of BBV that determines, through a fix-point program analysis, a set of basic block versions that are appropriate for the program and that covers all possible contexts without exceeding some versioning limit. This gives control over the code bloat induced by the multiple specializations of individual basic blocks in a way that avoids falling back on an unoptimized generic version of the basic block when the versioning limit is reached. SBBV is thus particularly interesting for use in AOT compilers and consequently it does not suffer from a warmup time.

At the core of the SBBV algorithm is a heuristic to drive the merging of previously generated versions to keep the number of versions within the allowed limit. We have shown through experiments that even a simple merge heuristic removes dynamic checks effectively in practice.

As a second contribution, we have shown in this paper how to extend the BBV approach to implement optimizations that go beyond the elimination of dynamic type checks. We have shown how to use it to remove integer overflow checks and bound checks effectively. By doing so, we have shown that the BBV approach can be viewed as a general programming analysis methodology that can be used to implement various optimizations that otherwise are implemented in isolation using dedicated techniques.

References

- 1 R7RS benchmarks. <https://github.com/ecraven/r7rs-benchmarks>, April 2024.
- 2 David F. Bacon, Susan L. Graham, and Oliver J. Sharp. Compiler transformations for high-performance computing. *ACM Comput. Surv.*, 26(4):345–420, 1994. doi:10.1145/197405.197406.
- 3 Jeff Bezanson, Jiahao Chen, Benjamin Chung, Stefan Karpinski, Viral B. Shah, Jan Vitek, and Lionel Zoubitzky. Julia: dynamism and performance reconciled by design. *Proc. ACM Program. Lang.*, 2(OOPSLA):120:1–120:23, 2018. doi:10.1145/3276490.
- 4 Craig Chambers and David M. Ungar. Customization: optimizing compiler technology for SELF, a dynamically-typed object-oriented programming language. In Richard L. Wexelblat, editor, *Proceedings of the ACM SIGPLAN’89 Conference on Programming Language Design and Implementation (PLDI), Portland, Oregon, USA, June 21-23, 1989*, pages 146–160. ACM, 1989. doi:10.1145/73141.74831.
- 5 Maxime Chevalier-Boisvert and Marc Feeley. Simple and effective type check removal through lazy basic block versioning. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming, ECOOP 2015, July 5-10, 2015, Prague, Czech Republic*, volume 37 of *LIPICs*, pages 101–123. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015. doi:10.4230/LIPICs.ECOOP.2015.101.
- 6 Maxime Chevalier-Boisvert and Marc Feeley. Interprocedural type specialization of JavaScript programs without type analysis. In Shriram Krishnamurthi and Benjamin S. Lerner, editors, *30th European Conference on Object-Oriented Programming, ECOOP 2016, July 18-22, 2016, Rome, Italy*, volume 56 of *LIPICs*, pages 7:1–7:24. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPICs.ECOOP.2016.7.

- 7 Maxime Chevalier-Boisvert, Noah Gibbs, Jean Boussier, Si Xing (Alan) Wu, Aaron Patterson, Kevin Newton, and John Hawthorn. *YJIT: a basic block versioning JIT compiler for CRuby*, pages 25–32. ACM, 2021. doi:10.1145/3486606.3486781.
- 8 Maxime Chevalier-Boisvert, Takashi Kokubun, Noah Gibbs, Si Xing (Alan) Wu, Aaron Patterson, and Jemma Issroff. Evaluating YJIT’s performance in a production context: a pragmatic approach. In Rodrigo Bruno and Eliot Moss, editors, *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, MPLR 2023, Cascais, Portugal, 22 October 2023*, pages 20–33. ACM, 2023. doi:10.1145/3617651.3622982.
- 9 Keith D. Cooper, Mary W. Hall, and Ken Kennedy. A methodology for procedure cloning. *Comput. Lang.*, 19(2):105–117, 1993. doi:10.1016/0096-0551(93)90005-L.
- 10 Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In Robert M. Graham, Michael A. Harrison, and Ravi Sethi, editors, *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*, pages 238–252. ACM, 1977. doi:10.1145/512950.512973.
- 11 Patrick Cousot and Radhia Cousot. Static determination of dynamic properties of generalized type unions. In David B. Wortman, editor, *Proceedings of an ACM Conference on Language Design for Reliable Software (LDRS), Raleigh, North Carolina, USA, March 28-30, 1977*, pages 77–94. ACM, 1977. doi:10.1145/800022.808314.
- 12 Iulian Dragos and Martin Odersky. Compiling generics through user-directed type specialization. In Ian Rogers, editor, *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems, ICPOOLPS 2009, Genova, Italy, July 6, 2009*, pages 42–47. ACM, 2009. doi:10.1145/1565824.1565830.
- 13 Matt D’Souza, James You, Ondrej Lhoták, and Aleksandar Prokopec. TASTyTruffle: Just-in-time specialization of parametric polymorphism. *Proc. ACM Program. Lang.*, 7(OOPSLA2):1561–1588, 2023. doi:10.1145/3622853.
- 14 Shimon Even and Yossi Shiloach. An on-line edge-deletion problem. *J. ACM*, 28(1):1–4, 1981. doi:10.1145/322234.322235.
- 15 Olivier Flückiger, Guido Chari, Ming-Ho Yee, Jan Jecmen, Jakob Hain, and Jan Vitek. Contextual dispatch for function specialization. *Proc. ACM Program. Lang.*, 4(OOPSLA):220:1–220:24, 2020. doi:10.1145/3428288.
- 16 ECMA International. Standard ECMA-262 - ECMAScript language specification, June 2015. 6th edition. URL: <http://www.ecma-international.org/ecma-262/6.0/>.
- 17 Richard Kelsey, William D. Clinger, and Jonathan Rees. Revised⁵ report on the algorithmic language Scheme. *ACM SIGPLAN Notices*, 33(9):26–76, 1998. doi:10.1145/290229.290234.
- 18 David Leopoldseider, Lukas Stadler, Thomas Würthinger, Josef Eisl, Doug Simon, and Hanspeter Mössenböck. Dominance-based duplication simulation (DBDS): code duplication to enable compiler optimizations. In Jens Knoop, Markus Schordan, Teresa Johnson, and Michael F. P. O’Boyle, editors, *Proceedings of the 2018 International Symposium on Code Generation and Optimization, CGO 2018, Vösendorf / Vienna, Austria, February 24-28, 2018*, pages 126–137. ACM, 2018. doi:10.1145/3168811.
- 19 Manuel Serrano. Bigloo. <http://www-sop.inria.fr/indes/fp/Bigloo/>, 2024.
- 20 Marc Feeley. Gambit. <https://gambitscheme.org>, 2024.
- 21 Meetesh Kalpesh Mehta, Sebastián Krynski, Hugo Musso Gualandi, Manas Thakur, and Jan Vitek. Reusing just-in-time compiled code. *Proc. ACM Program. Lang.*, 7(OOPSLA2):1176–1197, 2023. doi:10.1145/3622839.
- 22 Olivier Melançon, Marc Feeley, and Manuel Serrano. An executable semantics for faster development of optimizing Python compilers. In João Saraiva, Thomas Degueule, and Elizabeth Scott, editors, *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering, SLE 2023, Cascais, Portugal, October 23-24, 2023*, pages 15–28. ACM, 2023. doi:10.1145/3623476.3623529.

- 23 Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. SIAM, 2009. doi:10.1137/1.9780898717716.
- 24 Frank Mueller and David B. Whalley. Avoiding conditional branches by code replication. In David W. Wall, editor, *Proceedings of the ACM SIGPLAN'95 Conference on Programming Language Design and Implementation (PLDI), La Jolla, California, USA, June 18-21, 1995*, pages 56–66. ACM, 1995. doi:10.1145/207110.207116.
- 25 PLT Inc. Racket. <https://racket-lang.org/>, 2024.
- 26 R. Kent Dybvig. Chez Scheme. <https://www.scheme.com/>, 2024.
- 27 Baptiste Saleil and Marc Feeley. Interprocedural specialization of higher-order dynamic languages without static analysis. In Peter Müller, editor, *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain*, volume 74 of *LIPICs*, pages 23:1–23:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPICs.ECOOP.2017.23.
- 28 Baptiste Saleil and Marc Feeley. Building JIT compilers for dynamic languages with low development effort. In Stephen Kell and Stefan Marr, editors, *Proceedings of the 10th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, VMIL@SPLASH 2018, Boston, MA, USA, November 4, 2018*, pages 36–46. ACM, 2018. doi:10.1145/3281287.3281294.
- 29 Manuel Serrano. JavaScript AOT compilation. In Tim Felgentreff, editor, *Proceedings of the 14th ACM SIGPLAN International Symposium on Dynamic Languages, DLS 2018, Boston, MA, USA, November 6, 2018*, pages 50–63. ACM, 2018. doi:10.1145/3276945.3276950.
- 30 Manuel Serrano. Of JavaScript AOT compilation performance. *Proc. ACM Program. Lang.*, 5(ICFP):1–30, 2021. doi:10.1145/3473575.
- 31 Manuel Serrano and Marc Feeley. Property caches revisited. In José Nelson Amaral and Milind Kulkarni, editors, *Proceedings of the 28th International Conference on Compiler Construction, CC 2019, Washington, DC, USA, February 16-17, 2019*, pages 99–110. ACM, 2019. doi:10.1145/3302516.3307344.
- 32 Sam Tobin-Hochstadt and Matthias Felleisen. The design and implementation of Typed Scheme. In George C. Necula and Philip Wadler, editors, *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 395–406. ACM, 2008. doi:10.1145/1328438.1328486.
- 33 Sam Tobin-Hochstadt and Matthias Felleisen. Logical types for untyped languages. In Paul Hudak and Stephanie Weirich, editors, *Proceeding of the 15th ACM SIGPLAN international conference on Functional programming, ICFP 2010, Baltimore, Maryland, USA, September 27-29, 2010*, pages 117–128. ACM, 2010. doi:10.1145/1863543.1863561.

Appendix E

Float Self-Tagging Paper

This appendix presents Float Self-Tagging, an approach to object tagging that permits an immediate representation of 64-bit floats that still carries type information in its bit pattern. It leverages the non-uniform distribution of float values used in practice to implement a dual encoding: one for frequent, immediate floats, and one for less frequent, heap-allocated floats. It was published at the *International Conference on Object-Oriented Programming Systems, Languages, and Applications 2025* (OOPSLA'25) [43].

Most relevant to this thesis is that self-tagging introduces two distinct encodings of the same source-level flonum type. This is akin to the integer type in Scheme, which has two distinct encodings: the more common small integers are represented as immediate fixnums, while big integers are represented as heap-allocated bignums. In this thesis, SBBV and ΛV are shown to specialize integer arithmetic efficiently by generating hot paths specialized for fixnum arithmetic, eliminating the overhead of type checking small integers at each operation. This capability may be even more relevant for flonums represented with self-tagging, since their encoding incurs a more significant overhead than fixnums.

As a side note, the techniques described in the paper (self-tagging, NaN-boxing, and NuN-boxing) all rely on a data encoding being sound as long as encoded values map unambiguously to the values they represent, here IEEE 754 floats. A loose parallel can be drawn to code duplication, which similarly moves type information away from values to embed it in the program itself, in effect encoding it in the instruction pointer.

Contribution Statement

The candidate is the primary author of this paper, led the writing, conducted the experimental evaluation, and contributed to the implementation of Float Self-Tagging in the Gambit Scheme compiler. The core idea, superimposing value and tag bits, originated from the second author; the specific encoding, using high exponent bits to represent the most frequent floats as immediate values, originated from the candidate. The implementation in the Bigloo Scheme compiler was carried out by the second author.



Float Self-Tagging

OLIVIER MELANÇON, Université de Montréal, Canada

MANUEL SERRANO, Inria, France

MARC FEELEY, Université de Montréal, Canada

Dynamic and polymorphic languages attach information, such as types, to run time objects, and therefore adapt the memory layout of values to include space for this information. This makes it difficult to efficiently implement IEEE754 floating-point numbers as this format does not leave an easily accessible space to store type information. The three main floating-point number encodings in use today, tagged pointers, NaN-boxing, and NuN-boxing, have drawbacks. Tagged pointers entail a heap allocation of all float objects, and NaN/NuN-boxing puts additional run time costs on type checks and the handling of other objects.

This paper introduces self-tagging, a new approach to object tagging that uses an invertible bitwise transformation to map floating-point numbers to tagged values that contain the correct type information at the correct position in their bit pattern, superimposing both their value and type information in a single machine word. Such a transformation can only map a subset of all floats to correctly typed tagged values, hence self-tagging takes advantage of the non-uniform distribution of floating point numbers used in practice to avoid heap allocation of the most frequently encountered floats.

Variants of self-tagging were implemented in two distinct Scheme compilers and evaluated on four microarchitectures to assess their performance and compare them to tagged pointers, NaN-boxing, and NuN-boxing. Experiments demonstrate that, in practice, the approach eliminates heap allocation of nearly all floating-point numbers and provides good execution speed of float-intensive benchmarks in Scheme with a negligible performance impact on other benchmarks, making it an attractive alternative to tagged pointers, alongside NaN-boxing and NuN-boxing.

CCS Concepts: • **Software and its engineering** → **Source code generation**; **Polymorphism**.

Additional Key Words and Phrases: Dynamic Languages, Polymorphic Languages, Floating-Point Numbers, NaN-Boxing, Compiler Optimization, Scheme

ACM Reference Format:

Olivier Melançon, Manuel Serrano, and Marc Feeley. 2025. Float Self-Tagging. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 330 (October 2025), 27 pages. <https://doi.org/10.1145/3763108>

1 Introduction

In dynamic and other polymorphic languages, efficiently supporting floating-point numbers (floats) remains a challenging issue. These languages require attaching type information to values at run time, including numeric types like integers and floats. This requirement conflicts with the IEEE754 standard [1] for encoding floats, which uses the entire 64-bit word to represent a double-precision float, leaving no space for additional type information. A straightforward but suboptimal solution is to represent floats as heap allocated objects. To reduce the overhead of heap allocating floats, several alternative representations have been proposed, each offering different trade-offs.

Authors' Contact Information: [Olivier Melançon](#), Université de Montréal, Montréal, Canada, olivier.melancon.1@umontreal.ca; [Manuel Serrano](#), Inria, Sophia Antipolis, France, manuel.serrano@inria.fr; [Marc Feeley](#), Université de Montréal, Montréal, Canada, feeley@iro.umontreal.ca.



This work is licensed under a [Creative Commons Attribution 4.0 International License](#).

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/10-ART330

<https://doi.org/10.1145/3763108>

This paper presents a novel technique for encoding floats, called *self-tagging*, that avoids most float allocations without adversely affecting the cost of checking types and accessing objects, which are frequent operations of dynamic languages.

Self-tagging exploits the fact that any invertible bitwise transformation on an IEEE754 float will map some floats to a bit arrangement that contains the correct type information at the correct location, thus requiring no memory allocation for these floats. Using the fact that some ranges of floats appear more frequently in practical applications [4, 10, 28, 45], it is possible to define a transformation that avoids heap allocating almost all floats encountered in practice.

This technique does not require any program static analysis and could be integrated in most runtime systems of languages that need run time types. It is developed in the context of 64-bit runtime systems but is also applicable to 32-bit systems. The rest of this section presents the main encodings in-use as well as their trade-offs.

1.1 Tagged Objects

The classical and popular solution to preserve type information is to attach a tag to all objects [5, 16, 21, 25]. An N-bit machine word is used to encode an *object reference* (either a value or a pointer to a heap allocated value) and a small bit field is reserved in the word to store type information. For the rest of the paper the term *object* is used interchangeably with object reference. Tagging allows low-cost type checks and object access at the cost of losing a few bits for data. Aligning all heap allocated values to 64-bit machine words conveniently frees the low bits of pointers to store a 3-bit tag. On many architectures, accessing the fields of the object can be done at no additional cost by using an offset in the memory dereference instruction to take into account the tag for that type of object. Objects can thus be encoded using one of the three following representations.

- **Tagged values** store type information with a tag, and the object's value in the remaining bits. This representation does not require heap allocation.
- **Tagged pointers** also store type information with a tag, but values are stored in the heap. The objects are represented by pointers with a tag in their low bits.
- **Generic pointers** represent all remaining objects as pointers with a generic tag. The generic tag is associated with multiple types, and type information is stored in a header, in the heap.

Hence, a tag indicates a type, but also a memory representation. A tagged value representation is the most efficient as it needs no heap allocation and no memory read. A tagged pointer still allows efficient type checks, but requires heap allocation. This introduces the overhead of dereferencing the pointer, and adds a strain on the garbage collector. A generic pointer is the least efficient representation as it involves storing type information in the heap, which increases space usage and the cost of type checks and memory management. Consequently, specific tags are a precious resource that must be carefully assigned to the most frequent types observed in programs, such as small integers or floats. Figure 1 shows a typical memory layout of each representation.

As an example, consider an implementation using 3-bit tags. Due to the frequent use of small integers, a tagged value representation is appropriate for small integers and the choice of the tag 000 allows direct addition/subtraction. Since 3 bits are occupied by the tag, small integers are limited to 61 bits. This is generally considered an acceptable trade-off since a two's complement representation of signed integers can accommodate any number of bits, and 61 bits still offers a wide range of values. For larger values, generic pointers to heap allocated big integers can be used [25]. This provides a hybrid representation where the most common integers appear as efficient tagged values and less-frequent big integers are represented with more costly generic pointers.

Unfortunately, when it comes to the representation of floats, none of the aforementioned options are well-suited. Double-precision floats cannot be represented as tagged values since the IEEE754

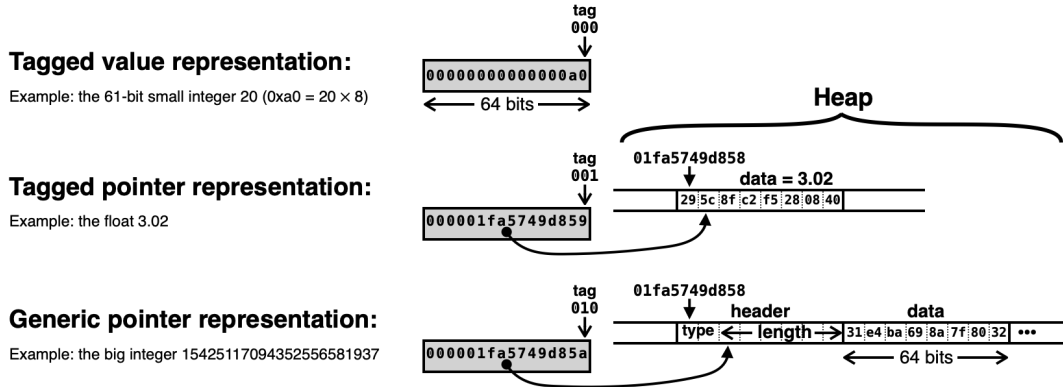


Fig. 1. The three representations in a tagged object system (here shown on a little-endian machine).

standard enforces the use of 1 sign bit, 11 exponent bits and 52 mantissa bits (see Figure 2a), totaling 64 bits and leaving no space for a tag [1]. Parting from this standard is impracticable due to the set of instructions offered by x86 [12] and ARM [26] architectures being specific to 32 or 64-bit floats.

Yet, tagged pointers introduce two major inefficiencies for floats. First, all uses of the value require fetching it from memory. Second, the heap allocation of all floats increases memory usage and the cost of computing float results. This is especially disconcerting considering that allocated floats are often short-lived intermediate results that add a strain on garbage collectors [22, 40]. This has costly implications for programs performing extensive numerical computations or languages such as JavaScript where floats are the only available primitive number type [20].

Still, tagged objects are straightforward to implement and are thus found in numerous compilers such as V8 [42], QuickJS [7], and Hopy [39] (JavaScript), the Lua interpreter [38] (Lua), CRuby [41] (Ruby), SBCL [35] (Lisp), and Gambit [30] and Bigloo [29] (Scheme). Recently, CPython also moved toward tagged pointers in the process of removing its global interpreter lock [15].

1.2 NaN-Boxing

NaN-boxing circumvents the drawbacks of tagged pointer floats by reclaiming unused bits in the encodings of floating-point NaN values to store data [16, 25]. As per the IEEE754 standard, a NaN value is represented by setting all 11 bits of the exponent to 1 and a non-zero mantissa (zero is used to encode Infinity). The mantissa's highest bit distinguishes between a quiet and signaling NaN, which determines whether the NaN should signal an exception or fall through operations.

A subset of NaNs can thus be reserved to encode non-float objects. A convenient subset is that of negative, quiet NaNs, illustrated in Figure 2b, which correspond to the interval from 0xfff8_0000_0000_0000 to 0xffff_ffff_ffff_ffff. This partitions floating-point numbers in two intervals. Bit patterns above 0xfff8_0000_0000_0000 are reserved for non-float objects. The bit patterns below 0xfff8_0000_0000_0000 are reserved for floats (including signaling NaN but not quiet NaN) and negative quiet NaN are mapped to the bit pattern 0xfff8_0000_0000_0000.

On current hardware, memory addresses typically fit in 48 bits. The 51 bits uncovered by NaN-boxing are thus sufficient for storing tagged pointers with 3-bit tags, and 32-bit small integers. It offers the advantage of unboxed floats, but impacts the performance of other, non-float objects due to higher-cost machine instructions to check the type and dereference NaN-boxed pointers.

This reliance on hardware specific details also interferes with other optimizations and portability. Moreover, on 32-bit architectures, NaN-boxing would cripple memory management because 32-bit float NaNs leave space for only 22-bit pointers, which only allows addressing 4 MiB.

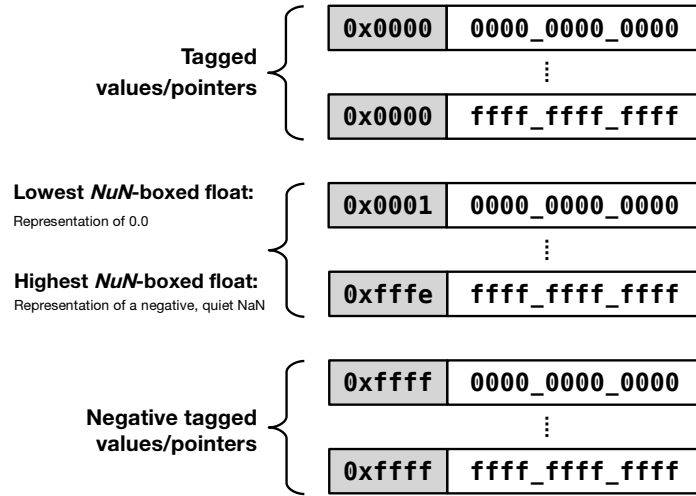


Fig. 3. Encoding of NuN-boxed values, after having applied the 0x0001_0000_0000_0000 bias to floats. The low and high ranges (whose 16 highest bits are 0x0000 or 0xffff) are used for non-float objects that can be represented as tagged values or pointers.

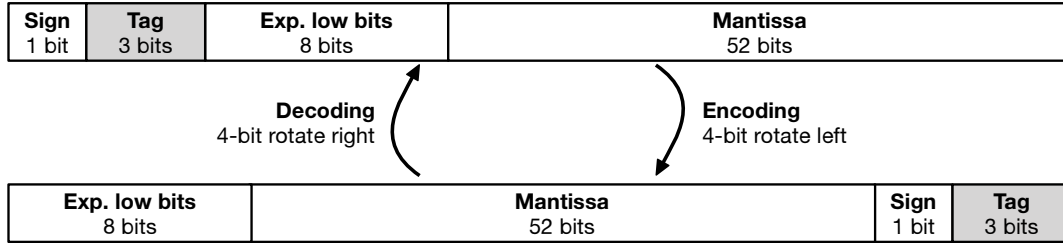


Fig. 4. A float self-tagging representation where the tag corresponds to the high bits of the exponent. The top bit sequence is a float where the tag is superimposed with the exponent's three most significant bits. The bottom sequence is the tagged value representation of the float where a 4-bit left rotation is applied to place the tag on low bits.

2 Self-Tagging

This section describes *self-tagging*, a tagging technique that exploits the fact that some values can be encoded to a tagged value by a bitwise transformation containing the tag corresponding to their type. Such self-tagged objects avoid the cost of heap allocation. In this paper, self-tagging is applied to the specific case of IEEE754 floats. Unless stated otherwise, double-precision floats and a 64-bit architecture are implied. However, self-tagging also pertains to 32-bit architectures, which is discussed in Section 5.

Consider the common case of tagged objects with 3-bit tags (8 available tags). Given tag T for floats, any invertible bitwise transformation will map $1/8$ of possible floats to a bit arrangement that contains T on its low bits. These floats can thus be encoded to tagged values with this transformation and decoded back to a IEEE754 representation by applying the inverse transformation. The set of floats that are mapped to a correctly tagged value by a given transformation are said to be *self-tagged* and require no memory allocation. Since only a subset of all floats can be self-tagged, the remaining floats must be heap allocated with tagged or generic pointers. Yet, the proportion of

self-tagged floats can be increased by assigning more than one tag to self-tagged floats. In general, using n tags for self-tagging allows avoiding heap allocation of $n/8$ of all possible floats.

As an example, consider the case where the encoding transformation is a 4-bit rotation to the left, which has the effect of using the 3 high bits of an IEEE754 float's exponent as tag for self-tagged floats (Figure 4). This transformation has the interesting property that each tag captures a contiguous range of floats. For instance, assigning the tags 000, 011 and 100 to self-tagged floats, avoids heap allocating all floats in the ranges $0.0 \dots 7 \times 10^{-251}$ and $1.7 \times 10^{-77} \dots 2.3 \times 10^{77}$ as well as the corresponding negative ranges (detailed calculations of these ranges are provided in Section 2.1).

This suggests that, while many transformations can be chosen to implement self-tagging, some choices are better than others. Three properties make for a useful self-tagging scheme. It must:

- **Capture intervals of common floats:** most values computed in practice are either ± 0.0 or centered around ± 1.0 [4, 10, 28, 45]. These should be prioritized to reduce memory allocation.
- **Have an efficient encoding/decoding:** the cost of the transformation must not outweigh the benefit of decreased memory allocations and strain on the garbage collector.
- **Use few tags:** it should reserve as few tags as possible to avoid having to defer to generic pointers for other important types. Ideally, a single tag should be used for all common floats, and uncommon floats should be represented either with tagged or generic pointers.

Section 2.1 presents self-tagging variants that offer different tradeoffs between captured intervals and number of reserved tags. Section 3 describes efficient implementations for each variant.

2.1 Float Coverage

Self-tagging introduces the notion of tags that indicate which values of a type are not heap allocated. This contrasts with standard object tagging that invites assigning tags with efficient handling of frequent types in mind, regardless of values. Consequently, self-tagging should be used for the *most common* floats to maximize the likelihood that a float will be self-tagged. Floats whose magnitude is concentrated near 1.0 have been observed to be more common [4, 10, 28, 45], with a relative occurrence that decreases sharply away from 1.0 [45]. Zeros are common, but subnormal numbers are rarely used [45].

These observations from [4, 10, 28, 45] are reproduced by instrumenting the R7RS benchmarks, the standard benchmark suite for Scheme [2], to create a profile of computed floats. All benchmarks labelled as *float* benchmarks were profiled (this includes fibfp=recursive Fibonacci with floats, fft=1024 points Fast Fourier Transform, mbrot=mandelbrot set, nucleic=3D structure determination of a nucleic acid, npoly=determine if a 2D point is in a 20 sided polygon, ray=raytrace a scene of 33 spheres, simplex=linear programming using simplex method, sumfp=add the floats 0 to 10^6 , and sum1=add a file of floats).

The results are shown in Figure 5. The table contains 32 rows, one for each combination of the 5 most significant bits of the float's 11-bit exponent field, which is indicated in the first column (for now, ignore the second column). Each of the right columns indicates what proportion of computed floats have a specific combination of most significant exponent bits. The computed floats are clearly clustered around 1.0, specifically in the range $1.1 \times 10^{-19} \dots 3.7 \times 10^{19}$, and the value 0.0 also occurs frequently for some programs. In fact, R7RS benchmarks have a distribution of floats that appears even more tightly concentrated near 1.0 than that reported by [45, Fig. 7], shown in the rightmost column. Hence, other scientific programs may exhibit profiles that use slightly wider ranges of floats. It is apparent that many programs will operate with floats that are predominantly those around 1.0 on a logarithmic scale, and also 0.0.

5 most signif. expo. bits	5 most signif. expo. bits + 1	Value range	fft	fibfp	mbrot	nucleic	pnpoly	ray	simplex	sumfp	sum1	[45]
00000	00001	0.0	79%	9%	0%	1%	11%	7%	11%	13%	0%	0%
		5e-324 .. 2.1e-289	-	-	-	-	-	-	-	-	-	0%
00001	00010	2.1e-289 .. 3.8e-270	-	-	-	-	-	-	-	-	-	0%
00010	00011	3.8e-270 .. 7e-251	-	-	-	-	-	-	-	-	-	0%
00011	00100	7e-251 .. 1.3e-231	-	-	-	-	-	-	-	-	-	0%
00100	00101	1.3e-231 .. 2.4e-212	-	-	-	-	-	-	-	-	-	0%
00101	00110	2.4e-212 .. 4.4e-193	-	-	-	-	-	-	-	-	-	0%
00110	00111	4.4e-193 .. 8.1e-174	-	-	-	-	-	-	-	-	-	0%
00111	01000	8.1e-174 .. 1.5e-154	-	-	-	-	-	-	-	-	-	0%
01000	01001	1.5e-154 .. 2.8e-135	-	-	-	-	-	-	-	-	-	0%
01001	01010	2.8e-135 .. 5.1e-116	-	-	-	-	-	-	-	-	-	0%
01010	01011	5.1e-116 .. 9.4e-97	-	-	-	-	-	-	-	-	-	0%
01011	01100	9.4e-97 .. 1.7e-77	-	-	-	-	-	-	-	-	-	0%
01100	01101	1.7e-77 .. 3.2e-58	-	-	-	-	-	-	-	-	-	0%
01101	01110	3.2e-58 .. 5.9e-39	-	-	-	-	-	-	-	-	-	1%
01110	01111	5.9e-39 .. 1.1e-19	0%	-	-	-	-	0%	-	-	-	3%
01111	10000	1.1e-19 .. 2	21%	28%	89%	61%	63%	22%	64%	13%	0%	61%
10000	10001	2 .. 3.7e19	0%	63%	11%	38%	26%	70%	25%	75%	100%	28%
10001	10010	3.7e19 .. 6.8e38	-	-	-	-	-	-	-	-	-	2%
10010	10011	6.8e38 .. 1.3e58	-	-	-	-	-	-	-	-	-	1%
10011	10100	1.3e58 .. 2.3e77	-	-	-	-	-	-	-	-	-	0%
10100	10101	2.3e77 .. 4.3e96	-	-	-	-	-	-	-	-	-	0%
10101	10110	4.3e96 .. 7.9e115	-	-	-	-	-	-	-	-	-	0%
10110	10111	7.9e115 .. 1.5e135	-	-	-	-	-	-	-	-	-	0%
10111	11000	1.5e135 .. 2.7e154	-	-	-	-	-	-	-	-	-	0%
11000	11001	2.7e154 .. 4.9e173	-	-	-	-	-	-	-	-	-	0%
11001	11010	4.9e173 .. 9.1e192	-	-	-	-	-	-	-	-	-	0%
11010	11011	9.1e192 .. 1.7e212	-	-	-	-	-	-	-	-	-	0%
11011	11100	1.7e212 .. 3.1e231	-	-	-	-	-	-	-	-	-	0%
11100	11101	3.1e231 .. 5.7e250	-	-	-	-	-	-	-	-	-	0%
11101	11110	5.7e250 .. 1.1e270	-	-	-	-	-	-	-	-	-	0%
11110	11111	1.1e270 .. 1.9e289	-	-	-	-	-	-	-	-	-	0%
11111	00000	1.9e289 .. 1.8e308	-	-	-	-	-	0%	-	-	-	0%
		Infinity/NaN	-	-	-	-	-	-	-	-	-	N/A

Fig. 5. The absolute value ranges captured by each combination of the 5 most significant exponent bits and the proportion of all floats computed by float-intensive R7RS benchmarks. Intervals of interest are highlighted. Empty entries indicate no float in this range, whereas 0% means that very few floats were generated (less than 0.5%). The last column shows the relative occurrence of float for each interval reported by [45, Fig. 7].

2.2 Self-Tagging with 3 Tags and 4 Tags

The float profile of Figure 5 has the property that almost all floats computed have the 3 most significant bits of the exponent field that are either 000 (green rows), 011 (dark blue rows), or 100 (light blue rows). Moreover, among the values captured by 000, only ± 0.0 are used.

These 3 exponent bits can be shifted using a bit rotation to align them with the lower 3 tag bits as shown in Figure 4. Thus a float is encoded to a tagged value by a 4-bit rotation to the left (or 60-bit to the right). A tagged value is decoded to a float with the inverse rotation. This corresponds to the encoding discussed in the previous section (Figure 4).

Test programs did not compute the float values $\pm\text{Infinity}$ and NaN, but those values would be assigned the tag 111 (gray rows). Assigning this fourth tag to self-tagged floats might be useful to avoid heap allocated floats in programs frequently computing these values.

2.3 Self-Tagging with 2 Tags and Preallocated Zeros

As is shown in the float profile of Figure 5 the tag 000 is only used to capture the values ± 0.0 . It can be freed for other uses by preallocating the values ± 0.0 using tagged pointer or generic pointer representation. When a float must be encoded as an object it is compared to zero, in which case one of the preallocated zeroes is returned, thus avoiding a new allocation. If this is done then only 2 tags are needed to cover the most frequent cases: 011 and 100. Unfortunately, this approach may raise the number of failed branch predictions due to the test for ± 0.0 (this is further discussed in Section 4.6).

2.4 Self-Tagging with 1 Tag

Self-tagging can also be achieved with a single tag by a different transformation of the most significant exponent bits. The 5 most significant bits of the exponent are added to 1 and the middle 3 bits are used as the tag (using a rotation 5 places to the left). This corresponds to the second column of Figure 5. Note that the tag 000 (bold pink rows) covers all the useful ranges, including $\pm\text{Infinity}$ and NaN that are not covered by the 2-tag and 3-tag variants.

It is noteworthy that there is nothing special about the tag 000. If for some external reason some other tag is more convenient it can easily be assigned to 1-tag self-tagged floats by adding $1 + 2 \times \text{tag}$ rather than 1 to the 5 most significant exponent bits.

Putting all of this together, the self-tagged encoding of a float whose IEEE754 64-bit representation is the integer n can be computed as $(n \oplus ((1 + 2 \times \text{tag}) \ll 58)) \ll_{\text{rot}} 5$, where $\oplus$ is addition modulo 2^{64} and $\ll_{\text{rot}}$ is the left rotation operator. The operations are reversed to perform the decoding of a self-tagged float to its IEEE754 64-bit representation.

This variant has the advantages of covering all the useful ranges of floats computed by R7RS benchmarks and about 90% of floats reported by [45], while using a single tag, which leaves more tags available for other types.

2.5 Self-Tagging with a Different Exponent Bias

A small change in the exponent bias of the 1-tag variant can be used to get another self-tagging variant. If $2 \times \text{tag}_1$ is added to the highest 5 bits and the middle 3 bits are examined, then when they are equal to tag_1 or $\text{tag}_2 = \text{tag}_1 - 1$ they are self-tagged floats. This effectively doubles coverage by including both the light pink and bold pink rows of the second column of Figure 5. The following ranges will be self-tagged: $0..3.8 \times 10^{-270} \cup 5.9 \times 10^{-39}..6.8 \times 10^{38} \cup 1.1 \times 10^{270}..\text{Infinity}/\text{NaN}$. To put this into perspective, this is a superset of the IEEE754 32-bit floats, so any floating point value that can be represented as a IEEE754 32-bit float can be self-tagged with this variant which uses 2

of the 8 3-bit tags. This is likely a better use of 2 tags than the 2-tag variant of Section 2.3, which does not cover the ranges containing 0, Infinity, and NaN.

When compared to the 1-tag variant, there will be almost no difference in heap allocations for programs with similar float profiles as Figure 5. However, as will be explained in the next section, it yields more compact code, because the bias can be added after the rotation, and the need for a bias disappears when $tag_1 = 000$.

3 Implementation

This section discusses implementation details for float-related operations in dynamic languages. The types f64 and f32 will refer to IEEE754 64-bit and 32-bit doubles and the types i64 and i32 will refer to 64-bit and 32-bit integers. Moreover, the type object will refer to values in the language (the sum type of floats, integers, booleans, and other objects). To illustrate the low-level implications of the implementation and give a sense of the execution time performance, assembly code for the Intel x86 architecture is provided. This section focusses on a 64-bit implementation where object is an i64 and floats are f64, but it is straightforward to implement the operations similarly on a 32-bit implementation where the type object is an i32 and floats are f32.

Testing the low tag bits of an object value is a basic need for float-related operations to convert f64 to and from object and for dynamic type checks. In particular it is possible to determine if a f64 can be self-tagged by going through the encoding process and checking that the resulting tag is one of the tags (or *the* tag) appropriate for the chosen self-tagging variant.

3.1 Single Tag Testing

Checking that the N low bits are equal to a specific tag can be achieved by masking those bits followed by a comparison. Assuming the object value is in the x86 64-bit register `rax`, then the following sequence will branch to label `tag_matches` when the 3 low bits are equal to `tag`:¹

```
and al, 7
cmp al, tag
jz tag_matches
```

Note that by using `al` only the lower 8 bits of `rax` are accessed, leading to smaller constants and instruction encodings. However, this sequence modifies `rax` so if the value is needed after the tag test, as often will be the case, then an additional instruction and register are required to keep a copy. In the special case of $tag=0$, the two first instructions can be replaced with “`test al, 7`”, which is a *bitwise-and* that tests if all 3 lowest bits of `rax` are zero without modifying `rax`. In the general case when $tag \neq 0$, the test instruction can be combined with a *lea* (*Load Effective Address*) instruction to copy the source register, plus a constant to cancel $tag \pmod{8}$, to a temporary register:

```
lea ebx, [eax+(8-tag)] # ebx = eax + (8-tag)
test bl, 7
jz tag_matches
```

3.2 Multiple Tag Testing

When it is necessary to check if an object’s tag is one of a set of tags it is often possible to use a single branch instruction rather than a sequence of single tag checks. This is particularly the case when the implementer has leeway in the assignments of tags, as is often the case.

Checking that $tag \in \{tag_1, tag_2\}$ where $tag_2 = tag_1 + 2^n \pmod{8}$ can be done just as efficiently as the single tag test by using the *lea* instruction to map tag_1 to 0 and tag_2 to 2^n and then using a

¹Intel assembler syntax is used throughout.

mask of $7 - 2^n$ in the test instruction to ignore the bit that may be 0 or 1. For example, for 2-tag self-tagging (Section 2.5) with the tags 010 and 110 the check is:

```
lea ebx, [eax+(8-2)] # 010 -> 000 and 110 -> 100
test bl, 3           # 3 = 7-4
jz tag_matches_010_or_110
```

4-tag self-tagging (Section 2.2) with the tags 010, 011, 110, and 111 is even simpler:

```
test al, 2
jnz tag_matches_010_or_011_or_110_or_111
```

The general case of checking that *tag* is in a set of tags can be done efficiently with the x86 *bit-test* instruction “bt *n*, *i*” that reads the bit at index *i* of *n* and puts it in the carry flag. The bt instruction comes in 16, 32, and 64 bit variants, but not 8 bit. For example, for 3-tag self-tagging (Section 2.2) with the tags 000, 011, and 100 the check is:

```
mov bx, 0x1919 # set of tags
bt bx, ax      # test bit at index ax of bx
jc tag_matches_000_or_011_or_100
```

The bit index in ax is obtained *mod 16*, so the byte 0b00011001=0x19 (all 0’s except at bit indices 0, 3, and 4) is repeated twice in register bx. Using the bt instruction has the advantage of neither modifying the rax register nor the register holding the set of tags, amortizing the cost of mov over multiple checks of that set of tags (a possibly near zero cost if that register is globally reserved).

On architectures without a bit test instruction, a dynamic count shift of the tag set register can achieve the bit indexing. If done by modifying the tag set register, the initialization can’t be shared by multiple tag tests. On most 3-address RISC architectures the shift can be done non-destructively. On ARM A64, this instruction sequence tests the 3 low bits of 64-bit register x1 for a tags 0, 3, or 4:

```
lslv w3, w2, w1
cmp w3, 0
bmi matching_tag
```

It assumes that the 32-bit register w2 has been preloaded with 0x98989898, the bit reversed tag set that aligns the bit for tag 0 with the sign bit.

The cost of the check does not depend on the number of tags tested. This can be advantageous if a tagged pointer is used for heap allocated floats, say with tag *h*. In that case, a check for a float (either self-tagged or heap allocated) can be done by adding *h* to the tag set. Once it is known that an object is a float, it is easy to check for the single tag *h* to discriminate between the self-tagged and heap allocated representations.

3.3 Boxing: Conversion f64→object

To convert a f64 to an object, it is converted to an i64, added to a constant *bias* modulo 2^{64} , and bit-rotated left by *r*. For the 1-tag variant, $bias = (1 + 2 \times tag) \times 2^{58}$ and $r = 5$. For the 2-tag, 3-tag, and 4-tag variants, $bias = 0$ (no bias) and $r = 4$. After this there is a check to verify if the resulting object’s tag is in the set of self-tagged tags. If this is the case the conversion is done, otherwise an out of line routine can be called to heap allocate a float and return an appropriately tagged pointer. As an example, if the f64 value is in the 64-bit float register xmm0, then the following x86 code will set register rax to the corresponding object when 3-tag self-tagging is used:

```
mov rax, xmm0 # rax ← xmm0
rol rax, 4     # rotate left 4 places
bt bx, ax     # assumes bx initialized to 0x1919 elsewhere
jnc heap_alloc_float # tag is not one of 000, 011, or 100?
done:
```

Assuming the initialization of the tag mask register (bx) is amortized over multiple float-related operations, the cost for the hot path is low: 3 register-to-register operations and an easily predictable conditional jump.

The following x86 code is appropriate for 1-tag self-tagging using 000 as the tag:

```

mov  rax, xmm0    # rax ← xmm0
add  rax, rbx      # assumes rbx initialized to 1<<58 elsewhere
rol  rax, 5        # rotate left 5 places
test al, 7
jnz  heap_alloc_float # tag is not 000?
done:

```

Here also, the cost for the hot path is just a bit more due to the bias: 4 register-to-register operations and an easily predictable conditional jump.

For the 2-tag variant with no special handling of zeroes (Section 2.5), the rotation can be done before adding the bias to use a compact addition instruction (because then the bias is the small constant tag_1). In the special case of $tag_1=000$ the bias vanishes. For example, here is the code when $tag_1=011$:

```

mov  rax, xmm0    # rax ← xmm0
rol  rax, 5        # rotate left 5 places
add  al, 3         # sufficient to modify the lower 8 bits of rax
bt   bx, ax        # assumes bx initialized to 0x0c0c elsewhere
jnc  heap_alloc_float # tag is not one of 010 or 011?
done:

```

The add instruction uses a small constant as a bias, contrary to the code for the 1-tag version (large constants typically need to be setup in a register, which is additional work even if it is amortized). Moreover the 1-tag version cannot eliminate the bias, but here the add instruction can be removed when $tag_1=000$ which makes it slightly faster than the 1-tag variant and as efficient as the 3-tag variant: 3 register-to-register operations and an easily predictable conditional jump.

3.4 Unboxing: Conversion object→f64

To convert an object (that is known to be a float) to a f64, the tag must be tested to see if it is a self-tagged float. If it is, the boxing operations are done in reverse order and inverted, replacing the rol by a ror (rotate-right), and if there is a bias, replacing the add by a sub (subtract). If it is not a self-tagged float, a memory read gets the f64 result out of the heap allocated float. As an example using the 3-tag variant, if the object value is in the 64-bit register rax, then the following x86 code will set register xmm0 to the corresponding f64:

```

bt   bx, ax        # assumes bx initialized to 0x1919 elsewhere
jc   self_tagged
mov  xmm0, [rax+offset] # xmm0 ← value field of float
jmp  done
self_tagged:
ror  rax, 4         # rotate right 4 places
mov  xmm0, rax      # xmm0 ← rax
done:

```

The number and type of instructions on the hot path is the same as for the boxing operation.

In summary, the self-tagging operations require few machine instructions but there are small variations depending on the instruction set and also the assignment of tags which can often be optimized for self-tagging without impacting the speed of operations on objects or small integers.

3.5 C Implementation

This section explains how the above operations can be implemented in C, as this is a common implementation language, in particular it is used by the Bigloo and Gambit Scheme to C compilers used in the experiments of Section 4.

Testing for a specific tag or a pair of tags is fairly straightforward to express in C and follows the same principles as the assembly code, so details are skipped. Testing for a set of tags is more challenging in portable C code. It can be implemented with the following pure C function, which is easily inlined by a C compiler (here testing for the tags 000, 011, and 100):

```
#define TAG_SET ((1<<0)|(1<<3)|(1<<4)) /* 0x19 for tags 000, 011, and 100 */

inline bool has_tag_0_or_3_or_4(int64_t n) {
    return (((uint32_t)1 << (n & 31)) & (~((uint32_t)0/0xff * TAG_SET))) != 0;
}
```

The expression $n \& 31$ computes a shift count *mod* 32 and the bitwise-and therefore tests the bit at index $n \bmod 32$ of its second operand. The code uses the compile-time constant $\sim((\text{uint32_t})0/0\text{xff} * \text{TAG_SET})$ that repeats the 8 bit mask TAG_SET 4 times to fill a 32 bit word, giving 0x19191919. This achieves the equivalent of testing the bit at index $n \bmod 8$ of TAG_SET.

Using this pure C definition with gcc version 13.2.0 and clang version 18.1.3 on x86-64 results in the non-destructive approach based on the bit test instruction. The expression $n \& 31$ is optimized out by those compilers because the bit test instruction on x86 implicitly does this operation. On older versions of those compilers that would implement this with a destructive shift instruction, the bit test approach can be achieved using the following definition that uses an asm statement:

```
inline bool has_tag_0_or_3_or_4(int64_t n) {
    bool carry;
    __asm__ ("bt %%ax, %2;" /* get one bit of mask into carry (ax is index) */
            : "=ccc"(carry)
            : "a"((uint16_t)n),
            : "r"((~((uint16_t)0/0xff * TAG_SET))); /* 0x1919 */
    return carry;
}
```

Boxing and unboxing operations need to convert between a float value and its bit representation. This can be achieved portably with a union type whose fields are of type f64 and i64. Moreover, boxing and unboxing need to rotate the bit representation of the float. Although C does not provide an operator for bit rotation, both gcc and clang recognize an equivalent pair of shifts and generate a single machine rotate instruction.

The implementation of boxing in C for the 3-tag variant is:

```
union di { double d; int64_t i; };

#define ROTL(n,s) (((int64_t)((uint64_t)n << s) | ((uint64_t)n >> (64 - s))))

inline int64_t f64_to_object(double f) {
    int64_t result = ROTL(((union di)f).i, 4);
    if (has_tag_0_or_3_or_4(result)) return result;
    return heap_allocate_float(f);
}
```

The implementation of unboxing for the 3-tag variant is:

```
inline double object_to_f64(int64_t o) {
    int64_t result = ROTL(o, 60);
    if (has_tag_0_or_3_or_4(result)) return ((union di)result).d;
    return value_of_heap_allocated_float(o);
}
```

4 Experiments

This section evaluates float self-tagging and demonstrates that it presents an interesting performance trade-off between programs that perform frequent operations on objects but few floating-point operations, and float-intensive programs where this balance is reversed. This evaluation is conducted through experiments that measure performance metrics across different object representations (both self-tagging and existing ones), multiple machine architectures, and two state-of-the-art Scheme implementations: the Bigloo (commit 5b1118) [29] and Gambit v4.9.7 (commit 768900) [30] compilers. These offer a level of performance that is competitive with other high-performance Scheme implementations such as Chez Scheme [2, 3, 33]. Using two independently developed compilers helps to demonstrate that self-tagging can be adapted to preserve the different design philosophies and historical implementation choices of these systems.

The following self-tagging variants are evaluated:

- **3-tag self-tagging** uses 3 tags for self-tagging floats corresponding to the green, dark blue, and light blue ranges from Figure 5. Remaining floats are heap allocated. Bigloo uses the tags 000, 011 and 100, and heap allocated floats are represented with generic pointers. Gambit offsets the tags by 3 (i.e. it uses 011, 110 and 111) so that the tag 000 is kept for tagging small integers, and tagged pointers with the tag 010 are used for the heap allocated floats.
- **4-tag self-tagging** (Gambit only) is like the 3-tag variant but uses the tag 010 to self-tag floats in the gray range from Figure 5. Allocated floats are represented with generic pointers.
- **2-tag self-tagging with preallocated zeros** (Bigloo only) tests the impact of expending the tag 000 for self-tagging. It only uses the tags 011 and 100 for float self-tagging the dark blue and light blue ranges from Figure 5 and reclaims the tag 001 to represent all heap allocated floats as tagged pointers. The floats ± 0.0 are preallocated and the float boxing operation contains an explicit zero check to return the preallocated zeros.
- **2-tag self-tagging** (Gambit only) tests the variant that uses the tags 010 and 110 for float self-tagging the bold pink and light pink ranges from Figure 5.
- **1-tag self-tagging** tests the variant that uses a single tag for self-tagging the bold pink ranges from Figure 5. Bigloo uses the tag 001 and Gambit uses the tag 110. heap allocated floats are represented with tagged pointers.

Both compilers can also be configured to represent all floats with heap allocated tagged objects or NuN-boxing. Bigloo also supports NaN-boxing. This provides a flexible comparison environment.

In this paper the C back end of the Bigloo and Gambit compilers are used. This way the resulting machine code benefits from the C compiler's optimizations and in particular excellent architecture specific instruction selection. In both cases the same version of the gcc C compiler is used.

4.1 Overview of the Bigloo Compiler

The Bigloo compiler conforms to the R5RS Scheme specification and adds several extensions, including optional type annotations. Exact rational numbers and complex numbers, which are optional in the R5RS Scheme specification, are not supported.

The Bigloo compiler builds a typed abstract syntax tree from the compiled Scheme program. Several analysis and optimizations refine the initial optional type annotations contained in the source program, the main ones being occurrence-typing [43] and storage use analysis [40]. During compilation, primitive values have a dual representation: one for polymorphic contexts and one for specific contexts. For instance, when the compiler can establish that a function is always invoked with floats, the compiler assigns the C `double` type to that function and, when needed, it introduces cast operations from and to the generic representation of floats and `double`.

The result of this compilation technique is that many local variables and local functions are precisely typed and avoid polymorphic representations. This contributes to removing otherwise necessary heap allocations and casts for floats. The compiler also tracks cases where only floats are stored into vectors, which are then transformed into arrays of C doubles.

Bigloo uses the Boehm-Demers-Weiser conservative mark-and-sweep garbage collector [8]. The complexity of collecting dead objects is proportional to the sum of the live and dead objects, contrary to copying collectors. For such a collector, avoiding allocating short lived floats is crucial for performance.

4.2 Overview of the Gambit Compiler

The Gambit compiler conforms to the R7RS Scheme language. It features the full numerical tower including arbitrary-precision integers, exact rational numbers and complex numbers. Arithmetic operators are generic and the result type depends on the specific values, for example multiplying two complex numbers may yield a number of any type. Like Bigloo, the compiler inlines the small integer and float cases of the dispatch and handles other cases in an out-of-line function.

First-class continuations (`call/cc`) and tail-calls are fully supported, and stack-overflows are gracefully handled. With the C back end, this requires the use of a trampoline and the management of the stack through an explicit array, stack pointer and stack-overflow checks. This causes a small slowdown when compared to a machine code back end where the trampoline would not be required. Memory management is based on a fast bump-allocator and a stop-and-copy garbage collector.

The compiler front end performs several source-to-source optimizations, such as constant folding, lambda lifting, and function inlining. It does not perform type analysis or type inference. However, the back end uses a simple mechanism to optimize float calculations by tracking the use of their results within basic blocks. The result of a float operation is kept in raw form so that other operations within the same basic block that consume it do not need to unbox the value. A float value is boxed only if it remains live at the end of the basic block.

4.3 Experimental Setup

Self-tagging variants are implemented in Bigloo and Gambit, and tested using a subset of the R7RS benchmarks, which is the standard benchmark suite for Scheme [2]. The experiment includes all macro-benchmarks (more than 500 lines of code) and benchmarks whose calculations mainly involve floats (tagged as such in the R7RS benchmarks suite). For the purpose of analyzing results, it is useful to distinguish between benchmarks that use few or no floats and float-intensive benchmarks (`fibfp`, `fft`, `mbrot`, `nucleic`, `npoly`, `ray`, `simplex`, `sum1`, and `sumfp`).

Experiments were repeated on the following machines that operate on distinct microarchitectures:

- INTEL - Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0
- AMD - AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0
- M2 - Apple M2 Max, 64GB, macOS 15.4.1, gcc 14.3.0
- RISC-V - riscv sifive, u74-mc, 8GB, Linux starfive 6.1.31-starfive riscv64, gcc 14.2.0

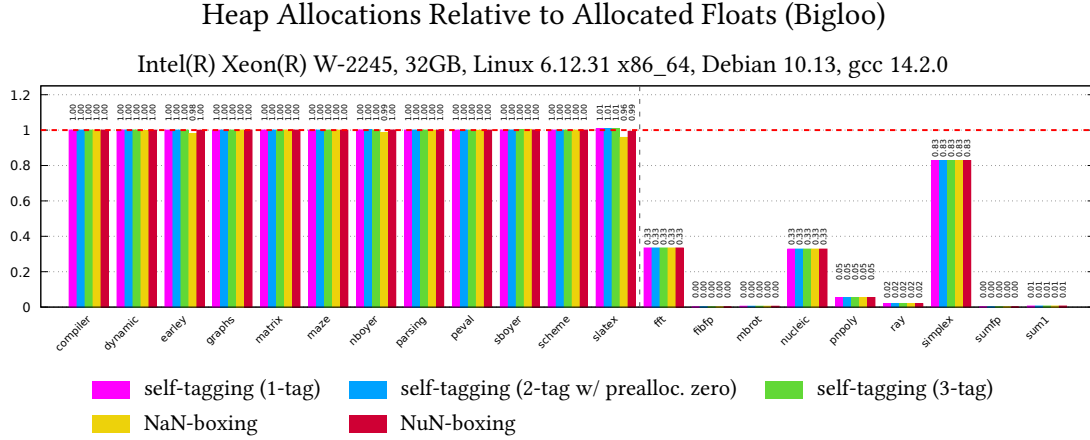


Fig. 6. Heap allocations of Bigloo with each variant of self-tagging, NaN-boxing, NuN-boxing, and allocated floats. The y-axis shows the number of allocated bytes relative to the version that heap allocates all floats (dashed horizontal red line). $1.00\times$ indicates no change, while $0.00\times$ indicates no heap allocation.

For each combination of machine, compiler and self-tagging variant, benchmarks are executed 10 times, and each repetition is configured to last at least 5 seconds. Repetitions of each variant and baseline are paired in order of execution to compute relative execution time for each pair. Upcoming sections report geometric means of these relative times, with geometric standard deviations.

To reduce variance on INTEL and AMD, benchmarks are executed on the same CPU core with taskset and address space randomization is disabled with setarch. On M2, no equivalent tools are available, hence the higher variance. On RISC-V, variance remains low despite not using taskset and setarch.

For measurements that were similar across all machines, a single microarchitecture is shown. Omitted figures can be found in a companion artifact [32].

4.4 Memory Profiling and Execution Time

Figure 6 shows the memory allocations of Bigloo’s self-tagging variants, NaN-boxing, and NuN-boxing compared to allocated floats. Since NaN/NuN-boxing are known to allocate no floats at all, these correspond to executions where heap usage is for non-float objects only. A few self-tagging variants allocate some floats on some benchmarks (see Figure 5), but these allocations are rare enough that memory profiling of all variants is nearly identical to that of NaN/NuN-boxing.

Detailed execution time comparisons with NuN-boxing are shown in Figures 7 and 8 (Bigloo and Gambit respectively). Figure 7 also shows time for NaN-boxing, which Bigloo implements. Figure 9 summarizes results by providing the geometric mean of execution times relative to NuN-boxing for each float encoding, compiler, and microarchitecture, for both float and non-float benchmarks.

As a general observation, none of the evaluated encodings is a better alternative overall; the best encoding depends on the nature of benchmarks, microarchitecture, and implementation. For instance with Bigloo, NaN-boxing is faster across microarchitectures on float-intensive benchmarks, but slowest on non-float benchmarks. Conversely, self-tagging generally fares better than NaN/NuN-boxing on non-float benchmarks since it introduces no overhead on non-float objects.

Bigloo’s self-tagging does not consistently match the performance of NuN-boxing on float benchmarks across microarchitectures (but is faster than allocated floats, as will be discussed in Section 4.5). The 1-tag and 3-tag variants offer the highest performance overall for Bigloo, with the 3-tag variant being faster for float benchmarks and slightly slower for non-float benchmarks.



Fig. 7. Execution time of Bigloo with each variant of self-tagging and NaN-boxing relative to NuN-boxing on four distinct microarchitectures (from top to bottom: AMD, INTEL, M2, and RISC-V). The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

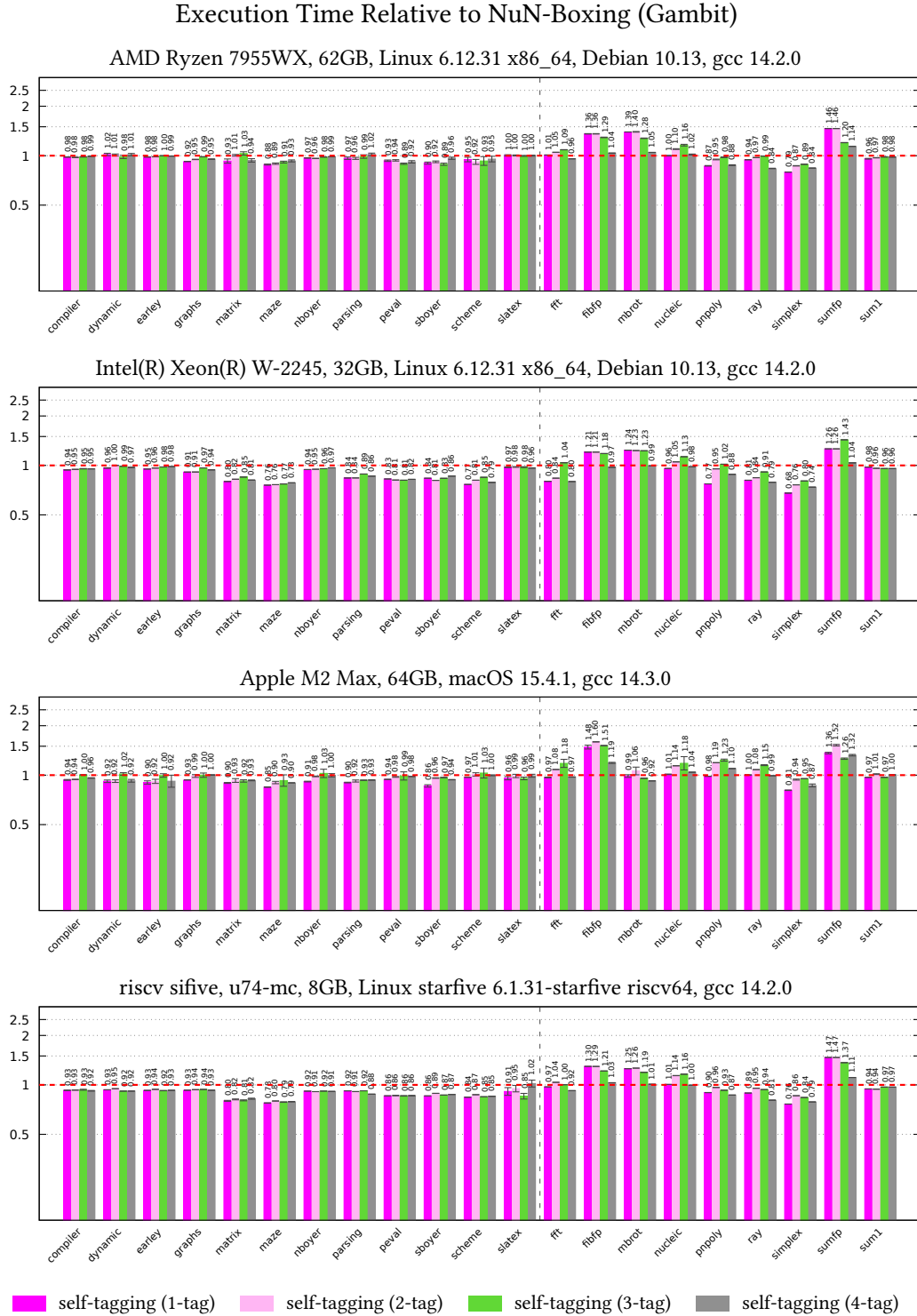


Fig. 8. Execution time of Gambit with each variant of self-tagging relative to NuN-boxing on four distinct microarchitectures (from top to bottom: AMD, INTEL, M2, and Risc-V). The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

Summary of Execution Time Relative to NuN-Boxing

		Non-float benchmarks				Float benchmarks			
		AMD	INTEL	M2	RISC-V	AMD	INTEL	M2	RISC-V
Bigloo	1-tag 	1.01×	.97×	.97×	.97×	1.20×	1.20×	1.25×	1.28×
	2-tag w/ prealloc. zero 	1.01×	.96×	.98×	.97×	1.22×	1.34×	1.33×	1.29×
	3-tag 	1.00×	1.00×	1.00×	1.02×	1.13×	1.09×	1.10×	1.19×
	NaN-boxing 	1.01×	1.02×	1.00×	1.06×	.87×	.84×	.90×	.90×
Gambit	1-tag 	.95×	.87×	.91×	.88×	1.06×	.94×	1.05×	1.03×
	2-tag 	.96×	.88×	.95×	.89×	1.11×	1.00×	1.16×	1.09×
	3-tag 	.96×	.90×	.98×	.88×	1.09×	1.06×	1.14×	1.06×
	4-tag 	.97×	.89×	.96×	.89×	.97×	.90×	1.04×	.94×

Fig. 9. Geometric mean of execution times relative to NuN-boxing of float and non-float benchmarks for each combination of float encoding, compiler, and microarchitecture. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

Bigloo’s 2-tag with preallocated zeros has similar average performance as 1-tag, however it shows more variability across benchmarks (see `fft` and `pnpoly` in Figure 7) since it has to test for ± 0.0 out-of-line. Hence, Gambit’s 2-tag variant (Section 2.5) should be preferred to that of Bigloo.

Gambit’s 4-tag variant provides consistently faster average execution times for both float and non-float benchmarks on all architectures except M2 where it is nearly the same speed as NuN-boxing. Gambit’s 1-tag variant also fares well on all benchmarks on INTEL and RISC-V.

Self-tagging and NaN/NuN-boxing have competitive performance and the best choice depends on the specific setting. When floating point computation performance is important but less so than operations on other objects, self-tagging is a good option to consider. Additionally, a significant portion of this study was spent adapting existing systems (Bigloo and Gambit) to new float encodings, and self-tagging was found to be qualitatively simpler to implement as it only affects the encoding of floats, while NaN/NuN-boxing has deeper object encoding implications.

4.5 Memory Management Considerations

Figures 10 and 11 show execution times of the best self-tagging variants on each implementation (Bigloo 1-tag, and Gambit 4-tag respectively) relative to allocated floats. For Bigloo, decreased memory allocations correlates with lower execution time on all architectures (see Figure 6) with the exception of `sum1`, an I/O bound benchmark.

Conversely, Gambit execution time slightly increases on many float benchmarks. This difference is caused by its use of a stop-and-copy garbage collector with bump allocation, while Bigloo uses the Boehm mark-and-sweep GC [8]. For programs with a nearly empty heap (default of R7RS benchmarks), the overhead of self-tagging’s encoding/decoding outweighs that of bump allocation.

However, since self-tagging improves performance by allocating fewer floats, its impact is more pronounced for programs that spend more time in memory management tasks. This section presents an experiment to measure execution time in a more realistic setting where benchmarks are executed with live data on the heap, thus putting more pressure on the garbage collector.

For this experiment, the R7RS benchmark suite is modified to allocate a vector before the execution of each benchmark. The size of the allocated vector is given as a parameter of the benchmark. Each float benchmark is then executed with vectors of 10^4 to 10^8 fields, which correspond to about 80 kB and 800 MB respectively. An execution is also done with no preallocated vector.

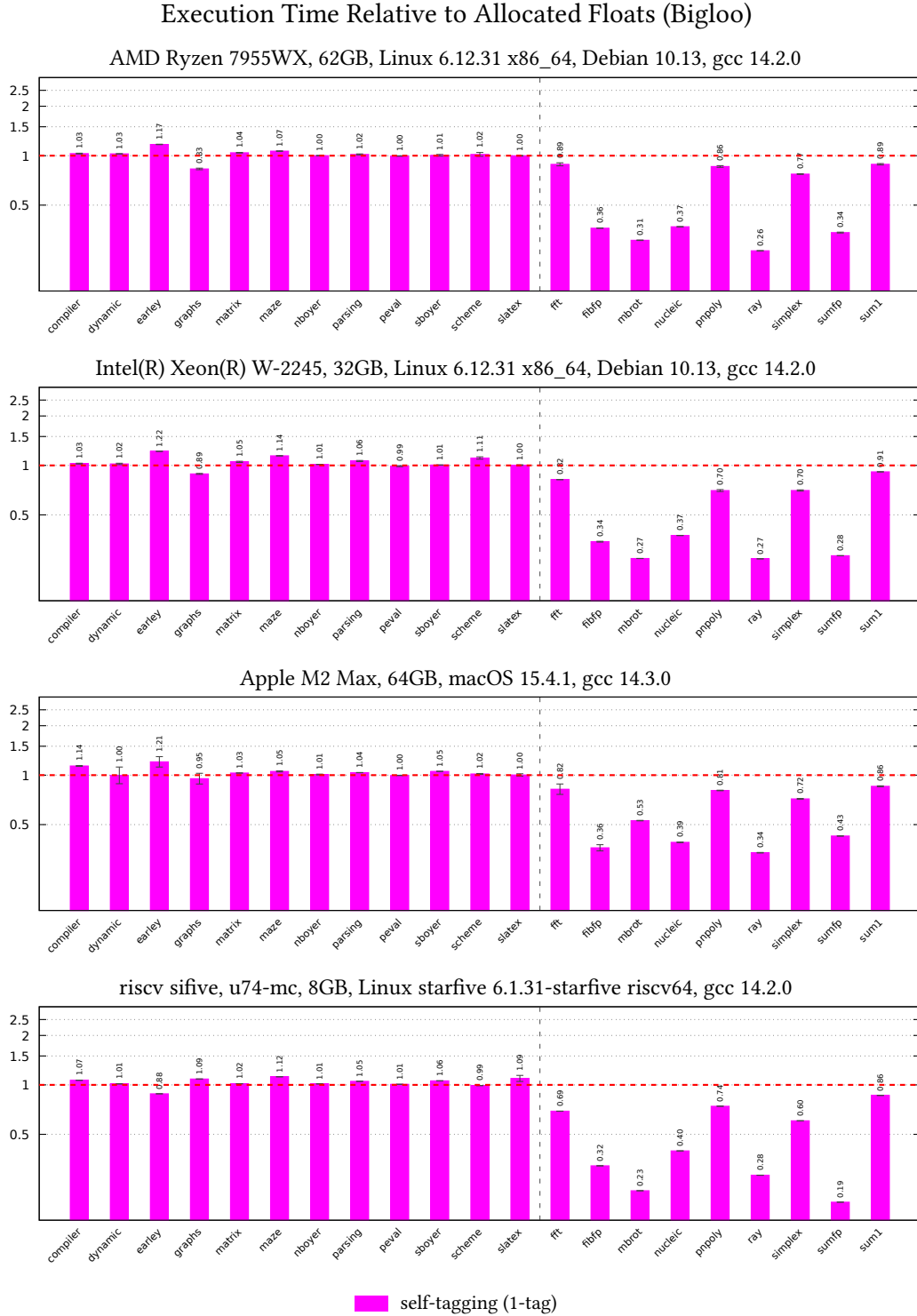


Fig. 10. Execution time of Bigloo with self-tagging (1-tag) relative to allocated floats on four distinct microarchitectures (from top to bottom: AMD, INTEL, M2, and RISC-V). The y-axis shows execution time relative to the version that heap allocates all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

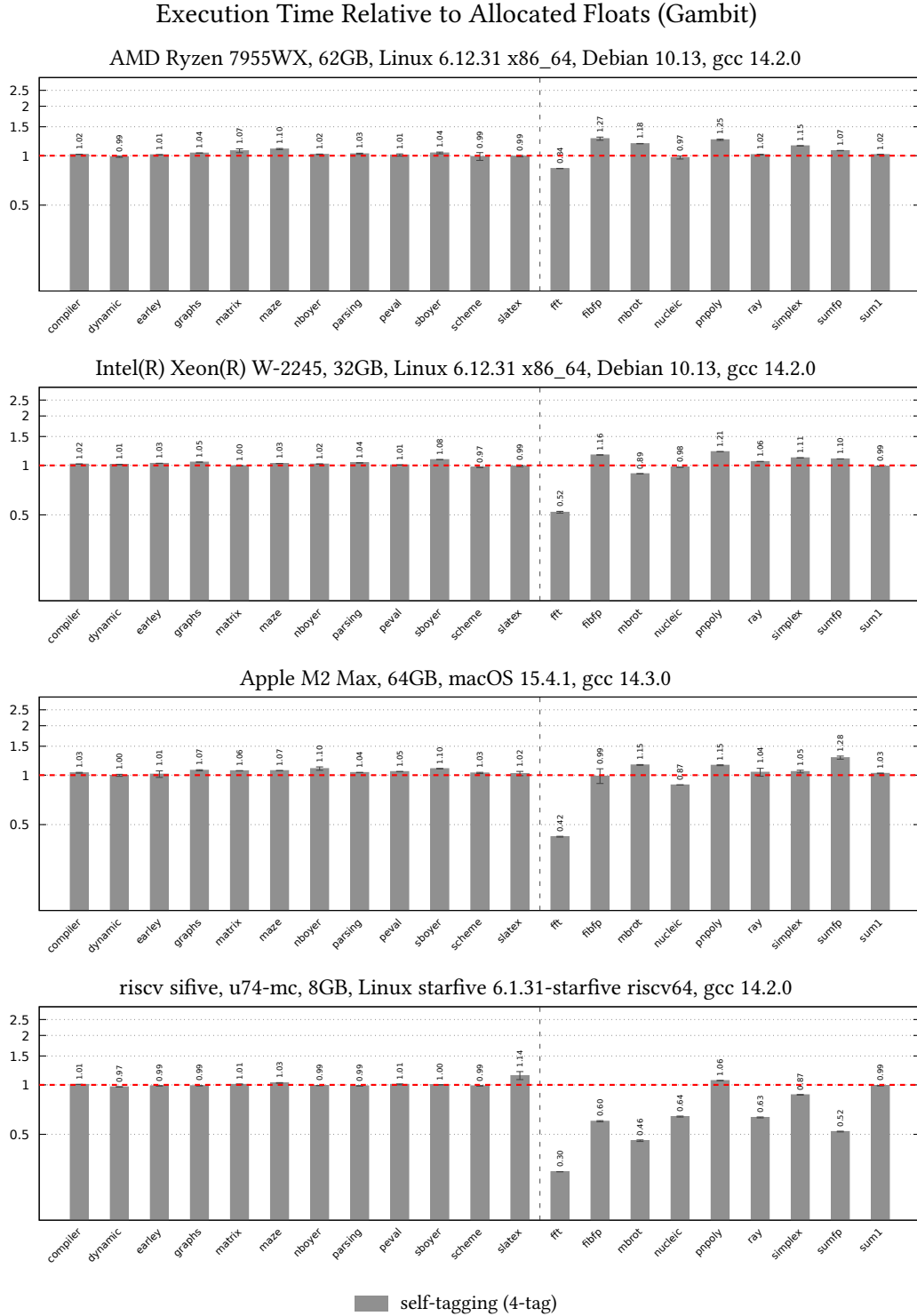


Fig. 11. Execution time of Gambit with self-tagging (4-tag) relative to allocated floats on four distinct microarchitectures (from top to bottom: AMD, INTEL, M2, and Risc-V). The y-axis shows execution time relative to the version that heap allocates all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

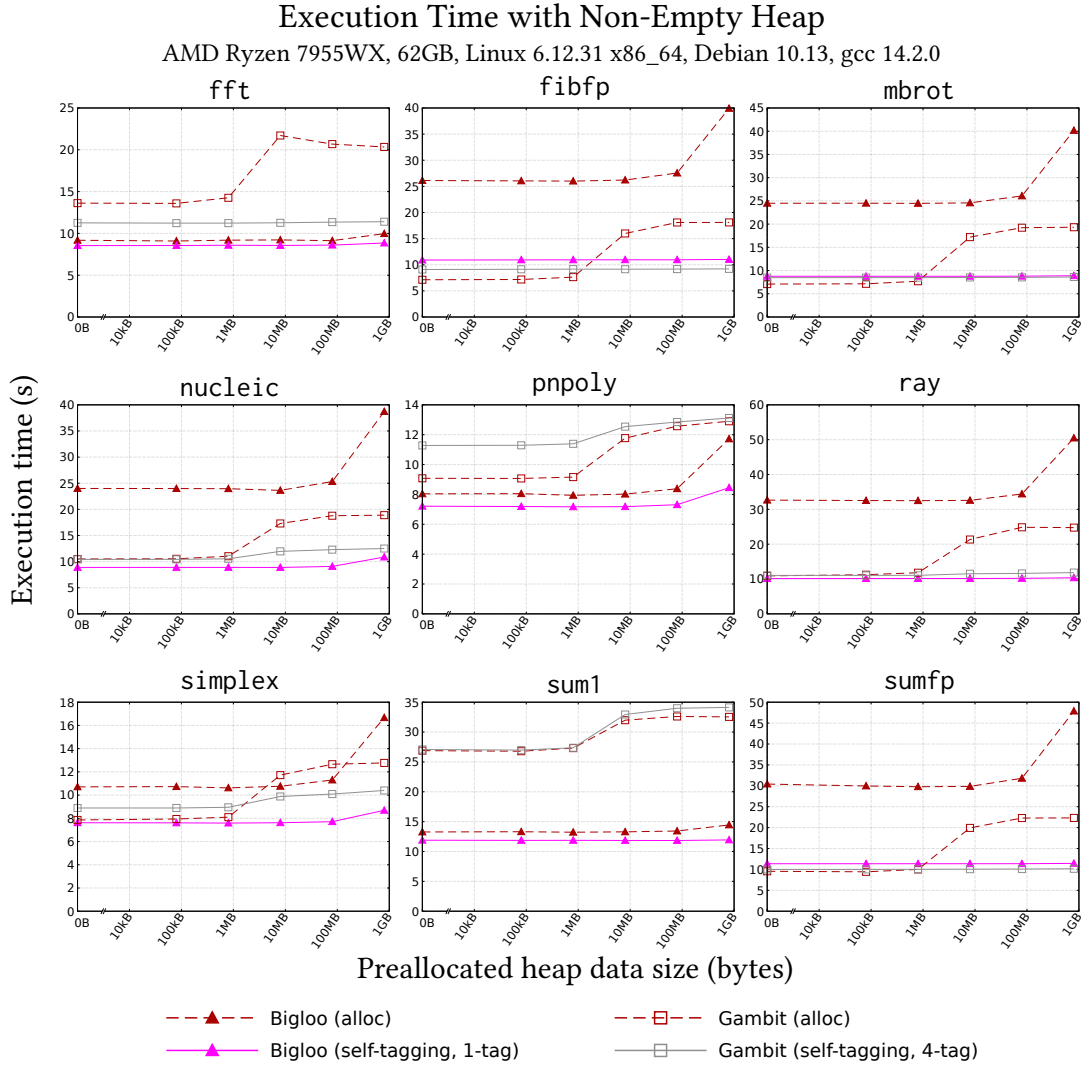


Fig. 12. Execution time of float benchmarks with extra data allocated on heap before execution with Bigloo and Gambit on AMD. The x-axis shows the size of preallocated data in bytes on a logarithmic scale. The y-axis represents execution time in seconds (mean of 10 repetitions). Execution time without preallocated data is added at $x = 0$. Dashed lines correspond to times with allocated floats. Solid lines are with self-tagging.

Figure 12 compares execution times for Bigloo and Gambit with self-tagging (1-tag and 4-tag respectively) and allocated floats with increasing vector sizes. With a nearly empty heap, and thus little strain on the garbage collector, the bump allocator of Gambit with allocated floats sometimes outperforms self-tagging since it does not incur the cost of encoding and decoding self-tagged floats. Once the heap is preloaded with about 1 MB (which is the size of the L2 cache per core on the AMD machine) the strain of the garbage collector starts to outweigh the cost of encoding and decoding self-tagged floats, thus self-tagging hereafter outperforms allocated floats.

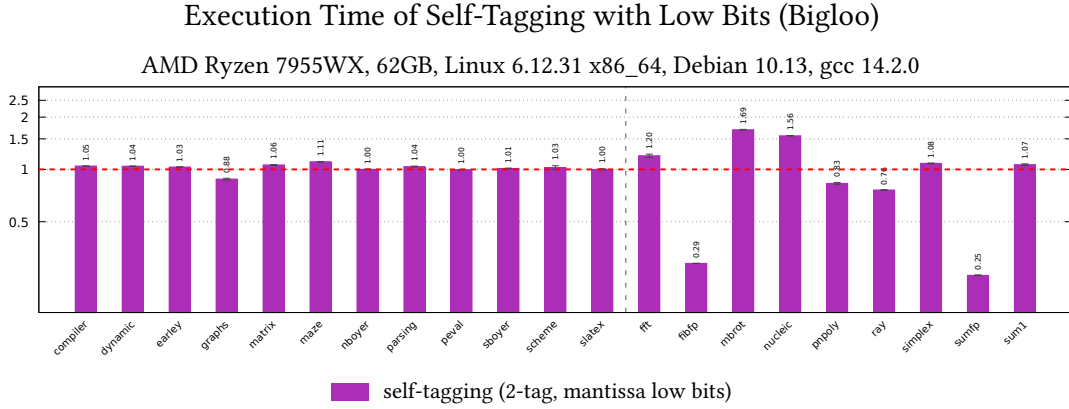


Fig. 13. Execution time of Bigloo with self-tagging using the mantissa low bits relative to allocated floats on the AMD microarchitecture. The y-axis shows execution time relative to the version that heap allocates all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than allocated floats.

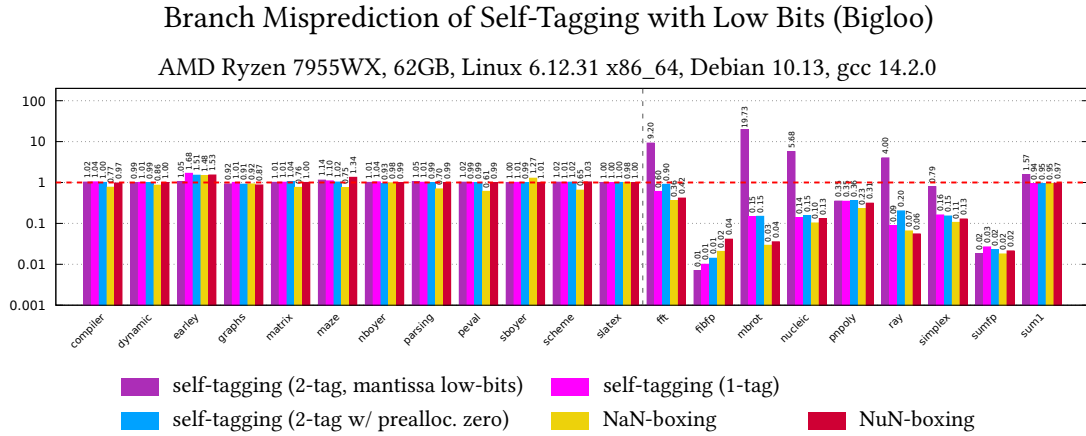


Fig. 14. Branch mispredictions of Bigloo with each variant of self-tagging, NaN-boxing, and NuN-boxing relative to allocated floats by the AMD microarchitecture. The y-axis shows branch mispredictions relative to allocated floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means less, and higher means more mispredictions.

4.6 Branch Prediction Considerations

This section discusses the impact of branch misprediction on the performance of self-tagging by presenting another variant of self-tagging that was developed based on intuitions that turned out to be wrong. This negative result is detailed here to guide implementers when adapting self-tagging.

Self-tagging variants from previous sections use high exponent bits of floats. This is convenient since it captures contiguous ranges of floats at the cost of a small encoding/decoding overhead. Alternatively, low bits of a float's mantissa can be used. For instance, by reserving the tag 000 for self-tagged floats and heap allocating all floats whose low mantissa bits are not 000.

This encoding has no overhead since a self-tagged float corresponds exactly to its IEEE754 value. It comes at the cost of no longer capturing contiguous ranges of floats, but rather a set uniformly spread across all possible floats. As in previous techniques, the proportion of captured floats can be

increased by assigning more tags to self-tagging. This variant is implemented in Bigloo with tags 000 and 100, thus self-tagging 1/4 of all floats, in particular all floats n that are integers $|n| \leq 2^{51}$.

Figure 13 compares execution times of this variant to allocated floats. As expected, benchmarks that extensively use floats corresponding to integers (fibfp and sumfp) execute faster, even faster than previous variants (see Figure 10), due to most floats avoiding heap allocation. However, other benchmarks (mbrot and nucleic) are slower despite allocating about 1/4 fewer floats.

This unpredictability is explainable by profiling the number of branch mispredictions caused by this variant. Figure 14 shows missed branch predictions of float encodings, including self-tagging with mantissa bits. This encoding causes a steep increase in missed branch predictions on some benchmarks, namely mbrot and nucleic. This stems from the fact that when tags occupy the low bits of the mantissa, even the smallest variations cause a float's representation to switch between self-tagged and tagged pointer in a hard to predict way. This is beyond the capabilities of the branch predictor and execution suffers costly misprediction penalties [11].

This result refutes the intuition that self-tagging with mantissa bits may be a good compromise due to its cheap encoding/decoding. It also hints at the necessity to avoid tests for special values in encoding/decoding, such as the test for ± 0.0 in the 2-tag self-tagging variant with preallocated zeros, which has a higher occurrence of branch mispredictions than 1-tag self-tagging in Figure 14.

5 Self-Tagging on 32-bit Architectures

Although previous sections only considered double-precision floats on 64-bit architectures, self-tagging has a natural adaptation for 32-bit architectures. Indeed, the core idea, which is to superimpose a tag to a sequence of bits likely to appear in practice, can be used for any machine word size. A popular object representation on 32-bit architectures is to use 2-bit tags (4 possible tags) and heap allocated objects aligned to 32-bit words. Figure 15 shows the ranges covered by each combination of the top 4 bits of the exponent, which is 8 bits wide in the IEEE754 32-bit representation.

The 1-tag approach described previously can be adapted to the narrower exponent of the IEEE754 32-bit representation field: the self-tagged encoding of a float whose IEEE754 32-bit representation is the integer n can be computed as $(n \oplus ((1 + 2 \times \text{tag}) \ll 27)) \ll_{\text{rot}} 4$, where $\oplus$ is addition modulo 2^{32} and $\ll_{\text{rot}}$ is the left rotation operator. If the two lowest bits are equal to tag then the float is self-tagged. This will cover all float values in the ranges: $0.0..3.9 \times 10^{-34} \cup 3.1 \times 10^{-5}..1.3 \times 10^5 \cup 1.0 \times 10^{34}..Infinity/NaN$ (bold pink rows of Figure 15 using $\text{tag}=00$).

The coverage can be doubled by using an adaptation of the 2-tag variant described in Section 2.5: $0.0..2.5 \times 10^{-29} \cup 4.7 \times 10^{-10}..8.6 \times 10^9 \cup 1.6 \times 10^{29}..Infinity/NaN$ (light pink rows of Figure 15 using $\text{tag}_1=00$ and $\text{tag}_2=11$). With this variant, it would be reasonable to reserve one tag for small signed integers in the range $-2^{29}..2^{29} - 1$ ($-536870912..536870911$), and the remaining tag for heap allocated objects, including the floats that can't be self-tagged.

6 Related Work

Handling floats is a long-standing issue in dynamic language implementation. In the last decades, little progress has been made improving the encoding of floats, with most implementations either using heap allocated floats or suffering from the overhead of NaN-boxing on pointers. Thus, more general strategies that use data flow analysis [18, 23, 27, 37, 40] and partial evaluation [44] have been developed to find locations where the type of a value is known across its lifecycle. This allows the generation of specialized code that safely handles fully unboxed, untagged values. Such strategies are not specific to unboxing floats. Rather, they tackle the more general problem of inferring types in dynamic languages, which allows dropping type information in either tagged pointers or NaN-boxed pointers. While more limited in scope, self-tagging solves the problem of heap allocated floats with a new encoding that involves no program analysis.

4 most signif. expo. bits	4 most signif. expo. bits + 1	Value	4 most signif. expo. bits	4 most signif. expo. bits + 1	Value
0000	0001	0.0	1000	1001	2 .. 1.3e5
		1.4e-45 .. 3.9e-34	1001	1010	1.3e5 .. 8.6e9
0001	0010	3.9e-34 .. 2.5e-29	1010	1011	8.6e9 .. 5.6e14
0010	0011	2.5e-29 .. 1.7e-24	1011	1100	5.6e14 .. 3.7e19
0011	0100	1.7e-24 .. 1.1e-19	1100	1101	3.7e19 .. 2.4e24
0100	0101	1.1e-19 .. 7.1e-15	1101	1110	2.4e24 .. 1.6e29
0101	0110	7.1e-15 .. 4.7e-10	1110	1111	1.6e29 .. 1.0e34
0110	0111	4.7e-10 .. 3.1e-5	1111	0000	1.0e34 .. 3.4e38
0111	1000	3.1e-5 .. 2			Infinity/NaN

Fig. 15. The positive value ranges captured by each combination of the 4 most significant exponent bits of the IEEE754 32-bit representation. The rows coloured in bold pink are covered by the 1-tag variant (tag 00) and the rows coloured in light pink are covered by the 2-tag variant (tags 00 and 11).

Self-tagging is straightforward to add to compilers that already use object tagging. Therefore, implementations that represent floats as tagged pointers could benefit from it with minimal implementation effort. Such popular implementations include CPython [15] and Google’s V8 [42]. Compilers that use NaN/NuN-boxing, such as Mozilla’s SpiderMonkey [14] and Apples’s JSC [19] can also benefit from the lower impact that self-tagging has on the performance of non-float types.

Due to the shortcomings of tagged pointers and NaN-boxing, additional strategies are used to avoid boxing. A straightforward approach is to provide homogeneous data structures such as TypedArray in JavaScript [20] and numpy’s float64 arrays in Python [17]. Type homogeneity then allows unboxing values within the collection. More generally, compilers can detect data homogeneity at run time and use context-dependent storage strategies to store data in a way that prevents pointer chasing [9]. Since self-tagging avoids heap allocating most floats in practice, it effectively prevents such pointer chasing in the case of floats without static or run time analysis.

Even for general-purpose languages not specifically designed for numerical computation, ensuring reasonably fast float operations is a major concern. All *production-quality* implementations surveyed for this paper devote significant efforts to optimizing them. Unfortunately, the academic literature is scarce when it comes to techniques used by these efficient implementations.

In many cases, the source code itself has to be studied to understand the implementation and compare it to self-tagging. The NuN-boxing implementation used in experiments in this paper is based on that of JSC.² NuN-boxing has the benefits over self-tagging that it encodes all floats uniformly and does not reserve tags for them. Its drawbacks are that it negatively impacts the type checking and accesses of all objects, it can represent integers with at most 48 bits (usually 32 bits for practical reasons), and it can’t be used on 32-bit architectures.

NaN/NuN-boxing are trade-offs between the performance of floats and pointers (NaN-boxing favors floats, NuN-boxing favors pointers). Self-tagging offers a different trade-off between the performance of frequently used floats and other floats. The fact that some ranges of floats are more commonly used has been studied in scientific computing where real world data typically fall into a narrow range of values [4, 10, 28, 45]. This is in accordance with the results presented in Figure 5.

²<https://github.com/WebKit/WebKit/commit/74c9b9bbc2c7527144ac366c3f62f84aaa1a8a4e>

The techniques closest to self-tagging discovered among surveyed implementations are those of CRuby,³ OpenSmallTalk⁴ and MonNom [34]. CRuby and OpenSmallTalk use a variant close to the 1-tag encoding of Section 2.4. The MonNom implementation from [34] uses a variant of the 2-tag encoding of Section 2.3 with 2-bit tags. Floats captured by these implementations all exclude ± 0.0 , and treat it as a special case. This imposes complex sequences of instructions for encoding/decoding that can hardly be inlined and risks increasing branch misses as discussed in Section 4.6.

To the best of the authors' knowledge, this is the first work exposing the principles of self-tagging, implementing it in optimizing compilers, and evaluating its benefits, drawbacks, and variants.

In this paper, self-tagging was discussed in the context of dynamic languages. Yet, polymorphic languages such as OCaml and Haskell also attach information to run time values and spend considerable effort to find efficient encodings for abstract data types [6, 24]. Self-tagging could be applied in these languages, either to floats or to the encoding of abstract data types.

7 Conclusion

Avoiding heap allocation of floating point numbers has always been a major concern for dynamic and polymorphic languages. This paper presents and evaluates the popular techniques deployed to that end (NaN and NuN-boxing) as well as a new approach, coined *self-tagging*, that allows representing certain floats without heap allocation.

The core idea is to apply a bitwise transformation that maps the most common floats used in practice to bit patterns that contain the required type information at the correct position. This allows encoding floats in specific ranges as tagged values instead of heap allocated objects, reducing the strain on the garbage collector and improving performance. Contrary to NaN-boxing and NuN-boxing, which are designed to optimize the handling of floats, self-tagging offers a different trade-off that adds no overhead to the handling of other types. Since it does not rely on the specificity of the IEEE754 NaN encoding, self-tagging is highly portable, including to 32-bit architectures.

It is also straightforward to retrofit in implementations that already use object tagging. The only required changes are to reserve one or more tags for float self-tagging and implement an encoding/decoding function for self-tagged floats. The rest of the runtime stays unchanged.

As shown by the experimental evidence presented, float self-tagging is a good candidate for general-purpose languages, such as JavaScript, whose specification relies heavily on floats, but where the performance of other types must not be sacrificed.

Given the purely bit-shuffling nature of float self-tagging boxing and unboxing, it may also be a good candidate for an essentially zero-cost implementation at the hardware level when floats are moved between float and integer registers.

Acknowledgment

The authors thank Erven Rohou for valuable insights and discussions that contributed to this work. This work was supported by the *Natural Sciences and Engineering Research Council of Canada* and the *Fonds de recherche du Québec*⁵.

Data Availability Statement

Data generated and analyzed for this paper is available in a companion artifact [32] that includes results for all microarchitectures listed in Section 4. The artifact also contains all source code used to run the experiments.

³<https://github.com/ruby/ruby/commit/b3b5e626ad69bf22be3228f847f94e1b68f40888>

⁴<https://clementbera.wordpress.com/2018/11/09/64-bits-immediate-floats>

⁵<https://doi.org/10.69777/366699>

References

- [1] 2019. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), 1–84. <https://doi.org/10.1109/IEEESTD.2019.8766229>
- [2] 2025. R7RS Benchmarks. <https://github.com/ecraven/r7rs-benchmarks>.
- [3] 2025. R7RS Benchmarks. <https://ecraven.github.io/r7rs-benchmarks/>.
- [4] Azim Afrozeh, Leonardo X. Kuffo, and Peter A. Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proc. ACM Manag. Data* 1, 4 (2023), 230:1–230:26. <https://doi.org/10.1145/3626717>
- [5] Andrew W. Appel. 1998. *Modern Compiler Implementation in C*. Cambridge University Press, Cambridge, UK, Chapter 16, 372–373.
- [6] Thaïs Baudon, Gabriel Radanne, and Laure Gonnord. 2023. Bit-Stealing Made Legal: Compilation for Custom Memory Representations of Algebraic Data Types. *Proc. ACM Program. Lang.* 7, ICFP (2023), 813–846. <https://doi.org/10.1145/3607858>
- [7] Fabrice Bellard and Charlie Gordon. 2024. QuickJS JavaScript Engine. <https://bellard.org/quickjs/>.
- [8] Hans-Juergen Boehm and Mark D. Weiser. 1988. Garbage Collection in an Uncooperative Environment. *Softw. Pract. Exp.* 18, 9 (1988), 807–820. <https://doi.org/10.1002/SPE.4380180902>
- [9] Carl Friedrich Bolz, Lukas Diekmann, and Laurence Tratt. 2013. Storage strategies for collections in dynamically typed languages. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013*, Antony L. Hosking, Patrick Th. Eugster, and Cristina V. Lopes (Eds.). ACM, 167–182. <https://doi.org/10.1145/2509136.2509531>
- [10] Ashley W. Brown, Paul H. J. Kelly, and Wayne Luk. 2007. Profiling floating point value ranges for reconfigurable implementation. In *Proceedings of the 1st HiPEAC Workshop on Reconfigurable Computing*. 6–16.
- [11] Randal E. Bryant and David R. O’Hallaron. 2016. *Computer Systems - A Programmer’s Perspective* (Third Global ed.). Pearson, Chapter 5, 577–579.
- [12] Intel Corporation. 2024. *Intel® 64 and IA-32 Architectures Software Developer’s Manual - Instruction Set Reference*. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [13] Robert Van Engelen. 2024. Lisp in 99 lines of C and how to write one yourself. <https://github.com/Robert-van-Engelen/tinylisp>.
- [14] Mozilla Foundation. 2024. SpiderMonkey 131.0 - Mozilla’s JavaScript and WebAssembly Engine. <https://spidermonkey.dev/>.
- [15] Sam Gross. 2023. PEP 703 - Making the Global Interpreter Lock Optional in CPython. <https://peps.python.org/pep-0703>.
- [16] David A. Gudeman. 1993. *Representing Type Information in Dynamically Typed Languages*. Technical Report. University of Arizona. <https://www.cs.arizona.edu/sites/default/files/TR93-27.pdf>
- [17] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [18] Fritz Henglein. 1992. Global Tagging Optimization by Type Inference. In *Proceedings of the Conference on Lisp and Functional Programming, LFP 1992, San Francisco, California, USA, 22-24 June 1992*, Jon L. White (Ed.). ACM, 205–215. <https://doi.org/10.1145/141471.141542>
- [19] Apple Inc. 2024. JavaScriptCore 2.46.3 - JavaScript Engine for WebKit. <https://webkit.org>.
- [20] ECMA International. 2015. *Standard ECMA-262 - ECMAScript Language Specification* (6th ed.). <https://www.ecma-international.org/ecma-262/6.0/>
- [21] Richard Jones, Antony Hosking, and Eliot Moss. 2012. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, Florida, USA, Chapter 11, 168–171.
- [22] Richard Jones, Antony Hosking, and Eliot Moss. 2012. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, Florida, USA, Chapter 9, 111–114.
- [23] Shiro Kawai. 2008. Efficient floating-point number handling for dynamically typed scripting languages. In *Proceedings of the 2008 Symposium on Dynamic Languages, DLS 2008, July 8, 2008, Paphos, Cyprus*, Johan Brichau (Ed.). ACM, 6. <https://doi.org/10.1145/1408681.1408687>
- [24] Andrew J. Kennedy and Dimitrios Vytiniotis. 2012. Every bit counts: The binary representation of typed data and programs. *J. Funct. Program.* 22, 4-5 (2012), 529–573. <https://doi.org/10.1017/S0956796812000263>
- [25] Daan Leijen. 2022. *What About the Integer Numbers? Fast Arithmetic with Tagged Integers - A Plea for Hardware Support*. Technical Report MSR-TR-2022-17. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/what-about-the-integer-numbers-fast-arithmetic-with-tagged-integers-a-plea-for-hardware-support/>
- [26] Arm Limited. 2009. *Instruction Set Assembly Guide for Armv7 and earlier Arm® architectures Reference Guide*. <https://developer.arm.com/documentation/100076/0200>.

- [27] Tobias Lindahl and Konstantinos Sagonas. 2003. Unboxed Compilation of Floating Point Arithmetic in a Dynamically Typed Language Environment. In *Implementation of Functional Languages*, Ricardo Peña and Thomas Arts (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 134–149. https://dx.doi.org/10.1007/3-540-44854-3_9
- [28] Peter Lindstrom and Martin Isenburg. 2006. Fast and Efficient Compression of Floating-Point Data. *IEEE Transactions on Visualization and Computer Graphics* 12, 5 (2006), 1245–1250. <https://doi.org/10.1109/TVCG.2006.143>
- [29] Manuel Serrano. 2024. Bigloo. <https://www-sop.inria.fr/index/fp/Bigloo>.
- [30] Marc Feeley. 2025. Gambit. <https://gambitscheme.org>.
- [31] Dave Mason. 2022. Design Principles for a High-Performance Smalltalk. In *Proceedings of the International Workshop on Smalltalk Technologies 2022 co-located with the 28th European Smalltalk User Group Conference (ESUG 2022)*, Novi Sad, Serbia, August 24th–26th, 2022 (CEUR Workshop Proceedings, Vol. 3325), Loïc Lagadec and Vincent Aranega (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-3325/regular2.pdf>
- [32] Olivier Melançon, Marc Feeley, and Manuel Serrano. 2025. Float Self-Tagging Artifact. <https://doi.org/10.5281/zenodo.16356364>
- [33] Olivier Melançon, Marc Feeley, and Manuel Serrano. 2024. Static Basic Block Versioning. In *38th European Conference on Object-Oriented Programming, ECOOP 2024, September 16–20, 2024, Vienna, Austria (LIPIcs, Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 28:1–28:27. <https://doi.org/10.4230/LIPICS.ECOOP.2024.28>
- [34] Fabian Muehlboeck and Ross Tate. 2021. Transitioning from structural to nominal code with efficient gradual typing. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–29. <https://doi.org/10.1145/3485504>
- [35] William Harold Newman, Christophe Rhodes, Nikodemus Siivola, Juho Snellman, Paul Khuong, Jan Moringen, and Douglas Katzman. 2024. Steel Bank Common Lisp 2.4. <https://www.sbcl.org>.
- [36] Mike Pall. 2024. LuaJIT 2.1 - Just-In-Time Compiler for Lua. <https://luajit.org>.
- [37] Leaf Petersen and Neal Glew. 2012. GC-Safe interprocedural unboxing. In *Proceedings of the 21st International Conference on Compiler Construction (Tallinn, Estonia) (CC'12)*. Springer-Verlag, Berlin, Heidelberg, 165–184. https://doi.org/10.1007/978-3-642-28652-0_9
- [38] Roberto Ierusalimsky. 2024. Lua 5.4. <https://www.lua.org>.
- [39] Manuel Serrano. 2021. Of JavaScript AOT compilation performance. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–30. <https://doi.org/10.1145/3473575>
- [40] Manuel Serrano and Marc Feeley. 1996. Storage Use Analysis and its Applications. In *Proceedings of the 1996 ACM SIGPLAN International Conference on Functional Programming, ICFP 1996, Philadelphia, Pennsylvania, USA, May 24–26, 1996*, Robert Harper and Richard L. Wexelblat (Eds.). ACM, 50–61. <https://doi.org/10.1145/232627.232635>
- [41] Pat Shaughnessy. 2013. *Ruby Under a Microscope: An Illustrated Guide to Ruby Internals*. No Starch Press, San Francisco, USA.
- [42] Igor Sheludko and Santiago Aboy Solanes. 2020. Pointer Compression in V8. <https://v8.dev/blog/pointer-compression>.
- [43] Sam Tobin-Hochstadt and Matthias Felleisen. 2010. Logical types for untyped languages. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (Baltimore, Maryland, USA) (ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 117–128. <https://doi.org/10.1145/1863543.1863561>
- [44] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. 2017. Practical partial evaluation for high-performance dynamic language runtimes. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18–23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 662–676. <https://doi.org/10.1145/3062341.3062381>
- [45] Niko Zurstrassen, Lennart M. Reimann, Nils Bosbach, Lukas Juenger, and Rainer Leupers. 2023. Evaluation of the RISC-V Floating Point Extensions. In *DVCon Europe 2023; Design and Verification Conference and Exhibition Europe*. 57–64. <https://ieeexplore.ieee.org/abstract/document/10461383>

Received 2025-03-25; accepted 2025-08-12